

Intro to C++ programming with SDL3

A complete guide to your first game

Max Friberg

2026-05-23

Contents

1 Acknowledgments	4
2 Foreword - How to learn how to program	5
3 SDL3 - part I	8
4 Introduction to C/C++ - Part I	38
5 Introduction to the Helix Editor - Part I	52
6 Introduction to SDL3 - Part II	56
7 Introduction to the Helix Editor - Part II	75
8 Core Loop - Part I	79
9 DLLs Memory and Hot Reloading - Part I	92
10 DLLs Memory and Hot Reloading - Part II	106
11 Rendering images	130
11.1 Updating our cmakeLists.txt	130
12 Savestates	152
13 Sokoban Programming I	156
14 Sokoban Programming II	177
15 Sokoban Programming IV	189
16 Command Pattern	196
17 Developer Tools with DearImGui	209
18 Better undo/redo	222
19 Animation Part I	226
20 Repeat Inputs	240
21 Camera	250

22 Asset Management Part I	257
23 Mouse input	267
24 Level Editor	273
25 Sokoban programming V	286
26 Animation Part II	321
27 Scratch Arena and Sprite Sorting	337
28 Spawn Commands and active/inactive entities	343
29 Scenes and transitions Part I	350
30 Tilemap parsing	365
31 Sokoban Programming V	388
31.1 Control deltatime	388
31.2 Select an active entity	389
32 Buttons Part I	414
33 Sokoban Programming VI	431
34 FMOD and Audio	444
35 Animation III	456
36 Music	465
37 Parallax	468
38 Text	473
39 Buttons Part II	485
40 Intermission I - Creating a release candidate	498
41 Intermission II - Debugging in Visual Studio	505
42 Intermission III - Github Part I	508

1 Acknowledgments

I would like to thank colleagues and friends for input and support during the creation of this material.

All content and intellectual property remain the sole work of the author.

© 2026 Max Friberg. All rights reserved.

2 Foreword - How to learn how to program

This course starts at the deep end, we will be doing a lot of manual work, learning from the ground up. This is difficult but rewarding. Your task, as a student is too:

- 1) Pay attention during lectures
- 2) Ask questions
- 3) Be curious
- 4) Accept the fact that you will be uncomfortable
- 5) Spend your school days outside of lectures revisiting material and practicing
- 6) Take notes during lectures
- 7) Start making games immediately!

Once this course is finished, you will have learnt so much that when we start working with game engines like Unity and Unreal you will breeze through it. The only thing you are not allowed to do is to give up, you are here to learn and by that very nature, you are here to struggle. You are entering a very, very(!) fun profession that other people also want to work in - we're going to prepare you for the actual job market.

Lets talk about the points above one by one:

- 1) Pay attention during lectures Not all courses and not all lectures are fun. But the result of learning those lectures is that you will be able to create really cool games - that is a good reward for your attention and patience. You decide your outlook and your mindset in the classroom.
- 2) Ask questions The lecturers are here to teach you, but they can't read your mind. Make sure you don't end up with gaps in your knowledge.

- 3) Be curious Programming is an infinitely deep subject that you can spend a lifetime learning and evolving in. A curious outlook will help you greatly!
- 4) Accept the fact that you will be uncomfortable If you expect to be great immediately then you will be very dissapointed. Embrace sucking and be kind to yourself.
- 5) Spend your school days outside of lectures revisiting material and practicing This program is full-time, meaning that the course material assumes that you will spend a full work day each day (8 hours) listening to lectures, working on assignments or studying course material. That is what is expected of you.
- 6) Take notes during lectures You will not remember everything, not all courses will feature reading material. You will be required to take notes to ensure that you can re-visit the material. I STRONGLY suggest you bring a lined A4 notebook and a Pilot Geltech pen to every class. I wish I could make this mandatory (and I might). The notebooks full of your ideas, lectures, questions and the occational grocery list will be one of the most important artifacts you will create during your time studying. Many people have terrible study-technique lets fix that!
- 7) Start making games immediately The best way for you to get better at making games is for you to make them. You should start right now!
- 8) (bonus) Programming is **not** black magic We might be unfomfortable with some hand-wavey decisions made by architects of programming languages, APIs game engines and tools. But all of them make sense, they can be learned and they are completely deterministic. If we take the time to understand them, they are completely knowable.

Best of luck, I believe in you!

3 SDL3 - part I

SDL3 is not a .EXE. it's a collection of scripts and DLL files that can be called on to access basic features like creating windows or accept input from the keyboard

NOTE: This course teaches game programming just about as “from scratch” as is educationally viable. Once we know how these systems operate and move through the semester we will off-load a bunch of this heavy lifting onto game engines and well groomed applications with a lot of large and shiny helpful buttons. This course aims to empower you by teaching you

- 1) how things work “behind the scenes”
- 2) how to not become dependant on a ready-made game engine (that might completely change their practices and routines)
- 3) if you can do this, then nothing a LIA will throw at you will feel difficult in comparison
- 4) how to survive in the uncomfortable role as someone who needs to learn new things With this in mind, dont worry, this is both difficult and easy at the same time. There is a right and a wrong way to do things AND it will only get easier and easier as the school progresses.

To start working with a new SDL3 project we need to store it in a folder. This folder will now be called our “root”

I have a folder called PROJECTS. inside this I keep all of my different game projects that I make using SDL.

in the root folder we will need the following folders

- * build
- * include

- * lib
- * assets
- * src

The build folder holds the compiled version of our game that is run and/or distributed

The include folder holds header files that tell us what functions we can call from the .DLL files

The lib (library) folder will hold all of our .DLL files

The assets folder has all our sound effects, PNGs etc

The src (source) folder has all the script files .cpp .h that we create ourselves
it's not uncommon to have a release folder as well, a version of the build folder that only release (to for example Steam) files are added to.

Once we have these folders we need to add the prerequisite .h and .DLL files to it. These have to be downloaded from Github where the makers of SDL are distributing them.

SDL on Github: [SDL](#)

from this link we can find the release page: [Release Page](#)

We will be downloading the `SDL3-devel-3.4.2-VC.zip` this `.ZIP` file, extracted using for example `7-ZIP` has the basic scripts and DLLs we need to work with SDL3. We are using x86_64-w64-VC this is the 64 bit version. The naming standards for x32 and x64 versions are confusing. If you see a i686 that is “the same” as x32. And we dont use x32

NOTE: [7-ZIP](#)

Back in our project root folder, inside the lib folder we've created we will be copying the SDL3.dll and SDL3.lib files found inside the lib/x64 of our downloaded .ZIP.

In our own include folder we will add all the .h files (for now) from the devel's include folder. I recommend grabbing the entire subdirectory (that is the name of a folder inside a folder) named SDL3 and copying that over

Our root folder should now look like this

```
root
-- build
-- include
---- SDL3
----- all the .h files
-- lib
---- SDL3.dll
---- SDL3.lib
-- assets
-- src
```

For this course, we will be writing our code in the Helix Editor

[Helix](#)

we will be going to the provided github link to download the pre-built binary. As we are on an x64 windows machine we will download the following version `helix-25.07.1-x86_64-windows.zip` and extracting the folder to a program folder on our computer.

NOTE: we need to remember this location as we will be referencing this specific adress on our computer soon

running the hx.exe will bring up the helix editor. To quit the editor (dont panic) type a `:` to bring up the command line, type a single `q` and press enter.

Additionally, for simpler text editing and pseudo-code examples we will be working with Sublime Text. Sublime Text can be downloaded here: [Sublime Text](#)

we will be starting the helix editor from the command line using Windows Powershell. If your computer doesn't already have powershell installed, then download it from [Powershell](#)

NOTE: here we have a x86 and a x64 version. We are on an 64-bit operating system so we will be using that one. The x86 version would give us the 32-bit version

We will run Powershell from the more modern Windows Terminal. Downloaded here

[Windows Terminal](#)

This supports tabs and more

There is some (and we have already started doing this) heavy lifting when setting up a development environment for the first time. Once we have everything installed and properly connected to each other we will be able to start having fun

as I mentioned earlier, we want to start the Helix Editor from our terminal using windows powershell. But powershell as no idea where

- A) The Helix editor is
- B) Where our projects live

To give Powershell access to our helix editor, we will need to create our first User Environment Variable. From the Control Panel on Windows we can find the System Properties, from there we have the Environment Variables. By adding a new entry to the **Path** list we can give Windows the ability to look inside our Helix Folder to find the files inside, for us this is the hx.exe. This new entry should have the exact path to the helix folder. So the Path should be something like this: **D:\helix-25.01.1-x86_64-windows**

NOTE: the hx.exe is not part of the Path

Once we have the User Environment Variable set up properly we can go ahead and type hx inside powershell to bring up the Helix Editor.

We are looking to make the powershell experience even better, being able to start specific projects in the Helix editor, build them and run our games without leaving our development environment.

NOTE: We already have the capability to, inside powershell, type **hx** followed by the address to a text file (scripts are just text files) this will open helix with that file selected. But typing the full address of a project file is very cumbersome.

We can create small functions that Powershell can call on. A function is a collection of commands that run in sequence. This is done by writing those functions to the profile being used by Powershell. This text file is already created on our system. By typing **\$profile** in Powershell it spits back its path.

Right now it's empty, we will need to add the functions ourselves. In Powershell we can type **hx \$profile** this will start the Helix Editor and open the Powershell profile.

Note: Helix is a text editor focused on manipulating code and can be uncomfortable to work with in the beginning. Until you have a more firm grasp of programming we will be using this editor for the course. By adding another User Environment Variable pointing at the folder where we installed Sublime Text we can type `subl $profile` instead of `hx $profile` to open the Powershell profile in Sublime Text instead

The first coding language we will “learn” is the Powershell Script language. it’s a cousin of C#, a higher level language than C++ and the language used in the Unity Editor.

Note: SDL3 uses C++ a lower level language than C#, meaning that it offers much more control but sacrifices some readability and ease-of-use to accomplish this.

Neither Sublime Text or Helix has native support for the specific syntax that Powershell Script uses. Though we can download a package for Sublime Text that adds this functionality. Inside Sublime Text, if we press CTRL+SHIFT+P we open up the command prompt. from here we can type `install package control` and press enter. Once that has installed we can go ahead and reopen the command prompt and type `install package` then after pressing enter we find a list of packages, type `powershell` and then press enter. Once the syntax highlighter has been installed we can select that specific mode from the bottom right of the screen, selecting `powershell` from the dropdown menu.

Note: Syntax refers to the way code has to be written and structured for the functions to execute and actually compile

To create our first function we need to create the functions name.

```
function hello {  
  
}
```

this is an empty function that we've named hello. Any code on a line between the curly braces { } will be executed in sequence when the function runs. Right now it's empty so nothing will happen if we run it.

NOTE: The powershell script language can be used to control so much about your computer. Though it's not something we use to make games. [strange powershell games](#)

Reading documentation is daunting but critical when wanting to understand any programming language or game framework or engine. The powershell documentation can be found here: [Powershell documentation](#)

by adding `Hello, sailor!` to the body of our hello function we have created our first function (!)

```
function hello {  
    `hello, sailor!`  
}
```

when we've made changes to our Powershell profile our functions are only updated if our \$profile is reloaded. type `. $profile` to reload the profile. Note the empty space between the `.` and `$profile`.

After the profile has been reloaded (or the program restarted) try typing `hello` and watch the terminal respond. we've called our first function starting your lifelong conversation with your computer!

We will create one more function right now, then return to our \$profile to add more functions later.

The first useful function we'll make is a "dev" function. it's job is to start

our text editor and open our game projects main script file. Called main.cpp

We haven't made this main.cpp yet, so lets go ahead and do that! in the `src` folder we created in the beginning of this lecture. in the adress bar, press to the right of the final word, then press backspace to erase the content of the adress bar, then type `powershell` this will open a new powershell window for you, but note that the text to the left of where you can type now has the address of your folder. This means that all functions we perform will happen to this folder and the files in it.

inside Powershell type `New-Item main.cpp` and press enter

NOTE: this is case-sensitive, note the uppercase letters (N and I). All code you will write will be case-sensitive, so best start learning to look for those errors.

we've created a new file! This main file is the entry point for our game, when we eventually run our game, the code found in the main file will be executed first. Currently it's empty, but that's fine.

Back to our dev function in the powershell \$profile we will create the following function:

```
function dev {  
    param([Parameter(Mandatory=$true)][string]$project)  
  
    $config = GetConfig $project  
    if ($null -eq $config) { return }  
  
    Set-Location -Path "$($config.path)\src"  
    $env:GAMEPROJECT = $project  
    hx $config.main_file  
}
```

You are not meant to know what all this means yet. So lets look at a pseudo-code version that explains what we want to happen in plain english.

- 1) require the person calling this function to also provide a project name
- 2) get a JSON file based on the provided project name
- 3) if the project had not been added to the JSON the “return” (aka do nothing)
- 4) save the name of the project to a local environment variable, this is very similar to what we did with our User environment variables earlier
- 5) start the helix editor using `hx` and supply the adress to our main.cpp found in the JSON file

Ok, our pseudo code version was simpler to follow, but we haven’t talked about JSON files. Later in the semester we will use JSON files to save our game data, both in SDL3 and Unity. Learning how to work with JSON files will be part of your skillset

Lets look at a JSON file now:

```
{
  "radio": {
    "path": "D:\\SDL_RADIO_GAME",
    "main_file": "main.cpp",
    "build_system": "cmake"
  },
  "pilot": {
    "path": "D:\\PROJECTS\\HEARTBURNER",
    "main_file": "main.cpp",
    "build_system": "cmake"
  }
}
```

We can spot two entries “radio” and “pilot”. each with their own collection of data. Reading this JSON should be pretty simple. We are storing the address path to the project folder, along with the name of our entry point (main.cpp script). The build system can be ignored for now.

Going back to our pseudo code we can imagine our user supplying either `radio` or `pilot` as the project name, and the code then fetching the relevant path and `main_file` name in order to properly start the helix editor. therefore turning:

```
dev pilot into hx D:/Projects/Pilot/src/main.cpp
```

But just having a JSON file somewhere on our computer won't make powershell know how to access its content. We need

- 1) Create a new file with the .json extension - we know how to do this now
- 2) Store the path to this JSON file in our \$profile
- 3) Create a new function that can fetch from this JSON based on the project name we provide it

Heading over to our PROJECTS folder we can use powershell and New-Item to create projects.json. All Json files start by adding a pair of curly braces

```
{  
}
```

followed by the relevant content. it's all a collection of names and data. And that data can be more data, that's when you add more curly braces one level deeper. A collection of data is stored as `"name": data` and if there are multiple ones we use a `,` at the end to list them.

NOTE: we decide the names on our own, there is nothing magic with me calling them `path` or `main_file` it's just to make it human readable

```
{
  "example": {
    "path": "PATH/TO/FOLDER",
    "main_file": "name_of_file.extention"
  },
  "second_game":{
    "path": "PATH/TO/ANOTHER/PROJECT/FOLDER",
    "main_file": "name_of_another_file.extention"
  }
}
```

JSON files can be written manually or generated by code. The fact that JSON files are so easy to read and understand means that should you find a save file in the JSON format you could open it in a text editor and play around with its content. The simplicity of how data can be converted to and from JSON makes it the natural choice for a bunch of data storage.

NOTE: another alternative to JSON is the XML, though also human readable its HTML-like syntax makes it more cumbersome to work with. And with the multitude of JSON parsers available we will rarely encounter XML files through the course material, if ever.

We want to store the location of our JSON inside our \$profile so we can access it easier. Lets create a variable to hold its value. A variable is a piece of memory that holds specific data, in our case a collection of characters called a string. We want to store the path to our projects.JSON so first we copy its path to our clipboard (CTRL-C)

At the top of our \$profile lets add:

```
$ConfigPath = "THE/COPIED/PATH/TO/THE/projects.json"
```

now whenever we type \$ConfigPath somewhere in our \$profile we are actually referencing the string with our path in the background.

NOTE: once again, there is nothing magic with the way we've named the

variable `$ConfigPath`, it could have been `$SpiderMonkey`, but that would be silly.

And now, with our JSON file prepared and its path known to `$profile` as a variable we can create a new function that uses this variable and the fact that Powershell has a JSON parser built in to help us extract the relevant data.

```
function GetConfig {  
    param([string]$projectName)  
    $config = Get-Content $ConfigPath | ConvertFrom-Json  
    return $config.$projectName  
}
```

Lets break this function down:

- 1) the user provides a project name when calling `GetConfig` that is stored in the variable `$projectName`
- 2) a new variable `$config` stores the content of the file found at `$ConfigPath` after it has been converted from JSON to a data type we can work with
- 3) we return, meaning we pass the following data back to whoever called the function, the data stored in the JSON file under the specified `$projectName`

Now we have returned a single data block from our JSON holding two pieces of data

- 1) the path to the project
- 2) the name of our entry point script (`main.cpp`)

lets go back to our dev function

```
function dev {
    param([Parameter(Mandatory=$true)][string]$project)

    $config = GetConfig $project
    if ($null -eq $config) { return }

    Set-Location -Path "$($config.path)\src"
    $env:GAMEPROJECT = $project
    hx $config.main_file
}
```

we can see that we, in common tv-chef fashion, already assumed we had created the GetConfig function. This function accepts the name of a project as a parameter, stored in the \$project variable, then **passes** this variable to the GetConfig function getting the specified JSON block back. Once that is done the data stored in the block can be used to

- 1) set the location to the src folder at that projects location on our machine
- 2) store the name of the project until we close down the terminal in a special environment variable that we call **GAMEPROJECT**
- 3) call on helix using **hx** and now that we've set the machines location to the src folder we can ask it to look for our main_file (main.cpp) in that location and open it when starting the editor.

NOTE: there is a hard to read if-statement in this function. This checks that the JSON block actually existed before trying to use it. If it wasn't found we **return** if we dont add anything to the right-hand side of a return statement that is the same as just not executing the rest of the function, stopping at that line instead.

Lets go through a checklist

- 1) we have a PROJECTS folder on our computer
- 2) inside that folder we have one or more project folders
- 3) those projects each contain a series of folders (src, include, library, build,

assets)

- 4) the src folder has a single file in it `main.cpp`
- 5) we have a projects.json file containing data blocks containing the path to our project as well as the name of its main entry point script (main.cpp)
- 6) we have functions inside powershell that allows us to find and parse this JSON data and then use it to start Helix on the relevant script to begin coding a fantastic new game
- 7) back in our project root folder we have copied over .h files and .dll files that we've downloaded from Github and put in the specified folder `include` and `lib`

Additionally we have

- A) created a few system environment variables
- B) installed a couple of useful programs (Sublime Text & Helix)

We need a few more things before we can consider our setup for SDL complete

- 1) We need a way to tell PowerShell to invoke our build system (Ninja) to build our game
- 2) We need to add some code to our entry point so we can get something to happen
- 3) We need to use CMake to generate build system files (build.ninja files for Ninja)
- 4) We need to use our build system (Ninja) to invoke the compiler (Clang) to compile our .cpp files, link them with SDL .lib files, and produce an executable .exe

Lets get cracking! first lets download the required software

cmake (Windows x64 Installer): [CMake](#)

ninja (ninja-win.zip): [Ninja](#)

LLVM [LLVM](#)

Visual Studio Build Tools **with Desktop Development with C++**
[Visual Studio Build Tools](#)

NOTE: **LLVM** contains and works with a compiler called **clang**, that is what we will be using in powershell later NOTE: we will talk about **clangd** (with a **d**) later, that's a different thing.

Visual Studio Build Tools are some required files and setup needed to make our other tools (clangd, cmake, ninja) know how to locate and find microsoft features and c++ standard libraries.

When installing Visual Studio Build tools we need to also check the box labeled **Desktop Development with C++**. Then press **install**.

It can be difficult at first to know if everything has been downloaded correctly and User environment variable having been correctly set up. A good way of testing if we can access our software through the command line is to type the following

```
`software` --version
```

into Powershell, substituting **software** for whatever we want to check
hx --version **subl --version** **cmake --version**

We want to ensure that our **clang** is of the type MSVC and not MinGW. This can be confirmed with by calling **clang --version** and making sure the target is: **x86_64-pc-windows-msvc**

If powershell throws an error here we have either not installed the software or we dont have a User environment variable pointing to its folder. We can also

use the command `where.exe software` to find where the .EXE is located on our computer.

NOTE: I strongly recommend setting up a logical folder structure for where to keep these things. And if you change that location at a later date, remember to update your User environment variable paths to match the new location.

Once everything is downloaded we need to set up our environment variables to their respective folders. Important to note is that when installing for example LLVM the installer adds the installation location as a SYSTEM environment variable for us. This works very similarly to a USER environment variable. If we just look at our User environment variables we might be surprised that even though we cant find the LLVM folder listed, we can still run `clang --version` and Powershell shows us the correct info instead of throwing an error. To list all the environment variables on our computer we can run `$env:PATH -split ';'` this takes each environment variable and prints it to its own line.

NOTE: if we remove the `-split ';'` then all paths just get printed to the same line - much harder to read

So with cmake and llvm hopefully automatically added to our system environment variables we just have to add a new user environment variable path to our Ninja folder.

These three pieces of software are responsible for taking our project and producing a .EXE. Each software used in order.

cmake: Build System Generator

ninja: Build system

clang: Compiler

cmake uses a text file we write ourselves, called `CMakeLists.txt` to learn

what content our project consists of, and what we want to do with these files and assets.

ninja then takes this set of instructions from cmake and using a compiler (clang++) performs actions on the files of the project based on the instructions provided by the build instructions created by cmake. Ninja tells the compiler what actions to perform and how those can be optimized for speed and making sure that things that need to compile first are done so.

our compiler, clang++, is responsible for actually turning our c++ code into binary (machine code) that the computer can run. All coding languages eventually compile into machine code, as computers cant work directly with the english syntax that we write.

A (very) crude way of looking at it is:

```
> Programmer = Architect - makes blueprints
> cmake = General contractor - Interprets blueprints, writes work orders for you
> Ninja = Foreman - Schedules workers, coordinates tasks, maximizes parallel wor
> Clang = worker - Actually builds the house
> Visual Studio Build Tools - necessary to work with windows and c++. Bricks and
```

we have a dev and a getConfig function. it's time to use our toolchain to prepare a function that builds our project

We can do this in two ways

- 1) tell cmake what we want directly in the function
- 2) create a JSON file with our settings and giving it this to work with instead

Below is a breakdown of method #1. Though this can be skipped as we will be working with method #2.


```

function build {
  param (
    [parameter(Mandatory=$true)][string]$project
  )

  $config = GetConfig $project
  $SourceDir = $config.path
  $BuildDir = "$SourceDir/build"

  $cmakeArgs = @(
    -S`, $SourceDir,
    -B`, $BuildDir,
    -G`, `Ninja`,
    -DCMAKE_CXX_COMPILER=clang++`
  )

  cmake @cmakeArgs
  cmake --build $BuildDir
}

```

the \$cmakeArgs is an array, like our JSON blocks, this is a list of data. Though in the case of our \$cmakeArgs the data is just conveniently stored together and under the hood all of these arguments are layed out as a series of instructions to cmake.

```

cmake @cmakeArgs
is the same as:
cmake -S $SourceDir -B $BuildDir -G Ninja -DCMAKE_CXX_COMPILER=clang++

```

NOTE: we've added parameters to our own functions before, if we wanted to have more than 1 parameter then we would keep adding them after one another just like the expanded cmake line above. Storing all the parameters in an array makes the line easier to read as well as simplify changing one of these parameters as they live on separate lines.

cmake \$cmakeArgs prepares our cmake generator with links and references to the correct folders and instructions on what generator (ninja) and what compiler (clang++) we plan to use

NOTE: this line `-DCMAKE_CXX_COMPILER=clang++` is pretty hard to parse

and remember how to write, that's why we have the documentation, lectures, AI-help and forums. How were you supposed to know that the syntax had to be written like that? you really weren't sadly, you basically just have to learn it, remember it or at least remember where to look it up.

For this course we will be working with method #2 instead:

We will create two files in the root folder of our project

- 1) CMakeLists.txt
- 2) CMakePresets.json

CmakeLists.txt is a set of instructions that will be performed by Ninja
CmakePresets is a settings file in JSON format that tells cmake what we will be using (ninja, clang) as well as additional information about our build based on what preset name we provide it.

NOTE: we can create multiple presets making swapping between a debug version and a release version as simple as swapping between -debug and -release when calling our cmake -build command

we want to use Powershell's New-Item function to create the two files listed above. But remember that any Powershell function will be executed in the directory (folder) we're currently working in.

To simplify our lives, we will create - you guessed it - another function

```
function goto {  
    param(  
        [string]$project  
    )  
    $config = GetConfig $project  
    Set-Location -Path $config.path  
}
```

now we can type `goto project_name` and the working directory will change

to the root folder of that project. Now if we use New-Item our newly created files will be created in that directory.

NOTE: the reason we use a JSON preset file is:

- 1) it is easily shared inside a team rather than each programmer having to set up their own. We share this by using Git
- 2) the reason above is why it is preferred by AAA teams. We cant afford that two programmers building the same thing yield two different results
- 3) it's good that we start learning JSON as much as possible
- 4) once set up it simplifies our lives

NOTE: if we make a type-o when calling dev, build, goto etc we currently dont have anything to handle that mistake. And we will get some nasty errors. Later in this course we will expand on each of these functions to put in some conditional checks to help protect us from “butter-fingering”

now that we are in the same working directory as our newly created JSON file we can open it in sublime text using `subl CMakePresets.json`

NOTE: to learn more about CMakePresets, check out the documentation: [CmakePreset Documentation](#)

lets create the minimum required configure preset and build preset to get started

```

{
  "version": 3,
  "configurePresets": [
    {
      "name": "default",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "CMAKE_CXX_COMPILER": "clang++"
      }
    }
  ],
  "buildPresets": [
    {
      "name": "default",
      "configurePreset": "default"
    }
  ]
}

```

First of, we have 2 presets, both named “default”. one is a configurationPreset and the other a build preset. The configuration preset is used in the first step of the cmake process, this dictates what generator (ninja) and what compiler (clang++) we’re using. The second preset, the build preset, is a smaller preset that tells cmake what configurePreset to check information about when actually telling ninja to build the project. Different build systems allow for different ways to handle the buildPresets and configurePresets - but Ninja needs us to have one buildPreset per configurePreset, meaning that the distinction becomes almost irrelevant and mostly a tedious chore to manage.

NOTE: This is known as boilerplate code and just has to be there for everything to function.

some of these we can name as we please, but most need to be written very exactly. The “version” field is not the version of our preset or the version of our game, it tells cmake what version of JSON we are using. This should always be 3. “configurePresets” and the name of the fields inside it “name”,

“generator” and the array “cacheVariables” have to be named exactly as they are - but we do get to put in whatever name we want for our preset on the “name” row. Here we use the name “default”.

NOTE: cmake syntax and powershell syntax are not the exact same, sadly. So even though both use \$ to specify a variable we have to put the variable inside a pair of curly braces {} when referencing it in our presets JSON. otherwise cmake cant read it.

now lets utilize this preset to simplify our build function

```
function build {  
  param(  
    [Parameter(Mandatory=$true)][string]$project,  
    [string]$preset_name = "default"  
  )  
  goto $project  
  cmake --preset $preset_name  
  cmake --build --preset $preset_name  
}
```

cmake without the `--build` looks for a configurePreset to work with. cmake `--build` looks for a buildPreset to use

look, we have 2 parameters, both with additional guardrails in place. The \$project parameter must be added or else the function wont run. And the \$preset_name can be skipped, but in that case it will use the provided value instead of nothing. We have provided it the default name “default”.

our build function is done, we have a preset that can be loaded to provide cmake with the settings we want. But we still have two more steps before we can build anything

- 1) add instructons to our CMakeLists.txt file that we created alongside our CMakePresets.json file
- 2) write a super small SDL game inside our main entry point (main.cpp)

without any information in our CMakeLists.txt our compiler has no idea what files it should add to our game, and how those files should be bundled. It won't make sure that any sprite or sound effect we've added to our asset folder will be available in our game and no code will be compiled to our .EXE and without a main.cpp to try and find a main() function inside of (this is the entry point) it will fail to create the EXE

lets `subl CMakeLists.txt` as we are already in the root.

NOTE: in case we've restarted our terminal or moved directory we can use our handy `goto project_name` function to get our directory back to the root folder

Below is our minimal version of the CMakeLists.txt

```
cmake_minimum_required(VERSION 3.25)
project(Heartburner LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)

# Automatically find all .cpp files in src/
file(GLOB_RECURSE SOURCES "src/*.cpp")

# Create executable
add_executable(${PROJECT_NAME} ${SOURCES})

# Add include directory
target_include_directories(${PROJECT_NAME} PRIVATE include)

# Automatically find and link all .lib files in lib/
file(GLOB_RECURSE LIB_FILES "${CMAKE_SOURCE_DIR}/lib/*.lib")
target_link_libraries(${PROJECT_NAME} PRIVATE ${LIB_FILES})

# Automatically copy all .dll files to executable directory
file(GLOB_RECURSE DLL_FILES "${CMAKE_SOURCE_DIR}/lib/*.dll")
add_custom_command(TARGET ${PROJECT_NAME} POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_if_different
        ${DLL_FILES}
        "$<TARGET_FILE_DIR:${PROJECT_NAME}>"
)
```

Before learning how this works we will take a break just to summarize all of

the things you've begun to learn

- JSON syntax
- Powershell coding syntax
- cmake syntax
- how to use the terminal to issue commands
- what steps are required to compile a game
- how to set up a development environment from scratch
- how to create and manage User Environment Variables
- the difference between system and user environment variables
- file and folder structure for game projects
- how to work with text editors (Helix and Sublime Text)
- the relationship between build systems (cmake, ninja) and compilers (clang++)
- how to download and integrate external libraries (SDL3)
- how to create and use functions with parameters
- how to work with variables and strings
- the concept of entry points in programming (main.cpp)
- how different file types work together (.dll, .h, .cpp, .json, .txt)
- basic project organization and directory management
- how to parse and read documentation
- the importance of case-sensitivity in programming
- how to navigate and manipulate file paths in Windows

That is seriously impressive! we've begun to lay the groundwork necessary to start making awesome games! In the next segment we will talk a bit more about `cmakelists.txt`, then create the most minimal program then compile and run it!

Our CmakeLists.txt does only what is absolutely necessary to build a mock-application. But let's talk about what is there and what is missing

```
cmake_minimum_required(VERSION 3.25)
project(Heartburner LANGUAGES CXX)
```

these two lines have to be the first things added to our cmakeLists.txt they specify what version of cmake will be used and what the name of the project is. Once we have added the game name we can keep using it in the cmakeLists.txt by using the variable `${PROJECT_NAME}` this is created automatically in the background from the `project()` function.

NOTE: We are making a C++ project, that is our language. The reason it says CXX and not C++ is because way back in the olden days, the operating system couldn't handle `++` symbols in text, so they used X instead.

```
set(CMAKE_CXX_STANDARD 20)
```

Languages, like C++ and C#, get new updates with additional functionality. This line tells cmake that we're using version 20 of C++. The reason we don't all just use the newest version is because changes in the syntax makes old code not compileable with newer versions of C++ without going through it and making the necessary changes.

```
# Automatically find all .cpp files in src/
file(GLOB_RECURSE SOURCES "src/*.cpp")
```

We ask it to, using the `file()` function to find all files in the `src` folder with the extension `.cpp`. here the asterisk `*` acts as a joker, allowing any word to be substituted for it. If we had multiple `.cpp` files (and we will later) this is required. `SOURCES` is the name we give a new variable (later used with `${SOURCES}`) this is created by the `file()` function. the `GLOB_RECURSE` parameter tells the build system to fetch all files even if they are in subdi-

rectories (folders) deeper than `src` this way we can clean up our `src` folder later by putting multiple scripts in different folders instead of having 100+ all layed out in the `src` folder.

```
# Create executable  
add_executable(${PROJECT_NAME} ${SOURCES})
```

Here we take our long list (eventually) of `.cpp` files found in our `src` directory and create the executable providing first its name and then the files that will be used to construct it

```
# Add include directory  
target_include_directories(${PROJECT_NAME} PRIVATE include)
```

Ok, so now we have to talk a bit about the structure of C++. We write two types of files

- 1) `.cpp` files
- 2) `.h` (header) files

Header files contain, with only a few exceptions, no code that will ever be run in the executable, rather a header file contains a list of names of functions and some more static logic (we will look more at this later) that we swear that any `.cpp` file that includes this header will have access to. it's common to have `bomb.h` and `bomb.cpp` with the `.h` file promising that any `.cpp` file that uses it will have access to a function called `Explode()`

When compiling our project, the content of the `.h` files are copied and pasted into all the `.cpp` files that have included them using the `#include <bomb.h>` syntax at the top of the `.cpp` file. So when we write a `.cpp` file we dont add the content of `.h` ourselves at the top, this is done at compile time.

The code above lets the build system know where it can find `.h` files inside our project so the compiler can add their contents to the top of all the `.cpp` files

NOTE: C# does not have .h files just .cs files (cs stands for c-sharp aka C#)

```
# find and link all .lib files in lib/
file(GLOB_RECURSE LIB_FILES "${CMAKE_SOURCE_DIR}/lib/*.lib")
target_link_libraries(${PROJECT_NAME} PRIVATE ${LIB_FILES})
```

.dll files are not programs but like our .exe contain machine code, compiled from a source code (one or more .cpp files). a dll is then able to be utilized by a program to run its machine code. a dll has no entry point, like an exe does, and relies on another program to call into it.

.lib files are import libraries that describe the functions exported by a .dll. They are used during linking so that a program can reference functions implemented in the DLL, which will be loaded and executed at runtime.

NOTE: Linking is a step that comes after compilation, where the different parts of a program are connected so they can reference each other and work together correctly.

NOTE: Some files may have the extension .dll.a. these work the same as .lib files, but indicate that you have downloaded the wrong version of the specific package. There are 2 primary targets for compilers

- 1) MSVC -> .lib
- 2) MinGW .dll.a

```
# Collect all .dll files inside `lib`
file(GLOB_RECURSE DLL_FILES "${CMAKE_SOURCE_DIR}/lib/*.dll")

# after our build is done, check each dll file and add it to the build folder if needed
add_custom_command(TARGET ${PROJECT_NAME} POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_if_different
        ${DLL_FILES}
        "$<TARGET_FILE_DIR:${PROJECT_NAME}>"
)
```

Since our program actually needs to run code found in .dll files we need to

make sure that these files are copied over to the build folder. First we collect all of them from our lib folder and store them in a variable we've named `DLL_FILES` then once our build is finished we ensure all of these are copied over to the build folder.

NOTE: this code is pretty dense, lets leave it for now and dive into the how and the why later

our `cmakelists.txt` is finished (for now). It does everything we need it to do and is configured for C++. Now we will:

- 1) open up our `main.cpp` and write a minimal program
- 2) use our powershell function to build the executable (.exe) by using our preset (`cmakepreset.json`) and our build instructions (`cmakelists.txt`) to tell our build system (ninja) to use our compiler (clang++) to compile the scripts into machine code and then link them together.
- 3) run our first program!

lets start development

```
dev pilot
```

NOTE: my project is called pilot in my `projects.json` read by my powershell function

NOTE: if the lecture series hasn't gone over VIM motions and helix, change the dev function to not call `hx` but instead `subl`. This will open the sublime text editor instead of helix if necessary.

inside our `main.cpp` we will add the following code

```
#include <windows.h>

int main() {
    Sleep(2000);
    return 0;
}
```

Ok! now because we have done so much from scratch we already know what this program is doing.

first we include a .h file called windows.h, this is part of windows and always available to be used. This .h file gives us access to the built in windows function Sleep(). Without this include our program would not know what Sleep() was and the compiler would not copy and paste the correct code to the top of our main.cpp when building.

then we create a function called `main` it has to be called exactly main, as our compiler will hunt for this function to make it our entry point. The code inside main() is the first code being executed when running our .exe later

the `int` in front of main is the c++ way of declaring it a function. In powershell we wrote function explicitly. int actually stands for integer (whole numbers) meaning that this function will eventually return a number - in this case it returns a `0`

the 2000 inside the sleep function is a parameter passed into it, representing the amount of milliseconds to wait before executing the next line of code. Once the program hits the `return 0` we “return” from our main entry point and the program closes.

NOTE: we use different integers to represent different conditions that caused the program to end. a 0 means it exited without any errors.

NOTE: c++ requires us to end each line of code with a semicolon `;`

So our program will do nothing (but still exist) for 2000 milliseconds AKA 2 seconds, then return 0 and at that point the program closes.

if we go ahead and save our `main.cpp` file then build our project using `build pilot` we will get an executable in our build folder. Double-clicking it will launch our program, the entry point being our `main()`. The program then does nothing for 2 seconds followed by it closing.

Boom!

We have now touched on ALL of the necessary (and some more advanced) systems, software, methods and functions used to create a game from scratch!

At this point the next steps ahead of us are:

- learn the basics of C++
- making fun systems through code
- add images and sound effects
- add more instructions to our `cmakelists.txt`
- add more presets to our `cmakepresets.json`
- add more helpful functions to our powershell `$profile`
- make the powershell functions we've already made more robust

It was hard, it was complex but this was the first, and biggest, step you will take on your journey to be a programmer! Going back to repeat steps and learning the ins-and-outs of these systems will be vital to actually understanding them. But now, just like our little program, it's time to `Sleep()`;

4 Introduction to C/C++ - Part I

Remember sdl3, cmake, and ninja, we're going to ignore those for now as we will need to learn more about c++ before we can honestly start working with SDL3. As soon 95%+ percent of all code we'll write will be c++, only writing powershell script or cmake-syntax from time to time.

The next step is to create a new empty .cpp file, open it, write some code, then use clang to compile it. Once that is done we run it. The reason we can do this is that we're just compiling a single c++ script, no need for a build system generator (cmake), instructions (cmakelists.txt) or a build system (ninja) because we require no linking or multiple files that need to be compiled together.

we need to update our directory to somewhere we can create and access a small .cpp file. This can be done by using the basic building blocks found in terminal programming

NOTE: a full exhaustive list can be found here [Windows Commands](#)

We will instead create a little function in \$profile (powershell) to help us get things set up.

NOTE: I like the convenience of the following setup as I will show you a bunch of coding examples throughout the course.

```
function practice {  
    Set-Location -Path "D:\PROJECTS\PRACTICE"  
    New-Item example.cpp -Force  
    subl example.cpp  
}
```

this function sets our working directory to a folder I've already created NOTE: the path will not be the same on your computer, it depends on where you

decided to create your folder I create a new file called example.cpp I then run Sublime Text opening up that newly created file NOTE: the -Force parameter will force our computer to re-create the file, therefore making it empty

now we need a function to help us compile and run this small program

```
function practice_run {  
    clang++ example.cpp -std=c++23 -o example.exe  
    if ($?) {  
        ./example.exe  
    }  
}
```

Ok, so this function first runs a line of code then it asks a question in form of an if statement. an if statement, used in all programming languages you will come across, is one of the most fundamental ways of controlling code flow. Lets explore it a bit

```
if (question) {  
    here we can do things only if the answer to the question was true  
}
```

we can put any question we want into the parenthesis following our 'if'. Lets for example imagine this being a nightclub

```
if (age > 18) {  
    welcome in  
}
```

we can also explore code that only runs in case the answer is false using 'else'

```
if (age > 18) {  
    welcome in  
}  
else {  
    get outta here!  
}
```

With this in mind, lets break down this function: using our system environment variable Path to the folder containing clang++ we get to call into it without

specifying ITS/FULL/PATH What follows are a set of instructions to clang:
* 'example.cpp' (the name of the file we want to compile) * '-std=c++23'
(tells clang++ to use c++ version 23, this will help us make some simpler
code later) * '-o example.exe' (specifies the name of the output file, namely
example.exe) NOTE: without -o, the program is given the default name 'a.exe'.
So whilst not a strictly necessary parameter, is help keep things tidy * 'if
(\$?)' is true if the code we ran above it succeeded (returned 0) NOTE: not
encountering an error means that the program exited with the exit code 0
- just like in our first SDL3 main.cpp example. 0 means 'all clear' had the
program returned any other int (integer) then the code inside the if statement
would not be executed * ./example.exe (look for a file in this directory called
example.exe and run it)

after saving our \$profile file we need to reload it in powershell before these
two newly created functions can be found. Running this simple line of code
does the trick

```
. $profile
```

lets add some code to our example.cpp

```
#include <print>

int main() {
    std::println("hello, sailor!");
    return 0;
}
```

All C++ programs needs an entry point. this is the function called main. It
has the int (integer) return type and at the last line of the main() function
we return a '0' meaning 'the code ran without issue'.

We want to be able to write text from our program into our terminal so
we want to use the println function (this stands for 'print on new line').

The `println` function is made available by the inclusion of `<iostream>` at the top of the program.

`'std::'` that comes before `println` is a safeguard put in place by the ISO C++ Standards Committee way back in the 90's. All functions added to the standard library are put into a namespace called `std` (stands for 'standard') this has two purposes 1) clearly show if a function was from the standard library by the addition of `std::` 2) through the use of a namespace, it allows us, the programmer, to write our own `println` function if we would like, and not causing a compile error when the compiler finds 2 functions with the same exact name.

writing the `'std::'` namespace indicator everywhere is pretty tedious. We can tell our program to look in the `std` namespace for functions by telling it to use that namespace. This is done by adding the following code

```
using namespace std;
```

making the new program:

```
#include <print>
using namespace std;

int main() {
    println("hello, sailor!");
    return 0;
}
```

We could then remove `'std::'` and just write `println()`.

NOTE: namespaces will be part of all games you will work on, including those using C# and game engines like Unity

You might have noticed that we add a semicolon to the end of all lines in C++. This is a required step dating back to the C programming language

and the inception of C++. It was added for clarity, to show when we are at the end of a line. It's something we just have to accept, for both C++ and C# (unfortunately).

NOTE: Thankfully our intellisense will catch us if we miss a ';'. And if it doesn't then our compiler will spit out a pretty clear error message when it fails to compile our program.

main() is a function, println() is a function. All functions have a pair of parenthesis after its name, this parenthesis hold the parameters we can send to the function.

Lets create a small function that adds two numbers together. To do this we leave the main() functions scope, marked by the curly braces '{}'

```
void AddNumbers(int a, int b){  
    int result = a + b;  
    println("{} ", result);  
}
```

a return type of void means that the function doesn't return anything. And therefore a 'return X' line is not added (as that would create an error). By introducing two integers (int a and int b) to the parenthesis of the function we have declared that anyone using this function must provide two integers separated by a ','

there are many ways of printing something to the console, the println function can't work with an integer directly. Other methods like 'cout' can. By adding "{}", as the first parameter to println it can take the second parameter (in our case the nubmer 15) and replace the placeholder "{}" with it later. But don't worry, there was really no way for you to know this, and no way from reading the code for us to understand this syntax, some parts of how code is written we kinda just gotta learn.

if we use the following program instead, including instead of and working with the older ‘cout’ syntax we can get the same result with a program that looks like this instead:

```
#include <iostream>
using namespace std;

void AddNumbers(int a, int b){
    int result = a + b;
    cout << result << endl;
}

int main(){
    AddNumbers(5, 10);
}
```

But notice those strange ‘<<’ we will not be seeing them in any other setting than these practice functions. Thankfully SDL3 has a helpful function that works like println but can accept more types of data without needing the strange placeholder ‘“{ }”’.

Lets learn about scope. The newly created integer variable “result” is only available inside the curly braces of the function. Once the code reaches the end of the last line all locally scoped variables are cleaned up. NOTE: a variable is a named piece of memory that stores something for us (a number, a word, a sentence etc)

updating our main() function we can take a look at our program:

```

#include <print>
using namespace std;

int main() {
    AddNumbers(5, 10);
    return 0;
}

void AddNumbers(int a, int b){
    int result = a + b;
    println("{} ", result);
}

```

Now we can clearly see the difference between the function itself and calling that function. Our main function uses the AddNumbers function inside its scope then outside of the main() function scope we write the actual function.

BUT! there is one problem, in C++ (not in C#) we can't use a function by another function before it has been seen by the compiler, and that happens in a top-down fashion. This feels silly and like something the computer should be able to handle, and here we touch on the idea of how programming languages are written by people and have different philosophies and trade-offs. By forcing the declaration of functions to be done in sequence the compiler can work faster.

NOTE: with even simple Unity projects taking AGES to compile, I would like to stress the importance of a design decision as this one.

Swapping the position of the AddNumbers() and main() functions then compiling and running our program it spits out 15, the combined total of the two values we passed to the function (5 and 10) that are then printed to the console via the println() function. After that we hit the return 0 and the program closes.

What we've created is sorta the world slowest and worst calculator. But it

has taught us a few things:

- 1) what the difference between an int and a void function is
- 2) what scope is
- 3) how to wrangle the println function and other ways we can print text to the console
- 4) what a namespace is
- 5) that we can compile a simple program with cmake or ninja
- 6) Function declaration order matters in C++
- 7) The difference between calling a function and defining a function
- 8) How parameters are passed to functions
- 9) how if statements control code flow
- 10) that we need semicolons at the end of lines

Introduction to C/C++ - Part II

This lecture will focus on how we use and write our own .h files. .h files, called headers. Are small(er) files containing function declarations but not their bodies. Meaning that we don't write the logic inside their scopes, just their names, return types and the parameters they accept

```
void Add(int a, int b);
```

We will use 'New-Item' to create a new .h file called example.h and the code written above will be the only content of this .h file (for now)

back in our example.cpp we can now include this .h file at the top.

```

#include <print>
#include "example.h"
using namespace std;

int main() {
    Add(5, 10);
    return 0;
}

void Add(int a, int b){
    int result = a + b;
    println("{} ", result);
}

```

NOTE: I also simplified the function name in the .h file and our .cpp file to just 'Add'

because our .cpp includes this .h file we get it added to the top of the file during compilation.

If you've noticed that when we include a .h file we do it by declaring its path (based on the root directory) using quotation marks "" but when we added print we used angle brackets '<>' angle brackets are used to include system files whos locations are already known to our program like those that are part of the standard library

Lets learn about return types, our Add function is poorly named because nowhere in the name 'Add' can we infer that it prints the result. Lets change the return type on our Add() function from void to int. This will require the function to use a 'return'

```

int Add(int a, int b){
    int result = a + b;
    return result;
}

```

our function no longer prints anything, instead it returns the integer we've named result. We have to do 2 things to get our program to compile and get

the number to print to the console First: update our .h file so the function declaration matches the changes we made in the .cpp file (changed void to int) Then: we update our main() function to print the result of the function itself, but we will use some fancy footwork to put the ‘println()’ and ‘add()’ functions on the same line.

```
int main(){  
    println("{} ", Add(5, 10));  
    return 0;  
}
```

Look we’ve added the function Add() as the parameter we pass to the ‘println()’ when our code executes it will call the Add function and whatever value we return at the end will be substituted in place of the function call. So it looks something like this in the end:

```
println("{} ", 15);
```

In case we find the function-as-parameter thing difficult at this time, here’s the same logic over a few more lines

```
int main(){  
    int toPrint = Add(5, 10);  
    println("{} ", toPrint);  
    return 0;  
}
```

here we store the result (the thing we return) of our Add function into the integer variable we’ve named “toPrint”, we then use that as the parameter in our ‘println’

Time to come clean, the ‘Add’ function is still not a good function. Doing basic arithmetic does not require a function. It can be done in just a normal line of code

```
println("{} ", 5 + 10);
```

this work just as well. (But of course we are just using these basic functions to learn the basics of code flow.)

Lets look at a series of basic building blocks: * int * float * double * bool

All four of these are types of variables, we've already familiarized ourselves with 'int'. the 'float' type is similar to an int but holds a number with decimal precision. An int can't store 1.5 but a float can.

A 'double' is also a variable type that stores a decimal number, but it's given more memory to work with than a float and can therefore be more precise (more decimal values stored)

a bool (short for boolean) holds one of two states, true or false. That's it.

when creating any variable we start with its 'type', followed by a 'name' and then an assignment operator '=' then its 'value' and lastly a ';'.

```
int wholeNumber = 5;
float percentageValue = 0.75;
double precisionValue = 0.75443341234114;
bool isThisCool = false;
```

NOTE: we will need to use the assignment operator '=' whenever we want to store the right side value in a left side variable.

There is a bit of syntax we need to learn, and it's in regards to 'float' type variables. the following way of writing decimal numbers '0.34' is interpreted as a double by the computer and then converted to a float when assigned to it. This means that our computer does a little conversion each time we assign a float like this. To tell the computer that the decimal number we've written is really a float we append a 'f' to the end of the decimal chain. Like this:


```
float aFloatFromTheStart = 0.34f;
```

lets look at a small program featuring a few of our variable types:

```
#include <print>
using namespace std;

int main(){
    int playerHealth = 10;
    int enemyDamage = 6;

    playerHealth -= enemyDamage;
    bool isPlayerDead = playerHealth <= 0;

    if(isPlayerDead == true){
        println("Ugh! I'm dead!");
    }
    else{
        println("is this all you got?");
    }
}
```

First we create two variables, both int's. Then we use a minus '-' and an assignment operator '=' to subtract the value of enemyDamage from PlayerHealth and storing the result back in playerHealth. If we just typed 'playerHealth - enemyDamage' without the assignment operator then the resulting value '4' would not be stored anywhere and no change would be assigned to playerHealth;

We then create a bool variable and because the statement after the assignment operator can only be either true or false. Either 'playerHealth' is above or equal to 0 or it isn't.

What follows is a common part of programming, an if statement followed by an else statement. The question being answered in the parenthesis of the if-statement is responsible for deciding if the code flow enters the scope of the 'if' or the 'else'.

The assignment operator '=' is different from the equality operator '==' the

equality operator doesn't assign a new value to the left hand side, instead it checks that the value on the left side and the right side are the same. So this if-statement asks if the value stored in 'isPlayerDead' is the same as 'true'. And with the playerHealth above 0 the value of isPlayerDead is false. The if-statement then looks like this: 'if(false == true)' and because these are not equal to each other we skip the if-scope and jump directly to the else-scope.

NOTE: so if we increased 'enemyDamage' above or equal to the value of playerHealth, then the if-statement would evaluate to 'true == true' and the code inside the if-scope would execute instead of the else-scope.

Lets talk about 'nesting'. Nesting refers to putting a scope within another scope. We have already done this by putting an if-statement inside a function. Though that is the shallowest nesting possible. a lot of nesting should be scrutinized as there is often a more readable solution. To much nesting will cause code flow that is hard to manage.

we can for example nest if-statements

```
void DealDamage(int damageAmount, bool isHardcore){
    playerHealth -= damageAmount;
    if(playerHealth <= 0){
        if(isHardcore){
            RetireHero();
            ReturnToTitleScreen();
        }
        else{
            ReturnToTitleScreen();
        }
    }
}
```

here we have an if statement within another if statement. Still no problem to read and parse. Though you can imagine that with 1-2 more if-statements our code would begin to drift right at an alarming rate. NOTE: the sideways drift towards right is sometimes referred to as a "pyramid of death"

We can use a 'return' to do an “early return” as well as ask the opposite question:

```
void DealDamage(int damageAmount, bool isHardcore){
    playerHealth -= damageAmount;
    if(playerHealth > 0){
        return;
    }
    if(isHardcore){
        RetireHero();
    }
    ReturnToTitleScreen();
}
```

look, we removed the nested if's. We also moved the ReturnToTitleScreen() call to outside of the if-statement as this was present in both the if and the else scope. By adding a 'return' in the case of the player still being alive we can guarantee that the code afterwards is only executed if the if-statement was false (aka the players health was in fact less than or equal to 0).

In this lecture we've looked at: * Writing and using our own .h files * how to #include standard library headers vs our own .h files * Return types * Variable types * Assignment vs equality operators * If/else statements * Nesting and early returns

5 Introduction to the Helix Editor - Part I

Helix is a code editor, and unlike the commonly used Visual Studio it is not also a debugger or has integrated build systems. Helix, compared to Visual Studio is:

- * Light-weight (memory footprint)
- * fast (starts almost instantly)
- * tailored for use with VIM-style motions and modes. Made to not use the mouse at all
- * Terminal-based

When we dive into programming applications in SDL3 we will be using Helix to edit our code and Visual Studio to debug it.

NOTE: Debugging in Visual Studio is another lecture.

If you have computer experience then you will quickly find that Helix is unlike any other software you've used. Just writing in it, before knowing how it works will feel alien and strange. You may eventually decide to move away from Helix and towards more mainstream and less opinionated editors. But for this course, you will be using the software that I use myself.

Helix uses a way of typing that was first introduced with the 'vi' text editor in 1976. It utilizes sequential keystrokes to change text by leveraging different editor modes: 1) Normal Mode 2) Insert Mode 3) Select Mode

Normal mode is used to move the 'caret' around. The Caret is the point in your file where text will be added as you type. In Normal Mode the user can't add any text by typing. This is the part that is the most confusing to new users as pressing keys will move the caret around or enter other modes. When working in Helix we also get access to Helix-specific menus by using the correct keystrokes in Normal Mode.

In Insert Mode the keys are used to type text like in any editor. Pressing Escape will return us out of Insert Mode and back into Normal Mode

In Select Mode we can select multiple pieces of text to be copied, moved and otherwise manipulated.

We enter Insert Mode using ‘i’ or ‘a’ or ‘o’ or ‘O’ (note how upper and lowercase are distinct from each other) We exit Insert Mode and Select Mode going back to Normal Mode by pressing ‘escape’

To code in both Visual Studio and Helix in a fashion that is at all acceptable we need to ensure that we have an english keyboard layout selected on windows. This means that ÅÄÖ are no longer available. The reason is that a lot(!) of symbols we will be typing when programming are very easy to access on the english layout and horrible to type on a swedish keyboard layout.

Inside Windows Settings we can add another keyboard layout by adding another preferred language. I added English (United States) as my secondary language and English (Swedish) as my primary. You will need to do the same. Swapping between the two is done by pressing ‘WIN+Space’

We will be pressing escape a lot, and because the escape key is so far away from the keyboards home row we will download Microsoft Powertoys to rebind our keyboard. ‘<https://apps.microsoft.com/detail/xp89dcgq3k6vld?hl=en-US&gl=SE>’

Once you’ve installed Powertoys you will have access to a bunch of (more or less) useful features. We will go to the ‘Keyboard Manager’ and add two remaps 1) Caps Lock to Escape 2) Escape to Caps Lock

now we turn caps ‘ON’ and ‘off’ using escape and exit ‘Insert Mode’ and ‘Select Mode’ using caps lock. This will, like many new things, feel strange at first. But this remapping is very common when using Helix or other VIM-style software. And now we’re using our computer as developers not hobbyists,

and that should naturally come with changes to how we use our hardware.

NOTE: I suggest unplugging your mouse when learning Helix if you can't help but reach for it all the time.

Helix and VIM style systems are so notorious that there are even a slew of memes relating to the fact that people don't know how to exit them. ('how to quit vim' on Google will yield a number of results). So lets learn how to close down Helix. This is done from the Helix Command Line. Which we access by typing ':' once we have done so, we can type a massive number of functions.

quitting helix from the command line is done by typing 'q' and pressing enter. You can also spell out 'quit' or typing a single 'q' then using 'tab' to cycle between the different quit commands.

You deserve a treat, reopen helix (if you closed it previously) write some sample code, just practice the main() function syntax. Once happy with a few lines. Open the Command Line and type 'theme' followed by a 'space' then cycle through the different color themes. Once you have found one you like, remember its name because we will make sure that each time you open your cool light-weight ultra-fast editor you will be greeted by it.

open the command line, type config, tab to 'config-open' and press enter. Go into Insert Mode and type:

```
theme = "your-chosen-theme"
```

Then we need to save the changes to this file. but out-of-the-box Helix saves using the 'write' command rather than our usual 'ctrl+S'. So go ahead and open the command line again, type 'w' and press enter

NOTE: you can also type 'write' or even 'write-quit-all' to save all your files at once then quit helix. or use the short form 'wqa'

////////////////////////////////////

We will be working with C++ files, and it would be very nice to catch errors before we try and compile. Luckily we can do just that. Once we compile its 'clang' that finds and spits out any errors. but using what is known as a 'language server' we can run background processes that look at and understands our code. This info is then given to Helix so it can display red errors for us.

In powershell we can run

```
hx --health
```

this will list all programming languages as well as any known language servers. Find the cpp row. If clangd (yes with a 'd') is not set as the language server then we need to download it and add it to our environment variable paths.

NOTE: 'clangd' is not the same as 'clang' it actually stands for clang daemon. a daemon is a silent background process that just listens to requests that come in then shuts down when not needed anymore. This specific language server daemon is a 'repackaged' part of clang that editors can talk to.

clangd can be downloaded from: <https://github.com/clangd/clangd>

once downloaded find the binary folder (bin), inside there should be a 'clangd.exe' add this folders path to your system environment variable PATHS.

once we have clangd up and running its runs in the background each time we open a .h or .cpp file. Then it lets us A) get diagnostics inside helix (red underlines, error message) B) we can use SPACE+A when our caret is over a part of our code with an error and clangd will offer fixes

6 Introduction to SDL3 - Part II

We've installed clang and added its path to our system environment variables. Now if we make a typo in our generic C++ code we will get helpful warnings and SPAC+A will give us suggestions on how to fix it. But as we try and #include files from our 'include' folder that we created as we started this project then clang will tell us that it can't find them.

We need to reopen our CMakeLists.txt and add some more build instructions. We are going to have clang generate a 'compile_commands.json' file for us. Up until now we've hand-rolled our .JSON files, but the far more common way is using a 'JSON serializer' to convert our data into a JSON format for us.

adding:

```
// cmake
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
```

to our CMakeLists.txt will create a 'compile_commands.json' in our build directory each time we compile our project using clang++.

Now we have 2 ways of ensuring that clangd can read this .json and use it to support our helix editor 1) we copy the file from our build folder into our root each time 2) we write a small '.clangd' file that tells clangd where to find our 'compile_commands.json'

We will create the '.clangd' file (option 2)

NOTE: clangd is used as a file extension here, and the lack of any text before the '.' tells us that the file has no name.

in our root directory (goto 'projectname') we add the .clangd file using

‘New-Item .clangd’. We then open it inside sublime text with `subl .clangd`.

We will write our `.clangd` file using a different data format than `.JSON`. The format is called `YAML` and has even less boilerplate than `JSON`, making it extremely human readable. But without braces to set the scope of a piece of data `YAML` instead relies on indentation to sort data. An indentation is what pushes new lines of text to the right on your monitor.

NOTE: like when we put an if-statement inside another if-statement, then we began drifting to the right.

For now, our `.clangd` file will be very small

```
// cmake  
  
CompileFlags:  
  CompilationDatabase: build/
```

Make sure that you adhere to the indentation, pushing “`CompilationDatabase`” one tab-step to the right. After saving our `.clangd` file we should head back to our `$profile` to add some helpful functionality to our `build` function. We want to add the ability to completely empty our ‘`build/`’ folder before compilation, just to ensure that everything we’re doing is working as intended and not relying on a previous compilation step adding necessary things for us that are now no longer present

By adding a new parameter and new logic to our ‘function `build`’ we can pass it as a parameter when calling ‘`build 'projectname'`’. The type of parameter inside powershell is called a ‘switch’ when it is not passed as a parameter the switch is set to false and when present it is set to true.

NOTE: this true/false data type is what we call a `bool` (boolean) in `C++`.

WARNING: before we look at the function below: we are going to start using

pretty strong commands to add or delete files from our computer. It's a good idea to, in the future, have backups for important stuff if you start experimenting with these functions on your own. Our function below is not an issue as we are doing a lot to ensure that we are in the proper directory.

Here's our updated 'function build'

```
// $profile

function build {
    param(
        [Parameter(Mandatory=$true)][string]$project,
        [switch]$clean,
        [string]$preset_name = "default"
    )
    goto $project

    $config = GetConfig $project
    $sourceDir = $config.path

    if((Test-Path $sourceDir) -eq $false){
        Write-Host "root directory not found. Aborting..."
        return
    }

    $buildDir = "$SourceDir/build"

    if((Test-Path $buildDir) -eq $false){
        Write-Host "no build directory found. Aborting..."
    }

    if($clean){
        Write-Host "Cleaning build directory..."
        Remove-Item -Recurse -Force $buildDir
    }

    cmake --preset $preset_name
    cmake --build --preset $preset_name
}
```

we've added a few new things. Like in other functions we've made we've sorted the sourceDirectory and buildDirectory in two variables. We have also added the '[switch]\$clean' parameter at the top and using in if-statement we check if \$clean was true.

Now we can begin programming in our main.cpp and adding a few SDL3 specific lines of code to spawn a window and fill it with a nice background color.

Remember how we previously had to ‘Sleep()’ our program for 2000 milliseconds in order to even see that our program ran before reaching the ‘return 0’ line at the end of our entry point? we will be using a new form of control flow statement to “run back over” the same piece of code over and over again, creating what is called a program loop.

For a game this loop is broken down into 3 distinct steps, Input, Update and Draw * Input handles key presses from a keyboard, controller or the input/motion of a mouse * Update takes the information about what the game state is and updates it based on both player inputs and the current state * Draw takes all objects that need to be rendered to the screen and draws them to the window. Then the loop starts over again, the more times this loop can be finished in a second, the higher our FPS is.

NOTE: this loop is present in all games and every game engine is built on it - even though Unity hides the Draw part of the loop from us.

The control flow statement is known as a ‘while’ and its syntax looks like this:

```
while (true) {  
    // Do stuff  
    // then more stuff  
}
```

Just like an if-statement is has the parenthesis where an expression is evaluated. As long as that expression is true then the code inside the while loop runs. At the end of the while loops scope it jumps back to the beginning of the scope and runs it again.

A naive version would look like

```
while (game_is_running){
    1. Handle_Input
    2. Update_The_Game
    3. Draw_The_Game
    ... and then repeat from step #1
}
```

lets update our main.cpp with everything we will need to test our while loop

```
#include "SDL3/SDL_log.h"
#include <windows.h>

int main() {

    bool running = true;

    while(running){
        SDL_Log("running...");
    }

    return 0;
}
```

Alright, so now our program doesn't quit automatically. Note how we've '`#include`' a new `.h` file. The reason the `.h` file is not added on its own but instead we've passed a path is because inside our 'include' folder we have a folder called 'SDL3' so our `#include` points at a specific file by passing in its path.

NOTE: the reason why our path is "`SDL3/SDL_log.h`" and not "`include/SDL3/SDL_log.h`" is because in our '`cmakelists.txt`' we set the include folder as directory that is included in the project with:

```
'target_include_directories(${PROJECT_NAME} PRIVATE include)'
```

NOTE: Remember the not-so-elegant syntax of the standard library header or '`cout`' from well now we can use the much more convenient '`SDL_Log()`'

instead!

We have no way of interrupting our program as there is nothing we can do within the scope of our 'while' that will turn 'running' from 'true' to 'false'.

NOTE: a while loop that never terminates or pauses will hog 100% of our CPU and make the program appear frozen, as it tries to keep running the same code over and over. It's up to us to either terminate a while loop or in this case, add logic to it so it has to 'yield'

All code is eventually represented as binary machine code. All languages like C# and C++ are an intermediary step that lets us write instructions in a far(!) more readable format. The chain is actually that C++ gets compiled into 'assembly' first, and then the assembly instructions are compiled into machine code. Our 'While()' loop' eventually becomes a 'jmp' assembly instruction telling the program to jump to another line. Another way of coding a while loop is like this:

```
int main(){
    Start:

    SDL_Log("Running...");

    goto Start;
}
```

The 'goto' instruction tells the code to jump (jmp) to the location specified by us. In this case the line we've added 'Start:' to.

Using the handy-dandy <https://godbolt.org/> website we can actually compare our C++ code for both example (while() and goto) and running both main() functions (without SDL stuff) we get the same output:

```
main:
    push    rbp
    mov     rbp, rsp
.L2:
    jmp     .L2
```

What this means is that when our program compiles, there is 0% difference between the while-loop and the goto solution. I touch on this to begin teaching you about what happens during the compilation step, introduce assembly and help empower you to dispel a lot of noise coming out of the programming community.

NOTE: it is considered back practice to use ‘goto’ and recruiters will probably not like seeing it. But just to be clear, it’s the exact same thing.

We will add some SDL boilerplate code to allow pressing ‘escape’ in order to flip ‘running’ to false and terminate the while loop and the program. Though before we can do this, we need to learn about one of the most essential parts of C++, a ‘pointer’

NOTE: pointers can be difficult to understand at first, re-reading is recommended.

lets look at a basic int

```
int number = 5;
```

so far so good. Now lets take a function that accepts an ‘int’ as a ‘parameter’ and adds 1 to it

```
void AddOne (int theNumber) {
    theNumber += 1;
}
```

then lets run this small program:

```
int number = 5;
AddOne(number);
SDL_Log(number);
```

Now the question is, is the number logged by ‘SDL_Log’ a ‘5’ or a ‘6’. The answer is still ‘5’ even though our ‘AddOne()’ function seemingly increased it by 1. So what we need to understand is that we can pass a variable ‘by value’ or ‘by pointer’. When we pass ‘by value’ we just send the number, not the place in memory that stores that number. If we instead pass the variable ‘by pointer’ we pass the location in memory that holds the variable, updating that will persist as we exit the scope of the function.

```
int number = 5;
AddOne(&number);
SDL_Log(number);

void AddOne(int* theNumber) {
    *theNumber += 1;
}
```

The ‘&’ before the variable name will pass the variable ‘by pointer’ instead of ‘by value’. The ‘int’ *with the* ” after the variable type indicates that we are working with a pointer (pointing to a place in memory) rather than a number. At this point we can’t actually add 1 to the variable as it is not itself a number, but rather something that points to a location in memory where a number lives. The ‘*’ before the variable name within the scope will ‘dereference’ the pointer, allowing us to modify the value stored in memory at that location.

Lets break it down:

int number = 5; <- this is a newly created variable. It is stored in memory somewhere with the value ‘5’

`int* aNumber <-` this is a pointer variable. It points not to a number, but the place in memory where we will store a number. To pass a point in memory to a function that expects a pointer we must pass the variable ‘by pointer’ using ‘&’

NOTE: without passing our number variable as a pointer we are actually just performing operations on a new number that lives only within the scope of the `AddOne()` function. As soon as we leave that scope the number stops existing.

`&number <-` we take number, and instead of passing it by value we get its place in memory and pass that along instead

`*number += 1;` <- this takes the reference to the memory address that the pointer was pointing to (where number lives) and ‘dereferences’ it, grabbing the value stored in it so we can manipulate and change it.

There is another way of passing ‘by reference’ that adds less new symbols, keeps things tidier and makes the compiler handle more of the heavy lifting for us

```
int number = 5;
AddOne(number);
SDL_Log(number);

void AddOne(int& theNumber){
    theNumber += 1;
}
```

Here we make it so the function is expecting a ‘int&’ this handles the pass-by-reference and dereferencing for us during compilation. Just by adding this single ‘&’ the program knows that any value being passed to the function is really passed by reference, and any changes to the passed variable in the

function will be made on the dereferenced value stored in that reference we passed along. It's more invisible and has both its advantages and disadvantages, it's less explicit because we 'hide' the fact that 'AddOne()' works by reference until we go to the function itself and spot the 'int&'. When in the more explicit version we wrote before we can clearly see '&number' being passed to the function, meaning that we know it's passed by reference without needing to look it up.

The three ways we can pass something in C++ to a function:

```
// 1. Pass by value - original unchanged

void AddOne(int theNumber) {
    theNumber += 1;
}
AddOne(number);

Result: number = 5

// 2. Pass by pointer - explicit, you can see the & at call site

void AddOne(int* theNumber) {
    *theNumber += 1;
}
AddOne(&number);

Result: number = 6

// 3. Pass by reference - cleaner, compiler handles it

void AddOne(int& theNumber) {
    theNumber += 1;
}
AddOne(number);

Result: number = 6
```

The “simplified” way of passing by reference requires less specific C++ syntax. And that convenience styled syntax is often referred to as ‘syntactic sugar’. Understanding how we pass something as either the memory address or the actual value is key to working with C++. So we will mostly be working with

‘2.’ as a learning tool. Though much of the C++ you will come across uses ‘3.’ So eventually you will still have to learn and understand both.

We can also convert our AddOne() function from a void to an int function to return whatever number we’ve created inside its scope. Allowing us to assign the new value to our number variable

```
int main(){
    int number = 5;
    number = AddOne(number);
    SDL_Log(number)
}

int AddOne(int theNumber){
    theNumber += 1;
    return theNumber;
}
```

With this setup the ‘Log’ function will output a 6, as the updated value of 5 -> 6 lives inside the scope of the ‘AddOne’ function but the value is later returned and then using the ‘=’ operator assigned back into the number variable inside ‘main()’.

NOTE: a ‘=’ operator always adds the value on its right to whatever is on its left.

Now that we know more about pointers and while loops we can better understand the SDL syntax (boilerplate) necessary to start working on a more proper example. What follows is a lot of new code, but we’ve touched on this syntax in many cases. We will break it all down once we’ve seen it in its entirety.

```

#include "SDL3/SDL_init.h"
#include "SDL3/SDL_events.h"

SDL_Window* window;

bool HandleRunning(SDL_Event event){
    if(event.type != SDL_EVENT_KEY_DOWN){
        return true;
    }
    if(event.key.key == SDLK_ESCAPE){
        return false;
    }
    else{
        return true;
    }
}

int main() {

    SDL_Init(SDL_INIT_EVENTS);
    bool running = true;

    window = SDL_CreateWindow("pilot", 650, 400, 0);

    while(running){
        SDL_Event event;

        while(SDL_PollEvent(&event)){
            running = HandleRunning(event);
        }
    }

    return 0;
}

```

This program does a few new things: 1) It actually creates a game window! 2) It initializes `SDL_Events` so we can get keyboard inputs 3) It stops running if we press the 'escape' key 4) It nests a while loop within another while loop 5) It passes multiple parameters to a single function 6) It includes some new .h files from SDL

Lets begin by looking at our entry point:

The 'main()' function has a lot of new parts to it. A lot of it is boilerplate

that SDL3 requires in order to start communicating with our computer. 'SDL_Init()' accepts a series of so called “flags” as parameters that tell it what systems from SDL3 to activate. In our case we’ve told it to initialize the “EVENTS” subsystem. This is required in order to get keyboard inputs to register as 'SDL_Event's we can query.

NOTE: “query” means “ask questions about”.

A flag is actually a datatype known as an “enum”, it is a series of numbers that are all represented as a name. what makes it a flag as well as an enum is that the numbers associated with each enum are one power of 2 larger than the previous.

```
enum WeaponBuffs {  
    NONE = 0,  
    POISON = 1,  
    FIRE = 2,  
    ICE = 4,  
    DARK = 8  
};
```

an enum that is not used as a flag does not need to specify its numbers, they are then treated as just growing by 1.

```
enum WeaponType {  
    NONE,  
    SWORD,  
    AXE,  
    HAMMER,  
    BOW  
};
```

When working with flags, by having each enum element as its own power of 2 we can combine them together and the number that is the result of that addition can only be the result of that exact combination. In our “WeaponBuffs” example, if we wanted to have a weapon with both a ‘Poison’ and an ‘ICE’ buff then this would be stored (in the background) as a ‘5’, an

no other combination of flags can produce that number.

enums are a great way of storing similar attributes in a way that is very easy to work with and very easy to read.

SDL3's 'SDL_Init()' function accepts multiple flags if we want, to send in multiple flags at once as a parameter to the function we use something called a 'bitwise operator' specifically this '|' it is called a bitwise 'OR'. The 'OR' operator combines bits if at least one of them is a '1' - Lets take a little detour and learn about bits.

When we are down in machine code, everything is represented as either a '0' or a '1'. Our computer calls these 'bits' each bit can either be 'on' or 'off' AKA '1' or '0'. The 'bool' variables we've created earlier can also either be 'true' or 'false' so in that sense they are similar, though a bool is stored in memory as a 'byte' which is the same thing as 8 'bits'. Our computer works on 'bytes' rather than 'bits' so even though all the necessary info about a bool just requires 1 'bit' we are still required to store all 8.

In the case of our 'WeaponBuffs' we can take each enum and because we used a sequence of 'power of 2's we get a very pleasing pattern of bits

```
NONE 00000000 POISON 00000001 FIRE 00000010 ICE 00000100 DARK
00001000
```

Look how each bit that is turned 'on' occupies its own column. If we use a "Bitwise OR" operator on these (the '|') we add combine them, keeping a '1' if ANY of the 'bits' in the 'byte' was a '1'

```
POISON 00000001 ICE 00000100 = Result 00000101
```

And now we can clearly see why an enum that is used as a flag needs to have each entry as a 'power of 2'. And the 'Result' above '00000101' is actually

the byte representation of the number '5'.

If we want to pass both 'SDL_INIT_EVENTS' and 'SDL_INIT_VIDEO' to our 'SDL_Init()' function then we add a bitwise 'OR' between the flags.

```
SDL_Init( SDL_INIT_EVENTS | SDL_INIT_VIDEO );
```

NOTE: you can find all the SDL_INIT flags here: https://wiki.libsdl.org/SDL3/SDL_Init

Lets keep looking at our program:

```

#include "SDL3/SDL_init.h"
#include "SDL3/SDL_events.h"

SDL_Window* window;

bool HandleRunning(SDL_Event event){
    if(event.type != SDL_EVENT_KEY_DOWN){
        return true;
    }
    if(event.key.key == SDLK_ESCAPE){
        return false;
    }
    else{
        return true;
    }
}

int main() {

    SDL_Init(SDL_INIT_EVENTS);
    bool running = true;

    window = SDL_CreateWindow("pilot", 650, 400, 0);

    while(running){
        SDL_Event event;

        while(SDL_PollEvent(&event)){
            running = HandleRunning(event);
        }
    }

    return 0;
}

```

`#include` at the top is used to add `SDL_init.h` and `SDL_events.h` to our program. These allow us to call the SDL functions we need from the ‘SDL3.dll’ in our ‘lib’ folder. Just below our `#includes` we store a pointer in memory the type being an `SDL_Window`. We know we’re storing a pointer and not the actual data as the ‘`SDL_Window`’ is followed by a `*`. This line on its own has not created a window for us. We need to actually create that window using the ‘`SDL_CreateWindow()`’ function. This function accepts 4

parameters. The first being the name of the window, then its the width and the height, followed by whatever option flags we want to pass along. Lets look at those now: https://wiki.libsdl.org/SDL3/SDL_CreateWindow

We should become comfortable with reading documentation, as this is really the only way of knowing how these kind of systems work. and as we can see, the `SDL_WindowFlags` enum can accept either a 0 (for NONE) or one or more flags combined with a bitwise ‘OR’ (`|`) We can also read that the function returns a ‘`SDL_Window`’ *this is then stored in the ‘`SDL_Window` window’* pointer variable we created at the top of our program.

We have another function besides our ‘`main()`’ it has the return type of ‘`bool`’ and is tasked with checking each event that SDL creates and when it finds that we’ve pressed the ‘escape’ key it returns ‘`false`’ instead of ‘`true`’. This flips running to ‘`false`’ and the infinite repeating ‘while loop’ is terminated and the program quits by returning 0.

The nested while loop inside ‘`main()`’ first stores a variable of the type `SDL_Event`, then it passes that place in memory to the ‘`SDL_PollEvent()`’ function, and by passing it by reference the ‘`SDL_PollEvents`’ can make changes to the variable that is stored in that same variable that we passed into the function. So that when we call our ‘`HandleRunning()`’ function we pass along that same event variable, now potentially modified by our ‘`PollEvents()`’

NOTE: The documentation for ‘`PollEvents`’ can be found here: https://wiki.libsdl.org/SDL3/SDL_PollEvent

looking at our ‘`HandleRunning()`’ function we can see a series of if and if-else statements asking questions about (querying) the `SDL_Event` parameter that was passed into it. As we are only interested in if the ‘escape’ key was pressed

we can avoid nesting by returning true (meaning ‘keep running’) if the event was not a keyboard event to begin with. Then if we did not return we know for a fact that it is a keyboard event we’re querying. Then we check the pretty nasty looking ‘key.key’ and compares the ‘key’ value with the SDL enum called `SDLK_ESCAPE` and if it is a match we return false.

‘`SDLK_`’ is the prefix for all keyboard events. The full list of buttons can be found here: https://wiki.libsdl.org/SDL3/SDL_Keycode

The value returned from the function is then stored into our running variable and if it was false it will stop the while-loop from continuing to run. Now we can actually quit our game by pressing escape.

NOTE: in the future we would of course not just close the program anytime someone accidentally presses ‘escape’ - but for now we’ll use this brutish approach.

So now we do the following things inside our program:

- 1) We initialize SDL
- 2) We create a window
- 3) We run our games core loop inside `main()`
- 4) We allow the core loop to terminate using ‘escape’

With this we’ve layed the foundation for a core loop (input, update, draw, repeat) !

7 Introduction to the Helix Editor - Part II

So far we've mostly been working in our `main.cpp` file, though this is the most important file, holding our programs entry point we will begin doing 2 things differently

- 1) work with `.h` files for function declarations
- 2) split our program up into multiple `.cpp` files

Helix can help us create files and give us the ability to get an overview of our code by splitting our editor into multiple smaller 'panes'.

To split our view into two panes that sit side by side we press the following button sequence (when in Normal Mode)

`space` - open the Helix menu `w` - open the window sub-menu `v` - hotkey to split the editor with a vertical right split

once we do this, the same file will be opened in both panes, if we modify one the other will update instantly. Though useful when working with very long files the far more common case is to have multiple different files open at once.

We can check what folder newly created files will get created in by pressing the following sequence:

`:` - open the helix console `pwd` - shortcut for the command "show-directory" 'pwd' stands for "print working directory"

If we have set things up correctly in powershell then this will print the path to our 'src' folder at the bottom of our Helix editor.

We can switch between our active pane by pressing

`space` - open helix menu `w` - open window submenu `left/right arrow` -

navigate to the left or right pane `h/l key` - can also be used to navigate to the left or right pane

once we are on a pane we can press

`: o` note the space after the o. `o` is the shortcut for the `open` command
`a-specific-file-name enter`

The helix fuzzy-search will look for and list all the files that match what you put in as the file name. You can flip between them using tab. With the file-name selected pressing enter will open it.

But the beautiful part is that if you attempt to open a file that doesn't exist it will be added to the pane anyways, but it is not yet in your 'src' folder. It lives only within helix.

NOTE: when Helix stores text before it has been written it is being stored in something known as a 'buffer'

Once you press

`: w` - write

You will have executed a 'write' command. This will save the file to disk, creating it if necessary. This allows us to open and create files as needed, without leaving Helix to use Powershells New-Item function.

If we are done with a pane, we need to decide if we want to 'write' its content to disk or if we want to discard the changes we've written.

if we try and close our pane using `: q` - quit

Helix will warn us and nothing will happen (if we have unsaved changes)

we can combine our write and quit (quit actually doesn't quit, it closes the

current view/pane)

: **wq** - write-quit

once we have multiple panes open at once, with multiple files, we can write all of them to disk at once using **:** **wa** - write-all

so lets say that we're just starting our workday, we want to begin devvin' in helix and start working on a new script called "bomb"

win press the windows key to open the start menu **pw** type 'pw' to search for powershell **enter** press enter to start powershell **dev project-name** type dev and then the name of our project to open it in helix **enter** press enter to execute the dev command opening helix **:** in helix, our dev function opened our main.cpp. use ':' to open the helix command **o bomb.cpp** this command will open a new buffer for a file we call bomb.cpp (not yet saved to disk) **enter** executes the open file command **space** open the helix menu **w** opens the helix window submenu **v** split the editor into two panes **:** open the helix command line again **o bomb.h** will open a new buffer for a new file **enter** executes the open file command write some code in the .h file' - an empty buffer won't save to disk **space** open the helix menu **w** open the helix window submenu **h** swap to the pane of the left (holding the buffer for bomb.cpp) write some code in the .cpp file' - an empty buffer won't save to disk **:** open the helix command line **wa** this is the write-all command **enter** executes the write-all command, saving bomb.cpp and bomb.h to disk in the src folder

To open and look through the currently active buffers we can press **space b** then step through them using **tab** and select which one to open with **enter**

To open and look through all files in the project we can press **space f** then step through them using **tab** and select which one to open with **enter**:

8 Core Loop - Part I

With the skillset we have currently we can begin constructing a core loop for a program.

NOTE: as it lacks a win-state or any sense of actual logic we'll call it a program for now and a game once we add those elements.

We're going to create a skeleton version of our core loop, including:

- 1) Recieving inputs
- 2) Updating logic based on time and our inputs
- 3) Displaying our game state

we will be setting up parts of this in our main.cpp but we'll also create other files that our main.cpp will call into.

NOTE: once we have gotten this core loop to work we will be changing a lot (almost all) of how we structure our program in the next couple of lectures. We will be re-writing things a couple of times, each time digging deeper into performance-focused C++ code!

At the end of this lecture we will have a colored rectangle that we can control on the screen using the arrow keys.

We could do all the following code inside our main() function, inside our while-loop. But we will be creating a new pair of .h and .cpp files and inside our while-loop we will call their functions.

Lets look at our main.cpp function


```

// main.cpp
#include "SDL3/SDL_init.h"
#include "SDL3/SDL_events.h"
#include "SDL3/SDL_timer.h"
#include "game.h"

SDL_Window* window;
SDL_Renderer* renderer;

Uint64 NOW;
Uint64 PREV;

bool HandleRunning(SDL_Event event){
    if(event.type != SDL_EVENT_KEY_DOWN){
        return true;
    }
    if(event.key.key == SDLK_ESCAPE){
        return false;
    }
    else{
        return true;
    }
}

int main() {

    SDL_Init(SDL_INIT_EVENTS);
    bool running = true;

    window = SDL_CreateWindow("pilot", 650, 400, 0);
    renderer = SDL_CreateRenderer(window, NULL);

    Core::Initialize();

    while(running){

        NOW = SDL_GetTicksNS();
        float dt = NOW - PREV;
        dt = SDL_NS_TO_SECONDS(dt);
        PREV = NOW;

        SDL_Event event;

        while(SDL_PollEvent(&event)){
            running = HandleRunning(event);
        }

        Core::Update(dt);
        Core::Draw(renderer);
    }

    Core::OnQuit(renderer);
    SDL_Quit();
    return 0;
}

```

A lot of new stuff happening. 1) we `#include` new `.h` files the `game.h` we wrote ourselves We've also written a `game.cpp` file that has the actual implementations of each function outlined in `game.h`

- 2) we store pointer variables to both a window and a renderer the renderer is tasked with taking textures and bitmap images and placing them into our window this is how we will color our window and render a rectangle inside of it this logic is found inside the `Draw()` function we've written inside `game.cpp` and declared inside `game.h` It is because we `#include` `game.h` that we can find and call this function note that we pass our renderer pointer to the `Draw()` function.
- 3) we have 2 new variables `NOW` and `PREV`. These are used to track how much time elapsed between the current and previous frame We check this by subtracting one from the other The `Uint64` is, like an `int` but can only hold positive values It is also 64 bits in memory compared to the (usually) 32 bits of an `int`. Meaning that it can store larger numbers The `U` stands for "unsigned" this means it only holds positive numbers note how we pass `deltatime` aka `dt` to our Update function
- 4) We use SDL functions `SDL_GetTicksNS()` and `SDL_NS_TO_SECONDS()` to work with a central part of all game logic, `dt` standing for `deltatime` Deltatime is used to scale values in relation to how quickly the computer can finish processing a `tick` the more `ticks` the higher the framerate and the smaller our `deltatime` is. Delta time is the time between the current and the last `tick`. Meaning that if it took a long time between ticks, then any equation that is multiplied by `deltatime` will be larger than if the time between ticks was very small. The result of this is that no matter how strong or

how slow our computers are, our bullets will still fly at the same speed.

Without deltetime, a gun in a fast computer would shoot faster bullets.

- 5) We call `Initialize` `Update` and `Draw` from a namespace we've named `Core`

Lets begin by looking at our game.h

```
#include "SDL3/SDL_renderer.h"
namespace Core{
    void Initialize();
    void Update(float dt);
    void Draw(SDL_Renderer* renderer);
    void OnQuit(SDL_Renderer* renderer);
}
```

This .h file outlines the functions that we will be writin the bodies for inside our game.cpp. It tells us what paremeters will be passed in and what type of function they are. `void` means that the function doesn't return any value. Because we know we will need to pass a pointer to the renderer in two of these functions we have to `#include` the `SDL_renderer.h` inside our .h file. This means that all files that include game.h also include SDL_renderer.h

All functions are collected in a `namespace` a namespace acts as a container for code, allowing multiple scripts to have the same name for functions. Imagine if we include 2 .h files, each with their own `Initialize()` function, without a namespace we would get an error during compilation telling us that it is unclear which function should be called. But keeping our `Initialize()` function inside a namespace forces us to specify the namespace as we call the function. We have already encountered a namespace earlier in this lecture series, when we decided to write the handy

```
using namespace std;
```

this allowed us to call the functions inside the namespace named `std` without

first writing `std::`. We can write `using namespace Core` at the top of our `main.cpp` and remove the `Core::` prefix from all function calls if we want.

NOTE: and in this project we don't have any other functions with these same names, so removing the namespace entirely would not cause compile errors.

Each function will have the following job:

<code>Initialization()</code>	-> Set up the necessary stuff
<code>Update()</code>	-> perform changes to the game using <code>deltatime</code> and keyboard
<code>Draw()</code>	-> With the changes from <code>Update</code> , render the relevant stuff t
<code>OnQuit()</code>	-> clean up before the application quits

Lets look at our `game.cpp` to find how each of these functions are implemented:

```

// game.cpp
#include "game.h"
#include "SDL3/SDL_keyboard.h"
#include "SDL3/SDL_render.h"

float xPos;
float yPos;
constexpr float SPEED = 100;
SDL_FRect box;

void Core::Initialize(){
    xPos = 100;
    yPos = 100;
    box.h = 50;
    box.w = 50;
    box.x = xPos;
    box.y = yPos;
}

void Core::Update(float dt){

    auto keys = SDL_GetKeyboardState(NULL);

    if(keys[SDL_SCANCODE_RIGHT]){
        xPos += SPEED * dt;
    }

    if(keys[SDL_SCANCODE_LEFT]){
        xPos -= SPEED * dt;
    }

    if(keys[SDL_SCANCODE_UP]){
        yPos -= SPEED * dt;
    }

    if(keys[SDL_SCANCODE_DOWN]){
        yPos += SPEED * dt;
    }

    box.x = xPos;
    box.y = yPos;
}

void Core::Draw(SDL_Renderer* renderer){
    SDL_SetRenderDrawColor(renderer, 0, 70, 0, 255);
    SDL_RenderClear(renderer);

    SDL_SetRenderDrawColor(renderer, 150, 0, 30, 255);
    SDL_RenderFillRect(renderer, &box);

    SDL_RenderPresent(renderer);
}

void Core::OnQuit(SDL_Renderer* renderer){
    SDL_DestroyRenderer(renderer);
}

```

At the top of `game.cpp` we write our list of variables that will be used. two for the position of our box, one for the actual box itself. and one is a `constexpr` variable called `SPEED`. Adding `constexpr` to a variable means that its value can't change when the program is running. Meaning that nothing or no one could accidentally write code that changes this value. The only place where the value of a `constexpr` can be set is at the same line it is being created.

Our `Initialize()` function takes all of our variables (except `SPEED`) and gives them an initial value. It also sets the internal values of `x` and `y` of the `SDL_FRect` to match our `xPos` and `yPos` variables as well as its width and height `w` and `h`.

an `SDL_FRect` is a ``struct`` holding the following data:

```
float x; - horizontal position
float y; - vertical position
float w; - width
float h; - height
```

a `struct` is just a collection of variables that we want to bundle together.

An apple struct could have the following variables

```
int prince
float weight
string speciesName
bool isRotten
```

Using structs is how we pass more complex data around without sending each variable one after the other

SDL can, using the info inside a `SDL_FRect` and the renderer, display a rectangle on the screen using a function called `SDL_RenderFillRect()` inside our `Draw()`.

Just like how we can create a new function or a new variable we can also create structs ourselves. The syntax is very simple

```
struct the_structs_name{
    int a_number,
    float a_decimal_number,
    bool a_true_or_false_thing
};
```

That's it, now we can pass and store multiple variables together. This will be used later to define things like bullets, enemies, players and more!

for example:

```
struct Bullet {
    int damage,
    bool fired,
    float travel_speed,
    float size
};
```

When we work with a struct, we can access its different internal variables by just using a `.`

```
Bullet a_bullet;

a_bullet.damage = 5;
a_bullet.fired = false;
a_bullet.travel_speed = 100;
// and so on
```

and if a function expects a Bullet struct we can pass it like so:

```
void FireBullet(bullet* a_bullet){
}
```

But as we're passing our bullet struct as a pointer instead of by value we do have to learn a bit more unintuitive C++ syntax. To access the values saved inside a pointer we can't use `.` we need to use `->`. Like this:

```
void FireBullet(Bullet* a_bullet){
    a_bullet->fired = true;
}
```

So we use: `.` when accessing the values on the actual variable `->` when accessing the values the pointer is pointing to `::` when accessing functions from a namespace

Lets look at our Update() function

```
void Core::Update(float dt){

    const bool* keys = SDL_GetKeyboardState(NULL);

    if(keys[SDL_SCANCODE_RIGHT]){
        xPos += SPEED * dt;
    }

    if(keys[SDL_SCANCODE_LEFT]){
        xPos -= SPEED * dt;
    }

    if(keys[SDL_SCANCODE_UP]){
        yPos -= SPEED * dt;
    }

    if(keys[SDL_SCANCODE_DOWN]){
        yPos += SPEED * dt;
    }

    box.x = xPos;
    box.y = yPos;

}
```

The first line holds a lot of new info for us: `const bool* keys` is similar to `bool* keys` which is a pointer to a place in memory where we store a bool. Adding const to it make it so that the value of the bool stored in memory at the address the pointer points to can't be changed - SDL does this because we are not supposed to change the status of the keyboard in code, we just read what it is.

But there is one more wrinkle. as SDL is written in C and not C++ we have some unintuitive syntax here. `bool* keys` (note the plural) is actually a pointer to the first bool out of many stored in sequence in memory. So if we had a micro-keyboard with only `A B C D E`. Then if we held the `B` key down our memory would look like this:

<code>TrueOrFalse:</code>	<code>[0]</code>	<code>[1]</code>	<code>[0]</code>	<code>[0]</code>	<code>[0]</code>
<code>Key</code>	<code>A</code>	<code>B</code>	<code>C</code>	<code>D</code>	<code>E</code>
<code>Index</code>	<code>0</code>	<code>1</code>	<code>2</code>	<code>3</code>	<code>4</code>

our `bool*` points to the first memory address (`A`) then we can check the value of a specific memory address using `[]` and an index

```
keys[0] (this is the true or false for `A`)
keys[1] (this is the true or false for `B`)
keys[4] (this is the true or false for `E`)
```

The first memory address is at index 0, not index 1. C++ and C# begins indexing at 0. Other languages like LUA for example start at 1.

But it would be really hard to remember the index of lets say the letter `U`. Thankfully SDL has an enum that helps us write in plain english and have the compiler substitute that for a number

```
keys[SDL_SCANCODE_B]
```

[Scancodes](#)

turns out the index for `B` was 5 and that is exactly what the number to `SDL_SCANCODE_B` was given.

So each tick SDL checks the status of all the keys on our keyboard then sets their value in memory to true or false, then we can check their status by

pointing to the first place in memory then shifting by the specific index to find the value of the key we are looking for.

```
if(keys[SDL_SCANCODE_RIGHT]){  
    xPos += SPEED * dt;  
}
```

here we check if the right arrow key is held down, and if it is we add to the xPos variable equal to our SPEED variable scaled by deltatime. This ensures that a fast and a slow computer will have the same speed applied to xPos over time.

We do the same for the other arrow keys. The only quirk is that the top left corner of our window is at position 0,0. So when we increase the value of yPos we actually move downwards. So we need to remember that when selecting to add `+=` or to remove `-=`

once we have changed the value of xPos and/or yPos we update the .x and .y value of our SDL_FRect “box”

The `draw()` function passes the render pointer to a bunch of SDL functions found in the `SDL_render.h`

```
void Core::Draw(SDL_Renderer* renderer){  
    SDL_SetRenderDrawColor(renderer, 0, 70, 0, 255);  
    SDL_RenderClear(renderer);  
  
    SDL_SetRenderDrawColor(renderer, 150, 0, 30, 255);  
    SDL_RenderFillRect(renderer, &box);  
  
    SDL_RenderPresent(renderer);  
}
```

`SDL_SetRenderDrawColor` sets the color through RGB (and `alpha` for transparency of the renderer) each of these are represented as a value between 0 and 255.

`SDL_RenderClear` clears whatever was previously drawn to the screen and fills it with the color we set for the renderer previously

`SDL_RenderFillRect` passes our `SDL_FRect` as a pointer, it then takes the values of the struct and uses those to draw a rectangle on the screen. We've changed the `RenderDrawColor` so the rectangle shows up as a different color than the background

`SDL_RenderPresent` is what actually puts every pixel on the window. First all pixels are prepared, then all of them are drawn at once using this function.

[Documentation](#): “SDL’s rendering functions operate on a backbuffer; that is, calling a rendering function such as `SDL_RenderLine()` does not directly put a line on the screen, but rather updates the backbuffer. As such, you compose your entire scene and present the composed backbuffer to the screen as a complete picture.

Therefore, when using SDL’s rendering API, one does all drawing intended for the frame, and then calls this function once per frame to present the final drawing to the user.”

So our basic draw loop is 1) `RenderClear` 2) Draw everything 3) `RenderPresent`

There we go, we have created our first core loop, with input handling, updating game logic and finally drawing things to the screen!

9 DLLs Memory and Hot Reloading - Part I

Ok! So we've managed to get things to interact, move and get rendered in SDL3. That is fantastic. It was a long journey to get here. During this and in upcoming lectures we will focus on 3 things:

- 1) Learning how to write gameplay code
- 2) Learning more about performance
- 3) Learning more about C++

This lecture will teach us how to expand our `Cmakelists.txt` to generate not only our `.exe` but also a `.dll` that will be responsible for holding most of our game, making our `.exe` just a very small entry point.

Why do we want to do this? Because we want to enable something called hot-reloading.

(<https://zylinski.se/posts/hot-reload-gameplay-code/>) “Hot reloading gameplay code means that you swap out the code that controls the behavior of your game while the game is running. Why? To improve and tweak your gameplay code without having to restart the game.”

Without this set up we have to stop running our `.exe` to make changes to the game. then recompile the game and run it again, getting back to the gamestate we're looking for. This becomes so useful when we want to make adjustments to parts of the game that happens a bit into our game, or requires a lot of tweaking to get right.

Here's the breakdown of how we will achieve this:

- 1) change our `cmakelists.txt` to compile our project differently, creating the `.exe` and our new `.dll`

- 2) set up what is called a memory arena to hold all the memory we are allowing our game to use
- 3) break that block of memory into pieces we can use
- 4) Call into our .dll from our .exe from our main() function and pass along our memory arena
- 5) Write a reload function for Powershell

Doing all these steps will break our program for a while as these changes are part of a bigger sweeping change. So for a while nothing will successfully compile.

Lets begin by looking at a new version of our cmakeLists.txt it has the following changes

- 1) It has been cleaned up and things are sorted in more manageable blocks
- 2) We create both a .EXE and a .DLL
- 3) We flag some scripts as being for the .EXE and the rest as belonging to the .DLL
- 4) We use comments to help distinguish the different parts of our cmakeLists.txt

Here it is in full

```

cmake_minimum_required(VERSION 3.25)
project(Heartburner LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)
set(DLL_NAME ${PROJECT_NAME}_game)

# Flag .EXE specific files
set(EXE_EXCLUSIVE
    ${CMAKE_SOURCE_DIR}/src/main.cpp
    ${CMAKE_SOURCE_DIR}/src/arena.cpp
)

# Create the file references
file(GLOB_RECURSE LIB_FILES "${CMAKE_SOURCE_DIR}/lib/*.lib")
file(GLOB_RECURSE DLL_FILES "${CMAKE_SOURCE_DIR}/lib/*.dll")
file(GLOB_RECURSE DLL_EXCLUSIVE "src/*.cpp")
list(REMOVE_ITEM DLL_EXCLUSIVE ${EXE_EXCLUSIVE})

# EXE
add_executable(${PROJECT_NAME} ${EXE_EXCLUSIVE})
target_include_directories(${PROJECT_NAME} PRIVATE include)
target_link_libraries(${PROJECT_NAME} PRIVATE ${LIB_FILES})

# GAME DLL
add_library(${DLL_NAME} SHARED ${DLL_EXCLUSIVE})
target_include_directories(${DLL_NAME} PRIVATE include)
target_link_libraries(${DLL_NAME} PRIVATE ${LIB_FILES})

# Copy DLLs
add_custom_command(TARGET ${PROJECT_NAME} POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_if_different
        ${DLL_FILES}
        "$<TARGET_FILE_DIR:${PROJECT_NAME}>"
)

set(DLL_NAME ${PROJECT_NAME}_game)

```

This lets us define a new variable called `DLL_NAME` and make it the same as
 ↪ ``PROJECT_NAME`` but with ``_game`` appended to the end of it. Now we can use this name
 ↪ in all places where we want to specify that we're talking about the .DLL and not the
 ↪ .EXE, without having to manually type out the name each time.

```

# Flag .EXE specific files
set(EXE_EXCLUSIVE
    ${CMAKE_SOURCE_DIR}/src/main.cpp
    ${CMAKE_SOURCE_DIR}/src/arena.cpp
)

```

Here we store all .cpp files we want to have included with our .EXE in a single array that we've named `EXE_EXCLUSIVE` should we need more files to

be compiled into our .EXE we will have to manually modify this list.

NOTE: Another method is to take all the .cpp files we want to have and store them in a separate subdirectory then point our functions towards that folder. But this time we'll do the manual work.

```
file(GLOB_RECURSE LIB_FILES "${CMAKE_SOURCE_DIR}/lib/*.lib")
file(GLOB_RECURSE DLL_FILES "${CMAKE_SOURCE_DIR}/lib/*.dll")
file(GLOB_RECURSE DLL_EXCLUSIVE "src/*.cpp")
list(REMOVE_ITEM DLL_EXCLUSIVE ${EXE_EXCLUSIVE})
```

These are all the collections of files we will need. Using the list() function we can make changes to a list, in this case we use the method REMOVE_ITEM to strip the EXE_EXCLUSIVE files from the DLL_EXCLUSIVE files, so they have no overlap between files.

```
# EXE
add_executable(${PROJECT_NAME} ${EXE_EXCLUSIVE})
target_include_directories(${PROJECT_NAME} PRIVATE include)
target_link_libraries(${PROJECT_NAME} PRIVATE ${LIB_FILES})

# GAME DLL
add_library(${DLL_NAME} SHARED ${DLL_EXCLUSIVE})
target_include_directories(${DLL_NAME} PRIVATE include)
target_link_libraries(${DLL_NAME} PRIVATE ${LIB_FILES})
```

the `add_executable` and `add_library` functions are the only part that is different between these. They specify if we should compile the following files into a .exe or a .dll. We specify the name of the file using our handy variables `PROJECT_NAME` and `DLL_NAME` then the EXE_EXCLUSIVE and DLL_EXCLUSIVE file lists are added respectively.

NOTE: `dll` stands for dynamically linked library

NOTE: our `add_library` creates both a .dll and a .lib file as both are needed for us to work with our .dll

```
# Copy DLLs
add_custom_command(TARGET ${PROJECT_NAME} POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_if_different
        ${DLL_FILES}
        "$<TARGET_FILE_DIR:${PROJECT_NAME}>"
)

```

unchanged from before, this function takes all the .dll files we located in our `lib` folder and copies them over to our build folder.

Inside our powershell \$profile we've added a new function

```
// $profile
function reload {
    param([Parameter(Mandatory=$true)][string]$project)

    $config = GetConfig $project
    $SourceDir = $config.path
    $BuildDir = "$SourceDir/build"

    $cachePath = "$BuildDir/CMakeCache.txt"
    $projectLine = Get-Content $cachePath | Select-String "CMAKE_PROJECT_NAME:STATIC"
    $projectName = $projectLine.Line.Split("=")[1]

    cmake --build $BuildDir --target "${projectName}_game"
}

```

- 1) It uses our GetConfig function to fetch the path to our build folder
- 2) Inside the build folder, it looks for a file called `CMakeCache.txt` this is automatically added by cmake when it is being configured inside our `build` function `cmake --preset $preset_name` NOTE: this reload function relies on us having built our project beforehand.
- 3) We read the contents of our Cache looking for the line of text that includes the `CMAKE_PROJECT_NAME:STATIC` text. Because on that line, just afterwards is a `=` followed by the name of our project set by our `cmakelists.txt` We then store the name of our game in the `$projectName` variable by splitting the line we found at the position of the first (and only) `=` then takes the second text. NOTE: arrays start a 0, so index

1 is actually the second entry

- 4) We then tell cmake to build the project again, but only the target named “NameOfOurProject_game”. So if our game is called “Asteroids” in our cmakeLists.txt then \$projectName will be “Asteroids_game” this is the name of our .dll, ensuring that that is the only thing being compiled.

So if we follow the syntax of having **_game** as a suffix for our created .dll for all projects then this will work just fine. If we ever need more flexibility with our naming scheme we can store info like this name in for example our projects.json.

Now we’ve configured our CmakeLists.txt and added a powershell function to help us later. The next step is to begin learning about memory management and how to set up a memory arena as well as learning more about pointers. we’ll be returning to our **practice example.cpp** that we added a long time ago to try small projects. This is a single file script that shows how we can work with what is called a memory arena and we’ll look at how we can think about memory and pointers pointing to memory.

```

// practice project
// part 1
#include <iostream>
#include <windows.h>
using namespace std;

struct MemoryArena {
    unsigned char* base;
    size_t size;
    size_t used;
};

void* Arena_Add(MemoryArena* arena, size_t size){
    if(arena->used + size > arena->size){
        return nullptr; // Safety so we can't go beyond our arena size
    }

    void* latest_point = arena->base + arena->used;
    arena->used += size;
    return latest_point;
}

void Arena_Initialize(MemoryArena* arena, void* start, size_t size){
    arena->base = (unsigned char*)start;
    arena->size = size;
    arena->used = 0;
}

void Arena_Reset(MemoryArena* arena){
    arena->used = 0;
}

struct Character {
    enum CHARACTER_TYPE {HERO, ENEMY};
    CHARACTER_TYPE my_character_type;
    int health;
    int damage;
    bool is_alive;
    char* name;
};

```

```

// practice project
// part 2
struct Game_Data {
    Character* characters;
    int character_count;
    int score;
};

Game_Data* gameData;
MemoryArena arena;

int main(){

    size_t memory_size = (1024 * 1024 * 4); // 4mb
    void* blob_of_memory = malloc(memory_size);

    if(blob_of_memory == nullptr){
        return 1;
    }

    Arena_Initialize(&arena, blob_of_memory, memory_size);

    gameData = (Game_Data*)Arena_Add(&arena, sizeof(Game_Data));
    gameData->character_count = 10;
    gameData->score = 0;

    void* characters_start_point = Arena_Add(&arena, sizeof(Character) *
    ↪ gameData->character_count);
    gameData->characters = (Character*)characters_start_point;

    gameData->characters[3].health = 32;

    cout << gameData->characters[3].health << endl;

    Sleep(2000);

    return 0;
}

```

In this program we:

- 1) create a few structs
 - MemoryArena
 - GameData
 - Character

Then we use these structs to hold variables, and our GameData struct holds Character structs itself. But notice how the variable name is `characters` in plural, but we only store a single pointer this should mean that we are only storing a single character. We are actually storing a collection of characters inside memory by pointing to the first character only. we'll get back to how that is set up once we have a better understanding of the program in its entirety.

2) we create our MemoryArena struct

Note: lets conceptualize a memory arena as a continuous block of memory, each thing layed out next to the previous.

This struct holds very little in terms of stuff, but is very powerful. Our three variables are `* unsigned char* base`; This a pointer to the first address in memory of our arena. We need to use an unsigned char* instead of a void* as this allows us to add a number to our base to move further down our block of memory. If we had our pointer as a void* it would need to be `cast` all the time before we attempt to do arithmetic (+, -, x, etc). NOTE: we will learn about casting a bit later in this lecture

- `size_t size`; `size_t` is a type of variable, like an int, that holds a whole number, but `size_t` is larger than an int and especially made to help us store how big something is. `size_t` is also unsigned, meaning that compared to a int it can't store a negative number.

This variable is meant to tell us how large our memory block is, whilst the unsigned char* pointer above just tells us where it starts.

- `size_t used`; We update this variable each time we specify what the next piece of memory is used for. So we know we aren't overwriting other

parts of our memory when adding new things to it. Also, by resetting this to 0 we actually delete all memory in the arena all at once. We do this in the `Arena_Reset()` function

3) We've created three functions

- `Arena_Add()` This function tells our memory arena to tag a portion of memory as used.
- `Arena_Initialize()` This function sets up the memory arena by setting up the values of our struct.
- `Arena_Reset()` This function sets the size of our `size_t used` variable to 0, meaning that to the memory arena no part of memory is tagged and should something new be added into memory then it will write it at the start of the memory block.

But before we can use this memory arena we need to find a place in memory where we can store our continuous chunk. In other applications where we create and free memory `willy-nilly` our memory lives all over our `heap` in this program we will store all our memory in one location and once we're done with it we will free it all at once. The upside to this is

- 1) we know that our program will not crash due to insufficient memory, if it starts up then we know we managed to allocate enough memory.
- 2) our memory lives in tidy blocks on our heap, making accessing them faster as the CPU doesn't have to go back to RAM as often to fetch a chunk of memory. This is perhaps the biggest performance win between our data-oriented approach and the object-oriented approach found in game engines like Unity.

```
size_t memory_size = (1024 * 1024 * 4); // 4mb
void* blob_of_memory = malloc(memory_size);
```

this code, that runs as the very first thing we do in our programs entry point finds a chunk of unused data that is 4mb large and sets it aside for us. It's just filled with junk data right now - but it is ours and no other program or background operation will use it.

Later, This blob of memory will be owned by our .EXE and passed to our .DLL each tick, allowing our game to work with the memory. But because the memory is owned by our .EXE, if we ever recompile the .DLL nothing will happen to our data, allowing our changed functions to immediately start using the same memory without us needing to restart our game.

the `void*` variable is a pointer to the first bit in memory at the start of our memory chunk. Used alongside our `memory_size` we know the start of our memory chunk and how large it is.

the `malloc` function sets aside a specified size in memory then returns a pointer to the first bit. 1kb (kilobyte) is 1024 bytes, a mb is 1024 of those. Then multiplying that by 4 we get our total of 4mb. A small footprint for our program, but in this example we're using almost none of it at all.

```
if(blob_of_memory == nullptr){
    cout << "failed to allocate memory" << endl;
    return 1;
}
```

This if-statement checks if our pointer returned from malloc actually successfully managed to find a place in memory to point to. If not we enter the scope of the if-statement and our main() returns with an error code of 1 after printing an error. from the Microsoft C++ documentation: "malloc returns a void pointer to the allocated space, or NULL if there's insufficient memory

available”. NULL is from C, a language that C++ builds off of. The modern C++ style wants us to use `nullptr` instead. It is safer as NULL is just another way of saying 0, whilst `nullptr` is not a number but the result of a non-viable operation. Meaning that if we accidentally had a NULL in our code and we tried to use it in arithmetic we would be allowed to. But a `nullptr` will throw an error, which is better, because we want to stop such undefined behaviour.

```
Arena_Initialize(&arena, blob_of_memory, memory_size);
```

here we take our `blob_of_memory`, AKA the pointer to the first byte in the memory chunk, along with the size of the memory chunk and the arena and pass all of these to our `Arena_Initialize()` function. It is then responsible for setting up the MemoryArena by assigning these values to the relevant variables (`base` and `size`) as well as setting `used` to 0, meaning that nothing has been declared inside this chunk.

Note: this has just set some initial state for our arena, it is yet to have anything actually useful inside of it

```
gameData = (Game_Data*)Add_To_Arena(&arena, sizeof(Game_Data));  
gameData->character_count = 10;  
gameData->score = 0;
```

Here we must take a moment to learn about `casting` casting is the process of telling our compiler that we want to take data that is one of type, and treat it as if it were another type. Not all types can be `cast` to each other, but the `void*` returned from our `Add_To_Arena` function can be cast into any other pointer. Just as for example a float value can be `cast` into an int, but of course in the process it gets stripped of its decimal value

```
float decimalValue = 2.5;
int cast_to_int = (int)decimalValue;
printf(decimalValue) // this prints 2.5
printf(cast_to_int) // this prints `2`
```

So what we're doing is allocating inside the arena enough memory to store the Game_Data struct, meaning that our memory holds one pointer to a character, and two ints. We store this space in memory using our gameData pointer. Now part of our memory arena has been allocated, this increases our `used` variable by that amount. So any new data allocated to the arena will start from this adjusted position (`base + used`) instead of at just `base`.

Now we need to allocate some memory for our collection of characters

```
size_t character_memory_footprint = sizeof(Character) * gameData->character_count;
void* characters_start_point = Add_To_Arena(&arena, character_memory_footprint);
gameData->characters = (Character*)characters_start_point;
```

A single instance of our Character struct takes up a certain amount of memory. The `sizeof()` function calculates this for us. We have decided to have 10 characters allocated into memory. This is stored in our `character_count` variable. We can therefore multiply the size of one Character by that amount to get the total amount of memory that 10 Characters will need. We then use our `Add_To_Arena` function to tag that amount of memory as being used to store our 10 Characters. The function returns a `void*` pointing to the first byte of our character memory chunk. Lastly we cast this `void*` into a `character*` so that the compiler knows what type of memory we have stored.

Now inside our memory arena we have layed out the memory for 10 characters sequentially. And because we know the size of a Character and we know they are packed next to each other in memory we can use the array `[]` symbols to fetch one of the ten characters by specifying its position in the memory 0-9.

NOTE: remember that arrays start at 0 instead of at 1. This means that

element 9 is the 10'th and last element.

```
gameData->characters[3].health = 32;  
cout << gameData->characters[3].health << endl;
```

Here we find the memory adress of the 4'th character and set the value of the health variable stored at this point in memory to 32. Then just to make sure we've successfully set everything up we print the value stored at that point to our console using `cout << value << endl;` So after we've assigned our gameData to our memory arena we can stop thinking about the arena and just work with the pointer as normal.

Now that we know more about how a memory arena, casting and memory layouts work we are ready to bring this into our SDL3 project to add new files and set up necessary boilerplate to allow us to work with our .EXE and .DLL solution

10 DLLs Memory and Hot Reloading - Part II

It's time to head back to our SDL3 project to set up the boilerplate necessary to use our .EXE + .DLL system.

In our previous example we had everything in one placeholder practice example.cpp. Now we will start breaking things into separate .cpp files along with corresponding .h files.

At the end of this lecture we will have the following files in our src folder:

- arena.cpp holds the implementation of functions from arena.h
- arena.h holds the declaration of functions for our memory arena as well as the arena struct
- common.h A helper .h file containing some helpful macros to figure out memory sizes in kb, mb and gb
- game.cpp Acts as the “entry point” for the DLL and performs our Input, Update, Draw routines
- game.h Holds the definitions for the functions used in game.cpp and has them tagged in such a way that we can find them from our main.exe
- gameState.h a .h file containing the struct with all variables used inside the game
- main.cpp our .exe entry point, initializes everything, sets up memory and the game loop. Calls into our DLL through functions found in game.h

The process of breaking out parts of code into its own files is industry standard, as it allows clearer boundaries between files and makes reasoning about them simpler.

As our .EXE has no access to game.cpp directly we need to specify where it can find each of its relevant functions:

- Initialize
- HandleEvents
- Update
- Draw

This must be done in a few steps:

- 1) flag the functions inside game.h in such a way as to be usable in this way
- 2) for each function, create a ‘function pointer’
- 3) create a struct to hold these ‘function pointers’ in one location
- 4) connect each ‘function pointer’ to the right function in the DLL
- 5) using the struct, call each function where appropriate

Once we have all the necessary boilerplate set up, we can actually ignore most of it, working instead as we normally would. So it’s a bit of up front costs for a lot of benefit later on.

The `arena.h` and `arena.cpp` files are very similar to our practice example, but lifted into their own files

```

// arena.h
#pragma once

namespace Memory {

    struct Arena {
        unsigned char* base;
        size_t size;
        size_t used;
    };

    void Initialize(Arena* arena, void* memory, size_t size);
    void* Allocate(Arena* arena, size_t size);
    void Reset(Arena* arena);

}

```

At the top of this .h file we write ‘#pragma once’ we will be doing this for ALL .h files we ever write. This is a not-so-nice feature of C++ where without it our .h file will be copied above all files that implement it, meaning that our .DLL or .EXE bloats unnecessarily. By adding ‘#pragma once’ our compiler knows to only add these once, which is enough.

We have encapsulated our struct and function declarations in a namespace we’ve named `Memory`. This means that when we `#include "arena.h"` we can only access the struct and functions by first specifying the namespace for example: `Memory::Initialize()`

the struct `Arena` holds the same three variables as our practice example and the three functions are the same as well. We first call `Initialize` to make sure our size variable is set, our `used` is zeroed and our `base` pointer `points` at the first byte in memory.

```

// arena.cpp
#include "arena.h"
#include <cstring>

void Memory::Initialize(Arena* arena, void* mem_start, size_t size) {
    arena->base = (unsigned char*)mem_start;
    arena->size = size;
    arena->used = 0;
}

void* Memory::Allocate(Arena *arena, size_t size) {
    void* front = arena->base + arena->used;
    arena->used += size;
    memset(front, 0, size);
    return front;
}

void Memory::Reset(Arena *arena){
    arena->used = 0;
}

```

We include `arena.h` at the top so we can create the functions declared in the `.h` file. But to work with those functions we need to remember to specify their namespace, otherwise we aren't connecting our functions to those inside the `.h` file, but instead creating new functions with the same names.

like in our practice example we are using the same functions, but have opted for the more appropriately named `Allocate` rather than `Add_To_Arena`. We've also added a new line of code to our `Allocate`

```
memset(front, 0, size);
```

this code makes sure that the block of memory that we've allocated here is free from garbage data by putting the value `0` across the board. This will stop undefined behaviour when if we've not been careful, we try and access data before we've set its values. This defensive pattern is called `zero-allocation` it makes all numbers `0` and all pointers become `nullptr`. Without this code we could find ourselves working with data that has been given random values. We add `#include <cstring>` as that is the header that holds `memset()`

that's it for our memory arena. Mostly it's all the same stuff, just in its own two files and with our defensive pattern added.

Lets look at our common.h

```
// common.h
#pragma once

#define KILOBYTES(n) ((size_t)n * 1024)
#define MEGABYTES(n) (KILOBYTES(n) * 1024)
#define GIGABYTES(n) (MEGABYTES(n) * 1024)

constexpr size_t GAME_MEMORY_ALLOWANCE = MEGABYTES(10);
```

once again we have our '#pragma once' at the top. what follows are three macros that simplify getting the correct `size_t` for different sizes of memory. Making it easier to remember that '1024 * 1024 * 4' means we're working with 4mb. Now we can write `MEGABYTES(4)` and it is compiled into the same thing. What define actually does is specify a preprocessor instruction that substitutes the text on the left in our project with that on the right. Meaning that for the compiler it was always the full text, but we, the programmers, were allowed a bit of a smoother experience, having to type less each time we want to use the specific piece of code.

Lastly we have set up a variable that can not change at runtime (because of the constexpr) this, whenever used, will be substituted by 10 megabytes. This part is not strictly necessary, but if we imagine our common.h to eventually hold a bunch of `settings` for our project. Then keeping the memory budget here starts to look more reasonable. we will be adding constexpr to values which have the 2 following conditions:

- 1) the value is not meant to change
- 2) we know the value at compile time

We only use this header file in our main.cpp currently to simplify our `malloc` (memory allocation)

and our minimal gameState.h

```
// gameState.h
#pragma once
#include "SDL3/SDL_rect.h"

struct GameData {
    SDL_FRect rect;
    float move_speed;
};
```

and just to hammer it home, we have ‘`#pragma once`’ at the very top of our .h file. Then we include ‘`SDL3/SDL_rect`’ so we can use the `SDL_FRect` struct.

Inside our `GameData` struct we currently just specify two variables, our rectangle and how fast we are going to want it to move. Note how we don’t do any setup or assign any value to these variables, that will be done in our actual game logic.

Ok, we’ve looked at four out of seven files, but those were the short and simple ones. `game.cpp` and `game.h` have some new logic but 90% of our boilerplate code lives inside our `main.cpp`.

lets tackle `game.h` and `game.cpp` next

```
// game.h
#pragma once

#include "SDL3/SDL_render.h"
#include "gameState.h"

extern "C" {
    __declspec(dllexport) void Initialize(GameData* data);
    __declspec(dllexport) bool HandleEvents(GameData* data, SDL_Event event);
    __declspec(dllexport) void Draw(GameData* data, SDL_Renderer* renderer);
    __declspec(dllexport) void Update(GameData* data, float dt);
    __declspec(dllexport) void OnQuit(SDL_Renderer* renderer);
}
```

Note the `#pragma once`. If we look past the very strange looking `extern "C"` and the `__declspec(dllexport)` prefix attached to each function declaration then this .h file looks very standard. Here it is just for posterity

```
// just an example without __declspec
#pragma once

#include "SDL3/SDL_render.h"
#include "gameState.h"

void Initialize(GameData* data);
bool HandleEvents(GameData* data, SDL_Event event);
void Draw(GameData* data, SDL_Renderer* renderer);
void Update(GameData* data, float dt);
void OnQuit(SDL_Renderer* renderer);
```

`extern "C"` ensures that during compilation, our functions won't have their names modified in any way. This is essential when we want to access them later by referencing their names exactly, otherwise the name will be changed in a linking process called name-mangling. (https://en.wikipedia.org/wiki/Name_mangling)

`__declspec(dllexport)` has that very strange syntax as it is not part of normal C++ but an addition by Microsoft to flag the function as relevant to the compiler, in this case making sure the function is made available to other

programs that are interested in calling it. The syntax is very strange, but fortunately we will basically only use it in this specific case. Meaning that as long as we can remember that the header file required some strange additions then we can search for those again later if we forget the exact syntax - that's very common.

```

// game.cpp
#include "game.h"

extern "C" {
    void Initialize(GameData* data){
        data->rect.x = 100;
        data->rect.y = 100;
        data->rect.h = 50;
        data->rect.w = 50;
        data->move_speed = 100;
    }

    bool HandleEvents(GameData *data, SDL_Event event){
        if(event.type != SDL_EVENT_KEY_DOWN){
            return true;
        }
        if(event.key.key == SDLK_ESCAPE){
            return false;
        }

        return true;
    }

    void Update(GameData* data, float dt){
        const bool* keys = SDL_GetKeyboardState(NULL);

        if(keys[SDL_SCANCODE_RIGHT]){
            data->rect.x += data->move_speed * dt;
        }

        if(keys[SDL_SCANCODE_LEFT]){
            data->rect.x -= data->move_speed * dt;
        }

        if(keys[SDL_SCANCODE_UP]){
            data->rect.y -= data->move_speed * dt;
        }

        if(keys[SDL_SCANCODE_DOWN]){
            data->rect.y += data->move_speed * dt;
        }
    }

    void Draw(GameData* data, SDL_Renderer* renderer){
        SDL_SetRenderDrawColor(renderer, 250, 70, 8, 255);
        SDL_RenderClear(renderer);

        SDL_SetRenderDrawColor(renderer, 150, 0, 100, 255);
        SDL_RenderFillRect(renderer, &data->rect);

        SDL_RenderPresent(renderer);
    }

    void OnQuit(SDL_Renderer* renderer){
        SDL_DestroyRenderer(renderer);
    }
}

```

our `game.cpp` is similar to last time. The changes are we work directly on the `SDL_FRect` struct instead of our own `xPos` and `yPos` variables that we later assigned to the `FRect`. We also wrap each function inside the `Extern "C"` scope to mirror the change in the `.h file`, which is necessary to ensure that both declaration and implementation avoid having the function names be name-mangled and no longer have the same name as each other to the linker. If the names were to differ to the linker then we would not be able to call them by `#include` the header.

lets look at the `Update()` function again:

```
// game.cpp
void Update(GameData* data, float dt){
    const bool* keys = SDL_GetKeyboardState(NULL);

    if(keys[SDL_SCANCODE_RIGHT]){
        data->rect.x += data->move_speed * dt;
    }

    if(keys[SDL_SCANCODE_LEFT]){
        data->rect.x -= data->move_speed * dt;
    }

    if(keys[SDL_SCANCODE_UP]){
        data->rect.y -= data->move_speed * dt;
    }

    if(keys[SDL_SCANCODE_DOWN]){
        data->rect.y += data->move_speed * dt;
    }
}
```

Now that we know more about pointers we can see that our `keys` variable holds a pointer to the first byte of the place in memory where all keys (bool's) are layed out sequentially. The `const` keyword means that the variables we find at the memory being pointed to can not be changed by us accidentally in code. It has become `read-only` meaning that we can read the value but not write to it.

Then we can use the array syntax `[]` along with an `enum` to handle the pointer offset to fetch the memory address of the specific keyboard key we are interested in. The enum `SDL_SCANCODE_XXX` is built into SDL and is made for this exact purpose.

meaning that our `if-statement` is true if the specific key is being held down this tick.

NOTE: we would need to store the result of the keys from the previous tick if we want to know if a key has been released or just pressed down this tick. But that is for a later lecture

Enough stalling, lets start digging into our new `main.cpp`

```
// main.cpp
#include <windows.h>
#include <fileapi.h>
#include <stdio>
#include "SDL3/SDL_init.h"
#include "SDL3/SDL_render.h"
#include "SDL3/SDL_timer.h"
#include "common.h"
#include "arena.h"
#include "gameState.h"
```

We're starting to include quite a lot of headers. And some of these headers will eventually start to clash with each other. You'll no doubt start getting messy error messages as your program grows. We will be looking into more robust solutions later, but for now we can resolve this inclusion chain by sticking to best practices when it comes to ordering our `.h` files.

- 1) Start with the global headers from windows and the standard library (that are passed by using `<>`)
- 2) After those we include all headers we have not written ourselves. In this case the SDL3 headers.

3) After that we add our custom made headers

```
// main.cpp
SDL_Window* window;
SDL_Renderer* renderer;
```

We will be passing the pointer to the renderer to our DLL, we also need to have an `SDL_Window` to assign the renderer to.

```
// main.cpp
Uint64 NOW = 0;
Uint64 PREV = 0;
```

Unsigned integers to store the time between ticks so we can calculate our `deltatime`

```
// main.cpp
constexpr const char* NAME_OF_DLL = "Heartburner_game.dll";
constexpr const char* NAME_OF_TEMP_DLL = "Heartburner_temp.dll";
```

A `char*` is a pointer that points to the first byte of a series of individual characters. The block in memory that holds the full character-sequence is automatically followed by a terminator, meaning that just by pointing at the first byte, then reading until we hit the terminator we can get a hold of the full character sequence.

NOTE: another data type that stores text is a `string` but many of the functions we are supplying `NAME_OF_DLL` to only accepts a `char*` and we would have to convert our `string` in order to pass it as a parameter.

A `const char*` means that the characters stored at the memory address are not allowed to change. In other words, the text itself is read-only.

However, without additional qualifiers, the pointer itself could still be reassigned to point somewhere else. That means the variable could later be made to reference a different string.

The `constexpr` keyword makes the variable a `compile-time constant`. This implies that the pointer itself cannot change after initialization, and the compiler knows its value during compilation.

now if we try and write

```
NAME_OF_DLL = "a_new_name";
```

we get this error

```
Cannot assign to variable `NAME_OF_DLL` with const-qualified type 'const char *const'
```

If we never make any mistakes when coding, then all of these qualifiers are unnecessary. But we are adding these to

- 1) safeguard against screwing things up
- 2) Write hireable C++
- 3) learn about `const` and `constexpr`
- 4) help other programmers know the intention of our program

```
// main.cpp
typedef void (*Function_Initialize) (GameData* data);
typedef bool (*Function_HandleEvents) (GameData* data, SDL_Event event);
typedef void (*Function_Update) (GameData* data, float dt);
typedef void (*Function_Draw) (GameData* data, SDL_Renderer* renderer);
typedef void (*Function_OnQuit) (SDL_Renderer* renderer);
```

We need a way of accessing the functions we've set as `__declspec(dllexport)` in our `game.h`. To do this we need to create what is called `function pointers` a `function pointer` is a way of passing a function as a variable. Meaning that we can store the address of a function and call it later. These five function pointers have the same **exact** return type and parameters as the functions inside `game.h`.

`typedef` allows us to take the structure of our specific functions, meaning their return type and parameters and allows us to store them with a more

easily typed name, like `Function_Initialize`

Without our `typedef` we would need to write

```
void (*initFunc)(GameData* data) = MyInitFunction;
```

with our top-level `typedef` we can just use the name we've provided as a substitute for all the behind-the-scenes stuff.

```
Function_Initialize initFunc = MyInitFunction

// main.cpp
constexpr const char* NAME_OF_FUNC_INIT = "Initialize";
constexpr const char* NAME_OF_FUNC_HANDLE_EVENT = "HandleEvents";
constexpr const char* NAME_OF_FUNC_UPDATE = "Update";
constexpr const char* NAME_OF_FUNC_DRAW = "Draw";
constexpr const char* NAME_OF_FUNC_QUIT = "OnQuit";
```

This series of `char*` stores the exact names of the functions found in `game.h` so we only have to write them correctly once, here in the variable declaration. Then we can use the variable anywhere and be sure that we didn't write a type-o anywhere. If we never wrote any type-o's and always remembered exactly the names of functions then we would not need these variables. But since we rarely remember things as well as we imagine we do, then we use these types of safeguards.

```
// main.cpp
struct DLL_INFO{
    HMODULE dll;
    FILETIME timestamp;
    Function_Initialize initialize;
    Function_HandleEvents handleEvents;
    Function_Update update;
    Function_Draw draw;
    Function_OnQuit quit;
};
```

We create a struct that holds all the data we will need to work with our .DLL. the `HMODULE` is, behind the scenes, a pointer to either a .DLL or a .EXE. Now by having access to it in memory we can tell it to do thing, like calling

functions.

`FILETIME` is a data type “Containing a 64-bit value representing the number of 100-nanosecond intervals since January 1, 1601 (UTC).” This might seem like insane precision for us, but we use it because there are handy windows functions that we can use to compare two `FILETIME` variables and we can also easily get the time a file was changed in the form of a `FILETIME` removing the need to convert it from another data type. Our timestamp will be used to check if our .DLL has been recompiled after we started running our game, and if it has been we re-link our dll and start sending the games data to it instead.

Then we have our `function pointers` that we will be pointing to the functions inside the `.DLL` can calling from our `.EXE`

```
// main.cpp
FILETIME GetTimestamp(){
    WIN32_FIND_DATA data;
    HANDLE handle = FindFirstFile(NAME_OF_DLL, &data);
    FILETIME time_of_last_change = data.ftLastWriteTime;
    FindClose(handle);
    return time_of_last_change;
}
```

We use this function to look for a file with the name we specified (`NAME_OF_DLL`) and then we can fetch the built in `LastWriteTime`. This comes in the `FILETIME` format by default. We pass along a pointer to the `WIN32_FIND_DATA` so the `FindFirstFile` function can store the information about the file it found in that variable. That’s why we’re required to pass it `by pointer` using the `address-of` operator `&` meaning that we pass not the value but instead the place in memory. once the `FindFirstFile()` function has filled the `data` struct with information we can get the `FILETIME`.

`FindFirstFile` allocates some memory when called. This is stored in a `HANDLE` variable, this is done no matter what by our operating system when it is asked to look for a file like this. This little chunk of memory needs to be told that it is no longer necessary to keep around by using the `FindClose()` function. If we don't free the memory from our `HANDLE` we will eventually run out of memory as every possible address in our memory is filled with an old no longer relevant `HANDLE`. Freeing memory is a core part of working with C++, but our memory arena lets us do most of that in one place rather than putting allocations and freeing calls all over our codebase.

```

// main.cpp
bool LoadDLL(DLL_INFO* info, int depth = 0){
    printf("loading dll");
    if(depth > 20){
        printf("failed to write temp DLL");
        return false;
    }
    bool success = CopyFile(NAME_OF_DLL, NAME_OF_TEMP_DLL, false);

    if(!success){
        Sleep(50);
        return LoadDLL(info, depth + 1);
    }

    info->dll = LoadLibrary(NAME_OF_TEMP_DLL);

    if(info->dll == nullptr){
        printf("could not load dll");
        return false;
    }

    info->initialize = (Function_Initialize)GetProcAddress(info->dll,
    ↪ NAME_OF_FUNC_INIT);
    info->handleEvents = (Function_HandleEvents)GetProcAddress(info->dll,
    ↪ NAME_OF_FUNC_HANDLE_EVENT);
    info->update = (Function_Update)GetProcAddress(info->dll, NAME_OF_FUNC_UPDATE);
    info->draw = (Function_Draw)GetProcAddress(info->dll, NAME_OF_FUNC_DRAW);
    info->quit = (Function_OnQuit)GetProcAddress(info->dll, NAME_OF_FUNC_QUIT);

    info->timestamp = GetTimestamp();

    return true;
}

```

This function has the job of finding our compiled .DLL, create a copy of it called `NAME_OF_TEMP_DLL` then store that .DLL in our provided `DLL_INFO` struct. We then take the `function pointers` in our `DLL_INFO` struct and point them to the functions we tagged as `__declspec(dllexport)` in our `game.h`.

The first thing, when reading the function that might seem strange is that the function does something bizarre that we haven't encountered before. In case the `CopyFile` function fails and the `success` bool is set to false, then we

wait for 50 milliseconds then we return the result of calling the function again, but incrementing the `depth` parameter by 1 as it is being passed to the re-run of the function. This is called a `recursive function call` meaning that the function has called itself. When trying to copy a .DLL there can, if we are unlucky, be background operations being performed by our operating system that locks the file, making reading it temporarily impossible. To make sure that we:

- 1) are allowed to eventually make the copy
- 2) stop in case some dramatic error has happened that will never allow the file to be copied

we only allow the function to call itself a total of 20 times before stopping.

What the recursive function does is this:

- 1) it tries to copy the .DLL. If it is successful then it just moves along as normal, doing the rest of the variable allocations we need.
- 2) But if it failed to copy it, we wait 50ms then the function returns, meaning that everything after the `return` will not be executed. We have aborted the function early. But we are at the same time starting up another instance of the function. This new instance will run from the top again, trying once more to copy the .DLL. If this once again is unsuccessful it will call another instance of the function and return whatever result that function gave back in its own `return`. Hopefully now the value being returned is the `return true` at the end of the function meaning that we successfully allocated our `DLL_INFO`. Each time a function calls itself we pass along `depth` but first we add 1 to it. Meaning that if ten of these functions call one another still trying to get a successful copy, then `depth` will have a value of `10`. If it ever reaches

20 the function returns `false` without calling itself again. This will stop the recursive loop. This means that the function will keep calling itself unless one of two things happen.

- 3) the .DLL successfully gets copied and its values stored in our `DLL_INFO` pointer.
- 4) the function calls itself for the 20'th time and returning `false` and not calling itself for a 21'st time again.

All recursive functions could be substituted by a `while-loop` or a `for-loop` and vise-versa. And recursive functions are powerful but can be a bit hard to conceptualize at first. I tend to find it helpful to think about each function digging deeper then once we reach one of the two stop conditions we crawl back out of the stack.

```
// main.cpp
void UnloadDLL(DLL_INFO* info){
    FreeLibrary(info->dll);
    info->dll = nullptr;
    DeleteFile(NAME_OF_TEMP_DLL);
}
```

This function accepts a pointer to our `DLL_INFO` struct, it then clears the memory holding the `HMODULE` aka the pointer to our .DLL it takes the `HMODULE` pointer and `null`s it. Meaning that the pointer no longer points at anything at all. It's important to note that just because we've freed the memory associated with our `HMODULE` then that doesn't mean that the memory is gone, we've just told our computer that it is allowed to overwrite it, until it does we could theoretically still use it. Then lastly we take the temporary copy of our .DLL created in the `LoadDLL` function and deletes the file.

```
// main.cpp
void* AllocateGameMemory(){
    void* blob = malloc(GAME_MEMORY_ALLOWANCE);
    if(blob == nullptr){
        printf("fatal error: could not allocate memory");
        return nullptr;
    }

    printf("memory succesfully allocated");
    return blob;
}
```

This function uses `malloc` to tag a block of memory, it then returns a pointer to the first byte of that memory so it can be referenced by our `memory arena`. If `malloc` was unsuccessful then it returns `nullptr` and if it does we have reached a fatal error and we will be terminating our program.

```
// main.cpp
void SDL_Setup(){
    SDL_Init(SDL_INIT_EVENTS);
    window = SDL_CreateWindow("pilot", 650, 400, 0);
    renderer = SDL_CreateRenderer(window, NULL);
}
```

this function as well as `AllocateGameMemory` are just functions that bundle some code we used to have one after the other in our `main()` function. We've just collected them in reasonable functions to make the `main()` function shorter and more descriptive. We could take the content of these functions and add them back into our `main()` as we had in an earlier lecture.

```
// main.cpp
void CalculateDeltaTime(float& dt){
    NOW = SDL_GetTicksNS();
    dt = NOW - PREV;
    dt = SDL_NS_TO_SECONDS(dt);
    PREV = NOW;
}
```

this function does the exact same thing, taking a pointer to a `float` and setting it to the calculated `deltatime`. Here we've used the `float&` rather

than `float*` this is what we called `passing by reference` rather than `passing by pointer` it is just syntactic sugar that lets us work with the value found at the memory address without the `->` and dereferencing it. If this looks confusing then rewriting the function to accept a pointer instead of a reference is as simple as this:

```
// main.cpp
void CalculateDeltaTime(float* dt){
    NOW = SDL_GetTicksNS();
    *dt = NOW - PREV;
    *dt = SDL_NS_TO_SECONDS(*dt);
    PREV = NOW;
}
```

Just a bit more to keep straight but as you can see it is very similar.

```
// main.cpp
void DLL_CheckStatus(DLL_INFO* dll){
    FILETIME timestamp = GetTimestamp();
    bool is_timestamp_changed = CompareFileTime(&dll->timestamp, &timestamp) != 0;
    if(is_timestamp_changed){
        UnloadDLL(dll);
        LoadDLL(dll);
    }
}
```

This function uses three of the functions we wrote earlier to fetch the `FILETIME`, then we compare it and if we find that the .DLL has changed we unload the old .dll with `UnloadDLL()` then copy and create the new one using `LoadDLL()`.

```

// main.cpp
int main() {
    void* game_memory = AllocateGameMemory();
    if(game_memory == nullptr){
        return 1;
    }

    Memory::Arena* arena = new Memory::Arena();
    Memory::Initialize(arena, game_memory, GAME_MEMORY_ALLOWANCE);
    GameData* gameData = (GameData*)Memory::Allocate(arena, sizeof(GameData));

    DLL_INFO dll;
    bool dll_successfully_loaded = LoadDLL(&dll);

    if(dll_successfully_loaded == false){
        return 2;
    }

    SDL_Setup();
    dll.initialize(gameData);

    bool running = true;
    float dt;
    while(running){

        DLL_CheckStatus(&dll);

        CalculateDeltaTime(&dt);

        SDL_Event event;
        while(SDL_PollEvent(&event)){
            running = dll.handleEvents(gameData, event);
            if(running == false){
                break;
            }
        }

        dll.update(gameData, dt);
        dll.draw(gameData, renderer);
    }

    dll.quit(renderer);
    SDL_Quit();
    return 0;
}

```

Here we do the same steps as in our previous lecture, but our `initialize()` `update()` and `draw()` functions are all called from our `DLL_INFO` struct

using the `function pointers` we set in `LoadDLL()` we also run the functions `SDL_Setup` and `AllocateGameMemory` to do initial setups that were previously written as-is in our `main()`. Whenever we call a function we could imagine taking all the code inside that function and just copy-pasting it into the call site. As a summary, this is what our `main()` does

- 1) we allocate a blob of memory
- 2) we create a memory arena and point it to the start of our memory blob
- 3) we load our DLL and set up our function pointers
- 4) we set up SDL, creating our window and assigning our renderer
- 5) we begin our core loop
- 6) inside our core loop we compare `FILETIME` for our DLL
- 7) then we calculate `deltatime`
- 8) we collect all `SDL_Events` and pass them to the `HandleEvent` function in our `.DLL`
- 9) we then call `update` followed by `draw` in our `.DLL`.
- 10) and if our core loop ever terminates we do some cleanup and return 0

We are done! We have now successfully added the boilerplate code necessary to work with our `.EXE` and `.DLL` setup. allowing us to do start developing a game in our next lecture. But before that we're going to see the magic of our system by doing a `hot-reload` of our `.DLL`.

Inside our `game.cpp` in the `Draw()` function we call `SetRenderDrawColor()`. We can now do the following steps

- 1) build the game using `build name_of_project` in powershell
- 2) run the game using `run name_of_project`
- 3) as the game is running, change the colors of our `SDL_SetRenderDrawColor()`

- 4) compile our .DLL using `reload name_of_project` prompting clang to just rebuild the .DLL
- 5) back in our still running program we can see that the changes we made to our game.cpp is directly visible in the game, without us having to close the game and running it again.

This ability to make live-changes to our game is such a huge win for us and will make development of any game so much more streamlined!

11 Rendering images

So far we've just rendered an `SDL_FRect` to the screen. But we of course want to have our own `.PNG` or `.BMP` files and use those. To accomplish this we must do the following things

- 1) Put an image into our `assets` folder
- 2) Expand our `CMakeLists.txt` to copy over our `assets` folder
- 3) prepare a portion of memory to store our images
- 4) use `SDL3_Image.dll` from its corresponding `SDL3_Image.h` file downloaded earlier In case you don't have both these files, then go ahead and download `SDL3_image-devel-3.2.4-VC` from https://github.com/libSDL-org/SDL_image/releases
- 5) swap our `SDL_FRect` to a texture

Opening any drawing software we can create a 32x32px square and fill it with whatever shapes and colors we please. I've created a red square with an 'X' running through it. I've saved it as "fallback.png" as this will be the sprite that gets loaded whenever I attempt to load a sprite that doesn't exist. I do this so I can continue testing and developing even if I lack the necessary assets still.

inside my `assets` folder I've created a subdirectory `sprites` and added my `fallback.png` to it.

11.1 Updating our `CMakeLists.txt`

at the top of our `CMakeLists.txt` we will be adding the following code to specify the location of our assets and the place we want to copy them to. We already have few `set` instructions up there so it's a nice place to keep

storing them

```
set(DIR_ASSETS_ORIGIN ${CMAKE_SOURCE_DIR}/assets)
set(DIR_ASSETS_DESTINATION ${<TARGET_FILE_DIR:${PROJECT_NAME}>}/assets)
```

The syntax is a little bit more confusing when we have to store a yet-to-be-known path. The location where our .EXE and files will end up is specified by our `cmakepresets.json` and when we add another preset for `release` we will be targeting another folder instead of `build`. So to not hard-code our paths we use `${<TARGET_FILE_DIR:${PROJECT_NAME}>}` to fetch the directory where the module called `${PROJECT_NAME}` ended up after compilation finished. So in my case that text gets replaced with `D:\PROJECTS\HEARTBURNER\build` because my `cmakepresets.json` sets `"binaryDir": "${sourceDir}/build"`,

`DIR_ASSETS_ORIGIN` points to the known path of our assets. In a project, our assets folder could eventually grow pretty sizeable, holding images, sound effects and music. Copying them over each time we compile will dramatically slow down our compile times. To get around this we will increase our `cmake_minimum_required(VERSION X.XX)` from `3.25` to `3.26` as cmake version 3.26 adds a very handy instruction `copy_directory_if_different` this according to the cmake documentation <https://cmake.org/cmake/help/latest/manual/cmake.1.html> does the following. We provide the `cmake_minimum_required` at the very top of our `cmakelists.txt`

```
"Copy changed content of <dir>... directories to <destination> directory. If
↪ <destination> directory does not exist it will be created."
```

Then at the very bottom of our `cmakelists.txt` we add

```

add_custom_command(TARGET ${PROJECT_NAME} POST_BUILD
    COMMAND ${CMAKE_COMMAND} -E copy_directory_if_different ${DIR_ASSETS_ORIGIN}
    ↪  ${DIR_ASSETS_DESTINATION} VERBATIM
)

add_custom_command(TARGET ${PROJECT_NAME} POST_BUILD(...)

```

sets up a new command that triggers on `POST_BUILD` meaning that only if the build was successful will this command fire. Inside its scope we run

```

COMMAND ${CMAKE_COMMAND} -E copy_directory_if_different <source> <destination>
↪  VERBATIM

```

`COMMAND ${CMAKE_COMMAND}` gets converted to just `cmake` but makes sure that the same cmake we used during our pre-compilation step is the one being used here. We could swap out `COMMAND ${CMAKE_COMMAND}` for just `cmake` but that runs the risk of causing issues down the road. So we do this little trade-off adding some more syntax to spare us a massive headache later. calling `cmake` is the same action we take inside our `build()` inside our `$profile` to use `cmake.exe` located by our `System Environment Variable` pointing to the `LLVM` folder with `cmake.exe` inside of it.

`-E` is a flag that stops cmake from running its usual build actions and instead executes the command as a internal command. Without it we would get stuck in a build-loop.

`copy_directory_if_different` checks the time when a file was modified and compares it to any file it finds with the same name. If the timestamp is newer it overwrites the file with the new one. Otherwise it does nothing

`${DIR_ASSETS_ORIGIN} ${DIR_ASSETS_DESTINATION}` we specify first the path to our files and then the path to where we want them to get copied over to. We created these variables just a little earlier in this course

`VERBATIM` this is a safety flag that makes sure that the paths we provided

are treated “as is” and nothing gets changed. Some operating systems will treat spaces and special symbols as something to modify or discard, possibly altering the paths we’ve set. VERBATIM stops any of this from happening.

With this updated `cmakelists.txt` we can start working with asset files. And with us having the same folder setup in our `build` folder as in our root we can use common sense paths to access these.

The next step is taking our big monolith `memory arena` and placing another arena inside of it, segmenting a section of memory to be the exclusive area to hold pointers to our sprites.

Note: `sprites` aka `textures` lives on the `GPU` inside our `VRAM` compared to our game data that lives on the `CPU`. We will need to convert each `.PNG` file into `SDL_Texture` storing it in `VRAM` and accessing it by pointer reference inside our memory arena.

NOTE: we could store our sprites as just pixel data in the data type `SDL_Surface` but this would not allow us to use our expensive GPUs to process our images. Instead putting all that work on the CPU. For simple projects this will work. But it will not scale well, due to the CPU being much slower when it comes to pushing a lot of these images to the screen at once - this is expressly the task that a GPU was created to do.

At the top of our `main()` we will be adding a new memory arena by allocating it directly inside our top-level memory arena. When we created our first memory arena we used

```
Memory::Arena* arena_main = new Memory::Arena();
```

The `new` keyword creates a struct of the specified type and returns a pointer to it. It places this piece of memory somewhere on the heap. But our new

memory arena will not be created in the same way, instead we will allocate it directly into this first `arena_main`

```
void* game_memory = AllocateGameMemory();
if(game_memory == nullptr){
    return 1;
}

Memory::Arena* arena_main = new Memory::Arena();
Memory::Initialize(arena_main, game_memory, GAME_MEMORY_ALLOWANCE);
GameData* gameData = (GameData*)Memory::Allocate(arena_main, sizeof(GameData));

Memory::Arena* arena_image = (Memory::Arena*)Memory::Allocate(arena_main,
↪ sizeof(Memory::Arena));
void* image_memory_start = Memory::Allocate(arena_main, GAME_MEMORY_IMAGES);
Memory::Initialize(arena_image, image_memory_start, GAME_MEMORY_IMAGES);
```

lets look closer at the allocation of the memory arena that will hold our images:

```
Memory::Arena* arena_image = (Memory::Arena*)Memory::Allocate(arena_main,
↪ sizeof(Memory::Arena));
void* image_memory_start = Memory::Allocate(arena_main, GAME_MEMORY_IMAGES);
Memory::Initialize(arena_image, image_memory_start, GAME_MEMORY_IMAGES);
```

Our first `Allocate()` function returns a `void*` to the first byte of memory, we then use a `cast` to convert this `void*` to a `Memory::Arena*`. This means that the block of memory allocated will be stored in our variable `arena_image` and due to us specifying `sizeof(Memory::Arena)` we can be sure that we allocated enough space for it. Note: This first allocation only adds the `arena struct` itself into memory. The `arena struct` is just:

```
struct Arena {
    unsigned char* base;
    size_t size;
    size_t used;
};
```

So we have not put the block of memory to store images, just the arena responsible for knowing about and adding to their place in memory.

the second `Allocate()` actually commits a block of memory inside `arena_main` to store the image arena. We then `Initialize()` the `arena_image` so it knows about this block of memory it has been given to manage.

Now we can free all memory inside our `arena_image` by just setting the `used` variable back to 0. Without this sub-arena we could only free the entire `main_arena` all at once.

Ok! Now we have a place in memory to store our image pointers. Now we have to create them and store them.

We will set up a new struct `Image` to hold the relevant variables. We will be storing this in a new `image.h` file

```
#pragma once
#include "SDL3/SDL_renderer.h"
#include "arena.h"

struct Image{
    SDL_Texture* texture;
    int width;
    int height;
};

namespace AssetManagement
{
    Image* LoadSprite(Memory::Arena* arena, SDL_Renderer* renderer, const char* path);
}
```

Our struct holds a pointer to an `SDL_Texture` we will, inside our `LoadSprite()` function fetch a .PNG file and convert it to this `SDL_Texture` format. It is a required step in order for a `SDL_Renderer` to draw it on the screen. We also store the `width` and `height` of the image alongside the `SDL_Texture` so we know how large it is when drawing it. We are putting our `LoadSprite()` function inside a namespace to avoid the

naming conflict from other headers having functions with the exact same name.

our `LoadSprite()` function accepts 3 parameters:

- 1) The arena to store the Image struct in
- 2) A pointer to the renderer that will be used during conversion from PNG to Texture
- 3) The file path for the .PNG

Our textures will live in `VRAM` on our GPU, and our `arena_image` will hold `Image` structs in sequence, each pointing to a different texture in `VRAM`.

The problematic part about creating `SDL_Textures` on the GPU is that we are not directly in control of that memory, the GPU will place these textures in VRAM in places it finds appropriate. We then point at it to use it. If our game is very simple we could even forgo using the GPU entirely, relying instead on the CPU to push pixels to the screen: in this case we use the intermediate `SDL_Surface` instead of `SDL_Texture`. Though using the CPU to put pixels in the screen is a lot slower than having the GPU do it, but it would allow us to store each `SDL_Surface` in our `memory arena` directly. If we want to have the GPU do the heavy lifting AND have complete control of how our sprites are stored in VRAM then we would use a Graphics API like `vulkan` - Though this increases coding complexity significantly compared to the other two methods.

Next we will need to add the actual code to `LoadImage`, then use it to create our texture, store it in VRAM and add its corresponding `Image` struct to our `arena_image`


```

#include <cassert>
#include <string>
#include "SDL3/SDL_render.h"
#include "SDL3_Image/SDL_image.h"
#include "image.h"
#include "arena.h"
using namespace std;

const char* DIRECTORY = "assets/sprites/";
const char* FALLBACK = "assets/sprites/fallback.png";

Image* AssetManagement::LoadSprite(Memory::Arena* arena, SDL_Renderer* renderer, const
↪ char* name){

    string path = DIRECTORY;
    path = path.append(name);

    SDL_Surface* surface = IMG_Load(path.c_str());

    if(surface == nullptr){
        surface = IMG_Load(FALLBACK);
    }

    assert(surface != nullptr);

    SDL_Texture* texture = SDL_CreateTextureFromSurface(renderer, surface);

    Image* img = (Image*)Memory::Allocate(arena, sizeof(Image));
    img->texture = texture;
    img->height = texture->h;
    img->width = texture->w;

    SDL_DestroySurface(surface);

    return img;
}

```

a `string` is very similar to a `char*`, meaning that it stores a sequence of text. But the string data type has a lot of quality-of-life functionality. We will be using it to allow ourselves to not pass in the full path to our .PNG but instead just its name `example_sprite.png` rather than `assets/sprites/example_sprite.png`

First we create the full path to the file using a `string`. We first set it to be text stored in `DIRECTORY` aka `assets/sprites/` then we use the

`append()` function to add our name to the end of the text.

But `IMG_Load()` doesn't support us passing it a `string` as a parameter, it only accepts a `char*`. the string data type comes with a handy function `c_str()` this converts the `string` to a `char*` allowing us to pass it into `IMG_Load()`.

NOTE: if we don't type `using namespace std` after our `#includes` then we can only access the `string` data type by referencing its namespace first `std::string`

We will be loading some (or maybe all) our sprites inside our .EXE and depending on our setup we might want to load some inside our .DLL as well. This means that our `image.cpp` and in case we want to modify our arena(s) inside our .DLL we will need to make sure that both our .EXE and our .DLL have access to these scripts. We will modify our `cmakelists.txt` to create a list of these shared scripts.

```
# Flag .EXE specific files
set(EXE_EXCLUSIVE ${CMAKE_SOURCE_DIR}/src/main.cpp
)

# Flag files that both our .EXE and .DLL need to know about
set(SHARED_SOURCES
    ${CMAKE_SOURCE_DIR}/src/image.cpp
    ${CMAKE_SOURCE_DIR}/src/arena.cpp
)
```

I have put each script reference on its own line, but that is not any different from just creating the worlds longest one-liner.

We can then create a new `.DLL` that we declare to be `STATIC` instead of `SHARED` as our game `.DLL` is. When we make the .DLL static it will share its contents with all other things we create. NOTE: when we mark a `DLL` as `STATIC` it will be copied into both our .EXE and our .DLL, meaning

that its contents will be found in both and the .DLL will not actually live inside our build folder, it is absorbed into them instead. We could mark our .DLL as `SHARED` instead of `STATIC` but then we would have to work with `__declspec(dllexport)` and stuff to access its functions. And that would not make any meaningful difference to our architecture right now.

Inside our `CMakeLists.txt` we first need to create a variable to store its name

```
set(SHARED_LIB_NAME ${PROJECT_NAME}_common)
```

then we create it

```
# SHARED STATIC DLL
add_library(${SHARED_LIB_NAME} STATIC ${SHARED_SOURCES})
target_include_directories(${SHARED_LIB_NAME} PRIVATE include)
```

This will make sharing scripts between our .EXE and .DLL easier as we can add them to the `SHARED_SOURCES` and then we're done.

With this setup we can now use our `image.cpp` and `arena.cpp` scripts inside both our `EXE` and our `DLL` allowing us to create `Image` structs in both and pass the memory arenas to our `DLL` if we want our DLL to allocate pointers and structs to them.

`IMG_Load` is part of the `SDL_image` .DLL that we downloaded earlier, without it we don't have the ability to convert any other image format other than .BMP to a `SDL_Surface`.

We want to be able to create the loading logic for sprites that are yet to be added to our assets folder. We accomplish this by having a fallback sprite that we load each time our `LoadSprite()` function fails to find the specified .PNG.

```
if(surface == nullptr) {...}
```

This if-statement is responsible for checking if what was returned from `IMG_Load()` was an `SDL_Surface` or if it failed and returned a `nullptr` instead. if that happens we instead try and load the fallback .PNG.

we then use something new, an `assertion` an assertion is similar to an if-statement in that we ask a question, in this case we use the `not-operator` `!` to check if assert that surface is in fact not a `nullptr` any longer. If it was a `nullptr` that means that neither our specified .PNG or our fallback .PNG managed to get loaded. At this point we want our program to fail. The `assert` will do just that. We forcibly decide that we will terminate our program so that we can catch these fundamental issues rather than having them fly under the radar.

afterwards we take our `SDL_Surface*` and using our `SDL_Renderer*` we convert it to an `SDL_Texture` that then gets automatically allocated to VRAM.

```
Image* img = (Image*)Memory::Allocate(arena, sizeof(Image));  
img->texture = texture;  
img->height = texture->h;  
img->width = texture->w;
```

We then store an `Image*` inside our `arena_image` and assign the `SDL_Texture` and its information to the variables held inside the `Image` struct.

Once we are done with that we have one job left to do, we have created a new `SDL_Surface` inside memory. Once this function terminates this point in memory is still allocated. Meaning that if this function ran enough times, our memory would be filled up with no longer necessary `SDL_Surfaces`. Calling `SDL_DestroySurface()` lets us free that part of memory. In a more

robust setup we would actually create another `memory arena` to hold this temporary data. If we do that then no part of our program would allocate any memory on the CPU other than the initial block of memory. We'll do this later in the course, for now we accept this temporary allocation.

at the end of the `LoadSprite()` function we return our `Image*` so that whatever system wanted to use it can do so.

Inside our `GameData` struct we could store a series of `Image*`, for a very small game like asteroids or pong it would be simple to have all the sprites referenced right there:

```
Image* ship;  
Image* asteroid_big;  
Image* asteroid_small;  
Image* background;
```

Note: we're not really adding these `Image*` they are just here to illustrate a concept.

But for larger projects we would do one of the following instead

- 1) Pass the image memory arena as part of the `GameData`, allowing whatever piece of code that wants to Load and allocate an `Image*` to do so at runtime. It still lives in the memory held by our executable, so hot-reloading still works. But the VRAM costs would be unknown after initialization and we could overflow the VRAM memory in a (much) larger project.
- 2) We create a central storage for all our images, loaded into memory during initialization. This is just a fancier way of finding them later, and skips the part where we have to explicitly type out each individual image variable ourselves, instead looping over each .PNG in our assets folder

and based in its name we can find the corresponding `SDL_Texture` by pointer reference later.

For right now, lets add our fallback texture to our `GameData` and expand our `Draw()` to render our .PNG instead of the default rectangle.

```
#pragma once
#include "SDL3/SDL_rect.h"
#include "image.h"

struct GameData {
    SDL_FRect rect;
    float move_speed;
    Image* fallback;
};
```

now lets use our memory arena and our new `LoadSprite()` function. inside our `main()` just after we've Initialized our `arena_image` and after we've done our `SDL_Setup()` . If we try to run the code before our `SDL_Setup()` then our `SDL_Renderer` is not initialized and we can't pass it as a parameter.

```
gameData->fallback = AssetManagement::LoadSprite(arena_image, renderer, "fallback.png");
```

Note how we need to specify that the `LoadSprite()` function comes from our `AssetManagement` namespace.

We should also update the size of our `arena_image` during initialization to a more reasonable size, as we know that it will only store `Image`

```
size_t IMAGE_ARENA_SIZE = sizeof(Image) * 1024;
Memory::Arena* arena_image = (Memory::Arena*)Memory::Allocate(arena_main,
    ↪ sizeof(Memory::Arena));
void* image_memory_start = Memory::Allocate(arena_main, IMAGE_ARENA_SIZE);
Memory::Initialize(arena_image, image_memory_start, IMAGE_ARENA_SIZE);
```

This allows us to store 1024 `Image` structs inside the arena. If we ever had 1025 `Image` structs they would not fit and we would have to give it more memory.

Now we can move to the `Draw()` function inside our `game.cpp`. Here we can swap the code that drew our Rect for our fallback texture.

```
void Draw(GameData* data, SDL_Renderer* renderer){
    SDL_SetRenderDrawColor(renderer, 0, 70, 8, 255);
    SDL_RenderClear(renderer);

    SDL_RenderTexture(renderer, data->fallback->texture, NULL, &data->rect);

    SDL_RenderPresent(renderer);
}
```

The third parameter of the `SDL_RenderTexture()` function allows us to specify a region of our texture to render instead of the whole thing. Passing in `NULL` is treated as us wanted to draw the entire thing. Our sprite, when converted to a `SDL_Texture` is presented on a `quad` this can be stretched and modified a bunch. So right now we pass our `rect` from our `gameData` as the fourth parameter, this allows us to specify the width and height of the resulting `quad` meaning that we can shrink and grow the texture as we see fit. Which is great if we want to create small for example pixel-art that we then scale up 10x to actually be rendered in an appropriate size.

Lets update our logic to make a simplified rendering function that any object tasked with rendering textures to the screen can use. We will be creating a new file `rendering.h`

```
#pragma once
#include "SDL3/SDL_render.h"

void RenderSprite(Image* sprite, SDL_Renderer* renderer, int xPos, int yPos, float scale
↪ = 1);
```

We create the function signature for a Render function, passing all the relevant variables. As well as setting a default value for the `float` variable `scale`. This means that we can omit passing this when calling the function and the function will give it a default value of 1.

This allows the following two calls to be functionally identical

```
RenderSprite(sprite, renderer, 50, 20);  
RenderSprite(sprite, renderer, 50, 20, 1);
```

The shorter call also has the scale set to `1` but this is done behind the scenes.

And in a newly created `rendering.cpp` we will write the `RenderSprite()` function

```
#include "rendering.h"  
#include "SDL3/SDL_renderer.h"  
  
void RenderSprite(Image* sprite, SDL_Renderer* renderer, int xPos, int yPos, float  
↪ scale){  
    SDL_FRect rect;  
    rect.x = xPos;  
    rect.y = yPos;  
    rect.h = sprite->height * scale;  
    rect.w = sprite->width * scale;  
  
    SDL_RenderTexture(renderer, sprite->texture, NULL, &rect);  
}
```

We create a `SDL_FRect` that will be passed as the fourth parameter to the `SDL_RenderTexture()` function. We position the rect at the specified `xPos` and `yPos` then fetch the size of the texture and multiply both `height` and `width` by `scale`. So if nothing was passed to the function then we will be multiplying them by `1`, making the height and width be their unmodified defaults. `SDL_RenderTexture()` expects the destination rect to be passed as a `SDL_FRect*` so we need to pass a pointer to the rect by adding the `address-of` operator `&` along with the parameter. Reading the SDL Documentation is the best way of learning these rules.

In a later lecture we will create another identically named function to `RenderSprite` that lets us pass even more parameters that we can use to render only a portion of the texture to the destination rect. This will be

useful to allow us to fit multiple frames of a sprite animation inside a single .PNG.

inside `game.cpp` our `Draw()` function now reads:

```
void Draw(GameData* data, SDL_Renderer* renderer){
    SDL_SetRenderDrawColor(renderer, 0, 70, 8, 255);
    SDL_RenderClear(renderer);
    RenderSprite(data->fallback, renderer, 50, 50);
    SDL_RenderPresent(renderer);
}
```

Short and sweet. Now we are able to keep allocating more `Image*` in our memory arena then passing them to our `game.cpp` living inside our `.DLL` by adding them to `GameData`. With only minimal work we could start to envision how we could make a visual novel if we could only render some text and control the game state more. So not quite there yet, but we are well on our way.

We're currently letting our application run wild, spinning our game loop as fast as it possibly can. Meaning that a few things are true

- 1) our framerate is as high as possible
- 2) our framerate will vary more as the CPU is tasked with doing more on certain ticks and less on others
- 3) we tax our CPU maximally with minimal upside

We will be implementing an enforced framerate. To do this we need to know how long a frame took in milliseconds to process then we need to stop the program from just going to the next tick, instead waiting until the specified time has elapsed. For a 60FPS game each tick gets 1/60 seconds to process. We would rather enforce a stable framerate than hit frame-spikes.

Our CPU can `sleep()` for a specified number of milliseconds, but there are

some nasty quirks of this system. The most egregious being that we can't ensure that the time we specify will be the exact time it sleeps, it will schedule the thread to continue as close to the specified millisecond as possible, but it might overshoot. Our `deltatime` will offset the effect of this being less than absolutely consistent. But we can implement a helpful pattern that will help us begin the next tick exactly on time.

First we will check if we've already taken longer than 1/60 seconds to process the current tick. If we still have time remaining we will `sleep` for a portion of the remaining time, then `spin` for the last couple of milliseconds. This ensures that our `sleep` doesn't overshoot and we tax the CPU only the amount necessary. If we can't hit our target framerate then we should lower it, as a stable 30 is preferred to an unstable 60.

Out of the box our `sleep()` function has a very rough granularity, and even if we specify a very precise number it will not go out of its way to work with very fine-tuned numbers. We will need to give more fidelity to the `sleep()` function by expanding our `cmakelists.txt` to add `winmm.lib` this is an old windows API that allows us to call `timeBeginperiod()` a function that controls the built in windows timer resolution. The finest resolution is setting the granularity to 1ms `timeBeginPeriod(1)`

In our `main()` function, after we've initialized our memory arenas we can add the following

```
MMRESULT result = timeBeginPeriod(1);
if(result == TIMERR_NOCANDO){
    printf("could not increase timer resolution");
    Sleep(2000);
    return 3;
}
```

We could add only `timeBeginperiod(1)` without the extra defensive

code, but then this change to the windows timer resolution could fail silently. I find it's better to ensure the change worked and if not be can close the application. We could also use `assert` here instead.

```
assert(result != TIMERR_NOCANDO)
```

NOTE: yes, the name of the result enum is very goofy.

Inside our `CMakeLists.txt` we need to add `winmm.lib`

```
set(WINDOWS_LIBRARIES winmm)
```

but as this is part of windows we don't have to add this `.lib` file to our `include` folder. We can just add it inside our `CMakeLists.txt`

```
target_link_libraries(${PROJECT_NAME} PRIVATE
    ${LIB_FILES}
    ${WINDOWS_LIBRARIES}
    ${SHARED_LIB_NAME}
)
```

We store winmm inside a variable we name `WINDOWS_LIBRARIES` we do this so we can add more of these libraries in one location then use the variable to add all of them at once. Then for only our .EXE we will add it. Because our game .DLL does not need to know about this code, we can skip linking this .lib file into it. We might come across a future where our .DLL needs this or another windows library, but because we've written our `CMakeLists.txt` from scratch such a change is no longer scary.

Now we need to know how much time a tick took to process. inside our `while(running)` game loop, after we've called `dll.draw()` we can, just as in our `CalculateDeltaTime()` fetch the time in nanoseconds since startup using `SDL_GetTicksNS()`

```
UInt64 frame_end_time_ns = SDL_GetTicksNS();
```

Then we remove the value stored in our `PREV` variable, which has the nanoseconds since startup stored just before we run our `update` and `draw` (its value gets set in our `CalculateDeltaTime()`). We store the result in a new variable `frame_time_spent_ns`. But in our `common.h` we store the target frame time in milliseconds not nanoseconds.

```
double frame_time_spent_ns = frame_end_time_ns - PREV;
```

We could change our `common.h` to include the calculations for `FRAME_TIME_NS` alongside `FRAME_TIME_MS` but I think this is bad practice for 2 reasons, `NS` is very close to `MS` and I can very much see myself making that mistake over and over again. We will only be needing the nanosecond version in this one location, so why pollute our code base with this extra variable.

We then convert and the millisecond representation of our `frame_time_spent_ns` in a new variable.

```
double frame_time_spent_ms = frame_time_spent_ns / 1e6;
```

`1e6` is the programmatic representation of a `1` followed by six zeroes `000000` aka one-million. Taking a number in nanoseconds and dividing it by 1'000'000 gives us the value in milliseconds instead.

We then, using our `FRAME_TIME_MS` as the maximum time to (hopefully) spend on a frame, then subtract the time in milliseconds that the frame actually took. Storing the result once again in a new variable. We could do some of these calculations all in a row on the same variable, but I've chosen to break it out like this for clarity.

```
double time_to_sleep_ms = FRAME_TIME_MS - frame_time_spent_ms;
```

If we took longer than our allowed time budget then `frame_time_spent_ms`

will be larger than `FRAME_TIME_MS` and our `time_to_sleep_ms` will hold a negative value. This allows us to use an `if-statement` to check the value before deciding what to do with it

```
if(time_to_sleep_ms > 0){
    SDL_Delay(time_to_sleep_ms);
}
else{
    printf("missed frame \n");
}
```

`SDL_Delay()` does the same thing as `Sleep()` but will work cross-platform. `Sleep()` only works on windows.

Now we have, with some (granularity issues still) a program that tries to wait for the specified time to elapse before running the next frame. The longer we want the time between frames to be, the larger `deltatime` grows, which we use to scale all movement, so that after 1 second has elapsed, no matter the framerate we still get the correct movement for all objects. You can experiment with this now by setting the `FPS` inside `common.h` to a really low value like `8` and watch as the program starts stuttering.

We can then move the code calculating our `time_to_sleep_ms` into a function, helping us keep our main loop clean. I happen to know that we will soon be needing this same code again, meaning that a function is very appropriate, but if this truly was a one-off block of code then having it live right where it is being used is often preferred.

```
void CalculateRemainingFrameTime_MS(double* milliseconds){
    Uint64 frame_end_time_ns = SDL_GetTicksNS();
    double frame_time_spent_ns = frame_end_time_ns - PREV;
    double frame_time_spent_ms = frame_time_spent_ns / 1e6;
    *milliseconds = FRAME_TIME_MS - frame_time_spent_ms;
}
```

Then we could remove this calculation from our `while(running)` loop and

just create a `double` to pass to the function. We create the `double` outside of the function and pass it in so the value can be assigned inside the function. Another method would be to change the signature of the function to `return` a `double` then instead of passing in a `double` we would assign to a double the value returned from the function. But in our case our new Delay logic looks like this

```
double time_to_sleep_ms;
CalculateRemainingFrameTime_MS(&time_to_sleep_ms);

if(time_to_sleep_ms > 0){
    SDL_Delay(time_to_sleep_ms);
}
else{
    printf("missed frame \n");
}
```

The logic is the same, but we've decided that keeping our `while(running)` loop shorter is worth the obfuscation that comes with breaking the logic out into its own function. It's a trade-off and different programmers will value these things differently.

We have one more issue we need to fix. We can't be sure that our `SDL_Delay()` actually perfectly hits our specified millisecond, making the timer more granular has helped this but not completely fixed it. We will be expanding the logic to `spin` for the last couple of milliseconds and not relying on `SDL_Delay()` to hit our mark. a `spin` or `spinning` is just another `while()` loop that runs, checking if we've reached our desired timestamp yet. This costs more CPU as it will run as many times as it needs, but this increased CPU load is well worth it as hitting our framerate should be an absolute goal of our program.

```

double time_to_sleep_ms;
CalculateRemainingFrameTime_MS(&time_to_sleep_ms);

if(time_to_sleep_ms > 0){
    if(time_to_sleep_ms > 1){
        SDL_Delay(time_to_sleep_ms - 1);
    }
    while (time_to_sleep_ms > 0) {
        CalculateRemainingFrameTime_MS(&time_to_sleep_ms);
    }
}
else{
    printf("missed frame \n");
}

```

We now check if we have more than `1` millisecond of sleeping to do and only then do we `SDL_Delay()` but(!) we sleep for `1` millisecond less than we will need, leaving a portion of very granular time left. This value will almost never be exactly `1` as we can't get that specificity from our `SDL_Delay()` then inside a `while-loop` we recalculate `time_to_sleep_ms` and once we've hit our target (0 or lower) we stop our `while loop` allowing the program to move on to the next frame.

And here we can see how we further managed to reduce both reading complexity and code duplication by using our `CalculateRemainingFrameTime_MS` function in another location. Now we can confidently say that moving this logic into its own function was the right call.

`printf(...)` is actually a pretty expensive function, so this could cause us to keep missing our frame window. We will move on to a more robust system of viewing our framerate in a later part of the course.

Now our application enforces a stable framerate! A lot of what we do in our `main.cpp` is `boilerplate-code` that will be reusable in just about every project.

12 Savestates

With our memory arenas set up and our game infrastructure being made almost entirely from scratch we can start to do some pretty impressive things. The first of these will be us saving and retrieving a complete snapshot of the state of the game.

To accomplish this we will need to

- 1) have a function that converts the game state into binary
- 2) write that binary data into a text file
- 3) read the binary data from the text file
- 4) and then... overwrite the binary data in our memory arena to the binary data we read

For now we'll copy over the entire block of memory into a `.bin` file. Its worth noting that this system is not currently equipped to handle being our official save/load system meant for consumers. But that is an issue we'll tackle later. a `.bin` file is just like a `.TXT` but the file type indicates to us humans that it's holding binary data only.

we will be adding:

- 1) one new `#include`
- 2) two new functions, both just 2 lines long
- 3) two if-statements inside our `while(running)` loop

that is it.

I can't stress enough that this system would be a complete and massive headache to ever even attempt inside Unity or Unreal Engine, the architecture is so obfuscated that any attempt at storing a full global state is so difficult

that another solution would make far more sense.

in our `#include` section we will add

```
#include <fstream>
```

this gives us access to built in functionality that lets us read and write files

We then create two functions

```
void StoreGameState(Memory::Arena* arena){
    std::ofstream file("temp_state.bin", std::ios::binary);
    file.write(reinterpret_cast<const char*>(arena->base), arena->size);
}

void RetrieveGameState(Memory::Arena* arena){
    std::ifstream file("temp_state.bin", std::ios::binary);
    file.read(reinterpret_cast<char*>(arena->base), arena->size);
}
```

`StoreGameState()` writes the contents of the provided `memory arena` to a file. `RetrieveGameState()` reads the content of a file and overwrites the contents of the `memory arena`. The nice part with our `memory arena` being a simple struct with a `pointer` to the first byte and a `size_t` for the total size of the arena is that this is precisely the two parameters needed to read the contents of the file into a place in memory.

with `<fstream>` included we get access to `ofstream` and `ifstream` responsible for writing and reading a file respectively. We create one of these `streams` stored in the `standard namespace` aka `std`.

Both an `ofstream` and an `ifstream` accepts 2 optional parameters. The first being a `char*` for the name and the second an option for how the data is supposed to be interpreted. In our case we store it as binary (0's and 1's) because that is the exact same thing our memory actually is!

we then call `.write` and `.read` and because `.write` expects to get a

`const char*` and our `arena->base` is an `unsigned char*` we need to use `reinterpret_cast<...>` to tell our program to treat it as if it were of the correct type. This is fine due to us not caring about the `arena->base` being anything other than a starting point for our arena and not actual relevant data. We use `unsigned char*` in our arena because it is the default case for when we want a collection of bytes without any padding or other behind-the-scenes stuff that could mess with our implementation.

And once we have stored our data in the file, use `file.read()` to read all the data starting at `arena->base` and continuing to read data with a total size of `arena->size`. We need to specify the size of the data we want to read as we could have stored multiple pieces of info in the same file and would need to be able to read only portions of the file in that case. for the `file.read()` we have to do a `reinterpret_cast<...>` to `char*` as that is the type required by the `.read()` function.

“Both `.write` and `.read` expect a `char*` in their own implementations. Our arena uses `unsigned char*` which is the appropriate type for raw memory. Since both `char` and `unsigned char` are exactly 1 byte we can safely use `reinterpret_cast` to tell the compiler to treat one as if it was the other. We do this conversion because `.read` and `.write` expects their data types to be exactly what they are meant to work with.

We need to call `file.close()` in the end of each function as the filestreams we’ve created have allocated memory on our computer and needs to be freed so other memory can be allowed to overwrite it.

With these functions set up we just have to call them when we press the keyboard. Inside our `while(running)` we expand our `while(SDL_PollEvent(&event))` to include two `if-statements` one for

pressing `F9` and one for pressing `F10`

```
while(SDL_PollEvent(&event)){
    running = dll.handleEvents(gameData, event);
    if(running == false){
        break;
    }

    if(event.type == SDL_EVENT_KEY_DOWN){
        if(event.key.key == SDLK_F9){
            StoreGameState(arena_main);
        }
        if(event.key.key == SDLK_F10){
            RetrieveGameState(arena_main);
        }
    }
}
```

We first check if the event was a `SDL_EVENT_KEY_DOWN` to not try and get the keystroke info from another event entirely. Then we check `event.key.key` the first `key` in `event.key` is a struct `SDL_KeyboardEvent` the second `key` in `event.key.key` is a variable inside `SDL_KeyboardEvent` that is of the type `SDL_Keycode` also unfortunately named `key`. But that is why we need to write `key` twice. We compare the keycode to the specified `SDLK` enum and if we get a match we call the specified function.

And with that we're actually done. We have everything needed to save and load our gamestate. Now we can go into our game, make any changes we want, save the gamestate with `F9` then whenever we press `F10` we are instantly back at that exact point again.

Boom!

13 Sokoban Programming I

We don't yet have for example architecture for working with sound implemented, but We'll hold off on that for a moment. Focusing instead on making some progress on game logic.

It's time we start implementing some gameplay logic. In this course we will be making a **Sokoban** style game. This is a **grid-based** game where you push blocks onto target cells. But(!) that is just the basics. The **Sokoban** base formula has been turned inside out creating some absolutely fantastic puzzle games with rich mechanics and surprising gameplay. To name a few favorites:

- A monsters expedition
- A good snowman is hard to build
- Steven Sausage Roll
- Baba is you
- Void Stranger
- Skipping stones to lonely homes

And in 2026 we will be getting the release of **Order of the Sinking Star**, poised to become the largest and probably most influential **Sokoban** game to date. Time will tell.

To make a Sokoban game we need to 1) Have a grid-based world that has floor and walls 2) Have entities on that grid that can move and be interacted with 3) load a level and populate it with the relevant entities

I will be creating three .PNG files, **ground.png** **player.png** and **wall.png** all are 32x32px squares. the ground will be brown, the player ice-blue and the walls grey. We'll add these to our **assets/sprites** folder.

We will be using a software called `Tiled` to create our levels. We could represent our levels in code directly, but this is not a smart way of handling level creation. Instead we'll download `Tiled` from `tiled.com`

Inside `Tiled` we'll create a new `tileset` importing our three pngs. Then we create two layers `level` and `entities`. In `level` we'll place `ground` and `wall` tiles. And in `entities` we'll place our `player`.

We can then create a map and using our `tileset` we can draw our level. Once we are happy with our test level we can export it from `file->Export As`, give it a name and export it as a JSON file that will have the file extension of `tmj`. a `.tmj` file is just a `.json` and is used in the exact same way. The name just indicates that it is from `Tiled` and I would bet that the file acronym stands for "TileMapJson"

opening our exported `.tmj` file inside `Sublime Text` we can look at the different fields.

The json element `layers` has two sub-elements, each with a couple of fields `data` and `id` are the most important to consider at the moment. Now that we know the structure of our `.json` file we can parse it.

But `Windows` or `SDL` does not have a native Json parser. Instead it is expected that we write our own or use one that someone else wrote. A very good Json parser comes from `nlohmann` and is a single `.h` file that has all the relevant functionality all in the same single location.

We download the `json.hpp` file from `https://github.com/nlohmann/json` I've placed this `.hpp` file in `include/Parsers/`.

Note: `.hpp` is just the dogmatic `C++` way of labeling a header file. The `C-standard` is `.h`. So just remember that when working with C++ `.hpp`

and `.h` are interchangeable.

With our new parser added to our include folder we can begin working with it. First we will need a `struct` that call hold the level data once we have `deserialized` it from `JSON`. We'll create this struct inside a new `.h` file `levels.h`.

```
struct LevelData{
    int w;
    int h;
    uint8_t* cells;
    const char* level_path;
    Entity* entityBuffer;
    int entityCount;

    uint8_t GetCellID(int x, int y){
        return cells[y * w + x];
    }
};
```

Ok, lets break down each part in order. `int w` and `int h` are the variables holding the width and height of our level. We get these from the `width` and `height` elements in our `.tmj` file that we exported from `Tiled`.

`uint8_t* cells` this is a pointer to the first cell stored in memory, this `LevelData` has a sequence of these layed out in memory one after the other, we use the `array indicators []` along with a specified with and height index to find the cell we're looking for in the grid.

`uint8_t` holds, like `char`, a 1 byte element. In this case numbers between 0 and 255. This allows us to have up to 255 unique cell types before we would need to expand to a `uint16_t` that can hold over 65 thousand unique numbers.

`const char* level_path` this will hold the name of the level. So we can reference it in other functions.

`Entity* entityBuffer` we will store a sequence of entities in memory. Using this pointer to the first entity along with `int entityCount` we can loop over each entity using the `array indicator []`

Note: We'll look at the `Entity struct` in just a moment. Until we have added all the relevant code our program won't compile for a while.

We've also created a helper function right inside of the struct, this means that when we're using the struct we can access this function like we could any of the variables stored within it. This function takes a `x` and `y` parameter and uses them to return what type of cell is stored at that grid position. our grid is layed out as a `2d grid`

`00000 01110 01010 01110 00000`

but in memory each cell is layed out in sequence `000000111001010011100000` we use a handy calculation to get the correct element calculated from its `2D` representation into `1D`

```
y * w + x
```

`y * w` sets us at the right row by skipping forward by the the full `width`, then we walk down the row the specified `x` steps. **Remember that multiplication is executed before addition**

Lets look at our `Entity` struct next, kept inside a newly created `entity.h`

```
#pragma once
#include <stdint>

struct Entity{
    uint8_t id;
    int x;
    int y;
};
```

That's it, just position and ID - neat!

Including `cstdint` is what gives us access to `uint8_t`. With this data-oriented programming methodology (as compared to object oriented) we operate on data through functions and model our data as only the relevant information, as compared to the operations being tied to the data itself.

Back in our `levels.h` we will be look at the included headers and two function declarations

```
#pragma once
#include "arena.h"
#include "entity.h"
#include <stdint>
using namespace Memory;

struct LevelData{
    // @MAX: Here's where we had our LevelData variables
    //I've just removed it to make the code block smaller.
};

void CreateLevel(Arena* arena, LevelData* level, const char* level_name);
void CreateEntities(LevelData* lvl_data, Arena* arena);
```

We include the `Memory` namespace so we can avoid typing `Memory::` before we can use `Arena`. This helps us reduce the length of the code in terms of raw characters to type.

`CreateLevel()` will grab a piece of memory within an arena to store the cells of the `LevelData` as well as assigning their IDs. This is done by parsing the `.tmj Json` that we will fetch from disk by using the `level_name` added to the function.

NOTE: we are doing no safeguarding at this stage, meaning that yes, our program will crash if the specified `.tmj` file is not found. We'll look at adding these safety measures to a bunch of functions in a later part of the course.

`CreateEntities()` will loop over the `entities` layer in our `.tmj` and when it finds an entity it will add it to the `entityBuffer` inside `LevelData` at the next slot. The result will be that the entities are packed tightly next to each other in memory. For the game we will be making, we will have zero trouble looping over this array as much as we want, meaning that even if we have to look over the entire array many many times per frame it will cost close to no time at all.

Inside a newly created `levels.cpp` we will be adding the contents of these two functions, as well as adding a third function found only inside the `.cpp`. This means that this function is not accessible by another class that includes the `levels.h` header. This is good practice when we want to break functionality into more discrete and reusable chunks without exposing these to the larger codebase. This concept is known as `encapsulation`

```
// levels.cpp
#include <cstdint>
#include <fstream>
#include <vector>
#include "levels.h"
#include "arena.h"
#include "Parsers/json.hpp"
#include "entity.h"
using namespace std;
```

We start by including our headers.

`cstdint` to get access to `uint8_t` `fstream` to allow reading from disk `vector` as this is the format of the Json Data we get back from parsing `levels.h` included to get access to `LevelData` `arena.h` included so we can pass in an `arena pointer` as a parameter `Parsers/json.hpp` the location of our `nlohmann Json parser` downloaded earlier `entity.h` to get access to the `Entity` struct

using namespace std; to skip specifying the std:: namespace everywhere

```
// levels.cpp
const int LEVEL_INDEX = 0;
const int ENTITIES_INDEX = 1;
```

we then create two constants, each referncing the `Tiled` layer of our `level` and `entities` layers found in our `.tmj` file. Making them `const` is a defensive pattern to make sure we don't accidentally change these numbers in our code.

```
//levels.cpp
void CreateLevel(Arena* arena, LevelData* level, const char* level_name){
    fstream stream(level_name);
    auto jsonResult = nlohmann::json::parse(stream);
    vector dataField = jsonResult["layers"][LEVEL_INDEX]["data"].get<vector<uint8_t>>();
    level->w = jsonResult["width"].get<int>();
    level->h = jsonResult["height"].get<int>();
    level->level_path = level_name;
    size_t size_of_cells = sizeof(uint8_t) * level->w * level->h;
    level->cells = (uint8_t*)Memory::Allocate(arena, size_of_cells);
    for (int i = 0; i < level->w * level->h; i++) {
        level->cells[i] = dataField[i];
    }
}
```

We create an `fstream` object that we named `stream`, this can, during creation, be given a path to the file it should stream data from. We will pass it the path of our `.tmj` file later `"assets/levels/testLevel.tmj"`.

The `nlohmann` namespace contains another layered namespace `json` inside it. Once we have gone into the first namespace, deeper into the second can we find the `parse()` function. We store the result of parsing the `stream` into a variable of type `auto`.

The `auto` variable is a variable that we rely on the compiler to know the correct type of. in actuality the `auto` variable in this case expands to

`nlohmann::basic_json<>` but all we want to know is that this `jsonResult` holds all the elements found in our `.tmj` file. The actual variable type is less interesting.

we can take our `jsonResult` and handle it like a nested set of arrays, each accesible by name. In our `.tmj` file we first had a list called `layers` inside that we had different layers `0` for `level` and `1` for `entities`. This is hard to remember, that's why we stored these two numbers in our constants earlier. Once we are in the right layer we need to find the `data` block this held the 2D grid representation of the level we drew in `Tiled`. This `data` is stored in the JSON as a `vector` of `uint8_t`.

a `vector` is like an `array`, but the size of this can be changed at runtime. Meaning that we can add and remove from this list at runtime without causing errors. the `.get<type>()` function is what takes the contents of our `json` and puts it inside the right `type`. Before we do this, the type is not known.

We take our `pointer` to our `LevelData` and store the `width` and `height` from our `.tmj`. These are stored in our `json` as whole numbers (`integers`) and therefore we should specify this in our `.get<type>()` function as `.get<int>()`

We store the path to the level in our `level_path` variable.

Then it's time to allocate our grid cells into memory. We take the size of a `uint8_t` aka `1 byte`, this is the same as a `char`. (We could even use these interchangeably). And multiply it by the combined `width and height` of our level. Giving us the memory footprint of all the cells. We then allocate this memory chunk to our arena and in the process fetching a pointer to the first cell in our `levels-cells` `pointer`.

It is named `cells` and not `cell` even though the pointer only points to one cell, we can, as we know the size of each cell and the fact that they are layed out sequentially in memory and therefore accesible with the `array indicator []`. If the name was singular this would be confusing.

we then loop an amount of times equal to the total number of cells (width times height of the level) and assign each of the cells in level their correct ID, that we can get from the `vector` aka our list of the same size and order, that we parsed from our `json` earlier and stored in `dataField`

with this we have read the contents of the JSON file and stored relevant information in our `LevelData` pointer that we passed into the function.

Our level represents our static game world, but our player, boxes and other objects will be stored as our `Entity` struct.

We will partition our Main Arena further so we can have an arena that holds just this data. Making clearing it a simple action. But making arenas inside arenas is not something new, we already know how to slice up our main arena further. But with each arena we will have to perform the same few lines of code each time. And when we find code that we write over and over again, we've found a clear candidate for a function.

inside our `Arena.h` we will add a new function `CreateSubArena()` this function accepts a parent arena and a size, then carves out that size of memory from the parent arena and stores it as a new arena.

```
Arena* CreateSubArena(Arena* parent_arena, size_t size);
```

and inside our `arena.cpp` we create the function

```
Memory::Arena* Memory::CreateSubArena(Arena* parent_arena, size_t size){
    Memory::Arena* sub_arena = (Memory::Arena*)Allocate(parent_arena,
↪    sizeof(Memory::Arena));
    void* memory_start = Allocate(parent_arena, size);
    Memory::Initialize(sub_arena, memory_start, size);
    return sub_arena;
}
```

First we allocate the size of the `arena struct` which are just the three variables that construct the arena. Not to be confused with the memory given to the arena to hold.

We then allocate the space of the new arena to the parent arena, committing that chunk of memory so it is not overwritten. Finally we Initialize the `sub_arena` telling it what chunk of memory it has access to and then we return it (as a pointer)

With this change we can simplify our arena creation inside our `main()` function inside `main.cpp`

```
Memory::Arena* arena_main = new Memory::Arena();
Memory::Initialize(arena_main, game_memory, GAME_MEMORY_ALLOWANCE);
GameData* gameData = (GameData*)Memory::Allocate(arena_main, sizeof(GameData));

size_t IMAGE_ARENA_SIZE = sizeof(Image) * 100;
gameData->arena_images = Memory::CreateSubArena(arena_main, IMAGE_ARENA_SIZE);
gameData->arena_levels = Memory::CreateSubArena(arena_main, MEGABYTES(3));
gameData->arena_entities = Memory::CreateSubArena(gameData->arena_levels, MEGABYTES(1));
```

With this we have taken our main arena and carved out specific arenas, meant to be responsible for specific parts of our game data. We will return to further create and subdivide these arenas later.

But in order for these arenas to be assignable we need to actually set up our `GameData` struct to hold the games data, and not our test data that we used previously.

```

struct GameData {
    Image* fallback;
    Image* wall;
    Image* ground;
    Image* player;
    Memory::Arena* arena_levels;
    Memory::Arena* arena_entities;
    Memory::Arena* arena_images;
    LevelData* levels;
    int levelCount;
    int currentLevel;
};

```

We have our sprites, references to our arenas, a pointer to the first level and the amount of levels and the current level index. With the pointer to `LevelData` and `levelCount` we can allocate the levels into our `arena_levels` and use the array symbols `[]` to fetch the correct level.

Note: We do the same thing inside `LevelData` with our `Entity* entityBuffer` and `int entityCount`.

We will create a helper function inside `LevelData` to help up fetch an `Entity`.

```

Entity* GetEntity(int x, int y){
    for (int i = 0; i < entityCount; i++) {
        if(entityBuffer[i].x == x && entityBuffer[i].y == y){
            return &entityBuffer[i];
        }
    }

    return nullptr;
}

```

We loop over all `Entity` structs by fetching them one at a time and comparing both their `x` and `y` to the parameters provided. If an entity is found we return a pointer to it using `&`. If no entity is found at the specified position.

We now need to update a few places inside our code and create a few new `.h` and `.cpp` files. These changes are necessary to render our levels and

entities. We will start by expanding `common.h` adding a few variables to help us know where on the screen we should draw our tiles as well as how big they should be.

```
const int SCREEN_WIDTH = 650;
const int SCREEN_HEIGHT = 400;
const int UPSCALE_FACTOR = 2;
const int CELL_SIZE_PX = 32 * UPSCALE_FACTOR;
```

We use `UPSCALE_FACTOR` to draw our tiles larger than they actually are, meaning that with a `UPSCALE_FACTOR` of 2 every `1x1` pixel is now a `2x2` pixel grid. This is necessary to actually see pixel art as our HD monitors would make them very very tiny otherwise.

`CELL_SIZE_PX` is the width (or height) of our tiles, adjusted for upscaling. This value is used to position the tiles next to each other. If we didn't account for `UPSCALE_FACTOR` then our tiles when drawn larger would overlap each other by 50%.

Inside `void SDL_Setup()` in our `main.cpp` we update our `SDL_CreateWindow()` to use `SCREEN_WIDTH` and `SCREEN_HEIGHT`

```
window = SDL_CreateWindow("pilot", SCREEN_WIDTH, SCREEN_HEIGHT, 0);
```

We will also be passing along our `SDL_Renderer` to our `Initialize()` function inside `game.h/.cpp` though this change, as it is part of the connection between our `.EXE` and `.DLL` requires a bit more work.

Inside `main.cpp` we will update our `typedef` to take a `SDL_Renderer*` as a second parameter

```
typedef void (*Function_Initialize) (GameData* data, SDL_Renderer* renderer);
```

this addition needs to be added to `game.h` as well

```
__declspec(dllexport) void Initialize(GameData* data, SDL_Renderer* renderer);
```

Then lets add the changes to `game.cpp`

```
void Initialize(GameData* data, SDL_Renderer* renderer){
    data->ground = AssetManagement::LoadSprite(data->arena_images, renderer,
↪  "ground.png");
    data->wall    = AssetManagement::LoadSprite(data->arena_images, renderer, "wall.png");
    data->player  = AssetManagement::LoadSprite(data->arena_images, renderer,
↪  "player.png");

    data->currentLevel = 0;
    CreateLevel(data->arena_levels, &data->levels[0], "assets/levels/testLevel.tmj");
    CreateEntities(&data->levels[data->currentLevel], data->arena_entities);
}
```

We load and store our `Image` structs for `ground` `wall` and `player` in our `GameData`. We hard-code our `currentLevel` to start at 0. Then we create level 0 and afterwards we create our entities related to the `currentLevel` (which is also level 0).

Lets add the contents of `CreateEntities` to `levels.cpp`


```

void CreateEntities(LevelData* lvl_data, Arena* arena){
    Reset(arena);
    lvl_data->entityCount = 0;
    fstream stream(lvl_data->level_path);
    auto result = nlohmann::json::parse(stream);
    auto entityData = result["layers"][ENTITIES_INDEX]["data"].get<vector<uint8_t>>();

    for (int i = 0; i < lvl_data->w * lvl_data->h; i++) {
        unsigned char entity_id = entityData[i];
        if(entity_id != 0){
            entityCount++;
        }
    }

    lvl_data->entityBuffer = (Entity*)Memory::Allocate(arena, sizeof(Entity) *
↪ lvl_data->entityCount);
    int index = 0;
    for (int i = 0; i < lvl_data->w * lvl_data->h; i++) {
        unsigned char entity_id = entityData[i];
        if(entity_id != 0){
            int x = i % lvl_data->w;
            int y = i / lvl_data->w;
            lvl_data->entityBuffer[index].id = entity_id;
            lvl_data->entityBuffer[index].x = x;
            lvl_data->entityBuffer[index].y = y;
            index += 1;
        }
    }
}

```

This function starts very similarly to `CreateLevel` but instead of providing a path to our `.tmj` file we retrieve the path we stored inside our `LevelData` struct as `level_path`. But before we do anything else we call `Memory::Reset()` on our `arena_entities` meaning that all entities that existed previously are instantly freed. We also reset our `entityCount` variable to acknowledge this. The first time we run this function we are already at `0`, but if we ever ran it again we would need to make sure everything was reset to the default.

We then find the JSON data from the layer related to entities (rather than the layer for the level as we did previously)

We then loop over all cells that we retrieved and whenever that cell is not 0 (meaning we found an entity) we increment our `entityCount`.

then after we have our block of memory allocated as an array of `Entity` structs we loop over the cells one more time, and when we find an entity (non-zero value) we find the `x` and `y` positions using some clever math that can produce a `2D point` from a `1D array` given that we know the width of the grid.

`x = i % lvl_data->w` finds the `x` coordinate by using the modulo operator to remove the width of the grid from `i` as many times as it can. Meaning that if the width is `5` and `i` is `11` then we remove `5` then we remove `5` again. Leaving an `x` value of `1`.

`y = i / lvl_data->w` we then find the `y` coordinate by taking the value of `i` and dividing it by the width. `11/5` this has the result of `2.2` but since an `int` can't store decimal values those are discarded, giving us a value of `2`. This means that at `i == 11` we get `x = 1` and `y = 2` and with arrays starting at `0` in c++ we know that our entity at `i = 11` is here

```
00000
00000
01000
00000
00000
```

and layed out in its `1D` representation we get

```
00000000000010000000000000
```

we then take our calculated `x y` and the `id` we found and store those at the position `index` which starts at 0 and increases by `1` each time we find an entity. This index shifts the array one step forward, filling each array element with the corresponding info. Lastly we increment `index` so that we,

during the next non-zero entity slot found, we put the corresponding data into the next array element.

With `LevelData` and its `EntityBuffer` loaded from our JSON and with the correct data filled we are ready to start rendering our level and entities.

We will render first the level then the entities ontop of it. We will create `levelRenderer.h/.cpp`

```
#pragma once

#include "gameState.h"

void RenderLevel(GameData* gameData, SDL_Renderer* renderer);
void RenderEntities(GameData* gameData, SDL_Renderer* renderer);
```

Then inside our `.cpp` we will add our `#includes` and write the functions

```
#include "levelRenderer.h" // our newly created .h file with our functions
#include "common.h"        // Has CELL_SIZE_PX
#include "rendering.h"     // needed to call the RenderSprite() function
#include <cstdint>          // to allow us to use uint8_t
```

```

void RenderLevel(GameData* gameData, SDL_Renderer* renderer){

    LevelData lvl = gameData->levels[gameData->currentLevel];

    int board_width_px_half = lvl.w * CELL_SIZE_PX / 2;
    int board_height_px_half = lvl.h * CELL_SIZE_PX / 2;

    for(int x = 0; x < lvl.w; x++){
        for (int y = 0 ; y < lvl.h; y++) {
            uint8_t cellType = lvl.GetCellID(x, y);

            Image* sprite;
            switch(cellType){
                case 1:
                    sprite = gameData->ground;
                    break;
                case 2:
                    sprite = gameData->wall;
                    break;
                default:
                    sprite = gameData->fallback;
                    break;
            }

            float xPos = x * CELL_SIZE_PX;
            float yPos = y * CELL_SIZE_PX;

            xPos += SCREEN_WIDTH / 2.0;
            yPos += SCREEN_HEIGHT / 2.0;

            xPos -= board_width_px_half;
            yPos -= board_height_px_half;

            RenderSprite(sprite, renderer, xPos, yPos);
        }
    }
}

```

Lets break down this function

```
LevelData lvl = gameData->levels[gameData->currentLevel];
```

We fetch the `levels` pointer then using our `array indicator` we fetch the level stored at the `currentLevel` position.

`board_width_px_half` takes the `w` meaning the amount of cells multiplies

it by the size of a cell `CELL_SIZE_PX` (which accounts for `UPSCALE_FACTOR`) then divides that total width of all the cells together by 2 to get half of this width. We need this and the corresponding “half-height” to push the rendered tiles back half the board width/height so that the level stays centered on the screen. Without adjusting our positions by these values a level would, as it grew, move down and to the right.

We nest a `for-loop` inside another `for-loop` as this will allow us to loop over each row one at a time, and each column in turn. We store the current row and column in the `x` and `y` variables.

```
uint8_t cellType = lvl.GetCellID(x, y);
```

Our helper function found inside our `LevelData` struct now gives us a simple way of finding the `ID` of the tile on the specified coordinate.

```
Image* sprite;
switch(cellType){
    case 1:
        sprite = gameData->ground;
        break;
    case 2:
        sprite = gameData->wall;
        break;
    default:
        sprite = gameData->fallback;
        break;
}
```

With this we can assign the correct `Image` to our `sprite` based on the `ID` we got back. the `default` case inside our `switch-statement` will be automatically selected if none of the other cases match. Meaning that instead of the program crashing we render our `fallback sprite` instead. Letting us know that something should be on that coordinate but we have not added it yet.

```

// Shift the tile based on its coordinate and the size of a single tile
float xPos = x * CELL_SIZE_PX;
float yPos = y * CELL_SIZE_PX;

// Without this the first tile would end up in the top-left corner of the window.
// this places it in the dead-center of our window.
xPos += SCREEN_WIDTH / 2.0;
yPos += SCREEN_HEIGHT / 2.0;

// Here we use board_width/height_px_half to shift the tile back so as
// all the tiles are rendered we keep them centered instead of growing
// off the screen
xPos -= board_width_px_half;
yPos -= board_height_px_half;

```

As the comments suggest, these operations on `xPos` and `yPos` calculate the correct position for the tile

```
RenderSprite(sprite, renderer, xPos, yPos);
```

With all the required data fetched we pass it as arguments to our `RenderSprite()` function inside `Rendering.cpp`. But we need to update it to actually use our `UPSCALE_FACTOR` if we don't we will need to pass it as a fifth parameter each time we want to render something

```

void RenderSprite(Image* sprite, SDL_Renderer* renderer, int xPos, int yPos, float
↪ scale){
    // ----- Non-changed code was omitted for brevity
    rect.h = sprite->height * UPSCALE_FACTOR * scale;
    rect.w = sprite->width * UPSCALE_FACTOR * scale;
    // ----- Non-changed code was omitted for brevity
}

```

Now `rect.h` and `rect.w` scale automatically by `UPSCALE_FACTOR` each time as well as still allowing us to supply `scale` as a fifth parameter to have custom scaling.

And our `RenderEntities()` function is in most ways very similar to our `RenderLevel()`. But instead of looping over the entire grid we loop over all entities in our `entityBuffer` then fetch the correct `Image*` from its

`id`. We then perform the same position adjustments as in `RenderLevel()` but I have not added the same `board_width_px_half` and instead opted to just write the equation where it is needed. This is something you can easily refactor to either go with the more readable name or the shorter function call.

```
//levelRenderer.cpp
void RenderEntities(GameData* data, SDL_Renderer* renderer){
    LevelData lvlData = data->levels[data->currentLevel];
    for(int i = 0; i < lvlData.entityCount; i++){
        Image* img;
        Entity entity = lvlData.entityBuffer[i];
        switch(entity.id){
            case 3:
                img = data->player;
                break;
            default:
                img = data->fallback;
                break;
        }

        int xPos = 0;
        int yPos = 0;

        xPos += SCREEN_WIDTH / 2.0;
        yPos += SCREEN_HEIGHT / 2.0;

        xPos -= data->levels[data->currentLevel].w * CELL_SIZE_PX / 2;
        yPos -= data->levels[data->currentLevel].h * CELL_SIZE_PX / 2;

        xPos += entity.x * CELL_SIZE_PX;
        yPos += entity.y * CELL_SIZE_PX;

        RenderSprite(img, renderer, xPos, yPos);
    }
}
```

All that is left now is to rewrite the contents of our `Draw()` function inside `game.cpp`

```

void Draw(GameData* data, SDL_Renderer* renderer){
    SDL_SetRenderDrawColor(renderer, 120, 70, 120, 255);
    SDL_RenderClear(renderer);

    RenderLevel(data, renderer);
    RenderEntities(data, renderer);

    SDL_RenderPresent(renderer);
}

```

We Set our `DrawColor` to any color we like, then calling `SDL_RenderClear` we fill the entire backbuffer with that color. Afterwards we render our level tiles using `RenderLevel` and then our entities on top of that backbuffer. Finally we finish by calling `SDL_RenderPresent` which takes everything we've drawn to our backbuffer and blits it to the window so it can be rendered.

With this we have our level rendering to the screen! In the next chapter we will start adding gameplay logic!

14 Sokoban Programming II

It's time to get a player character moving on the screen. For this we will need to work with our data inside of our `Update()` inside `Game.cpp`. We will add behaviour as `flags` to our entities then based on those behaviours we will treat them differently.

Lets look at our updated `Entity.h`

```
#pragma once
#include <cassert> // So we can use assert()
#include <cstdint> // So we have access to uint8_t

enum Behaviour : uint32_t {
    NONE = 0,
    CAN_MOVE = 1 << 0,
    IS_PLAYER = 1 << 1,
    RESPOND_TO_INPUT = 1 << 2
};

enum class ID : uint8_t {
    GROUND = 1,
    WALL = 2,
    PLAYER = 3
};
```

We're working with a new concept here `enum` and right from the start we're using two different versions `enum` and `enum class`. an `enum` is a named number. Looking at `ID` we can see that each of our tiles have been designated a number. We do this to sync the `id` from our `.tmj` file with our code, so that they match. This is a pretty flimsy setup because if we change the order of our tiles in `Tiled` then their `id` will change and this will no longer match up. We will look at improving this system later, but for now, as long as we keep the id-to-enum setup correct we are good to go. by adding the `class` attribute we make it so we can only access our `enums` by first specifying the `class` like so `ID::GROUND` this is very similar to a namespace.

We also have a new operator `<<` used for our `Behaviour` its known as one of many `bitwise` operators. A `uint32_t` holds `32 bits` to create its number as opposed to a `uint8_t` that holds `8 bits`

```
00000000
```

This is the bits of a `uint8_t` they can either be `0` or `1`. We can turn a number of these `on/off` by setting them to `1`

```
01000101
```

This is like a list of 8 booleans, and that is how we're treating them, we are representing one unique behaviour of an entity with one of these bits. For example if the second bit (right to left) has a value of `1` then `IS_PLAYER` is `true`. if the value is `0` then `IS_PLAYER` is false. With a `uint32_t` we can store 32 booleans in a single location, using the `enum` name to fetch them.

Lets look at the numbers `0, 1, 2, 3, 5, 10 and 32` in bit-format

```
00000000 = 0
00000001 = 1
00000010 = 2
00000011 = 3
00000101 = 5
01000000 = 32
```

Each time we add `1` we flip the rightmost bit to `1` if it was already `1` we flip it back to `0` then flip the bit to the left of it to `1`, if that was also a `1` already we flip its leftside bit (and so on). This means that each bit to the left of the previous is tasked with holding a number twice as large

```
128 64 32 16 8 4 2 1
0 0 0 0 0 0 0 0
```

if all the bits are set to `1` in a `uint8_t` then the total value would be `256` a value of `255` would have all bits except the rightmost set to a `1`. As our

ID enum is not a series of flags but an indicator of the type of tile from TILED we don't need more than a uint8_t as it will be quite a long time before we need more than 256 tiles.

for our behaviour flags to work each number used has to be a unique bit. This means that we can store 8 enum behaviour flags in a uint8_t and 32 of them in a uint32_t.

knowing the value of our bits when flipped to 1 we could write our enum Behaviour like this:

```
enum Behaviour : uint32_t {  
    NONE = 0,  
    CAN_MOVE = 1,  
    IS_PLAYER = 2,  
    RESPOND_TO_INPUT = 4  
};
```

Keep in mind that we did not “miss” 3 we are not allowed to use that number as it could be created by combining 1 and 2 together, meaning that it is not uniquely represented by a single bit. Therefore the sequence is 0,1,2,4,8,16,32,64,128 ...and so on.

Using the bitwise operator << we can learn our first action that manipulates bits. This is a left-shift operation that pushes a bit a certain amount of steps to the left

```
IS_PLAYER = 1 << 1
```

This moves a 1 toggled bit 1 step to the left. Resulting in 00000010 aka a 2.

```
RESPOND_TO_INPUT = 1 << 2
```

this moves a 1 toggled bit 2 steps to the left. Resulting in 00000100 aka

a 4.

Now we can see how our `bitwise left shift` and our simpler `= 0,1,2,4,8...` versions are the same.

Inside our `struct Entity {}` we've added a new variable as well as changing our `uint8_t id` to `ID id`

With the changes from `uint8_t` to our `ID` enum we need to update `levels.cpp` and `levelRenderer.cpp`

```
lvl_data->entityBuffer[index].id = (ID)entity_id;
// lvl_data->entityBuffer[index].id = entity_id;
```

in `levels.cpp` we need to cast our `entity_id` to `ID` so it can be set to our `id`

```
//case 3:
case ID::PLAYER:
```

in `levelrenderer.cpp` we switch from the hard-coded "3" to our `ID`, being much more specific. Avoiding putting what is known as `magic numbers` in our code

```
ID id;
int x;
int y;
Behaviour behaviour;
```

Then inside our struct we add a series of functions. These live inside our struct so we can access them from an `Entity` like `'entity->function();`

```
bool HasBehaviour(Behaviour flags){
    return (behaviour & flags) == flags;
}
```

`HasBehaviour()` takes a flag (or flags as they are collected in one single variable) and checks an `&` operation between them. This means that wherever

there is a `1` in both sequences a `1` will be added to the output.

```
01001001
00001011
=
00001001
```

This boolean function only returns true if all the bits in `flags` were also set to `1` in `behaviour`. This allows us to check multiple `flags` at once.

```
void InitializeBaseBehaviour(){
    assert(id != ID::NONE);
    switch (id) {
        default:
            SetBehaviour(NONE);
            break;
        case ID::PLAYER:
            SetBehaviour((Behaviour)(CAN_MOVE | IS_PLAYER | RESPOND_TO_INPUT));
            break;
    }
}
```

From our `CreateEntities()` function in `levels.cpp` we call `InitializeBaseBehaviour`. This checks what `ID` we've assigned to the entity and then give it its relevant starting behaviours. We are able to pass along multiple `behaviour flags` at once by adding them together using the `bitwise or` operator, making the resulting `bit` a `1` if either of the `bits` checked were a `1`.

We also use `assert` from `<cassert>` this allows our program to, if the assertion was false, to fail. This will let us learn that we have a critical error and halt our program. We use this to catch errors during development. And we can remove these once we build the final version of the game. The reasoning is that if our `assert` fails then we don't want to continue running our application as we've introduced a fatal error that needs to be addressed. In this case the `assert` checks that all `entities` that we initialize have had

their `ID` set.

```
void SetBehaviour(Behaviour flags){
    behaviour = flags;
}
```

Our `SetBehaviour()` function overwrites the current flags with the flags added as a parameter

```
void AddBehaviour(Behaviour flags){
    behaviour = (Behaviour)(behaviour | flags);
}
```

Add behaviour combines the current flags with another flag (or multiple flags) using the `bitwise or`.

```
void RemoveBehaviour(Behaviour flags){
    behaviour = (Behaviour)(behaviour & ~flags);
}
```

The `RemoveBehaviour()` function uses the `bitwise and` `&` that sets the `bit` to `1` of both the bits checked were `1`. And the `bitwise not` `~` that inverts all `1's` and `0's` turning ones into zeroes and vice-versa. The combination of these `bitwise operators` will remove `flags` from `behaviour`.

Lets look at how we've updated `CreateEntities()`

```
if(entity_id != 0){
    int x = i % lvl_data->w;
    int y = i / lvl_data->w;
    lvl_data->entityBuffer[index].id = (ID)entity_id;
    lvl_data->entityBuffer[index].InitializeBaseBehaviour();
    lvl_data->entityBuffer[index].x = x;
    lvl_data->entityBuffer[index].y = y;
    index += 1;
}
```

We've added a call to `InitializeBaseBehaviour()` as well as fixed a cast from `entity_id` to `ID`, a necessary change to compile our project now that `id` is an `enum`.

The next step is writing code inside `game.cpp` fetching the currently pressed, held and released keyboard keys then using that info to manipulate our entities.

in our `GameData` struct we need to store an array of the status of all keys on the previous tick. We will compare this previous key array to the current one.

if a key were pressed in both previous and current then it will be treated as `held` if a key was pressed this frame but not last frame then it will be treated as `pressed` if a key was pressed last frame but not on the current frame then it will be treated as `released`

the `SDL_SCANCODE` enum has a `SDL_SCANCODE_COUNT` this is not a key on the keyboard but instead the last value, letting us know how many keys are present in total. We can now allocate a block of memory to hold, in sequence, this amount of `bools`.

We add a pointer inside `GameData`

```
bool* keys_previous;
```

then allocate this block of memory in our `main.cpp`

```
gameData->keys_previous = (bool*)Memory::Allocate(gameData->arena_levels, sizeof(bool)
↳ * SDL_SCANCODE_COUNT;
```

inside our `update()` in `game.cpp` we can not fetch the current array of `bools` and at the end of the function we can assign the now old values to `keys_previous`

```

// at the top of the Update function
const bool* keys = SDL_GetKeyboardState(nullptr);

...

// at the bottom of the Update function
memcpy((void*)data->keys_previous, keys, SDL_SCANCODE_COUNT * sizeof(bool));

```

our `keys` pointer points to the place in memory where SDL upon initialization holds the state of the keyboard. we could pass an optional `int*` pointer to this function to retrieve the length of the returned array, this is not necessary for us, so we pass `nullptr` instead.

the `memcpy` function simplifies what would otherwise be a for-loop. Instead of looping over all our `SDL_SCANCODE_XXXX` we take a `destination` and a `location` in memory as well as a `size to copy`. This takes the data in `keys` and copies those values to the position in memory starting from `keys_previous`. If we had just assigned `keys_previous = keys` these pointers would both point to the same place in memory, both updating simultaneously as they are actually referencing the same bit of memory. With this `memcpy` we are just grabbing the values. And as we know that we allocated `SDL_SCANCODE_COUNT * sizeof(bool)` amount of memory and that holds every single key, then we can safely use it here to tell `memcpy` how much memory to read and copy.

Now we can use `keys` and `keys_previous` to create our functions that can tell us if a key was pressed, released or held.

We will open `game.h` and, outside of our `extern "C" block` we will create the function signatures for 3 functions. These three functions are not meant to be called from outside the DLL so they don't use any of those tags for them.


```
bool KeyPressed(SDL_Scancode key, const bool* current, const bool* previous);
bool KeyHeld(SDL_Scancode key, const bool* current, const bool* previous);
bool KeyReleased(SDL_Scancode key, const bool* current, const bool* previous);
```

Each of these functions have the same parameters passed into them. First the `SDL_SCANCODE` that we are interested in, followed by the current values of each key, lastly the state of each key on the previous frame.

Back in `game.cpp` we can create the functions, once again outside of our `extern "C"` block.

```
bool KeyPressed(SDL_Scancode key, const bool* current, const bool* previous){
    if(previous == nullptr){
        return current[key];
    }
    return current[key] && !previous[key];
}
bool KeyHeld(SDL_Scancode key, const bool* current, const bool* previous){
    if(previous == nullptr){
        return false;
    }
    return current[key] && previous[key];
}
bool KeyReleased(SDL_Scancode key, const bool* current, const bool* previous){
    if(previous == nullptr){
        return false;
    }
    return !current[key] && previous[key];
}
```

when we use the `and operator` `&&` we can do some simple `boolean logic`

`false && true = false` and `true && true = true`

our `KeyPressed` checks if `current[key]` is true and `previous[key]` is false using the `not operator` `!`

we could also write it as

```
return current[key] == true && previous[key] == false;
```

each of the three functions also have a defensive part where we first check if

`previous` was `nullptr` this is true if:

- a) we have messed something up
- b) it's the very first frame of the game, and nothing has been stored in `keys_previous` yet.

We will be accessing the `entitybuffer` and `cells` in the `currentLevel` a lot. At this point the only way we can access the level we're on is by the following code:

```
data->levels[data->currentLevel]
```

having to write this long-ish line, referring to `data` twice is a bit clumsy. Lets simplify our lives a bit by adding a `GetCurrentLevel()` function to our `LevelData` struct

```
struct GameData {  
    // non-relevant variables omitted  
    LevelData* levels;  
    int currentLevelIndex;  
  
    LevelData* GetCurrentLevel(){  
        return &levels[currentLevelIndex];  
    }  
}
```

It's important that we return the specified level as a pointer reference using `&` and making the return type a `LevelData*` pointer. Otherwise we would be returning not the specified level but a copy of it. This copy would be discarded as soon as it goes out of scope. Meaning that any changes will not be reflected in the actual level.

With our keyboard logic set up as well as our entity behaviour we can start asking questions about our entities and the state of our inputs to drive our game.

Lets look at our `update()` in `game.cpp`

```
void Update(GameData* data, float dt){

    const bool* keys = SDL_GetKeyboardState(nullptr);

    for (int i = 0; i < data->GetCurrentLevel()->entityCount; i++) {
        Entity* entity = &data->GetCurrentLevel()->entityBuffer[i];
```

The first bit of code stores the current keyboard keys, we then use our new `GetCurrentLevel()` to fetch the `entityCount`. We use this to loop over our entities by going from 0 to `entityCount`. Inside our `for-loop` we fetch a pointer to an entity by taking the entity stored in the current `i` location. Both `entityCount` and `entityBuffer` are part of `LevelData` and is fetched from our `GameData* data`.

```
if(entity->HasBehaviour((Behaviour)(Behaviour::RESPOND_TO_INPUT |
↳ Behaviour::CAN_MOVE))){
    int xChange = 0;
    int yChange = 0;
    if(KeyPressed(SDL_SCANCODE_RIGHT, keys, data->keys_previous)){
        xChange = 1;
    }
    else if(KeyPressed(SDL_SCANCODE_LEFT, keys, data->keys_previous)){
        xChange = -1;
    }
    else if(KeyPressed(SDL_SCANCODE_UP, keys, data->keys_previous)){
        yChange = -1;
    }
    else if(KeyPressed(SDL_SCANCODE_DOWN, keys, data->keys_previous)){
        yChange = 1;
    }
}
```

Looping over all entities we check if the flags `RESPOND_TO_INPUT` and `CAN_MOVE` are both set to 1 using `hasBehaviour()`. Only if both of these are 1 do we continue.

We then create 2 variables, meant to store the direction we will be travelling in based on keyboard inputs. We use `if-else` statements so that we can't accidentally go diagonally if we pressed up and right on the same exact frame. We send in the correct `SDL_SCANCODE` and our keyboard data to our

`KeyPressed()` function.

```
if(xChange != 0 || yChange != 0){
    int stepInto_x = entity->x + xChange;
    int stepInto_y = entity->y + yChange;
    Entity* stepInto_entity = data->GetCurrentLevel()->GetEntity(stepInto_x,
↪ stepInto_y);
    uint8_t stepInto_tile_id = data->GetCurrentLevel()->GetCellID(stepInto_x,
↪ stepInto_y);
    if(stepInto_entity == nullptr){
        if(stepInto_tile_id == (uint8_t)ID::GROUND){
            entity->x = stepInto_x;
            entity->y = stepInto_y;
        }
    }
}
}
}

memcpy((void*)data->keys_previous, keys, SDL_SCANCODE_COUNT * sizeof(bool));
}
```

We then check if either our `x` or `y` position should change. If yes, then we start fetching relevant data, like what the new cell position would be using `stepInto_x/y`. We then check if we have an entity already standing on the new cell with `stepInto_entity`. We also get the type of tile we're aiming for with `stepInto_tile_id`. We do this by using the helper functions we've added to our `levelData` struct.

Then if no entity was found in the new position and the type of tile we're moving into was `ground` and not `wall` then we update the `x` and `y` value of the specified entity.

lastly, after having looped over all entities, we store the keyboard state in our `keys_previous`.

With this our player entity can move around the level using the arrow keys!

15 Sokoban Programming IV

As we often do, it's time to `refactor` our code. we're going to break the movement logic out into its own function then learn about something called `recursive functions` which we will need to help us push boxes around on the level.

A recursive function is a function that calls itself. It needs one or more exit conditions or else the function might call itself forever. This would completely stall our program and eventually it would crash.

A recursive function is really just a looping function where we take some state and pass it along to the latest iteration of the function. We can always turn any recursive function into a for-loop (and vice-versa) but once the concept of a recursive function is understood it has a lot more visual clarity.

An example of a small recursive function

```
int Factorial(int nmbr){
    if(nmbr == 1){
        return 1;
    }

    return nmbr * factorial(nmbr - 1);
}
```

This function will multiply all numbers from `nmbr` to `1` together. if we start with `nmbr = 5` we get the following output `5 x 4 x 3 x 2 x 1`. What is important to understand is that as long as we don't reach the end of the recursive chain `nmbr == 1` then we will keep calling `factorial` until our `base case` is reached. This means that the first iteration of this function to get resolved is the fifth iteration, where we passed in `nmbr - 1` and number was `2`. Then we calculate `nmbr - 1` to be `1` and instead of

calling `factorial()` again we just return `1`. Now in the earlier iteration we get `return 2 * factorial(2 - 1)` and with the knowledge that we returned `1` we get `return 2 * 1` aka `2`. The earlier iteration now becomes `return 3 * 2`, the next `return 4 * 6` and finally we return to the very first iteration with `return 5 * 24`. With no earlier iteration to return back to, we can leave our recursive loop having climbed back up from the deepest iteration, each time taking the result back with us to the previous iteration. So this function computes `5! aka 5 factorial` and correctly gets the result `120`.

We are going to use a recursive function to help us push boxes around. We do this because before the player character can move into the cell of the box she pushed, we need to know if the box could move. We could check the tile 1 extra space away, but what if we decide we should be able to push a box that in turn push another box? Does this not start to sound a bit recursive?

Here's what we'll need to do:

- 1) Add a sprite for the box both to our game and `Tiled`
- 2) Remove the old movement logic and put it in a new `TryMove` recursive function inside `game.h/cpp`
- 3) set up the behaviour for the box so it behaves correctly

We can start by adding the box to our list of `IDs` inside `entity.h`

```
NONE = 0,  
GROUND = 1,  
WALL = 2,  
PLAYER = 3,  
BOX = 4 // <- new
```

Then we add the `ID::BOX` case to the `switch case` inside the `InitializeBaseBehaviour()` function

```

void InitializeBaseBehaviour(){
    assert(id != ID::NONE);
    switch (id) {
        default:
            SetBehaviour(NONE);
            break;
        case ID::PLAYER:
            SetBehaviour((Behaviour)(CAN_MOVE | IS_PLAYER | RESPOND_TO_INPUT));
            break;
        case ID::BOX: // <- new
            SetBehaviour((Behaviour)CAN_MOVE);
            break;
    }
}

```

We want to have the box be able to move, but should not get moved by us pressing the arrow keys. So we just give it the `CAN_MOVE` behaviour.

In `gameState.h` we add an `image*` pointer to a box.

```

Image* player;
Image* box; // <- new

```

Then we can load a texture and store the result in our `box` pointer. In our `Initialize()` function inside `game.cpp` we can load it

```

data->player = AssetManagement::LoadSprite(data->arena_images, renderer, "player.png");
data->box = AssetManagement::LoadSprite(data->arena_images, renderer, "box.png"); // <-
↳ new

```

I decided to create a new `.TMJ` file with a level containing more space to move around, a player and a box. I saved this new `.TMJ` inside my `assets/levels` folder. Then inside the same `Initialize()` function we load this new level as well as updating `currentLevelIndex` to refer to this new level

```

data->currentLevelIndex = 1; // updated to `1` from `0`
CreateLevel(data->arena_levels, &data->levels[0], "assets/levels/testLevel.tmj");
CreateLevel(data->arena_levels, &data->levels[1], "assets/levels/testLevel_box.tmj"); //
↳ <- new
CreateEntities(&data->levels[data->currentLevelIndex], data->arena_entities);

```

In `levelRenderer.cpp` we select our box `Image*` to be the texture rendered from `RenderEntities()`

```
switch(entity.id){
    case ID::PLAYER:
        img = data->player;
        break;
    case ID::BOX: // <- new
        img = data->box;
        break;
    default:
        img = data->fallback;
        break;
}
```

Finally we'll add a new function to `game.h`

```
bool TryMove(Entity* mover, LevelData* level, int xDir, int yDir);
```

This uses the info we can get from `LevelData` as well as the direction we are hoping to move the `entity`. It returns a `bool` so that different cases can return either `true` or `false`, controlling the recursive loop.

in our `Update` inside `game.cpp` we'll strip out the logic that was responsible for moving our player and instead add a call to our new `TryMove()` function. We'll keep the for-loop going over each entity and the keyboard-checks


```

for (int i = 0; i < data->GetCurrentLevel()->entityCount; i++) {
    Entity* entity = &data->GetCurrentLevel()->entityBuffer[i];

    if(entity->HasBehaviour((Behaviour)(Behaviour::RESPOND_TO_INPUT |
↪ Behaviour::CAN_MOVE))){
        int xChange = 0;
        int yChange = 0;
        if(KeyPressed(SDL_SCANCODE_RIGHT, keys, data->keys_previous)){
            xChange = 1;
        }
        else if(KeyPressed(SDL_SCANCODE_LEFT, keys, data->keys_previous)){
            xChange = -1;
        }
        else if(KeyPressed(SDL_SCANCODE_UP, keys, data->keys_previous)){
            yChange = -1;
        }
        else if(KeyPressed(SDL_SCANCODE_DOWN, keys, data->keys_previous)){
            yChange = 1;
        }

        if(xChange != 0 || yChange != 0){
            TryMove(entity, data->GetCurrentLevel(), xChange, yChange); // <- new
        }
    }
}

```

with this the final step is to write the contents of the `TryMove()`

```

bool TryMove(Entity* mover, LevelData* level, int xDir, int yDir){
    if(mover->HasBehaviour(CAN_MOVE) == false){
        return false;
    }
}

```

first we check if the `entity` we are trying to move actually is allowed to move based on its behaviour flags

```
// TryMove() continued

int test_x = mover->x + xDir;
int test_y = mover->y + yDir;
Entity* stepInto_entity = level->GetEntity(test_x, test_y);
ID stepInto_tile_id = (ID)level->GetCellID(test_x, test_y);
if(stepInto_entity == nullptr){
    if(stepInto_tile_id == ID::GROUND){
        mover->x = test_x;
        mover->y = test_y;
        return true;
    }
    return false;
}
```

Then we get the `x` and `y` positions that the entity would move into (if they are allowed to move). We check if there is an entity on that spot as well as retrieving the type of tile it is, storing this in `stepInto_entity` and `stepInto_tile_id`. We store it as `ID` requiring us to cast the `uint8_t` that we get from `GetCellID` into `ID`.

Then if there was no entity blocking us `== nullptr` and the tile was `GROUND` then we perform the move, updating `x` and `y`. With this we have successfully performed a move and can return `true`. Otherwise if the tile was not `GROUND` we can now return `false`.

```
// TryMove() continued

if(stepInto_entity->HasBehaviour(CAN_MOVE)){
    if(TryMove(stepInto_entity, level, xDir, yDir)){
        mover->x = test_x;
        mover->y = test_y;
        return true;
    }
}

return false;
}
```

Now at this point we know that we are walking into another `entity`. We

can then check if that entity is allowed to move, if not we can return `false`. If it is allowed to move then we recursively call `TryMove()` again, using that entity instead. This will then return either `true` or `false` letting us know if the cell is available to get moved into by our original mover.

And with this we can now push a box around the level recursively! We're making steady progress towards the basic logic of a Sokoban game!

16 Command Pattern

Now that we have our desired functionality we will (once again) `refactor` it. We're going to take the concepts for moving our entities on the level and create a structure that allows us to undo and redo our movement.

When we move our player (or box) their `x` and `y` both update, but they have no memory of where they stood previously. We need to keep some sort of data that tracks entities and where they have gone. Then we need to be able to go back (and forth) in this chain using `Z` on our keyboard.

One could imagine many solutions on how to store all the previous positions of an entity though there exists a common solution, called a `pattern`, that we can leverage.

a `pattern` is just a way to structure code that is tailored made to solve a specific issue. These have been developed as a lot of codebases face similar challenges and these have proven to be a good fit to solve them.

We'll be implementing the `Command Pattern`. This is designed to store all the relevant data associated with the execution of some code, allowing us to keep the action as data to be accessed later.

Lets begin by setting up our `command.h`

```
// command.h

#pragma once // always add this
#include <stdint> // allows us to use uint8_t

// we make it a `class` to require us
// to type `CMD_TYPE` when accessing it
enum class CMD_TYPE : uint8_t {
    NONE = 0,
    MOVE = 1
};

struct Command {
    CMD_TYPE type;
};
```

Our `Command` struct does very little, it stores an `enum` that we can set to specify what type of Command it is going to be. Currently our logic only calls for a way to store and execute the movement of an entity, so our `CMD_TYPE` only has one relevant value `MOVE`. But as a game of this type would expand, so would this list.

We will be creating new `Command` structs that `inherit` from this base struct. By doing so all future command structs will have access to the `CMD_TYPE type` variable. We'll be using this to determine what type of Command it was that we tried to undo.

```
// command.h

struct MoveCommand : Command {
    Entity* entity;
    int xDir;
    int yDir;
};
```

Here we have our `MoveCommand` responsible for holding an entity and the direction it is going to move in. This is the same data that we fetch from our `Update()` function in `game.cpp` and then use in the `TryMove` function.

We'll need a way of storing our different commands as an array as well as a

way of knowing which command we're currently trying to undo/redo. We'll accomplish this by allocating all our commands all at once in an arena. We'll store them in a new struct inside `command.h`

```
// command.h

struct CommandBuffer{
    AnyCommand* allCommands;
    int capacity;
    int index;
    int head;
};
```

You'll notice that `AnyCommand` is a new type that we haven't talked about yet. `capacity` holds the bounds of the array by setting a count. `index` is an indicator between `0` and `capacity` that tells us which command we're on. we'll also store the value of `index` in `head`. This will allow us to know how much we are allowed to redo once we begin walking index backwards as we undo our commands.

`AnyCommand` is a new type called `union`. It helps us solve an otherwise annoying problem. Our commands, depending on their variables, will be of different sizes. But the only way of pre-allocating them and accessing them with an array indicator is to have each command take up the same space in memory.

```
// command.h

union AnyCommand {
    Command command;
    MoveCommand move;

    AnyCommand(MoveCommand mv){
        move = mv;
    };
};
```

The `union` keyword makes this new `AnyCommand` have the same size as

the largest struct it could represent. This means that when we allocate AnyCommands we allocate the largest command meaning that we're sure that each slot in memory is large enough to fit any of the commands we're using. Without this we would get less data than we needed when fetching large commands if we had allocated the base `Command` struct.

We're also creating what is called a `constructor` for `AnyCommand` this is needed to allow a command like `MoveCommand` to be `cast` into `AnyCommand`. We'll be needing this in order to simplify creating a new `MoveCommand`. Lets compare the syntax needed if we use or skip this `constructor`

```
// without the constructor
```

```
AnyCommand command;  
command.move.xDir = 1;  
Push(command);
```

```
// with the constructor
```

```
MoveCommand move;  
move.xDir = 1;  
Push(move);
```

without the constructor we have to explicitly create `AnyCommands` then access the correct command from it.

The constructor is a function without a custom name, just the type directly. In this case `AnyCommand`, we then pass in necessary data, the `MoveCommand` we'll be creating. Inside the `Constructor` function we then assign the values of the variable `move` with the provided `mv`.

We could continue without these constructors, but it makes the code we'll write later easier as we can more or less forget about the `AnyCommand` struct and work with the commands directly.

Finally we need to create three functions, these are responsible for adding a

new command to the array, undoing a command and redoing a command

```
// command.h

void Push(CommandBuffer* buffer, AnyCommand cmd);
void Undo(CommandBuffer* buffer);
void Redo(CommandBuffer* buffer);
```

We've opted for calling the function that adds a new command to the array `Push()` as this is the normal syntax we'll find if we work with something called a `queue` data type. This logic we've set up imitates the same logic as a `queue`.

Inside `command.cpp` we'll add the bodies to these functions as well as creating a new `Execute()` function that takes `AnyCommand` and runs the logic that we want. In our case, moving the player and box(es). The reason we don't have our `Execute()` function in our `command.h` is because we don't want any script other than `command.cpp` to be able to call this function.

```
// command.cpp

void Execute(AnyCommand cmd){
    switch(cmd.command.type){
        case CMD_TYPE::NONE:
            break;
        case CMD_TYPE::MOVE:
            MoveCommand mv = cmd.move;
            mv.entity->x += mv.xDir;
            mv.entity->y += mv.yDir;
            break;
    }
}
```

the `Execute()` function uses the `.type` enum held in the `Command` base class to determine which type of `Command` we've passed in. We then use a `switch case` to run the correct code.

a `switch case` allows us to define multiple cases that something could be, then only run the code inside the relevant case.

for example

```
// example

switch(player.health) {
case <= 0:
    cout << "you're dead" << endl;
    break;
case player.maxHealth:
    cout << "you feel great" << endl;
    break;
default:
    cout << "you're hurt but alive" << endl;
    break;
}
```

in this example `player.health` is checked to be either `at or below zero` or at the maximum `player.maxHealth`. We also use the `default` syntax. This is selected when there is no other case that fits. The `break` sets the end of a case, so the code doesn't continue into the next case.

in our `Execute` function we can use the `switch` to determine what command we're working with.

```
// command.cpp
case CMD_TYPE::MOVE:
    MoveCommand mv = cmd.move;
    mv.entity->x += mv.xDir;
    mv.entity->y += mv.yDir;
    break;
```

We take the `MoveCommand` from the `AnyCommand` `union` and then work with its data. The `MoveCommand` holds a pointer to an entity as well as the direction to move it. We used to update the entity `x` and `y` inside `game.cpp` but we're moving it here instead.

If we create more Commands we need to add them to our `AnyCommand` union, create their `Constructor` then use their variables inside our `Execute` function to actually do something. It is extra code, but it's actually very

manageable. But(!) it's very very important to understand that this code has made our game logic less simple, we've created a layer of abstraction in our system. We're doing this because this makes the logic responsible for undo/redo trivially easy - that is why it is worth it.

Next, let's look at our `Push()` function

```
// command.cpp  
  
void Push(CommandBuffer* buffer, AnyCommand cmd){  
    buffer->allCommands[buffer->index] = cmd;  
    buffer->index++;  
    buffer->head == buffer->index;  
    Execute(cmd);  
}
```

We take the `AnyCommand` that we've passed in as a parameter and assign it to the specific element in our array indicated by our current `index`. We do this storage step to later allow us to undo the command.

We then increment our `index` by `1` by using the `increment operator` `++`. the code below does the same thing.

```
// increment operator example  
  
buffer->index += 1;  
buffer->index++;
```

We then store this new value of `index` in `head`. We only update the value of `head` when we `Push()` a new Command, meaning that it is always synced with the last pushed command in our chain.

lastly we call `Execute()` and pass along our `cmd`.

Let's look at our `Undo()` function next

```
// command.cpp

void Undo(CommandBuffer* buffer){
    if(buffer->index == 0){
        return;
    }
    buffer->index--;

    AnyCommand cmd = buffer->allCommands[buffer->index];
    switch(cmd.command.type){
        case CMD_TYPE::NONE:
            break;
        case CMD_TYPE::MOVE:
            MoveCommand mv = cmd.move;
            mv.entity->x -= mv.xDir;
            mv.entity->y -= mv.yDir;
            break;
    }
}
```

First we check if we're currently at the very first command `index is 0` if we are then there is nothing left to undo and we `return` early.

otherwise we decrement index by `1` using the `decrement operator --`. Like with the `increment operator` this removes `1` just as if we'd written `index -= 1`

Once we have set our index to point to the previous command, which is actually the last command we executed we do the very same `switch case` syntax but this time we use the variables stored in our `MoveCommand` to reverse what happened during `Execute()`. In the case of a `MoveCommand` we move the `entity` back in the opposite direction using `-=` instead of `+=`.

Then as we add new commands we will make sure to add the actual logic to both switch cases in `Execute()` and `Undo()`.

Lastly we have `Redo()`

```
// command.cpp

void Redo(CommandBuffer *buffer){
    AnyCommand cmd = buffer->allCommands[buffer->index];
    if(cmd.command.type == CMD_TYPE::NONE){
        return;
    }
    if(buffer->index == buffer->head){
        return;
    }
    buffer->index++;
    Execute(cmd);
}
```

it is almost exactly the same as our `Push` except we don't pass in a `Command` to assign to the array. We just fetch the current one by `index`. if we already are at our furthest point aka our `head` then we return early.

then we increment `index` and `Execute()` the command again. The command stored at `index` is the last one we undid. By not syncing `head` to `index` as we do in `Push()` we maintain the furthest point we're allowed to redo. only when we push a new command does head update. This means that if we have made 100 moves, then undone 40 of those our `head` is at 100 and our `index` is at 40. meaning we have 60 commands that we are allowed to redo. but if we push a new `Command` our `index` will increment to 41 and our `head` will sync back with `index` making the commands between 41-100 unaccessible as we've started on a totally new path and the old commands beyond 40 are no longer relevant. This solution allows us to overwrite the contents of the `Commands` stored after 41.

Now we need to add our `CommandBuffer` pointer and a new `Memory::Arena*` to our `GameData` struct

```
// gameState.h
struct GameData {
    // other variables inside struct removed from clarity
    Memory::Arena* arena_commands;
    CommandBuffer* commandBuffer;
};
```

We will allocate our `CommandBuffer` to our `arena_main` so that it is never removed. Then our `AnyCommand*` array will be allocated to our new `arena_commands` that itself is a sub-arena of `arena_levels`

so inside our `main.cpp` we add these allocations

```
// main() inside main.cpp
Memory::Arena* arena_main = new Memory::Arena();
Memory::Initialize(arena_main, game_memory, GAME_MEMORY_ALLOWANCE);
GameData* gameData = (GameData*)Memory::Allocate(arena_main, sizeof(GameData));

size_t IMAGE_ARENA_SIZE = sizeof(Image) * 100;
gameData->arena_images = Memory::CreateSubArena(arena_main, IMAGE_ARENA_SIZE);
gameData->arena_levels = Memory::CreateSubArena(arena_main, MEGABYTES(3));
gameData->arena_entities = Memory::CreateSubArena(gameData->arena_levels,
↪ MEGABYTES(1));
gameData->arena_commands = Memory::CreateSubArena(gameData->arena_levels,
↪ MEGABYTES(1)); // <- new

gameData->levelCount = 5;
gameData->levels = (LevelData*)Memory::Allocate(gameData->arena_levels,
↪ sizeof(LevelData) * gameData->levelCount);
gameData->keys_previous = (bool*)Memory::Allocate(gameData->arena_levels,
↪ sizeof(bool) * SDL_SCANCODE_COUNT);

gameData->commandBuffer = (CommandBuffer*)Memory::Allocate(arena_main,
↪ sizeof(CommandBuffer)); // <- new
gameData->commandBuffer->capacity = 2000; // <- new
size_t COMMAND_SIZE = sizeof(AnyCommand) * gameData->commandBuffer->capacity; // <-
↪ new
gameData->commandBuffer->allCommands =
↪ (AnyCommand*)Memory::Allocate(gameData->arena_commands, COMMAND_SIZE); // <- new
```

We have just hard-coded `capacity` to be 2000. Our biggest command `MoveCommand` holds 2 integers and a pointer. this gives us a total of 16 bytes of memory to store a single `Command`. With 1 megabyte of memory (1 million bytes) allocated to the `arena_command` we can actually store closer to 62500

commands. I've just lazily set the current bounds at 2000.

Now we just have to worry about `game.h/cpp` where we will be using this new logic.

First we have to update our `TryMove()` function signature inside `game.h` to also pass in a `CommandBuffer*` pointer

```
// game.h
// bool TryMove(Entity* mover, LevelData* level, int xDir, int yDir); // old
bool TryMove(Entity* mover, LevelData* level, CommandBuffer* cmd_buffer, int xDir,
↳ int yDir); // <- new
```

then inside `game.cpp` inside our `TryMove()` function we update the signature to match then remove the code that updated the `x` and `y` of the entity (this happens in two places) and instead we create a new `MoveCommand` and passes it to our `Push()` function.

Note: we need to `#include "command.h"` to access these.

```

// game.cpp
bool TryMove(Entity* mover, LevelData* level, CommandBuffer* cmd_buffer, int xDir,
    ↪ int yDir){
    // some code hidden from clarity

    if(stepInto_entity == nullptr){
        if(stepInto_tile_id == ID::GROUND){
            MoveCommand mv;           // <- new
            mv.type = CMD_TYPE::MOVE; // <- new
            mv.entity = mover;        // <- new
            mv.xDir = xDir;           // <- new
            mv.yDir = yDir;           // <- new
            Push(cmd_buffer, mv);     // <- new
            return true;
        }
        return false;
    }

    if(stepInto_entity->HasBehaviour(CAN_MOVE)){
        if(TryMove(stepInto_entity, level, cmd_buffer, xDir, yDir)){
            MoveCommand mv;           // <- new
            mv.type = CMD_TYPE::MOVE; // <- new
            mv.entity = mover;        // <- new
            mv.xDir = xDir;           // <- new
            mv.yDir = yDir;           // <- new
            Push(cmd_buffer, mv);     // <- new
            return true;
        }
    }
    return false;
}

```

So we create a new `MoveCommand` assign its variables and pass it into `Push()`. Note that `mv.type` comes from `Command` and is accessible because `MoveCommand` inherits from `Command`.

Now at our callsite for `TryMove()` we have to update the signature as well. This is done inside `Update()`

```
// game.cpp

if(xChange != 0 || yChange != 0){
    // TryMove(entity, data->GetCurrentLevel(), xChange, yChange); // old
    TryMove(entity, data->GetCurrentLevel() , data->commandBuffer, xChange, yChange); //
    ↪ <- new
}
```

and to use our undo/redo functionality we only have to check if we are pressing or holding the right keys in Update()

```
// game.cpp
if(KeyPressed(SDL_SCANCODE_Z, keys, data->keys_previous)){
    if(KeyHeld(SDL_SCANCODE_LSHIFT, keys, data->keys_previous)){
        Redo(data->commandBuffer);
    }
    else{
        Undo(data->commandBuffer);
    }
}
```

So if we press **Z** we undo, and if we press **Z** whilst holding **Left Shift** we redo.

With this we have implemented undo/redo functionality by leveraging the battle-tested **Command Pattern** and despite there being quite a lot of text in this chapter to help explain what we're doing there is surprisingly little actual new code and we only had to make changes to a handful of our previously existing script files.

17 Developer Tools with DearImGui

So far, we've added quite a few quality of life features to our game. We can `store a game state`, we can `undo/redo actions`, we can `hot-reload our code` by splitting our program into an `exe` and a `dll`.

But! The largest differentiating factor between our development environment and an off-the-shelf engine like Unity or Unreal Engine is the lack of a visual development ui. Something where info about our game and buttons, gizmos, sliders and text boxes could live.

We're going to solve that today by adding `Dear ImGui` to our project. `Dear ImGui` is an `immediate mode GUI` framework that allows us to, with very very little code, get a developer window up and running.

This window is not meant to act as actual game UI, but is instead only meant to hold our development tools. `Dear ImGui` uses a game engine style approach where no state is copied over to the gui, instead all of the data is being fed to the gui each frame. This ensures that there is no desync between what the gui visualizes and what the data of the game is.

We will download `Dear ImGui` from: <https://github.com/ocornut/imgui/releases>

At time of writing the latest release was `v1.92.8`

We've come to expect that everything we download and add to our program is a bunch of .h files and .lib or .dll files. But this framework comes just as a series of .h/cpp files.

This is not really a problem and we'll have it up and running in no time.

In the root of our project I'll add a new directory called `src_external` this

is because I don't want to have these new .cpp files mingle directly with my own. It also helps if I should decide I don't want to include these .cpp files in my .exe later.

inside `src_external` I'll add a new subdirectory just named `imgui`. Inside it I'll fetch the following files from the `Dear ImGui .ZIP` I downloaded earlier

- `imgui.h`
- `imconfig.h`
- `imstb_truetype.h`
- `imstb_rectpack.h`
- `imstb_textedit.h`
- `imgui_internal.h`
- `imgui_impl_sdl3.h`
- `imgui_impl_sdlrenderer3.h`
- `imgui.cpp`
- `imgui_draw.cpp`
- `imgui_tables.cpp`
- `imgui_widgets.cpp`
- `imgui_impl_sdl3.cpp`
- `imgui_impl_sdlrenderer3.cpp`

Note the six `.cpp` files, we'll need to reference these in our `CMakeLists.txt` in order to give access to both our `.exe` and `.dll`. We use a `GLOBAL` command

to fetch all the .cpp files from our normal `src` folder, and we can do the same action for this new directory that we named `src_external` or we can manually reference them. To show how we would go about this, and because the files won't change after this point lets look at adding them by direct name reference.

```
// cmakeLists.txt

# Collect imgui cpp files
set(IMGUI
    src_external/imgui/imgui.cpp
    src_external/imgui/imgui_draw.cpp
    src_external/imgui/imgui_tables.cpp
    src_external/imgui/imgui_widgets.cpp
    src_external/imgui/imgui_impl_sdlrenderer3.cpp
    src_external/imgui/imgui_impl_sdl3.cpp
)
```

We create the variable `IMGUI` above that holds all the `.cpp` files we've added.

```
// cmakeLists.txt

add_executable(${PROJECT_NAME} ${EXE_EXCLUSIVE} ${IMGUI})
target_include_directories(${PROJECT_NAME} PRIVATE include src_external)
```

Then for both our `.exe` and our `.dll` we make sure that `${IMGUI}` is added to the list of files that they can access as well as where they are allowed to look for `.h` files. In this case our newly created `src_external` folder.

```
// cmakeLists.txt

add_library(${DLL_NAME} SHARED ${DLL_EXCLUSIVE} ${IMGUI})
target_include_directories(${DLL_NAME} PRIVATE include src_external)
```

Later we will look at how we can limit access to `Dear ImGui` if we build a `Release` version rather than a `Debug` version. But for now, these are all the additions we need to add to our `cmakeLists.txt`

Next we'll create `dev_gui.h/cpp`.

```
// dev_gui.h

#pragma once
#include "SDL3/SDL_render.h"
#include "SDL3/SDL_video.h"
#include "imgui/imgui_impl_sdl3.h"
#include "gameState.h"

namespace DEV{
    void Initialize(SDL_Window* window, SDL_Renderer* renderer);
    void ProcessEvents(SDL_Event* event);
    void PreDraw();
    void Draw(GameData* data, SDL_Renderer* renderer);
}
```

Because of the generic names of the functions I've put them in their own namespace. The other option is naming them `gui_functionName`. Without one of these two solutions we get errors if we try and include two different `.h` files that both implement functions with the same name.

`Initialize` will be used to set up required `Dear ImGui` boilerplate. `ProcessEvents` will grab the `SDL_Event*` pointer that holds information about if the mouse was clicked or a key was pressed. This will hook into the `ImGui` code to make it so we can drag it around and interact with it. `PreDraw` is a step that we do before each `Draw`. the predraw sets up the `frame`. Finally in `Draw` we actually call all of our `ImGui` code responsible for putting our menus, buttons and sliders on the screen - that's why we pass along a `GameData*` pointer to `Draw`.

Now lets implement them

```
// dev_gui.cpp - part 1

#include "dev_gui.h"
#include "gameState.h"
#include "command.h"
#include "imgui/imgui_impl_sdlrenderer3.h"
#include "SDL3/SDL_render.h"
#include <string>

using namespace std;

void DEV::Initialize(SDL_Window* window, SDL_Renderer* renderer){
    ImGui::CreateContext();
    ImGui_ImplSDL3_InitForSDLRenderer(window, renderer);
    ImGui_ImplSDLRenderer3_Init(renderer);

    ImGuiIO& io = ImGui::GetIO();
    int w, h;
    SDL_GetWindowSize(window, &w, &h);
    io.DisplaySize = ImVec2((float)w, (float)h);
}
```

The `Initialize()` function creates what is known as a `context`. This is required as it is what is responsible for holding all the info about our `ImGui`. Because `Dear ImGui` is code we haven't written ourselves it becomes a bit more difficult to break down every part of it, as what happens in the background is a bit beyond the scope of this lecture series. In `Helix` you can press `g-d` to jump to the function under the caret. If you're interested you can dive into the `ImGui` code and see what it does under the hood. But for brevity we need to actually have a `context` to have `ImGui` able to do anything.

`ImGuiIO` is a part of the `context` struct. It holds a lot of data that `ImGui` uses to understand how it is supposed to work. One thing it needs to know is how large the game window is. We use the handy `SDL function` called `SDL_GetWindowSize` to get the window size, the `width` and `height` are stored in the `w` and `h` variables that we pass along by `reference`. So the `GetWindowSize` function actually sets new values for `w` and `h` that we

then pass along to the `ImGuiIO`. `ImVec2` is just a struct that `ImGui` has created that holds 2 `floats` but has some additional functionality that helps `ImGui` check that everything is working behind the scenes. So we cast our `ints` to `floats` as that is the type `ImVec2` expects.

Next we do setup that comes ready-made with `ImGui` - send `window*` and `renderer*` to helper functions that hook `ImGui`'s backend up with `SDL3`.

How were we supposed to know this in advance? we weren't. This is what example projects and code documentation is for.

```
// dev_gui.cpp part 2
void DEV::ProcessEvents(SDL_Event* event){
    ImGui_ImplSDL3_ProcessEvent(event);
}
```

Nice and simple, `ImGui` provides a function that we can use to pass along our `SDL_Event*` pointer. We could use their function directly, but using our `dev_gui` like a simplified remote control is helpful as it allows us to put all code that interfaces with `ImGui` in one spot.

```
// dev_gui.cpp part 3
void DEV::PreDraw(SDL_Renderer* renderer){
    ImGui::NewFrame();
}
```

We call the `NewFrame()` function that lives inside the `ImGui` namespace. This is to safeguard as the function name is very generic (very similar to our own naming standard)

```
// dev_gui.cpp part 4

void DEV::Draw(GameData* data, SDL_Renderer* renderer){
    ImGui::Begin("Dev Tools");

    // Our specific ImGui code will go here

    ImGui::End();
    ImGui::Render();
    ImGui_ImplSDLRenderer3_RenderDrawData(ImGui::GetDrawData(), renderer );
}
```

between `Begin` and `End` is where we will add all our code that lets us add buttons, sliders etc to our `Dev window`. Each `Begin+End` pair will produce its own dev window.

Once we have created all of our stuff we call `Render()` and right afterwards we call the specific `SDL3` helper function `RenderDrawData()` that takes (behind the scenes) everything that `Render()` set up in a generic way, and displays it using `SDL3's` render system.

Now lets set up three functions inside our `.cpp` that we'll call inside our `Begin+Draw` area.

```
// dev_gui.cpp part 5

void Draw_ImGui_Arena_Usage(Arena* arena, std::string name_of_arena){
    float fraction = (float)arena->used / (float)arena->size;
    string barText = name_of_arena
    barText += " " + to_string(arena->used);
    barText += " / " + to_string(arena->size);
    ImGui::ProgressBar(fraction, ImVec2(-1,0), barText.c_str());
}
```

I split up the creation of `barText` to three rows to help with reading clarity. But it could all have been added together on one line.

We use common division to find out how much of an `Arena's` memory budget is being used. Then create a `ProgressBar` that is filled in to that percentage

and writes `barText` inside of it. Passing in `ImVec(-1,0)` allows the bar to stretch the entire width of the dev window. the `c_str()` function converts a string into a `const char*` which is the data type that `ProgressBar` expects.

With this we can add

```
// inside Begin+End block
Draw_ImGui_Arena_Usage(data->arena_images, "images");
Draw_ImGui_Arena_Usage(data->arena_levels, "levels");
Draw_ImGui_Arena_Usage(data->arena_commands, "commands");
Draw_ImGui_Arena_Usage(data->arena_entities, "entities");
```

To visualize how much of each of these arenas are currently being used. I leveraged this to bump my `capacity` for the `commandBuffer` from `2000` to `20000` for example. Remember that `arena_commands` is a subarena inside `arena_levels`.

Next we'll do some magic with undo/redo

```
// dev_gui.cpp part 6
void Draw_History(CommandBuffer* buffer){
    int sliderPos = buffer->index;

    if(ImGui::SliderInt("history",&sliderPos, 0, buffer->head)){
        while(buffer->index > sliderPos){
            Undo(buffer);
        }
        while(buffer->index < sliderPos){
            Redo(buffer);
        }
    }
}
```

We create a `SliderInt` which goes between 0 and the amount of `Commands` we've created. This lets us call `Undo` and `Redo` as we scrub the slider back and forth, letting us perform mass-undo and mass-redo operations by just sliding. Because over the course of a single tick, our `SliderPos` could jump more than 1 spot, we need to put our `undo/redo` calls in `while` loops so

that we keep calling them until `index` has caught up to the `sliderPos`. the `SliderInt()` function returns a bool that is `true` only if the slider has changed value since the last `tick`. This means that we only run the code inside the `{}` curly braces of the `if-statement` if this was the case.

Lastly (for now) we'll display the games `fps`

```
void DrawFPS(float dt){
    ImGui::Text("FPS: %0.f", 1 / dt);
}
```

this formats the text to have no decimals and `1 / dt` gives us how many times `dt` goes into `1` aka how many `frames` can run per `1 second`. also known as our `frames per second`

We have to pass along `dt` to this function somehow though. We will do this by adding a new variable to our `GameData`

```
// gamestate.h
struct GameData {
    // other variables are just hidden for clarity
    const float* dt;
};
```

We make sure that our pointer to the memory where `dt` is stored is `const` this means that no code is allowed to change the value stored at the point in memory being pointed too. We do this as part of a defensive coding strategy, to help us catch if we would have accidentally modified this value through this pointer that we've just added for dev window convenience.

in `main.cpp` right after we create our `float dt` before our `main loop` we assign its address to this pointer

```
// main.cpp
float dt; // this was already in place
gameData->dt = &dt;

while(running) ... // this comes just after
```

this takes a pointer to `dt` and gives it to `gameData` by using the `&` `get-pointer-to-symbol`

finally with all this our `Draw()` function looks like this:

```
// dev_gui.cpp

void DEV::Draw(GameData* data, SDL_Renderer* renderer){
    ImGui::Begin("Dev Tools");
    ImGui::Text("memory arena usage");

    Draw_ImGui_Arena_Usage(data->arena_images, "images");
    Draw_ImGui_Arena_Usage(data->arena_levels, "levels");
    Draw_ImGui_Arena_Usage(data->arena_commands, "commands");
    Draw_ImGui_Arena_Usage(data->arena_entities, "entities");

    Draw_History(data->commandBuffer);

    DrawFPS(*data->dt);

    ImGui::End();

    ImGui::Render();
    ImGui_ImplSDLRenderer3_RenderDrawData(ImGui::GetDrawData(), renderer );
}
```

Now we need to call these `DEV::Functions()` from our `game.cpp`. So inside `game.cpp` we include `dev_gui.h`.

then we call `DEV::Initialize(window, renderer);` from our `void Initialize()` function. But before we can do that we need to update our `Initialize()` in `game.h/cpp` to pass in `SDL_Window* window` as a new parameter

```
// game.h
// 'SDL_Window* window' is a new parameter
__declspec(dllexport) void Initialize(GameData* data, SDL_Window* window, SDL_Renderer*
↪ renderer);
```

and

```
// game.cpp
void Initialize(GameData* data, SDL_Window* window, SDL_Renderer* renderer){
    DEV::Initialize(window, renderer); // new

    // the other code was hidden for clarity
}
```

we call `DEV::ProcessEvents(&event);` at the top of

```
bool HandleEvents()
```

and we update our `Draw()` to call both `DEV::PreDraw()` and `DEV::Draw()`

```
// game.cpp
void Draw(GameData* data, SDL_Renderer* renderer){
    DEV::PreDraw(); // new
    SDL_SetRenderDrawColor(renderer, 120, 70, 120, 255);
    SDL_RenderClear(renderer);

    RenderLevel(data, renderer);
    RenderEntities(data, renderer);

    DEV::Draw(data, renderer); // new
    SDL_RenderPresent(renderer);
}
```

we need to do `DEV::Draw()` just before `SDL_RenderPresent()` and after our other Render functions to make sure the dev window gets rendered on top of everything else.

With this our new dev window works! we can drag it around and make it larger by dragging the bottom right corner.

there is only one problem now: if we perform a hot-reload the `ImGui context` that was created during `Initialize()` will dissappear, and because we don't

rerun `Initialize()` during a hot-reload we won't have a context after the load and our program will crash.

Thankfully there's a simple fix!

We have to expand `GameData` with 1 new variable and put a safety check in during `PreDraw()`

```
// gamestate.h
struct GameData {
    // other variables hidden for clarity
    ImGuiContext* ImGui_context;
};
```

then in `game.cpp` during `Initialize()` we store a this context pointer in `GameData` that lives in our `.EXE` instead of our `.DLL`

```
// game.cpp
void Initialize(GameData* data, SDL_Window* window, SDL_Renderer* renderer){

    DEV::Initialize(window, renderer);
    data->ImGui_context = ImGui::GetCurrentContext(); // new

    // code below this point was hidden for clarity
```

Then we change the function signature of `PreDraw()` to take in a `ImGuiContext*` pointer

```
// dev_gui.h
void PreDraw(ImGuiContext* saved_context);
```

NOTE: we need to update the function signature in our `dev_gui.cpp` as well.

Then we pass the context we stored during `Initialize()` to `PreDraw()`

```
// game.cpp
// inside void Draw()
DEV::PreDraw(data->ImGui_context);
```

and finally in `PreDraw()` we check if our current context has been lost (is currently a pointer pointing to null aka nothing)

```
// dev_gui.cpp

void DEV::PreDraw(ImGuiContext* saved_context){
    if(ImGui::GetCurrentContext() == nullptr){
        ImGui::SetCurrentContext(saved_context);
    }

    ImGui::NewFrame();
}
```

if the context was a `nullptr` we set it manually to the `ImGuiContext*` pointer we stored in `GameData` and passed to the `PreDraw()` function.

Now if we hot-reload our `.DLL` and the context would get lost, we set it back.

And a `nullptr` check is very computationally cheap.

With this we've added `Dear ImGui` to our game engine and created our first dev tool!

Now as we expand our dev GUI we can visualize and help us build ANYTHING we want!

18 Better undo/redo

Currently you might have noticed that after we push a block and press undo. We end up in a state that we can't naturally create in game without undoing first. Our block is still pushed away but our player has taken an undo step backwards.

We will solve this by adding a new variable to `GameData` and our base `Command` called `timestamp`. This variable will be the same for all commands created during the same `Update()` call. This will allow us to keep undoing/redoing until the next command either doesn't exist or has a different `timestamp` number assigned to it.

```
// GameData struct inside gamestate.h
struct GameData{
    // other variables hidden for clarity
    uint32_t command_timestamp;
}
```

We will increase this `uint32_t` by `1` each time we run `Update()` inside `game.cpp`.

```
// game.cpp

void Update(GameData* data, float dt){

    // undo/redo keypress code hidden for clarity

    data->command_timestamp += 1; // new

    for (int i = 0; i < data->GetCurrentLevel()->entityCount; i++) {
        // ...
        // ...
    }
```

at 60 fps this variable will fill to capacity only after hundreds of days, and at that point it just wraps back to `0` and continues again. So lets just not worry about it.

Lets add a similar variable to `Command`

```
// command.h

struct Command {
    CMD_TYPE type; // old
    uint32_t timestamp; // new
};
```

Then we need to add a `uint32_t` parameter to our `Push()` command in `command.h/cpp`

```
// command.h
void Push(CommandBuffer* buffer, AnyCommand cmd, uint32_t timestamp);
```

and in `.cpp` we update the signature and assign the command the provided timestamp

```
// command.cpp
void Push(CommandBuffer* buffer, AnyCommand cmd, uint32_t timestamp){
    buffer->allCommands[buffer->index] = cmd;
    buffer->allCommands[buffer->index].command.timestamp = timestamp; // new
    buffer->index++;
    buffer->head = buffer->index;
    Execute(cmd);
}
```

We can't assign the `timestamp` to `cmd` directly as that is a temporary variable that we copy to the `allCommands` array. As soon as we leave the `Push()` function `cmd` stops existing.

We also add the same parameter to our `TryMove()` function inside `game.h/cpp`

```
// game.h
bool TryMove(Entity* mover, LevelData* level, CommandBuffer* cmd_buffer, int xDir,
    ↪ int yDir, int timestamp);
```

Now at all locations in `game.cpp` where we call `TryMove()` and `Push()` we need to provide the timestamp from our `GameData* data`.

Helix will provide us with errors at all locations where this has not been done yet. To go between errors in Helix press `space-D` (note the capital letter for D). From this menu we can find all the calls for `TryMove()` and `Push()`. We can also go to the function declaration and with the caret over the function name we can press `g-r` to get a list of everywhere the function is being used.

With this done we need to add logic to our `Undo()` and `Redo()` functions inside `command.cpp`

```
// command.cpp

void Undo(CommandBuffer* buffer){
    if(buffer->index == 0){
        return;
    }
    buffer->index--;

    AnyCommand cmd = buffer->allCommands[buffer->index];
    uint32_t timestamp = cmd.command.timestamp; // new
    switch(cmd.command.type){
        // cases inside switch case hidden for clarity
    }

    if(buffer->index > 0){ // new
        if(buffer->allCommands[buffer->index - 1].command.timestamp == timestamp){
            Undo(buffer);
        }
    }
}
```

We check if `index` is larger than zero before comparing the `timestamp` of the earlier `Command` with the one we just undid. And if that was `true` we call `Undo` again recursively.


```

// command.cpp
void Redo(CommandBuffer *buffer){
    AnyCommand cmd = buffer->allCommands[buffer->index];
    if(buffer->index == buffer->head){
        return;
    }

    Execute(cmd);
    buffer->index++;

    int timestamp = cmd.command.timestamp; // new

    if(buffer->index != buffer->head){ // new
        AnyCommand nextCommand = buffer->allCommands[buffer->index];
        if(nextCommand.command.timestamp == timestamp){
            Redo(buffer);
        }
    }
}

```

and for redo we check if we are not already at the very latest `Command` by comparing `index` to `head`. Then we fetch the next `Command` and if the timestamps are the same we recursively call `Redo`.

And because our `Dear ImGui` does not itself change variables but only calls our gameplay functions like `Undo()` and `Redo()` this change already works with our `undo-redo-slider`!

Now our undo and redo can't put the game in an unnatural state.

19 Animation Part I

This chapter covers code related to animating our entities, as well as how to buffer inputs for a smoother gameplay experience.

Before we do that we will do a small piece of housekeeping. We're moving our `memcpy()` function from the bottom of `Update()` in `Game.cpp` to `main.cpp`. We'll call `memcpy()` on the line after we call `dll->Update()`. The reasoning being that this is part of the foundation of our game engine and should never be accidentally removed or skipped due to us making big changes to `game.cpp`

```
// main.cpp

while(running){
    // other code hidden for clarity
    dll.update(gameData, dt);

    memcpy((void*)gameData->keys_previous, SDL_GetKeyboardState(nullptr),
    ↪ SDL_SCANCODE_COUNT * sizeof(bool));
}
```

This was the same function we called inside `game.cpp` but we pass along `SDL_GetKeyboardState` directly instead of having it saved to the earlier variable we named `keys`

With that out of the way, what we want to do is having our entities slide across the screen instead of teleport to their new location.

To do this we'll need to store two sets of variables

- 1) where they are currently
- 2) where they previously were

with this we can `linearly interpolate` between them. This is a way of making a third value that slides between the two extremes.

Linear interpolation is almost always referred to as `lerp` and has 3 basic components, `a`, `b`, and `t`.

here's some pseudo-code

```
float milesTravelled = 0;
milesTravelled = lerp(0, 1000, 0.4);
```

in this example, `milesTravelled` can have any value between 0 and 1000. the variable `t` set to `0.4` makes the `lerp()` function return the 40% point between `a` and `b`. In this case 400.

First we'll add four new variables to `struct Entity`

```
// entity.h
struct Entity{
    ID id;
    int x;
    int y;
    int x_prev; // new
    int y_prev; // new
    float progress_01; // new
    Behaviour behaviour;
```

`x_prev` and `y_prev` will store the previous position of our entity. `progress_01` will go between 0 and 1 and act as the value we assign to `t` in our `Lerp()` later.

With this we can go to our `RenderEntities()` function in `LevelRenderer.cpp`

```
// levelRenderer.cpp
#include <cmath>
```

we need this header to be included to get access to a `Lerp()` function.

```

// levelRenderer.cpp
// inside RenderEntities()

    int xPos = 0;
    int yPos = 0;

    xPos += SCREEN_WIDTH / 2.0;
    yPos += SCREEN_HEIGHT / 2.0;

    xPos -= data->levels[data->currentLevelIndex].w * CELL_SIZE_PX / 2;
    yPos -= data->levels[data->currentLevelIndex].h * CELL_SIZE_PX / 2;

    float x_animated = std::lerp(entity.x_prev, entity.x, entity.progress_01); // new
    float y_animated = std::lerp(entity.y_prev, entity.y, entity.progress_01); // new

    xPos += x_animated * CELL_SIZE_PX; // modified
    yPos += y_animated * CELL_SIZE_PX; // modified

    RenderSprite(img, renderer, xPos, yPos);

```

previously we used `entity.x` and `entity.y` directly when calculating `xPos` and `yPos`. We now `Lerp()` between `x/y_prev` and `x/y` using `progress_01` and store the moving position in `x/y_animated`. Then we use that to adjust `x/yPos`.

Now we need to make sure that `Progress_01` increases whenever we issue a move command.

Before we make some large scale changes to `Update()` in `game.cpp` there is some more logic we need to set up.

We're going to be constructing a new way of storing our data. We'll be using a `ring buffer` to hold all our arrow key inputs. But we might be pressing the arrow keys thousands of times per level and we're only really interested in the 2-5 next inputs that are yet to be animated. Once these have been animated we are free to discard this info.

A `ring buffer` is a limited sized array that loops back on itself once its full - therefore overwriting its oldest elements.

We'll add this `input ring buffer` to our `GameData` in `gameState.h`

```
// gameState.h

struct GameData{
    // other variables hidden for clarity

    Position* input_buffer;
    int input_buffer_capacity;
    int input_buffer_write_count;
    int input_buffer_read_count;
}
```

You'll notice that `Position` is a new variable. We'll take a quick detour to `entity.h` and add this very (very) simple struct first.

```
// entity.h

struct Position{
    int x;
    int y;
};
```

As a position is always both an `x` and a `y` value we've collapsed them into a single struct to make reasoning about them simpler.

our four new variables in `GameData` are:

`Position* input_buffer`: this is an array like we're used to.
`input_buffer_capacity`: the size of this array. We'll be keeping this very small on purpose
`input_buffer_write_count`: this is a running tally of how many inputs have ever been added to the `ring buffer` so if the buffer can hold 5 input elements and we've added 500. We can still only access the latest 5, but this integer will let us know how many we've ever added.
`input_buffer_read_count`: each input is read only once its time for that input to be processed. Meaning that if we press up twice, we'll immediately begin moving up. But only after we've arrived at our destination will we move up for the second time. `read_count` will lag behind `write_count`

and with each move performed by our `entities` this will increase by `1` until `read` and `write` are at the same value.

To leverage our `ring buffer` we'll be using our `_capacity` variable alongside our `_read` and `_write` to know which element out of the looping few in the `ring buffer` to use. To do this we'll use the `modulo operator` - `%`.

The modulo operator takes the first value and loops it over the second value as many times as it can. And once it can't loop the value any longer it returns whatever was left.

If we have 5 spots in a ring buffer and we are adding 7 things to it. We'll `modulo` 7 into 5

$$7 \% 5 = 2$$

If we wanted to add 24 we'll `modulo` 24 into 5

$$24 \% 5 = 4$$

this strips away 5 from 24 for as long as is possible before returning the remaining value. So 5 is stripped away four times for a total of 20. before we return 4.

With this we can reason about `_capacity` and `_read/_write`

```
front = input_buffer_read_count % input_buffer_capacity;
```

the value of `front` would be the total value ever added to `_read_count` that has been `modulo'd` with `_capacity`.

Lets look at an example

```
front = 1536 % 20;
```

in this scenario our made up variable `front` gets a value of `16`. As we can strip away 20 from 1536 a total of 76 times $20 * 76 = 1520$ then we have 16 remaining which is the value stored in `front`.

Inside `main.cpp` we'll allocate the `ring buffer` to our `arena_levels`

```
// main.cpp

gameData->input_buffer_capacity = 50;
size_t RING_BUFFER_SIZE = sizeof(Position) * gameData->input_buffer_capacity;
gameData->input_buffer = (Position*)Memory::Allocate(gameData->arena_levels,
↪ RING_BUFFER_SIZE);
```

This allows our `ring buffer` to hold `50` inputs before looping. Should we ever find that we need more, we can just increase this number. The memory footprint of our `Position` struct is extremely(!) small.

We'll be making large changes to our `Update()` function inside `game.cpp`.

The only code we'll save for now is:

```
// game.cpp
void Update(GameData* data, float dt){

    const bool* keys = SDL_GetKeyboardState(nullptr);

    if(KeyPressed(SDL_SCANCODE_Z, keys, data->keys_previous)){
        if(KeyHeld(SDL_SCANCODE_LSHIFT, keys, data->keys_previous)){
            Redo(data->commandBuffer);
        }
        else{
            Undo(data->commandBuffer);
        }
    }
}
```

everything else we can safely delete. This way of programming where we first ensure we get something working, and only once we have a concrete need for a new feature do we actually code that system is by far the most reasonable way of working and the act of rewriting code aka `refactoring` is a cornerstone of programming. So with the rest of `Update()` removed we

can no longer move our entities in game.

What we'll do next is start filling our `ring buffer`

```
// game.cpp
// Update() function

if(KeyPressed(SDL_SCANCODE_RIGHT, keys, data->keys_previous)){
    data->input_buffer[data->input_buffer_write_count++ % data->input_buffer_capacity] =
    ↪ {1, 0};
}
else if(KeyPressed(SDL_SCANCODE_LEFT, keys, data->keys_previous)){
    data->input_buffer[data->input_buffer_write_count++ % data->input_buffer_capacity] =
    ↪ {-1, 0};
}
else if(KeyPressed(SDL_SCANCODE_UP, keys, data->keys_previous)){
    data->input_buffer[data->input_buffer_write_count++ % data->input_buffer_capacity] =
    ↪ {0, -1};
}
else if(KeyPressed(SDL_SCANCODE_DOWN, keys, data->keys_previous)){
    data->input_buffer[data->input_buffer_write_count++ % data->input_buffer_capacity] =
    ↪ {0, 1};
}
```

the `KeyPressed` calls are very similar to our older code, but now we do some fancier stuff with it. We utilize something called `syntactic sugar` to make the creation of our `Position` struct smaller.

there is a **lot** going on in this code but you'll notice that it's actually repeating four times with just a change to the `if-statement` and the `= {int, int}` at the end.

Lets start with the small `= {1, 0}`. This is the `syntactic sugar` I mentioned earlier. `Position(1, 0)` can be simplified down to just `{1, 0}`. If you find it confusing to not write the type then you can easily substitute out the `sugar'ed` version.

We assign the relevant `Position` to the correct element in our `Ring Buffer` by taking the `_write_count` and adding a `Modulo` with `_capacity`. The

`increment operator` aka `++` will increase `_write_count` by 1 **after** the line of code has resolved. This is the same as writing:

```
data->input_buffer[data->input_buffer_write_count % data->input_buffer_capacity] = {1,  
↪ 0};  
data->input_buffer_write_count += 1;
```

If you find the single line to be “doing to much” you can easily remove the `increment operator` and add a `+= 1` on a line below.

With each arrow key adding its own `Position` to the buffer we can begin looking at taking these inputs and one-by-one resolving them - to actually make our entities move by adding the following code after we’ve checked what keys are pressed

```
// game.cpp  
// update() function  
  
bool are_entities_moving = false;  
for (int i = 0; i < data->GetCurrentLevel()->entityCount; i++) {  
    Entity* entity = &data->GetCurrentLevel()->entityBuffer[i];  
    if(entity->HasBehaviour(CAN_MOVE) && IsMoving(entity)){  
        entity->progress_01 += MOVE_SPEED * dt;  
        if(entity->progress_01 >= 1){  
            entity->progress_01 = 0;  
            entity->x_prev = entity->x;  
            entity->y_prev = entity->y;  
        }  
        if(IsMoving(entity)){  
            are_entities_moving = true;  
        }  
    }  
}
```

First, by creating `are_entities_moving` and setting it to `false` at the start, we can know that if we get past the upcoming `for-loop` and it is still `false`, then we found no entity that was moving. And if no entity was moving, we can check if we have any more buffered inputs to resolve.

We loop over all entities in the current level and then we check if they

are allowed to move `CAN_MOVE` and has a move its currently performing `IsMoving()` .

`IsMoving()` is a new function we've added to `entity.h` . But we've added it outside of our `Entity struct` . Instead it is added to a newly created `Entity.cpp` . Later we'll move more of the functions found inside our `Entity struct` to our `entity.cpp` instead. The logic will be very similar, but it will be more in line with our overall code structure.

lets look at our newly created `entity.cpp`

```
// entity.cpp
#include "entity.h"

bool IsMoving(Entity* e){
    return e->x != e->x_prev || e->y != e->y_prev;
}
```

This means that `IsMoving()` returns true if either the `x` or `y` values were different from `x/y_prev` . This is only the case when a move is under way. Once we have arrived at our location `x/y_prev` will catch up to `x/y` and have the same value.

back in our `Update()` we can now see that our `if-statement` asks that the entity both is moving and is allowed to move. If this is the case we update its `progress_01` by adding `MOVE_SPEED` multiplied by `delta time` .

`MOVE_SPEED` is a new variable added to `common.h`

```
// common.h
const float MOVE_SPEED = 6.0;
```

This means that anytime we multiply `delta time` aka `dt` by this value, we make it go from 0 to 1 in 1/6 of a second. and because `dt` aligns with our `framerate` we can ensure that it takes 1/6 of a second no matter how

powerful the computer running the game is.

if `progress_01` is at or above a 1 we catch up `x/y_prev` to `x/y` and reset `progress_01` so that it can begin a new move sequence later.

Then we check if `IsMoving()` is still true after having added to `progress_01` and if so we flip the `are_entities_moving` boolean to `true`. Note that nothing inside this `for-loop` can set it to `false`.

The next step of our `Update()` is to call `TryMove()` again using our `Ring Buffer`

```
if(are_entities_moving == false){
    if(data->input_buffer_read_count == data->input_buffer_write_count){
        return;
    }

    data->command_timestamp += 1;

    for (int i = 0; i < data->GetCurrentLevel()->entityCount; i++) {
        Entity* entity = &data->GetCurrentLevel()->entityBuffer[i];
        if(entity->HasBehaviour((Behaviour)(RESPOND_TO_INPUT | CAN_MOVE))){
            int xDir = data->input_buffer[data->input_buffer_read_count %
                ↪ data->input_buffer_capacity].x;
            int yDir = data->input_buffer[data->input_buffer_read_count %
                ↪ data->input_buffer_capacity].y;
            TryMove(entity, data->GetCurrentLevel(), data->commandBuffer, xDir, yDir,
            ↪ data->command_timestamp);
        }
    }
    data->input_buffer_read_count++;
}
```

First we check if `are_entities_moving` was `false`. Meaning that all entities have arrived at their location.

Then we compare our `_read` with our `_write`. If `_read` has caught up then we have no further inputs to resolve and we can `return` early. This means that if we hit our return we leave our `Update()` function and no code below it will run.

If we manage to get past the `read / write if statement` we know that we have at least one input to process. We can therefore increase `command_timestamp` by 1 - giving all Commands created during this tick a new timestamp. Note that with this system our `command_timestamp` only increases in this case, meaning that it won't increase each tick on its own as it did in our previous code.

We then `for-loop` over all our entities again this time checking for `RESPOND_TO_INPUT` as well as `CAN_MOVE`. if we find an entity with both these behaviours we collect the `x/y` from the `Position` currently being pointed to in our `Ring Buffer`. We do this by taking `_read` and using `modulo` on it with our `_capacity`. This gives us the `Position` struct that we can then fetch `x/y` from.

We then call `TryMove()` as normal.

once we have looped over all our entities we increase `_read_count` by 1 using the `increment operator` `++`. Meaning that the next time we check `are_entities_moving` we will be evaluating the next element in the `ring buffer`.

Finally we have to update `command.cpp` to assign our `x_prev` and `y_prev` values. As well as adding what is known as an `optional parameter`

```
// command.cpp
void Execute(AnyCommand cmd, bool from_redo = false){ // optional
    switch(cmd.command.type){
        case CMD_TYPE::NONE:
            break;
        case CMD_TYPE::MOVE:
            MoveCommand mv = cmd.move;
            mv.entity->x_prev = mv.entity->x; // new
            mv.entity->y_prev = mv.entity->y; // new
            mv.entity->x += mv.xDir;
            mv.entity->y += mv.yDir;
            if(from_redo){ // new
                mv.entity->progress_01 = 1;
            }

            break;
    }
}
```

we're adding an **optional parameter** to **Execute()** meaning that we can skip passing this bool when we call the function if we want. In that case the value will default to the value set in the Function itself. In this case **false**.

Before we update **x/y** we store the old values in **x/y_prev**. Then we check if the **optional parameter** was **true** **if(from_redo)** and if it was we set **progress_01** to **1** immediately. Meaning that it will instantly arrive at its new destination. We will be setting this **optional parameter** to **true** when calling **Execute()** from our **Redo()** function.

We'll do something similar in **Undo()**

```
// command.cpp

void Undo(CommandBuffer* buffer){
    // code above hidden for clarity
    switch(cmd.command.type){
        case CMD_TYPE::NONE:
            break;
        case CMD_TYPE::MOVE:
            MoveCommand mv = cmd.move;
            mv.entity->x -= mv.xDir;
            mv.entity->y -= mv.yDir;
            mv.entity->progress_01 = 1; // new
            break;
    }
    // code below hidden for clarity
}
```

we also put `progress_01` to `1` if we call `Undo()`.

Finally our change to `Redo()` looks like

```
// command.cpp

void Redo(CommandBuffer *buffer){
    // code above hidden for clarity
    Execute(cmd, true); // new
    // code below hidden for clarity
}
```

we have only added `true` as an optional parameter when calling `Execute`.

if we look at our `Push()` function we can see that because we added this as an `optional parameter` we didnt have to make any changes to the call to `Execute()` inside it.

```
// command.cpp

void Push(CommandBuffer* buffer, AnyCommand cmd, uint32_t timestamp){
    buffer->allCommands[buffer->index] = cmd;
    buffer->allCommands[buffer->index].command.timestamp = timestamp;
    buffer->index++;
    buffer->head = buffer->index;
    Execute(cmd); // look how we didn't have to pass in `false`
}
```

With these final changes our entities now slide across the game board and our inputs can be buffered, meaning that we can press our arrow keys as fast as

we want and inputs will be registered and acknowledged once the animations have caught up to them!

20 Repeat Inputs

In this chapter we'll add the ability to hold down a key and get our entities to keep moving, and undos to keep undoing. Sparing us from having to press a key each time we want to perform an action (though that functionality will of course remain)

We're also going to do some housekeeping and move key press logic out of `game.cpp` and firmly into its own script - as this fits better as part of our boilerplate.

We're going to create a new `struct` that will live inside a newly created `input.h`

```
// input.h
#pragma once // always do this for any .h file

struct Input{
    const bool* keys_current;
    const bool* keys_previous;
    float* keys_held_time;
};
```

`keys_current/previous` are set to `const` as we are not looking to allow their contents to be changed individually. but `keys_held_time` will be updated on an individual level. Each of these variables are used as an array, indicated by the plural `keys` rather than `key`.

`keys_held_time` is an array of `floats` that track how long each key has been held down. We're going to use this to simplify checking how long a key has been held.

We'll add one of these structs to our `GameData` inside `gameState.h`. We're also removing the variables related to keys that lived directly inside `GameData` earlier. We also previously quite lazily allocated our key arrays

into `arena_level` but we don't really want to free the memory of these arrays. But we might want to reset their values. We're also going to add a new `arena_input` and create this new `subarena` from `arena_main`.

```
// gameState.h

#include "input.h" // we need to include the input header file to access `Input`

struct GameData {
    // other variables hidden for clarity
    Input input; // new
    Arena* arena_input; // new
    // bool* keys_previous; <- removed
```

inside `main.cpp` we'll create this subarena and allocate our arrays to it.

```
// main.cpp
GameData* gameData = (GameData*)Memory::Allocate(arena_main, sizeof(GameData)); // old

size_t INPUT_ARENA_SIZE = 0;
INPUT_ARENA_SIZE += sizeof(bool) * SDL_SCANCODE_COUNT * 2;
INPUT_ARENA_SIZE += sizeof(float) * SDL_SCANCODE_COUNT;
INPUT_ARENA_SIZE += 128;
gameData->arena_input = Memory::CreateSubArena(arena_main, INPUT_ARENA_SIZE);

gameData->input.keys_current = (bool*)Memory::Allocate(gameData->arena_input,
    ↪ sizeof(bool) * SDL_SCANCODE_COUNT);
gameData->input.keys_previous = (bool*)Memory::Allocate(gameData->arena_input,
    ↪ sizeof(bool) * SDL_SCANCODE_COUNT);
gameData->input.keys_held_time = (float*)Memory::Allocate(gameData->arena_input,
    ↪ sizeof(float) * SDL_SCANCODE_COUNT);
```

We collect the total size for all our arrays and add them to `INPUT_ARENA_SIZE`, making sure two multiply by 2 to get the size of both our `bool*`. Then we allocate our `SubArena` and finally allocate the three arrays into that arena. We also lazily add on 128 bytes as our `Allocate()` function is a bit naive and does not take into account that sometimes a new allocation will be padded a little. creating a gap of a few bytes. This is only necessary because

- 1) our `Allocate()` is a bit naive

2) we are slicing of juuuuust enough memory to hold the three arrays

Now inside our `input.h` we'll add the functions that previously lived inside `game.cpp` (as well as 4 new ones). These functions live outside of the `struct`

```
// input.h

bool KeyPressed(const Input* input, SDL_Scancode key); // moved from game.cpp
bool KeyHeld(const Input* input, SDL_Scancode key); // moved from game.cpp
bool KeyReleased(const Input* input, SDL_Scancode key); // moved from game.cpp
bool KeyHeld_ForTime(const Input* input, SDL_Scancode key, float min_length); // new
void UpdateKeys(Input* input, float dt); // new
void ResetKeyHeldTime(Input* input, SDL_Scancode key); // new
void ResetAll(Input*); // new
```

Note that the parameters for each function have changed and that there are generally fewer. Previously we passed `keys` and `keys_previous` as separate variables each time. Now we send `Input` that holds both of these. we pass in `Input*` as a `const` as we are not letting those functions modify the arrays.

Inside `input.cpp` we create the content of each of these functions

```

// input.cpp
#include "input.h"
#include <cstring>

bool KeyPressed(const Input* input, SDL_Scancode key){
    if(input->keys_previous == nullptr){
        return input->keys_current[key];
    }
    return input->keys_current[key] && !input->keys_previous[key];
}

bool KeyHeld(const Input* input, SDL_Scancode key){
    if(input->keys_previous == nullptr){
        return false;
    }
    return input->keys_current[key] && input->keys_previous[key];
}

bool KeyReleased(const Input* input, SDL_Scancode key){
    if(input->keys_previous == nullptr){
        return false;
    }
    return !input->keys_current[key] && input->keys_previous[key];
}

```

these functions are the same as the ones we used inside `game.cpp` but we access `keys_current/previous` from our `Input` parameter that was passed in.

```
// input.cpp

bool KeyHeld_ForTime(const Input* input, SDL_Scancode key, float min_length){
    return input->keys_held_time[key] >= min_length;
}

void UpdateKeys(Input* input, float dt){
    for (int i = 0; i < SDL_SCANCODE_COUNT; i++) {
        if (input->keys_current[i]){
            input->keys_held_time[i] += dt;
        }
        else{
            input->keys_held_time[i] = 0;
        }
    }
    memcpy((void*)input->keys_previous, input->keys_current, SDL_SCANCODE_COUNT *
↪ sizeof(bool));
}

void ResetKeyHeldTime(Input* input,SDL_Scancode key){
    input->keys_held_time[key] = 0;
}

void ResetAll(Input* input){
    memset((void*)input->keys_current, 0, sizeof(bool) * SDL_SCANCODE_COUNT);
    memset((void*)input->keys_previous, 0, sizeof(bool) * SDL_SCANCODE_COUNT);
    memset((void*)input->keys_held_time, 0, sizeof(float) * SDL_SCANCODE_COUNT);
}
```

For `KeyHeld_ForTime()` we pass along a `float` then find the specified key from our `keys_held_time[]` array and check if the timer for that key exceeded the time we passed in.

`UpdateKeys()` is our consolidated function that runs over all keys and increased the held timer by `deltatime / dt` if it was held this frame. If not we reset the `held_timer` for that key. After that is done we take `keys_current` and copy over all of those values to `keys_previous`. We'll call this function from `main.cpp` after we've called `Update()`

`ResetKeyHeldTime()` accepts a key then sets the timer for that key to `0`

`ResetAll()` uses the `memset()` function to fill every memory address for

our three arrays with zeroes. This means that they still exist but nothing is stored. We'll use this later to clear the inputs as we load or reload a level. We have to cast our `bool*` and `float*` to `void*` as that is the parameter that `memset` uses.

Lets head to `main.cpp` and add this new functionality

```
// main.cpp

gameData->input.keys_current = SDL_GetKeyboardState(nullptr); // new
dll.update(gameData, dt); // old
UpdateKeys(&gameData->input, dt); // new

dll.draw(gameData, renderer); // old
// memcpy((void*)gameData->keys_previous, SDL_GetKeyboardState(nullptr),
// ↪ SDL_SCANCODE_COUNT * sizeof(bool)); // removed
```

So before `Update()` we collect the keys being pressed. then after `Update()` we update our `held_timers` and copy over `keys_current` to `keys_previous` in preparation for the next tick using our new `UpdateKeys()` function. We also remove the old `memcpy` we created last chapter as we do this inside `UpdateKeys()` now.

We also have to do a small amount of coding inside our `CMakeLists.txt`. Currently our `exe` does not get access to `input.cpp` but it calls `UpdateKeys()`.

```
set(SHARED_SOURCES
    ${CMAKE_SOURCE_DIR}/src/image.cpp // old
    ${CMAKE_SOURCE_DIR}/src/arena.cpp // old
    ${CMAKE_SOURCE_DIR}/src/input.cpp // new
)
```

Now we can remove the old `Pressed/Held/Released` function from `game.cpp` and instead `#include "input.h"` and our new simplified functions. We are also removing the old functions (Pressed, Held, Released) from `game.h`

```
// game.cpp

if(KeyPressed(&data->input, SDL_SCANCODE_Z)){
    if(KeyHeld(&data->input, SDL_SCANCODE_LSHIFT)){
        Redo(data->commandBuffer);
    }
    else{
        Undo(data->commandBuffer);
    }
}

if(KeyPressed(&data->input, SDL_SCANCODE_RIGHT)){
    data->input_buffer[data->input_buffer_write_count++ % data->input_buffer_capacity] =
    ↪ {1, 0};
}
else if(KeyPressed(&data->input, SDL_SCANCODE_LEFT)){
    data->input_buffer[data->input_buffer_write_count++ % data->input_buffer_capacity] =
    ↪ {-1, 0};
}
else if(KeyPressed(&data->input, SDL_SCANCODE_UP)){
    data->input_buffer[data->input_buffer_write_count++ % data->input_buffer_capacity] =
    ↪ {0, -1};
}
else if(KeyPressed(&data->input, SDL_SCANCODE_DOWN)){
    data->input_buffer[data->input_buffer_write_count++ % data->input_buffer_capacity] =
    ↪ {0, 1};
}
```

With this we can clearly see how our housekeeping has simplified calling these functions. It currently does the same thing, but it's easier to digest.

Now we're going to expand these functions using the `or evaluator` written as `||`. This will make an `if-statement` be `true` if either of the conditions checked is true.

```
// or-evaluator example

if((weapon.damage >= enemy->health) || cheats->max_damage){
    KillEnemy(enemy);
}
```

This pseudo-code would kill the enemy if either our weapon had enough damage or the `max_damage` bool from `cheats` were set to true.

```
// game.cpp
if(KeyPressed(&data->input, SDL_SCANCODE_Z) || KeyHeld_ForTime(&data->input,
↳ SDL_SCANCODE_Z, UNDO_REPEAT_TIME)){
    ResetKeyHeldTime(&data->input, SDL_SCANCODE_Z);
    if(KeyHeld(&data->input, SDL_SCANCODE_LSHIFT)){
        Redo(data->commandBuffer);
    }
    else{
        Undo(data->commandBuffer);
    }
}

if(KeyPressed(&data->input, SDL_SCANCODE_RIGHT) ||
↳ KeyHeld_ForTime(&data->input, SDL_SCANCODE_RIGHT, (1 / MOVE_SPEED) * 1.15)){
    ResetKeyHeldTime(&data->input, SDL_SCANCODE_RIGHT);
    data->input_buffer[data->input_buffer_write_count++ % data->input_buffer_capacity] =
↳ {1, 0};
}
else if(KeyPressed(&data->input, SDL_SCANCODE_LEFT) ||
↳ KeyHeld_ForTime(&data->input, SDL_SCANCODE_LEFT, (1 / MOVE_SPEED) * 1.15)){
    ResetKeyHeldTime(&data->input, SDL_SCANCODE_LEFT);
    data->input_buffer[data->input_buffer_write_count++ % data->input_buffer_capacity] =
↳ {-1, 0};
}
else if(KeyPressed(&data->input, SDL_SCANCODE_UP) ||
↳ KeyHeld_ForTime(&data->input, SDL_SCANCODE_UP, (1 / MOVE_SPEED) * 1.15)){
    ResetKeyHeldTime(&data->input, SDL_SCANCODE_UP);
    data->input_buffer[data->input_buffer_write_count++ % data->input_buffer_capacity] =
↳ {0, -1};
}
else if(KeyPressed(&data->input, SDL_SCANCODE_DOWN) ||
↳ KeyHeld_ForTime(&data->input, SDL_SCANCODE_DOWN, (1 / MOVE_SPEED) * 1.15)){
    ResetKeyHeldTime(&data->input, SDL_SCANCODE_DOWN);
    data->input_buffer[data->input_buffer_write_count++ % data->input_buffer_capacity] =
↳ {0, 1};
}
```

suddenly there is more code here, but remember for each direction we press the code is actually almost identical.

we've also just added `UNDO_REPEAT_TIME` to our `common.h`

```
// common.h
const float UNDO_REPEAT_TIME = 0.15;
```

this new `or evaluator` in our undo `if-statement` will allow us to repeat-

edly undo once every `0.15` seconds as long as the `Z key` is being held down. Once the `if-statement` evaluates to true we call `ResetKeyHeldTime` to set the timer keeping track of the `Z key` back to `0`. So that we need to wait another `0.15` seconds for the next undo.

lets look at the first of the right/left/up/down input blocks (as the rest are just copies)

```
// game.cpp

if(KeyPressed(&data->input, SDL_SCANCODE_RIGHT) ||
↪ KeyHeld_ForTime(&data->input,SDL_SCANCODE_RIGHT, (1 / MOVE_SPEED) * 1.15)){
    ResetKeyHeldTime(&data->input, SDL_SCANCODE_RIGHT);
    data->input_buffer[data->input_buffer_write_count++ % data->input_buffer_capacity] =
↪ {1, 0};
}
```

we use the same `||` to allow for a new input. But the float we pass as a parameter to `KeyHeld_ForTime()` is a bit more complex. We actually pass in the time it takes for a move animation to finish but increased by 15%. This is honestly not a very good approach but it will do for now. It means that the next input will only be logged after a full movement has elapsed, (plus a little extra). We'll definitely return to this little equation later and improve it.

We pass `&data->input` using the `pointer to symbol` aka `&` we do this because our `data->input` is not a pointer but our `KeyPressed()` function expects a pointer. We pass by pointer to avoid copying over the array each time, as this creates unecesary overhead (CPU work).

If we are allowed inside the `if-statement` we reset the timer for the specific key then add the correct `Position*` to our `ring buffer` and increase `_write_count` by `1` afterwards using the `increment operator` aka `++`.

Now we can hold our undo and movement keys instead of clicking all the time. We have also put our input logic inside our boilerplate and simplified calling `Pressed/Held/Released`.

21 Camera

In this chapter we'll implement a naive camera as well as refactor rendering code to simplify asking questions about positions as well as simplifying the render functions inside `levelRenderer.h/cpp`.

A camera in a 2D game is, at its simplest, a position in space. We'll be taking that position and shifting everything we render by that amount multiplied by `-1`. This means that as the camera shifts right, everything drawn shifts left.

We're setting this up to later simplify mouse controls when we start working on additional dev tools. But the best part is that once we're done we can create new functions responsible for rendering with a fraction of the code we currently have in `RenderEntities()` and `RenderLevel()`. So once we're done with this chapter, if we've set everything up right the game will look exactly the same - and that's a good thing.

We're setting up a new `camera.h/cpp`

```
// camera.h

#pragma once

#include "levels.h"

struct Camera{
    float camera_x;
    float camera_y;
};

namespace camera {
void GridToWorld(float* x, float* y, const LevelData* lvl);
void WorldToGrid(float x_world, float y_world, int* x, int* y, const LevelData* lvl);
bool GetIsPointInsideGrid(float x, float y, const LevelData* lvl);
};
```

for now, our `Camera` struct has only two floats, responsible for storing the position of the camera. Later we'll expand this list of variables as we create

additional camera features.

we're also creating three useful helper functions. `GridToWorld()` will let us specify a position on the game board and get back the actual position in the game window. `WorldToGrid()` does the opposite and takes any point in space and finds what cell this would belong to on the game board. Note that this can give us positions that are outside of the level bounds. To easily reason about what is inside and outside of the grid we will also be using the `GetIsPointInsideGrid()`

The implementation in our `camera.cpp` looks like:

```

// camera.cpp
#include "camera.h"
#include "common.h"

bool camera::GetIsPointInsideGrid(float x, float y, const LevelData* lvl){
    int x_grid;
    int y_grid;
    WorldToGrid(x, y, &x_grid, &y_grid, lvl);
    return x_grid >= 0 && y_grid >= 0 && x_grid < lvl->w && y_grid < lvl->h;
}

void camera::GridToWorld(float* x, float* y, const LevelData* lvl){
    *x *= CELL_SIZE_PX;
    *x += SCREEN_WIDTH / 2.0;
    *x -= lvl->w * CELL_SIZE_PX / 2.0;
    *y *= CELL_SIZE_PX;
    *y += SCREEN_HEIGHT / 2.0;
    *y -= lvl->h * CELL_SIZE_PX / 2.0;
}

void camera::WorldToGrid(float x_world, float y_world, int* x, int* y, const LevelData*
↪ lvl){
    *x = x_world;
    *y = y_world;
    *x += lvl->w * CELL_SIZE_PX / 2.0;
    *x -= SCREEN_WIDTH / 2.0;
    *x /= CELL_SIZE_PX;
    *y += lvl->h * CELL_SIZE_PX / 2.0;
    *y -= SCREEN_HEIGHT / 2.0;
    *y /= CELL_SIZE_PX;
}

```

Our `WorldToGrid()` function accepts two floats that specify the point in space to check, then two pointers to integers, these integer pointers are being modified by the function. So for our `GetIsPointInsideGrid()` we create two new integers then pass along pointer references to them using `&`. The `return` looks a bit long and scary, but we're just making sure that the `x/y_grid` is larger than zero and smaller than the width `w` and height `h` of the level we passed in. We can chain multiple `and operators` aka `&&` to make our expression only evaluate to true if all conditions were true.

`GridToWorld()` actually does the same arithmetic as our

`RenderEntities()` and `RenderLevel()` did in `levelRenderer.cpp` but with the change that we're operating on the two float pointers we passed in. And to do that we need to make changes to the values they point to using the **dereference operator** aka putting a `*` before the variable name. Inside `GridToWorld()` we do the following steps for both `x` and `y`

- 2) we take the size of a cell in pixels and multiply it with the cell coordinate.
Shifting our coordinate into pixels
- 3) add half of the width of the screen to make the `0,0` position be at the center of the screen instead of at the top left corner
- 4) we take the width of the level aka the total amount of cells then convert that number to a length in pixels and remove half of it from `x/y`. This shifts the position so that the center of the level is at the center of the screen.

for `WorldToGrid()` we do the same operations but in reverse to get the same result back.

we pass `LevelData*` as a const pointer to indicate that we're not supposed to make any changes to it inside these functions, just use its variables.

We're making some small changes to our `rendering.h/cpp` to include our `Camera` struct in our rendering step

```
// rendering.h
#pragma once
#include "SDL3/SDL_render.h"
#include "camera.h"
#include "image.h"
#include "levels.h"

void RenderSprite_World(Image* sprite, SDL_Renderer* renderer, const Camera* camera,
    ↪ float x, float y, float scale = 1);
void RenderSprite_Grid(Image* sprite, LevelData* lvl, SDL_Renderer* renderer, const
    ↪ Camera* camera, float x, float y, float scale = 1);
```

We have taken what was previously a single `RenderSprite()` function and made a distinction between `_World` and `_Grid`. This is to give us a simplified way of rendering using an entities grid position instead of always having to manually do the conversion between grid and world.

```
// rendering.cpp

#include "rendering.h"
#include "SDL3/SDL_render.h"
#include "common.h"

void RenderSprite_World(Image* sprite, SDL_Renderer* renderer, const Camera* camera,
↳ float x, float y, float scale){
    SDL_FRect rect;
    rect.x = x;
    rect.y = y;
    rect.h = sprite->height * UPSCALE_FACTOR * scale;
    rect.w = sprite->width * UPSCALE_FACTOR * scale;
    rect.x -= camera->camera_x;
    rect.y -= camera->camera_y;

    SDL_RenderTexture(renderer, sprite->texture, NULL, &rect);
}

void RenderSprite_Grid(Image* sprite, LevelData* lvl, SDL_Renderer* renderer, const
↳ Camera* camera, float x, float y, float scale){
    camera->GridToWorld(&x, &y, lvl);
    RenderSprite_World(sprite, renderer, camera, x, y, scale);
}
```

We can see how `RenderSprite_World()` is the same as our old `RenderSprite()` except we adjust the final `rect.x/y` by the camera's position. `RenderSprite_Grid()` just uses our newly created `GridToWorld()` function before calling `RenderSprite_World` making this just a small helper function really.

finishing up we're adding a `Camera` struct variable to our `GameData` inside `gameState.h`. We need to pass this along to our `levelRenderer.h/cpp` functions, but

```
// gameState.h

struct GameData {
    // other variables hidden for clarity
    Camera camera;
}
```

next we add `#include "camera.h"` in `levelRenderer.cpp` and simplify our Render functions a lot.

```
// levelRenderer.cpp

#include "levelRenderer.h"
#include "common.h"
#include "rendering.h"
#include <cmath>

void RenderLevel(GameData* gameData, SDL_Renderer* renderer){

    LevelData lvl = gameData->levels[gameData->currentLevelIndex];

    for(int x = 0; x < lvl.w; x++){
        for (int y = 0 ; y < lvl.h; y++) {
            uint8_t cellType = lvl.GetCellID(x, y);

            Image* sprite;
            switch(cellType){
                case 1:
                    sprite = gameData->ground;
                    break;
                case 2:
                    sprite = gameData->wall;
                    break;
                default:
                    sprite = gameData->fallback;
                    break;
            }
            RenderSprite_Grid(sprite, &lvl, renderer, &gameData->camera, x, y);
        }
    }
}
```

Now instead of having the grid to pixel calculations in `RenderLevel()` we just fetch the relevant sprite and call `RenderSprite_Grid()` .

```
// levelRenderer.cpp

void RenderEntities(GameData* data, SDL_Renderer* renderer){
    LevelData lvl = data->levels[data->currentLevelIndex];
    loop(i, lvl.entityCount){
        Image* img;
        Entity entity = lvl.entityBuffer[i];
        switch(entity.id){
            case ID::PLAYER:
                img = data->player;
                break;
            case ID::BOX:
                img = data->box;
                break;
            default:
                img = data->fallback;
                break;
        }

        float x_animated = std::lerp(entity.x_prev, entity.x, entity.progress_01);
        float y_animated = std::lerp(entity.y_prev, entity.y, entity.progress_01);

        RenderSprite_Grid(img, &lvl, renderer, &data->camera, x_animated, y_animated);
    }
}
```

We still perform our `lerp` logic inside `RenderEntities()` to get the point between `x/y_prev` and `x/y`. But then it's very similar.

That's it. We have done a bit of cleanup and layed the foundation for a camera and simplified our render logic!

22 Asset Management Part I

We're going to refactor out our `image.h/cpp` and create at least a slightly more robust way of loading sprites. The fact that we currently have our `Image*` pointers lying flat inside our `GameData` then loaded one-by-one in our `Initialize()` function inside `game.cpp` makes it very obvious that we should refactor as this solution is very transparent BUT more cumbersome than necessary.

Another issue is that we have to pass each `Image*` manually when we want to pass them to a function or pass the entire `GameData` struct.

We're making two changes to start with, we're removing everything inside `image.h/cpp` and instead creating `spriteLibrary.h/cpp`. We're also taking our `Image` struct and changing its name to `Sprite`.

Note: We're renaming `Image` to `Sprite` using the built in `rename` command in `Helix` this is performed by having the caret over the word and pressing `space+r`. Then in the command line at the bottom of the screen we just erase/type the name we want to change it to. Finally pressing `enter` confirms the change. This will update the word across our entire codebase. (which is nice).

We'll soon be moving from our generic Sokoban game, with a player and a box, and adding some actual game design to this base shell. One of the first changes we're making is adding more contextually relevant `IDs` inside `entity.h`

```
// entity.h

enum class ID : uint8_t {
    NONE = 0,
    GROUND = 1,
    WALL = 2,
    DEMON = 3,
    ROCK = 4,
    MEDUSA = 5,
    GHOST = 6,
    GOLEM = 7,
};
```

We've added `Medusa`, `Ghost` and `Golem` as well as I've renamed `Player` to `Demon` and `Box` to `Rock`. With these changes we currently can't build our project because we have compilation errors. But after our refactor is finished we will be able to.

```
// spriteLibrary.h

#pragma once

#include "SDL3/SDL_render.h"
#include "entity.h"

struct Sprite{
    SDL_Texture* texture;
    int width;
    int height;
};
```

so first we're copying over our `Image` struct and renaming it `Sprite`. in a later chapter we'll expand on this to help us with animation.

```
// spriteLibrary.h
```

```
enum class SPRITE_ID{  
    Fallback,  
    Ground,  
    Wall,  
    Rock,  
    Demon,  
    Medusa,  
    Golem,  
    Ghost  
};
```

we're also creating a new `enum` that will look very (very) similar to our `Entity ID` enum that we wrote earlier. Currently we're mapping our entities to a sprite 1-to-1 but later we'll be adding more `SPRITE_IDS` like `Demon_walking` or `Golem_pushing`. But for now we'll keep it like this.

```
// spriteLibrary.h
```

```
struct SpriteDataEntry{  
    SPRITE_ID id;  
    const char* path;  
};  
  
Sprite* GetSpriteFromID(ID id, Sprite* spriteBuffer);  
  
namespace AssetManagement  
{  
    void LoadSprite(Sprite* spriteBuffer, SpriteDataEntry entry, SDL_Renderer* renderer);  
    void LoadAllSprites(Sprite* spriteBuffer, SDL_Renderer* renderer);  
}
```

We're creating a `SpriteDataEntry` struct that pairs a `SPRITE_ID` with a corresponding `path` to where in the assets folder it is supposed to be found.

We're also creating a helper function `GetSpriteFromID()` that allows us to pass an `ID` and get back a `Sprite*`. The `spritebuffer` will be added to our `GameData` as a way of fetching every sprite we've loaded.

We're protecting our `LoadSprite` and new `LoadAllSprites` with the

`AssetManagement` namespace. This is just to highlight that these functions have a specific place within our program. We're also changing our `LoadSprite` function to work with our allocated `spriteBuffer` instead of being passed our `Memory::Arena` directly

With everything added to `spriteLibrary.h` we can remove everything from `image.h` and remove `image.cpp` for our `SHARED_SOURCES` inside our `CmakeLists.txt`.

```
set(SHARED_SOURCES
  # ${CMAKE_SOURCE_DIR}/src/image.cpp <--- remove this
  ${CMAKE_SOURCE_DIR}/src/arena.cpp
  ${CMAKE_SOURCE_DIR}/src/input.cpp
)
```

We'll be loading all of our sprites from inside our .DLL with saving their pointers in a new `Sprite* spriteBuffer`

```
// gameState.h

struct GameData {
    // Image* fallback
    // Image* wall
    // Image* ground
    // etc
    Sprite* spriteBuffer;

    // other code hidden for clarity
```

We've deleted all of our direct `Image*` references from `GameData` and replaced them with a single `Sprite*` array, Now we need to go to our `main.cpp` and allocate our sprite buffer

```
// main.cpp

int SPRITE_COUNT = 256;
size_t IMAGE_ARENA_SIZE = sizeof(Sprite) * SPRITE_COUNT;
gameData->arena_images = Memory::CreateSubArena(arena_main, IMAGE_ARENA_SIZE);
gameData->spriteBuffer = (Sprite*)Memory::Allocate(gameData->arena_images,
↪ sizeof(Sprite) * SPRITE_COUNT);
```

So in our little ocean of arena allocations we're allocating a subarena (just like before) then allocating our `spriteBuffer` to it. Currently each `Sprite` is `16 bytes` meaning that our `arena_images` is exactly 4096 bytes (or ~4kb) in size.

We're also removing the line in `main.cpp` where we loaded `fallback` manually. This gets handled automatically from now on. (and if not removed will not let us compile our program as `image.cpp` is no longer part of our Executables known files)

Now lets look at our `spriteLibrary.cpp`

```
// spritelibrary.cpp

#include "spriteLibrary.h"
#include "SDL3_Image/SDL_image.h"
#include <cassert>

const char* FALLBACK_PATH = "assets/sprites/fallback.png";

static const SpriteDataEntry all_sprite_data[] = {
    {SPRITE_ID::Fallback, FALLBACK_PATH},
    {SPRITE_ID::Wall, "assets/sprites/wall.png"},
    {SPRITE_ID::Demon, "assets/sprites/player.png"},
    {SPRITE_ID::Rock, "assets/sprites/box.png"},
    {SPRITE_ID::Ground, "assets/sprites/ground.png"},
    {SPRITE_ID::Medusa, "assets/sprites/medusa.png"}
};
```

Here we have first a path to where our fallback sprite is located. We store this to help us with our `LoadSprite` in case we have an issue with a sprite not exiting.

We then create a `static` array of our `SpriteDataEntry`. Using the c++ way we're allowed to outline a struct with just a pair of curly bracer `{}` we can add the contents of this array right here inside the script. the `static` keyword makes the array scoped only to our `spriteLibrary.cpp` meaning that no other file can ever access it and even if another file had a variable with the exact same name it would not create a compile conflict. This variable is constructed for us before `main()` even runs and will live for the duration of the program. adding `const` makes this piece of memory `read-only` as nothing is allowed to change it at runtime.

We might revisit this setup later and try and automate it a bit more.

With this setup we can declare the contents of our array up front

```
// example  
  
static const VariableType name[] = {  
    {},  
    {},  
    {}  
}
```

in this example code we have an array of 3 elements (empty for the sake of simplicity). The array can't be modified by code and the array will at compile-time get the correct size `3` automatically as the compiler finds three items in it. Each item is separated by a `,` comma and any extra newline is just to help us humans lay it out in a more easily understood way.

our `SpriteDataEntry` struct has two variables. first a `SPRITE_ID` then a `const char* path`. This has to be taken into account when using just the `{}` setup. As the variables are added in the order they show up inside the struct.

```
// spritelibrary.cpp

Sprite* GetSpriteFromID(ID id, Sprite* spriteBuffer){
    switch (id) {
        case ID::NONE:
            return nullptr;
        case ID::GROUND:
            return &spriteBuffer[(int)SPRITE_ID::Ground];
        case ID::WALL:
            return &spriteBuffer[(int)SPRITE_ID::Wall];
        case ID::DEMON:
            return &spriteBuffer[(int)SPRITE_ID::Demon];
        case ID::ROCK:
            return &spriteBuffer[(int)SPRITE_ID::Rock];
        case ID::MEDUSA:
            return &spriteBuffer[(int)SPRITE_ID::Medusa];
        case ID::GHOST:
            return &spriteBuffer[(int)SPRITE_ID::Ghost];
        case ID::GOLEM:
            return &spriteBuffer[(int)SPRITE_ID::Golem];
            break;
        default:
            return &spriteBuffer[(int)SPRITE_ID::Fallback];
            break;
    }
}
```

The next part is to add the logic for our `GetSpriteFromID` function. It accepts an `ID` and passes along our `spriteBuffer`. Then it maps each of the `IDs` to its corresponding `SPRITE_ID`, it also returns our fallback sprite in the `default` case. This happens when we pass an `ID` to the function that we have not created a `case` for yet. We need to `cast` our `SPRITE_ID` to `int` as arrays expect ints for their index. Then we take a pointer to that sprite inside the `spriteBuffer` using the `&` operator.

Right now, as mentioned previously, this just maps each entity to a sprite 1-to-1. But this will be refactored later when we add animations.

```
// spriteLibrary.cpp
namespace AssetManagement{
    void LoadAllSprites(Sprite* spriteBuffer, SDL_Renderer *renderer){
        for (SpriteDataEntry entry : all_sprite_data) {
            LoadSprite(spriteBuffer, entry, renderer);
        }
    }

    void LoadSprite(Sprite* spriteBuffer, SpriteDataEntry entry, SDL_Renderer* renderer){
        SDL_Surface* surface = IMG_Load(entry.path);
        if(surface == nullptr){
            surface = IMG_Load(FALLBACK_PATH);
        }
        assert(surface != nullptr);
        SDL_Texture* texture = SDL_CreateTextureFromSurface(renderer, surface);
        Sprite* sprite = &spriteBuffer[(int)entry.id];
        sprite->texture = texture;
        sprite->height = texture->h;
        sprite->width = texture->w;

        SDL_DestroySurface(surface);
    }
}
```

next, inside the same `namespace` we have in our `spriteLibrary.h` we add the logic for our `LoadAllSprites()` and `LoadSprite()` functions.

`LoadAllSprites` accepts the `spriteBuffer` and will pass it along each time it calls `LoadSprite()` from its `for-loop`

It then uses a nifty simplified way of writing a `for-loop` that first asks us about the type we want to loop over `SpriteEntryData` in this case. It asks us to give it a name `entry` then asks us what array we're working with. we pass it our `all_sprite_data` array that we wrote. This is much simpler to write as we're asking the compiler to infer the size of the array. Otherwise we would have to either write the array size manually or use a bit of code to calculate it.

For each of the `SpriteDataEntry` elements in the `all_sprite_data` array we call `LoadSprite()` and we pass along the `entry` so that our

`LoadSprite()` function can fetch the correct `path` and `ID`. To update the Sprite that the `SpriteBuffer` points to we need to fetch the pointer to the value, we do this by using the `pointer to` operator `&`. Now when we make changes to `texture, height and width` these will persist at the location pointed to by `spriteBuffer` at that index.

Other than the change to what parameters are passed in and how `path` is fetched from our `entry` the `LoadSprite()` is similar to its original version.

Now it's time to fully deprecate `Image.h/cpp` I have opted for just making these files empty to help with continuity in the lecture series. But I recommend deleting these files all together.

At this point you will have errors related to `#include "image.h"` that was found in more than a few of our files. We can safely delete those lines and if a file can't find `Sprite` its because we have not included `#include 'spritelibrary.h'`

we'll go to our `game.cpp` and erase all lines where we try and set our old `Image*` manually. Swapping them out for our new logic

```
// game.cpp

void Initialize(GameData* data, SDL_Window* window, SDL_Renderer* renderer){

    // Image* fallback = LoadSprite(... ..) <---- all manual sprite loading
    ↪ removed
    // Image* box = ... .. <----- all manual sprite loading
    ↪ removed
    AssetManagement::LoadAllSprites(data->spriteBuffer, renderer);
    // other code hidden for clarity
```

Finally we can go to our `levelRenderer.cpp` and simplify both `RenderLevel()` and `RenderEntities()`

```

// levelRenderer.cpp

void RenderLevel(GameData* gameData, SDL_Renderer* renderer){

    LevelData lvl = gameData->levels[gameData->currentLevelIndex];

    for(int x = 0; x < lvl.w; x++){
        for (int y = 0 ; y < lvl.h; y++) {
            uint8_t cellType = lvl.GetCellID(x, y);
            Sprite* sprite = GetSpriteFromID((ID)cellType, gameData->spriteBuffer);
            RenderSprite_Grid(sprite, &lvl, renderer, &gameData->camera, x, y);
        }
    }
}

void RenderEntities(GameData* data, SDL_Renderer* renderer){
    LevelData lvl = data->levels[data->currentLevelIndex];
    loop(i, lvl.entityCount){
        Entity entity = lvl.entityBuffer[i];
        Sprite* sprite = GetSpriteFromID(entity.id, data->spriteBuffer);

        float x_animated = std::lerp(entity.x_prev, entity.x, entity.progress_01);
        float y_animated = std::lerp(entity.y_prev, entity.y, entity.progress_01);

        RenderSprite_Grid(sprite, &lvl, renderer, &data->camera, x_animated, y_animated);
    }
}

```

now instead of each function needing a switch case to fetch the correct `Image*` (now `Sprite*`) we can use our helper function `GetSpriteFromID` instead.

With these changes we have refactored our asset loading system and how we fetch sprites.

23 Mouse input

Before we can write our level editor we're going to need a way of accessing the state of our mouse. We can do this very naively by not storing the mouse state between frames, but then any code that would check if a mouse button was pressed would fire every single tick.

We're going to return to our `input.h/cpp` and add the relevant variables to track our mouseState and then a few functions to simplify checking the current mouse state. In a lot of ways this will be similar to how we work with keys, but a bit less “cool”.

```
// input.h

struct Input{
    const bool* keys_current;
    const bool* keys_previous;
    float* keys_held_time;
    SDL_MouseButtonFlags mouse_current; // new
    SDL_MouseButtonFlags mouse_previous; // new
    float* mouse_held_time; // new
    float mouse_x; // new
    float mouse_y; // new
};
```

We will track and compare `_current` and `_previous` just as with our keyboard keys. We're also storing the position of the mouse as `mouse_x/y`. These two floats will be passed in to an SDL function that updates their values for us. the `SDL_MouseButtonFlags` are `bitwise flags` that we will be able to use `bitwise operators` to check against.

```
// input.h

enum class MouseButtons{
    LEFT = 0,
    MIDDLE = 1,
    RIGHT = 2,
};
```

We're going to be using SDL's `bitmasks` to compare our mouse buttons, but I've opted for this more human readable version as an in-between to make calling our new functions a bit simpler. You can decide for yourself if you find this extra step harder or easier to parse.

```
// input.h

bool MousePressed(const Input* input, MouseButton button);
bool MouseReleased(const Input* input, MouseButton button);
bool MouseHeld(const Input* input, MouseButton button);
bool MouseHeld_ForTime(const Input* input, MouseButton button, float min_length);
void UpdateMouse(Input* input, float dt);
```

very similarly to our `KeyPressed()` functions we're creating one for each of the relevant checks `Pressed`, `Held` and `Released` as well as a way of checking for how long our mouse buttons have been held down.

In our `input.cpp` we'll be adding a helper function to handle the `MouseButton` to `SDL_flag` conversion. We're keeping this in the `.cpp` so no file that includes our `.h` file can access it.

```
// input.cpp
SDL_MouseButtonFlags ButtonToFlag(MouseButtons button){
    switch(button){
        case MouseButtons::LEFT:
            return SDL_BUTTON_LMASK;
        case MouseButtons::MIDDLE:
            return SDL_BUTTON_MMASK;
        case MouseButtons::RIGHT:
            return SDL_BUTTON_RMASK;
        break;
    }
}
```

We pass in a button and return the corresponding `SDL_BUTTON_L/M/RMASK`. You can check out the definition of each `Mask` by putting the caret over them and pressing `space+d`. The bitwise logic is a bit more forced in my opinion. I'll be happy to have this `tv-remote style interface` to simplify

accessing them.

```
// input.cpp

bool MousePressed(const Input* input, MouseButton button){
    SDL_MouseButtonFlags flag = ButtonToFlag(button);
    return (input->mouse_current & flag) != 0 && (input->mouse_previous & flag) == 0;
}

bool MouseReleased(const Input* input, MouseButton button){
    SDL_MouseButtonFlags flag = ButtonToFlag(button);
    return (input->mouse_current & flag) == 0 && (input->mouse_previous & flag) != 0;
}

bool MouseHeld(const Input* input, MouseButton button){
    SDL_MouseButtonFlags flag = ButtonToFlag(button);
    return (input->mouse_current & flag) != 0 && (input->mouse_previous & flag) != 0;
}

bool MouseHeld_ForTime(const Input* input, MouseButton button, float min_length){
    SDL_MouseButtonFlags flag = ButtonToFlag(button);
    return input->mouse_held_time[flag] >= min_length;
}
```

We fetch the `SDL_MouseButtonFlag` from our helper function (that we have to have above these functions as it only exists in our .cpp and is compiled top-to-bottom). Then we do a slightly different kind of boolean comparison here. We are checking if the bits of the Mask of our `mouse_current/previous` and `flag` overlap. If the result was not 0 aka `!= 0` then at least one bit remained. if the result was 0 `== 0` then the two flags shared no bits. Because the `L/M/RMASKS` only set one bit to 1 each, then we can treat the `(mouse & flag) == 0` as false and `(mouse & flag) != 0` as true. With this the logic is identical to our keys.

```
// input.cpp

void UpdateMouse(Input* input, float dt){
    if(MouseHeld(input, MouseButton::LEFT)){
        input->mouse_held_time[(int)MouseButton::LEFT] += dt;
    }
    else{
        input->mouse_held_time[(int)MouseButton::LEFT] = 0;
    }

    if(MouseHeld(input, MouseButton::MIDDLE)){
        input->mouse_held_time[(int)MouseButton::MIDDLE] += dt;
    }
    else{
        input->mouse_held_time[(int)MouseButton::MIDDLE] = 0;
    }

    if(MouseHeld(input, MouseButton::RIGHT)){
        input->mouse_held_time[(int)MouseButton::RIGHT] += dt;
    }
    else{
        input->mouse_held_time[(int)MouseButton::RIGHT] = 0;
    }

    input->mouse_previous = input->mouse_current;
}
```

Ok, this is not the prettiest function, but trying to make aesthetically pleasing code is not something to strive for in and of itself. I don't envision this code changing for the foreseeable future and even though it repeats itself three times it's easy to parse. We check if we're holding down a mouse button, if we are then we increment the `mouse_held_time` element at that position in the array using the number associated with the `MouseButton` enum. If the mouse button was not held we reset the value back to 0.

Lastly we take the contents of `mouse_current` and set `mouse_previous` to match.

Inside `main.cpp` we need to do some allocation and then call the relevant functions.

```
// main.cpp

gameData->arena_input = Memory::CreateSubArena(arena_main, INPUT_ARENA_SIZE);

gameData->input.keys_current // old (shortened for clarity)
gameData->input.keys_previous // old (shortened for clarity)
gameData->input.keys_held_time // old (shortened for clarity)
gameData->input.mouse_held_time = (float*)Memory::Allocate(gameData->arena_input,
↳ sizeof(float) * 3;
```

We have **hard-coded** the value **3** as that is how many mouse buttons we're working with. This could warrant either a small comment or an actual variable. So let's add one to show our options

```
// main.cpp documentation example

// either we write a comment telling us that `3` represent the three mouse buttons
↳ (Left, Middle, Right)
gameData->input.mouse_held_time = (float*)Memory::Allocate(gameData->arena_input,
↳ sizeof(float) * 3;

// or we add a reminder variable
int mouseButtonCount = 3;
gameData->input.mouse_held_time = (float*)Memory::Allocate(gameData->arena_input,
↳ sizeof(float) * mouseButtonCount;
```

now inside our **main loop** we can add the logic to fetch and update our mouse data

```
// main.cpp

gameData->input.keys_current = SDL_GetKeyboardState(nullptr); // old
gameData->input.mouse_current = SDL_GetMouseState(&gameData->input.mouse_x,
↳ &gameData->input.mouse_y); // new

dll.update(gameData, dt); // old

UpdateKeys(&gameData->input, dt); // old
UpdateMouse(&gameData->input, dt); // new

dll.draw(gameData, renderer); // old
```

SDL_GetMouseState both returns the **SDL_MouseButtonFlags** for the current tick and accepts a pointer to a **x** float and a **y** float. These values

will be set inside the function itself to reference the current position of the mouse. By passing the `mouse_x/y` we can ensure that these variables are accessible from inside our Update and Draw loops by fetching them from `gameData->input.mouse_x/y`.

Now our game engine can handle basic mouse inputs. In the next chapter we'll be using this to help us create a level editor

24 Level Editor

our use of the Tiled level creator has many positives, it's a visual way of laying out our levels and it provides us with a tool that an artist can learn to manage on their own. It exports to a handy .JSON file that is easy for us to consume through code.

but, there is some friction in our pipeline currently. I would like to avoid touching Tiled when I am testing mechanics and making the first implementation of new entities.

We're going to create a first version of a level editor. In this lecture it won't be able to store any of the changes we make to a level, but we can still place entities to test if they behave correctly. In a future chapter we will expand on the capabilities of our level editor.

We'll be using DearImGui to help us get some visuals up on screen. We want to have a bar containing all of our entities/tiles so that we can select them then place them anywhere on our game board. One could easily imagine more functionality, like being able to change the size of the game board etc, but for now we'll settle for having an easy way of testing entities.

adding an entity or a tile to a position on the board is not something we currently have an easy way of doing. We call `CreateLevel()` and `CreateEntities()` from `level.cpp` but those all use the `.JSON` file that we've exported from `Tiled`. That is fine for the normal case. But for our purposes we need to have a simpler way of handling this. So first we'll refactor parts of `level.h/cpp`

```
// levels.h

Entity* GetNextAvailableEntitySlot(Entity* entityBuffer);
void AddEntity(ID entity_id, int x, int y, LevelData* level);
void removeEntity(int x, int y, LevelData* level)
```

we're adding three new functions to our `level.h`. giving us an easier way of adding/removing entities to the level.

Inside `level.cpp` we'll add the function bodies

```
Entity* GetNextAvailableEntity(LevelData* level) {
    for (int i = 0; i < level->entityCount; i++) {
        if(level->entityBuffer[i].id == ID::NONE){
            return &level->entityBuffer[i];
        }
    }

    return &level->entityBuffer[level->entityCount++];
}
```

We first check if any of the entities that we have at one point spawned has been set to `ID::NONE` again, meaning that they are no longer in use. If that is the case we return this `gap-entity`. If no gap entity was found we instead return the forward most index using `entityCount` then after we've done so we increment it by `1` so that it's ready to perform the same function next time.

Note how our for loop runs for `i < level->EntityCount` if we did `<=` we would always run up to the forward most slot and return that without incrementing `entityCount`.

```
// levels.cpp

void AddEntity(ID entity_id, int x, int y, LevelData *level){
    Entity* entity = level->GetEntity(x,y);

    if(entity == nullptr){
        entity = GetNextAvailableEntity(level);
    }

    entity->x = x;
    entity->y = y;
    entity->x_prev = x;
    entity->y_prev = y;
    entity->id = entity_id;
    entity->InitializeBaseBehaviour();
}
```

We'll be setting up a way of changing the level tiles like `ground` or `wall` but I have opted not to give that logic its own function as it is one line of code and will only be used in one place currently.

our `AddEntity()` first checks if it can find an entity already located on the specified coordinate. Then only if it didnt does it go ahead and use our `GetNextAvailableEntity()` function to fetch the most appropriate index.

Then just like we previously did inside `CreateEntities` we assign the basic variables to our `Entity`. And by calling `InitializeBaseBehaviour()` as well as updating the `entity_id` we've made sure that the entity is either completely added or the old entity totally overridden.

now we can simplify our `CreateEntities()` function to use this new `AddEntity()`

```
// levels.cpp
void CreateEntities(LevelData* lvl_data, Arena* arena){
    Reset(arena);
    lvl_data->entityCount = 0;
    fstream stream(lvl_data->level_path);
    auto result = nlohmann::json::parse(stream);
    auto entityData = result["layers"][ENTITIES_INDEX]["data"].get<vector<uint8_t>>();

    lvl_data->entityBuffer = (Entity*)Memory::Allocate(arena, sizeof(Entity) * 256);

    for (int i = 0; i < lvl_data->w * lvl_data->h; i++) {
        unsigned char entity_id = entityData[i];
        if(entity_id != 0){
            int x = i % lvl_data->w;
            int y = i / lvl_data->w;
            AddEntity((ID)entity_id, x, y, lvl_data);
        }
    }
}
```

we're also adding the function body for our `RemoveEntity()` function

```
// levels.cpp
void RemoveEntity(int x, int y, LevelData* level){
    Entity* entity = level->GetEntity(x, y);
    if(entity == nullptr){
        return;
    }

    *entity = {};
}
```

We try and fetch an entity pointer at the specified position. if none is found we can just return. Otherwise we take the value at the pointer reference and set it to its `struct default` using an empty pair of curly braces `{}`. This will zero out all variables inside the struct.

We no longer collect `entityCount` then allocate that specific amount. Instead we've simplified our code to just add enough room for 256 entities. This is still a bit flimsy and we can think about refactoring this later.

We need a way of telling our game that we want to use our editor and a way

of turning it off. We'll add a bool to our `GameData`

```
// gameState.h

struct GameData {
    // other variables hidden for clarity
    bool edit_level;
}
```

then in `Update()` inside `game.cpp` we'll toggle it between true and false.

```
// game.cpp

void Update(GameData* data, float dt){
    if(KeyPressed(&data->input, SDL_SCANCODE_F2)){
        data->edit_level = !data->edit_level;
    }

    // other code below hidden for clarity
```

We can use the `not operator` aka `!` to return the inverse of what the value actually was. So this line tells the `edit_level` bool to be set to whatever it was not. So false becomes true and then true becomes false again. This use of the `not operator` to create a toggle is very common.

The bool does nothing right now as the state of `edit_level` is never checked against.

Let's set up our `leveleditor.h/cpp`

```
// leveleditor.h

#pragma once

#include "camera.h"
#include "input.h"
#include "spriteLibrary.h"

struct Editor{
    ID object_to_place_id;
};

namespace EDITOR{
    void DrawObjectPanel(Editor* editor, Sprite* spriteBuffer);
    void PlaceObject(const int x, const int y, Editor* editor, LevelData* level);
    void Update(Editor* editor, Input* input, LevelData* level);
    void DrawPreview(Editor* editor, Input* input, SDL_Renderer* renderer, LevelData*
        ↪ level, Camera* camera, Sprite* spriteBuffer);
}
```

We'll most likely be adding more variables to our `Editor` struct, but right now we just have to keep track of what type of entity or tile we're trying to place down.

`DrawObjectPanel` will create a new window for us from which we will display all the entities/tiles we can place on our level.

`PlaceObject` will put an entity or tile on the level

`Update` will be called each tick and checking our mouse inputs will decide what to do

`DrawPreview` accepts a whole bunch of parameters and will draw a transparent version of the entity/tile we're placing, to help us see that things are working properly.

We also need to add one of these new `Editor` structs to `gameData`

```
// gameState.h

#include "leveleditor.h"

struct GameData {
    // other variables hidden for clarity

    bool edit_level; // added recently
    Editor editorData; // new
}
```

Lets start adding these to `leveleditor.cpp`

```
// leveleditor.cpp
#include "leveleditor.h"
#include "imgui/imgui.h"
#include "rendering.h"

namespace EDITOR {
```

First we do our `#includes` as usual then we make sure that all our functions are wrapped inside the same `EDITOR` namespace as we declared in `leveleditor.h`

```

// leveleditor.cpp
void DrawObjectPanel(Editor* editor, Sprite* spriteBuffer){
    ImGui::Begin("objects");
    ImVec2 size = {32, 32};

    if(ImGui::ImageButton("Ground", (ImTextureID)GetSpriteFromID(ID::GROUND,
        ↪ spriteBuffer)->texture, size)){
        editor->object_to_place_id = ID::GROUND;
    }
    ImGui::SameLine();
    if(ImGui::ImageButton("Wall", (ImTextureID)GetSpriteFromID(ID::WALL,
        ↪ spriteBuffer)->texture, size)){
        editor->object_to_place_id = ID::WALL;
    }

    ImGui::SameLine();
    if(ImGui::ImageButton("Rock", (ImTextureID)GetSpriteFromID(ID::ROCK,
        ↪ spriteBuffer)->texture, size)){
        editor->object_to_place_id = ID::ROCK;
    }

    ImGui::SameLine();
    if(ImGui::ImageButton("Demon", (ImTextureID)GetSpriteFromID(ID::DEMON,
        ↪ spriteBuffer)->texture, size)){
        editor->object_to_place_id = ID::DEMON;
    }
    ImGui::SameLine();
    if(ImGui::ImageButton("Medusa", (ImTextureID)GetSpriteFromID(ID::MEDUSA,
        ↪ spriteBuffer)->texture, size)){
        editor->object_to_place_id = ID::MEDUSA;
    }

    ImGui::End();
}

```

This code, as you can tell, repeats itself identically for each `ImGui::ImageButton`. With the amount of tiles we have this is more than fine. We can look at creating a streamlined automatic solution later. But for now we just want to get things up and running. `ImGui::ImageButton` returns `true` if it was pressed this frame, it also allows us to add an `ImTextureID` to it to set what the button will display. And because we have included `imgui_impl_sdlrenderer3.h` in our `src_external` folder we have given `ImGui` the ability to convert an `SDL_Texture` into the

format it needs.

So we use our `GetSpriteFromID()` function to retrieve the `Sprite*` pointer, then we take the `texture` stored within the struct. Now we have our `SDL_Texture`, but before we can use it we need to cast it to `ImTextureID`. We also set a size `ImVec2` at the top of the function and pass it in to `ImageButton` to set the size of the button. By calling `ImGui::SameLine()` after each button we make it so each `ImageButton` is layed out in a row rather than in a tall column. We do this as we want to place this horizontal bar at the bottom of our screen. We wrap all of our `ImGui` calls in `ImGui::Begin/End` to make this its own window.

As each button is pressed we update our `object_to_place_id` so that we can use it later.

```
// leveleditor.cpp
void PlaceObject(const int x, const int y, Editor* editor, LevelData* level){
    if(editor->object_to_place_id == ID::GROUND || editor->object_to_place_id ==
    ↪ ID::WALL){
        level->cells[y * level->w + x] = (int)editor->object_to_place_id;
    }
    else{
        AddEntity(editor->object_to_place_id, x, y, level);
    }
}
```

So this function might also undergo a refactoring step later as currently we're hard coding a check to see if what we're placing is a `Tile` or an `Entity`. This wont scale if we add `Water, Lava, Dirt, Ice` etc. But with just `GROUND` and `WALL` we can afford it.

So first we compare the `ID` of our `object_to_place_id` then if it was `GROUND` or `WALL` we set the specific cell in our `cells` array at that position (using our handy 2D->1D equation) to the integer value represented by the `enum` (with a cast to int using `(int)`).

If we on the other hand had selected an Entity we go ahead and call `AddEntity()` that we prepared earlier.

```
// leveleditor.cpp

void DrawPreview(Editor* editor, Input* input, SDL_Renderer* renderer, LevelData*
↪ level, Camera* camera, Sprite* spriteBuffer){
    int x;
    int y;
    camera::WorldToGrid(input->mouse_x, input->mouse_y, &x, &y, level);
    Sprite* preview = GetSpriteFromID(editor->object_to_place_id, spriteBuffer);
    if(preview != nullptr){
        RenderSprite_Grid(preview, level, renderer, camera, x, y, 1, 0.5);
    }
}
```

Even though our `DrawPreview` accepts a lot of parameters the actual function is very small. We get the grid position based on the world position of our mouse, then store the grid position in the `int x/y` variables we pass along by pointer reference. Then we use `object_to_place_id` to fetch the `Sprite*` associated with it. And if this `preview` sprite had an actual value we call `RenderSprite_Grid()`. The cool part is that in object-oriented systems we would probably have such tight coupling for our Sprites-to-Entities that it would be a lot less clean to just fetch and render a sprite, as IF it was a real entity/tile.

Note how we pass along `0.5` as a new final parameter to `RenderSprite_Grid`. This is a `float` value between `0` and `1` that we'll be using to control the alpha of the rendered texture. To do this we need to update `rendering.h/cpp`

```
// rendering.h

void RenderSprite_World(Sprite* sprite, SDL_Renderer* renderer, const Camera* camera,
↪ float x, float y, float scale = 1, float alpha = 1);
void RenderSprite_Grid(Sprite* sprite, LevelData* lvl, SDL_Renderer* renderer, const
↪ Camera* camera, float x, float y, float scale = 1, float alpha = 1);
```

after our optional `scale` parameter we've added `float alpha` and given it a default value of `1`. Meaning that if it is not specified then it will be set to `1` automatically.

```
// rendering.cpp
void RenderSprite_Grid(Sprite* sprite, LevelData* lvl, SDL_Renderer* renderer, const
    ↪ Camera* camera, float x, float y, float scale, float alpha){
    camera->GridToWorld(&x, &y, lvl);
    RenderSprite_World(sprite, renderer, camera, x, y, scale, alpha);
}
```

Our `RenderSprite_Grid()` function is the same, except we pass along `alpha` as the final parameter to `RenderSprite_World()`.

```
// rendering.cpp

void RenderSprite_World(Sprite* sprite, SDL_Renderer* renderer, const Camera* camera,
    ↪ float x, float y, float scale, float alpha){
    SDL_FRect rect;
    rect.x = x;
    rect.y = y;
    rect.h = sprite->height * UPSCALE_FACTOR * scale;
    rect.w = sprite->width * UPSCALE_FACTOR * scale;
    rect.x -= camera->camera_x;
    rect.y -= camera->camera_y;

    SDL_SetTextureAlphaModFloat(sprite->texture, alpha); // new
    SDL_RenderTexture(renderer, sprite->texture, NULL, &rect);
}
```

With the only change being a call to `SDL_SetTextureAlphaModFloat` we're modifying the alpha of the texture before it's being drawn. Note that this change persists as it updates the texture being pointed to. So if we would do this only once and not repeatedly each time we call the function then every time the texture is rendered it would have the latest alpha value that was set.

```

// leveleditor.cpp
void Update(Editor* editor, Input* input, LevelData* level){
    if(MousePressed(input, MouseButton::LEFT)){
        if(camera::GetIsPointInsideGrid(input->mouse_x, input->mouse_y, level)){
            int x;
            int y;
            camera::WorldToGrid(input->mouse_x, input->mouse_y, &x, &y, level);
            PlaceObject(x, y, editor, level);
        }
    }
    else if(MousePressed(input, MouseButton::RIGHT)){
        if(camera::GetIsPointInsideGrid(input->mouse_x, input->mouse_y, level)){
            int x;
            int y;
            camera::WorldToGrid(input->mouse_x, input->mouse_y, &x, &y, level);
            RemoveEntity(x, y, level);
        }
    }
}
}
}

```

similarly we use the same code twice to either add an entity/tile or remove an entity. To remove a `Wall` we have to replace it with a `Ground` tile. So that's why we don't have a `RemoveTile()` function or more logic inside `RemoveEntity` to handle these cases.

We retrieve the grid position of our mouse after we've determined that it is inside our game world, then we call the corresponding function.

This is everything needed to start working with our leveleditor now we just have to call these new functions we've created from our game.

We're calling this through `dev_gui.cpp`

```
// dev_gui.cpp

void DEV::Draw(GameData* data, SDL_Renderer* renderer){
    // code for drawing memory usage hidden for clarity

    if(data->edit_level){
        EDITOR::DrawObjectPanel(&data->editorData, data->spriteBuffer);
        EDITOR::DrawPreview(&data->editorData, &data->input, renderer,
↪ data->GetCurrentLevel(), &data->camera, data->spriteBuffer);
    }

    ImGui::Render();
    ImGui_ImplSDLRenderer3_RenderDrawData(ImGui::GetDrawData(), renderer );
}

```

if our toggled `edit_level` was true, then we call `DrawObjectPanel()` and `DrawPreview()` from our `EDITOR` namespace.

and lastly we call the `EDITOR::Update` from `game.cpp`

```
// game.cpp
void Update(GameData* data, float dt){

    if(KeyPressed(&data->input, SDL_SCANCODE_F2)){ // old
        data->edit_level = !data->edit_level; // old
    }
    if(data->edit_level){ // new
        EDITOR::Update(&data->editorData, &data->input, data->GetCurrentLevel()); // new
    }
}

```

Now our level editor is set up, we can now go ahead and test logic without having to enter `Tiled` and set up/export/parse!

25 Sokoban programming V

We're going to be adding functionality specific for the certain game we're making. To outline it we're creating a cast of characters that have different gameplay abilities. The starting point will be the game **Heroes of Sokoban 1, 2 and 3** by **Jonah Ostroff** (https://sites.math.washington.edu/~ostroff/puzzles/Heroes_of_Sokoban.html)

The heroes of sokoban are:

Red (warrior) pushes blocks Green (thief) drags blocks Blue (wizard) swaps position with blocks in view yellow (priestess) the priestess is unkillable purple (bard) pushesx and drags along entities in the squares around her green (druid) turns blocks into foliage and vice versa

on a level one or more of these characters will be present, and the player will be allowed to swap between them using an action button. Then each level is cleared when all characters present on the level are standing on a designated goal square. Each of these abilities are **compulsory** meaning that they are not activated by the player and is instead an intrinsic part of the character - for good and for bad.

We'll be adding another cast of characters with their own abilities instead

Golem

- > Can push blocks
- > Can push any amount of blocks
- > Can't be pushed

Medusa

- > Can push 1 block

- > Turns objects she looks at into pushable rocks
- > then they transform back if she no longer looks at them

Siren

- > objects on the same row or column attempt to move in her same direction
- > can't push objects herself

Demon

- > Can walk on lava
- > Can push 1 block

So for the `Golem` we need the logic to control how many blocks a character is allowed to push.

For `Medusa` we need to keep track of the facing direction of entities, and update these as they turn to move. We also need a `Petrified` state to help us transform objects

For the siren we need to complicate our `TryMove` to then issue new moves on all objects found. We're also adding a `Charmed` state to track this.

For the `Demon` we need specific allowances to make lava situationally walkable.

First, before we start on all this fun stuff (sorry) I want to refactor some of the code in `entity.h/cpp`. We're currently working with the `C-standard` paradigm with the goal of keeping `structs` as plain data and pass them along to functions to modify them.

So currently inside of `struct Entity` in `entity.h` we have 5 functions related to working with `Behaviour`. We're going to move them out of the struct and add a first parameter to each of them where we pass along an `Entity*` pointer.

```
// entity.h

bool HasBehaviour(Entity* entity, Behaviour flags);
void InitializeBaseBehaviour(Entity* entity);
void SetBehaviour(Entity* entity, Behaviour flags);
void AddBehaviour(Entity* entity, Behaviour flags);
void RemoveBehaviour(Entity* entity, Behaviour flags);
```

We create the function declarations inside `entity.h`. Then we add the same content of these functions that we used to have inside the struct to `entity.cpp`.

```
// entity.cpp

bool HasBehaviour(Entity* entity, Behaviour flags){
return (entity->behaviour & flags) == flags;
}

void InitializeBaseBehaviour(Entity* entity){
assert(entity->id != ID::NONE);
switch (entity->id) {
default:
SetBehaviour(entity, NONE);
break;
case ID::DEMON:
SetBehaviour(entity, (Behaviour)(CAN_MOVE | IS_PLAYER | RESPOND_TO_INPUT));
entity->strength = 2;
break;
case ID::ROCK:
SetBehaviour(entity, (Behaviour)CAN_MOVE);
break;
}
}

void SetBehaviour(Entity* entity, Behaviour flags){
entity->behaviour = flags;
}

void AddBehaviour(Entity* entity, Behaviour flags){
entity->behaviour = (Behaviour)(entity->behaviour | flags);
}

void RemoveBehaviour(Entity* entity, Behaviour flags){
entity->behaviour = (Behaviour)(entity->behaviour & ~flags);
}
```

so instead of calling our functions from our `Entity struct` itself we had to

refactor our code to pass the entity along instead. Other than that, the code is identical.

Now on each `call site` inside `game.cpp` and `levels.cpp` where we used to call these functions from the struct we instead pass the struct along.

for example:

```
// example of refactored changes

//old
if(entity->HasBehaviour(CAN_MOVE) && IsMoving(entity)){ ... }

//new
if(HasBehaviour(entity ,CAN_MOVE) && IsMoving(entity)){ ... }
```

If you try and build the game you'll get an error message if you still have remaining places to update the syntax.

This change is just to keep the logic more self-similar across our files.

Next, let's add the features for the `Golem`

we add an `int strength` to our `Entity` struct

```
// entity.h

struct Entity{
    ID id;
    int strength; // new
    int x;
    int y;
    int x_prev;
    int y_prev;
    float progress_01;
    Behaviour behaviour;
};
```

I also noticed that for both `Entity ID` and `SPRITE_ID` I had `GHOST/Ghost` set up instead of `SIREN/Siren`. So I went to both these sites in `entity.h` and `spriteLibrary.h` and renamed it.

Then in the switch case inside our `InitializeBaseBehaviour()` we make sure to set the `strength` values and add the four characters we've outlined. We'll be returning to this function as we keep adding more logic. For now we're giving `Medusa` and `Demon` a `1` in strength, the `Siren` a `0` and `999` for our `Golem` (should be enough I imagine).

```
// entity.cpp
void InitializeBaseBehaviour(Entity* entity){
    assert(entity->id != ID::NONE);
    switch (entity->id) {
        default:
            SetBehaviour(entity, NONE);
            break;
        case ID::DEMON:
            SetBehaviour(entity, (Behaviour)(CAN_MOVE | IS_PLAYER | RESPOND_TO_INPUT));
            entity->strength = 1;
            break;
        case ID::GOLEM:
            SetBehaviour(entity, (Behaviour)(CAN_MOVE | IS_PLAYER | RESPOND_TO_INPUT));
            entity->strength = 999;
            break;
        case ID::MEDUSA:
            SetBehaviour(entity, (Behaviour)(CAN_MOVE | IS_PLAYER | RESPOND_TO_INPUT));
            entity->strength = 1;
            break;
        case ID::SIREN:
            SetBehaviour(entity, (Behaviour)(CAN_MOVE | IS_PLAYER | RESPOND_TO_INPUT));
            entity->strength = 0;
            break;
        case ID::ROCK:
            SetBehaviour(entity, (Behaviour)CAN_MOVE);
            break;
    }
}
```

Now lets update our `TryMove()` to demand we pass along a `strength` value. We can then decrease this value by `1` each time we call a `TryMove()` recursively inside itself, meaning that each time a block pushes a block that pushes a block we reduce this value by `1`. If we ever reach a block in our chain and strength is no longer above `0` then we know we're trying to push to many things at once and we can return `false`, meaning that the move

fails.

```
// game.h
```

```
bool TryMove(Entity* mover, LevelData* level, CommandBuffer* cmd_buffer, int xDir, int  
↪ yDir, int timestamp, int strength);
```

then inside `game.cpp` we update by passing the `entity->strength` to `TryMove()` inside our `Update()` function.

```
// game.cpp
```

```
for (int i = 0; i < data->GetCurrentLevel()->entityCount; i++) { // old  
    Entity* entity = &data->GetCurrentLevel()->entityBuffer[i]; // old  
    if (HasBehaviour(entity, (Behaviour)(RESPOND_TO_INPUT | CAN_MOVE))) { // old  
        int xDir = data->input_buffer[data->input_buffer_read_count %  
↪ data->input_buffer_capacity].x; // old  
        int yDir = data->input_buffer[data->input_buffer_read_count %  
↪ data->input_buffer_capacity].y; // old  
        TryMove(entity, data->GetCurrentLevel(), data->commandBuffer, xDir, yDir,  
↪ data->command_timestamp, entity->strength); // new  
    }  
}  
data->input_buffer_read_count++;
```

Then inside the `TryMove()` function itself, at the recursive call site we pass along `strength` but only after we've reduced its value by `1` using the `decrement operator` aka `--`. Meaning that each time we recursively call `TryMove()` we will be passing along a lower value for `strength`.

```
// game.cpp
```

```
if (HasBehaviour(stepInto_entity, CAN_MOVE)) {  
    if (TryMove(stepInto_entity, level, cmd_buffer, xDir, yDir, timestamp, --strength)) {  
        MoveCommand mv;  
        mv.type = CMD_TYPE::MOVE;  
        mv.entity = mover;  
        mv.xDir = xDir;  
        mv.yDir = yDir;  
        Push(cmd_buffer, mv, timestamp);  
        return true;  
    }  
}
```

then at the top of `TryMove()` we'll make an `if-statement` that reacts to

the value of `strength`. But notably its not the strength of the entity, its the value of the variable named `strength` that was passed into the function as one of its parameters

```
// game.cpp  
  
if(strength < 0) {  
    return false;  
}
```

That's it, now the `Golem` is really strong, the `Siren` can't push at all and the `Demon` and `Medusa` can push one block.

We're also doing some refactoring to `GetSpriteFromID()` inside `spriteLibrary.cpp`. We want to always make sure we return fallback if we didn't find a sprite at the specified index of our `spriteBuffer`. Previously we could have invisible objects. Now they all at least show up. I've also taken the liberty to prepare for when we have art for the different characters.

```

//spritelibrary.cpp
Sprite* GetSpriteFromID(ID id, Sprite* spriteBuffer){
    Sprite* sprite_to_return = nullptr;

    switch (id) {
    case ID::NONE:
        sprite_to_return = nullptr;
        break;
    case ID::GROUND:
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Ground];
        break;
    case ID::WALL:
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Wall];
        break;
    case ID::DEMON:
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Demon];
        break;
    case ID::ROCK:
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Rock];
        break;
    case ID::MEDUSA:
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Medusa];
        break;
    case ID::SIREN:
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Siren];
        break;
    case ID::GOLEM:
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Golem];
        break;
    }

    if(sprite_to_return == nullptr || sprite_to_return->texture == nullptr){
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Fallback];
    }

    return sprite_to_return;
}

```

So we create a `sprite_to_return` pointer and assign to it using our `switch-case`. Then at the end if we didn't enter a switch case to give it a value or whatever was returned to us didn't have a texture set aka was `nullptr` we can set it to the `ID::Fallback` sprite.

We are allowed to stack our conditions inside our `if-statement` to include `->texture` in the second condition. This would cause a crash if we changed

from `||` to `&&` as `&&` evaluates all statements and would sometimes not find the `texture` pointer in the second condition as the `sprite_to_return` might be a `nullptr` and can therefore not have any variables set. the `||` operator will evaluate the leftmost condition first, and if it was `false` it never checks the next condition. Making it so that at the evaluation of the second condition we can be sure that `sprite_to_return` was in fact not a `nullptr`.

Lets tackle the fact that we want to limit `Medusas` petrification ability to only her facing direction, as well as actually making the ability work as intended. This is a larger feature and will require more new code than the `Golem` did.

We're going to start with a series of refactoring steps to improve our codebase a bit, as well as help us with the foundation for the `Medusa` ability inclusion.

We're adding a new `Command` a `RotateCommand` and a new `CMD_TYPE` enum. This will be created inside of our `command.h` file.

```
// command.h

enum class CMD_TYPE : uint8_t{
    NONE = 0, // old
    MOVE = 1, // old
    ROTATE = 2 // new
};

struct RotateCommand : Command{
    Entity* entity;
    Direction from;
    Direction to;

    RotateCommand(Entity* entity, Direction from, Direction to){
        this->entity = entity;
        this->from = from;
        this->to = to;
        type = CMD_TYPE::ROTATE;
    }
};
```

We inherit from `Command` (just as we do with `MoveCommand`). We store an entity pointer and two `Direction` enums to keep track of where we used to look and where we are looking now. then we create a `constructor` for this `Command`. A constructor is a function with no return type that has the exact same name as the actual struct. This function, once it exists, is suddenly a required function to call when creating a new struct of this type. We pass in three parameters that correspond with those stored in the struct itself. Because the parameters we pass in have the same name as our structs parameters we have to use the `this->` keyword to specify which is from the struct and which one is a parameter being passed in. If this is confusing you can add a prefix to the parameters aka `Entity* _entity` or similar.

We also set the `type` directly from the `constructor`.

lets go ahead and give `MoveCommand` a constructor as well

```
// command.h

struct MoveCommand : Command {
    Entity* entity;
    int xDir;
    int yDir;

    MoveCommand(Entity* entity, int xDir, int yDir){
        this->entity = entity;
        this->xDir = xDir;
        this->yDir = yDir;
        type = CMD_TYPE::MOVE;
    }
};
```

same variables as before, now just with a **constructor** that passes in the specified variables and sets type on its own.

We're also making an addition to **AnyCommand** so it holds a **RotateCommand** and so that our new **RotateCommand** can be treated as an **AnyCommand** when passed along.

```
// command.h

union AnyCommand {
    Command command;
    MoveCommand move;
    RotateCommand rotate; // new

    AnyCommand(MoveCommand mv){
        move = mv;
    };

    AnyCommand(RotateCommand rc){ // new
        rotate = rc;
    };
};
```

This is exactly what we did for **MoveCommand** earlier in the series.

If we return to our base **Command** struct we can give **type** a default value of **NONE**. This will help us catch a nasty bug.


```
// command.h
struct Command {
    CMD_TYPE type = CMD_TYPE::NONE;
    uint32_t timestamp;
};
```

Now unless a new Command remembers to set its `type` it will be set to `NONE`. Then in our `command.cpp` we can `abort` our program if this ever happens. Because if we ever push a command without a type we know that we have screwed up and need to fix the issue

```
// command.cpp
void Push(CommandBuffer* buffer, AnyCommand cmd, LevelData* level, uint32_t timestamp){
    assert(cmd.command.type != CMD_TYPE::NONE);
    // other code hidden for clarity
}
```

This will just terminate our entire program and flag the issue for us. A good safeguard against forgetting to set up our struct correctly. This is a defensive coding habit that we can leverage to help us spend less time hunting strange bugs that we don't know the cause of.

Note how we've added `LevelData*` as a parameter to our `Push` function. We are refactoring our `Push, Execute and Redo` functions inside `command.h/cpp` to have `LevelData*` as a parameter, we'll be needing this later to help our entities use their abilities on the board after a move or rotation.

```
// command.h
void Push(CommandBuffer* buffer, AnyCommand cmd, LevelData* level, uint32_t timestamp);
void Redo(CommandBuffer* buffer, LevelData* level);
```

Then inside `command.cpp` we need to update the same parameters as well.

```

// command.cpp
void Execute(AnyCommand cmd, LevelData* level, bool from_redo = false){
    // code hidden for clarity
}

void Push(CommandBuffer* buffer, AnyCommand cmd, LevelData* level, uint32_t timestamp){
    // code hidden for clarity
    Execute(cmd, level);
}

void Redo(CommandBuffer *buffer, LevelData* level){
    //a lot of code hidden for clarity
    Execute(cmd, level, true); // added level

    //a lot of code hidden for clarity
    Redo(buffer, level); // added level
}

```

With this change we need to update the code where we call our `Push` and `Redo` functions in `game.cpp` and `dev_gui.cpp` to also pass along `level`.

```

// dev_gui.cpp
void Draw_History(CommandBuffer* buffer, LevelData* level){
    int sliderPos = buffer->index;

    if(ImGui::SliderInt("history",&sliderPos, 0, buffer->head)){
        while(buffer->index > sliderPos){
            Undo(buffer);
        }
        while(buffer->index < sliderPos){
            Redo(buffer, level);
        }
    }
}

```

we needed to pass along `LevelData*` to our `Draw_History` function as `Redo()` needs this parameter.

Also Inside `game.cpp` we're creating two `MoveCommands` lets update those callsite to instead use our new `constructor`

```
// game.cpp
bool TryMove(Entity* mover, LevelData* level, CommandBuffer* cmd_buffer, int xDir, int
↪ yDir, int timestamp, int strength){
    // code above hidden for clarity
    if(stepInto_entity == nullptr){
        if(stepInto_tile_id == ID::GROUND){
            MoveCommand mv(mover, xDir, yDir);
            Push(cmd_buffer, mv, level, timestamp); // note our level parameter
            return true;
        }
        return false;
    }

    if(HasBehaviour(stepInto_entity, CAN_MOVE)){
        if(TryMove(stepInto_entity, level, cmd_buffer, xDir, yDir, timestamp,
↪ --strength)){
            MoveCommand mv(mover, xDir, yDir);
            Push(cmd_buffer, mv, level, timestamp); // note our level parameter
            return true;
        }
    }

    return false;
}
```

previously we set each variable (and the type manually) on individual rows, now we pass the variables all in one place from the **constructor** call.

```
// example
ATypeOfStruct aVariableName(variable_1, variable_2, ... etc);
```

This is the syntax for creating a struct and passing along variables to its constructor.

Next we're going to **levels.h** where we will move the functions from inside the **Level** struct outside of it and just placing their declarations in the **.h** and their bodies in the **levels.cpp** file. With this change we also have to add a **LevelData* level** parameter to each function as we now have to pass the **LevelData*** into it rather than having it placed inside our struct. We're also creating a new function **RaycastFirstEntity()**

```

// levels.h
#pragma once
#include "arena.h"
#include "entity.h"
#include <stdint>
using namespace Memory;

struct LevelData{
    int w;
    int h;
    uint8_t* cells;
    const char* level_path;
    Entity* entityBuffer;
    int entityCount;
};

void CreateLevel(Arena* arena, LevelData* level, const char* level_name);
void CreateEntities(LevelData* lvl_data, Arena* arena);
Entity* GetNextAvailableEntity(Entity* entityBuffer, int bufferSize);
void AddEntity(ID entity_id, int x, int y, LevelData* level);
void RemoveEntity(int x, int y, LevelData* level);
uint8_t GetCellID(LevelData* level, int x, int y); // previously inside the struct
Entity* GetEntity(LevelData* level, int x, int y); // previously inside the struct
// new
Entity* RaycastFirstEntity(int x_origin, int y_origin, Direction direction, LevelData*
↪ level, bool ignore_walls = false);

```

We'll get back to our new `Raycast` function soon. But first we'll update our `levels.cpp` with the functions previously inside our `LevelData` struct.

```

// levels.cpp

uint8_t GetCellID(LevelData* level, int x, int y){
    return level->cells[y * level->w + x];
}

Entity* GetEntity(LevelData* level, int x, int y){
    for (int i = 0; i < level->entityCount; i++) {
        if (level->entityBuffer[i].x == x && level->entityBuffer[i].y == y){
            return &level->entityBuffer[i];
        }
    }

    return nullptr;
}

```

They are identical except that we now have to use `level->` to fetch the

necessary variables.

With this update to `levels.h/cpp` all the places where we call `GetCellID()` and `GetEntity()` are now broken. This is because none of these call sites pass along `LevelData*` as a parameter and all of them try and access the function from the `level` variable itself `level->GetCellID()`.

Here's a list of the affected files

- `levels.cpp`
- `game.cpp`
- `levelrenderer.cpp`

In each of these files we need to make the following change

```
// example of change to level functions  
  
// old  
level->GetCellID(x, y);  
  
// new  
GetCellID(level, x, y);  
  
// old  
level->GetEntity(x, y);  
  
// new  
GetEntity(level, x, y);
```

Open each file and find the broken callsites then adjust them to match the new syntax.

With that done we can create our new `Raycast` function inside `levels.cpp`. A raycast is a “laser” that we fire from a point in space in a specific direction (also called a `vector`) then if that “laser” hits something then we return what it was. With us having four directions we’re not duplicating our code four times to handle all of these cases. Instead we do a bit of clever coding to

allow all directions to work with the same function

```
// levels.cpp
Entity* RaycastFirstEntity(int x_origin, int y_origin, Direction direction, LevelData*
↪ level, bool ignore_walls){
    Position facingVector;
    switch (direction) {
    case Direction::RIGHT:
        facingVector = {1, 0};
        break;
    case Direction::LEFT:
        facingVector = {-1, 0};
        break;
    case Direction::UP:
        facingVector = {0, 1};
        break;
    case Direction::DOWN:
        facingVector = {0, -1};
        break;
    }

    int x_search = x_origin + facingVector.x;
    int y_search = y_origin + facingVector.y;

    while(x_search > 0 && x_search < level->w && y_search > 0 && y_search < level->h){
        ID cellID = (ID)GetCellID(level, x_search, y_search);
        if(cellID == ID::WALL && !ignore_walls){
            break;
        }

        Entity* entity_search = GetEntity(level, x_search, y_search);
        if(entity_search != nullptr){
            return entity_search;
        }

        x_search += facingVector.x;
        y_search += facingVector.y;
    }

    return nullptr;
}
```

we're creating a `Position` variable that we set to store a different `x` and `y` value depending on the `Direction` variable we provided. This will be the direction our laser travels. We then use our `x/y_search` integers to act as the point we're evaluating. These numbers will increase by the contents

of our `facingVector` each time the loop runs. We start of by immediatly adding `facingVector.x/y` to it as we don't want to evaluate the cell that we started from, as that would mean that we always shot our laser into the origin cell and return back the entity that is standing there.

Our `while-loop` has four condtions that all have to be `true` for the loop to continue. because this function handles movement in all directions we have to check that `x` and `y` remain inside the level bounds both in the positive and negative directions.

Once inside the `while-loop` we check if we have encountered a wall, and if we're not allowed to `ignore_walls` then we `break` the loop causing us to move down and `return nullptr`. If we do not stop at a wall then we continue and check if there is an `Entity` at the specific position. If it did we can return that entity and stop the raycast - we found our closest target from the origin moving in the specified vector/direction.

If we didn't encounter a wall or an entity we add the value of `facingVector` to the `x/y_search` variables. And with only one of these always being `0` and the other being either `1 or -1` we ensure that we continue searching in the same direction.

We will need a `Behaviour` to toggle whether or not an `Entity` has been turned to stone. Lets update our enum inside `entity.h`

```
// entity.h
enum Behaviour : uint32_t {
    NONE = 0,
    CAN_MOVE = 1 << 0,
    IS_PLAYER = 1 << 1,
    RESPOND_TO_INPUT = 1 << 2,
    IS_PETRIFIED = 1 << 3, // new
    CAN_ROTATE = 1 << 4, // new
    UNPUSHABLE = 1 << 5 // new
};
```

We're adding 3 new Behaviours at this stage, one to control if we are allowed to rotate the entity (rocks won't rotate). One to check if the entity is **petrified** aka turned-to-stone and the last is a future addition that we'll use to make the **Golem** impossible to push due to being very heavy.

Next we're going to add our **RotateCommand** to **Game.cpp**. We want to rotate an entity only if they moved due to a player input and we want to rotate them even if they did not manage to actually perform a move.


```

// game.cpp
for (int i = 0; i < data->GetCurrentLevel()->entityCount; i++) {
    Entity* entity = &data->GetCurrentLevel()->entityBuffer[i];
    if (HasBehaviour(entity, (Behaviour)(RESPOND_TO_INPUT | CAN_MOVE))) { // old
        if (HasBehaviour(entity, Behaviour::IS_PETRIFIED)) { // new
            continue;
        }
        int xDir = data->input_buffer[data->input_buffer_read_count %
            ↪ data->input_buffer_capacity].x;
        int yDir = data->input_buffer[data->input_buffer_read_count %
            ↪ data->input_buffer_capacity].y;

        Direction new_facing = DirectionFromXY(xDir, yDir);
        if (new_facing != entity->facing) {
            RotateCommand rotate(entity, entity->facing, new_facing);
            Push(data->commandBuffer, rotate, data->GetCurrentLevel(),
            ↪ data->command_timestamp);
        }

        TryMove(entity, data->GetCurrentLevel(), data->commandBuffer, xDir, yDir,
            ↪ data->command_timestamp, entity->strength);
        }
    }
    data->input_buffer_read_count++;
}

```

We can now stop `petrified` entities from being allowed to move/rotate on their own due to player inputs by checking if the `IS_PETRIFIED` flag was set and if so continue immediately to the next entity.

We then use a helper function `DirectionFromXY` to convert the direction we're moving in into our `Direction` enum. As this is a super small function I've opted to place it inside the `entity.h` directly which is possible using the `inline` attribute

```
// entity.h

enum class Direction {
    RIGHT,
    LEFT,
    UP,
    DOWN
};

inline Direction DirectionFromXY(int xDir, int yDir){
    assert(xDir * yDir == 0);
    if(xDir == 1) { return Direction::RIGHT; }
    if(xDir == -1) { return Direction::LEFT; }
    if(yDir == 1) { return Direction::UP; }
    else          { return Direction::DOWN; }
}
```

This small function first does a safety assertion (defensive coding style) where we make sure that only either `xDir` or `yDir` had a non-zero value. We can do this by multiplying them together. Only if both had a non-zero value will the result not be zero. We have also added `class` to our `Direction` enum so that we need to type `Direction::` in order to use them. We do this to reduce the chance of accidental mixups.

Then we check which values were passed in and return the relevant `Direction` enum.

Back in `game.cpp` we compare this `new_facing` with the current `facing` direction of the entity. If they were different we go ahead and create a new `RotateCommand` using its constructor and then `Push` it to the `commandBuffer` to be executed.

After the rotation we call our `TryMove` as usual.

Right now the `Execute()` of our `RotateCommand` does nothing as we've not updated our `Execute()` function inside `command.cpp` yet

```
// command.cpp

void Execute(AnyCommand cmd, LevelData* level, bool from_redo = false){
    switch(cmd.command.type){
        case CMD_TYPE::NONE:
            break;
        case CMD_TYPE::MOVE: {
            MoveCommand mv = cmd.move;
            mv.entity->x_prev = mv.entity->x;
            mv.entity->y_prev = mv.entity->y;
            mv.entity->x += mv.xDir;
            mv.entity->y += mv.yDir;
            if(from_redo){
                mv.entity->progress_01 = 1;
            }
            break;
        }
        case CMD_TYPE::ROTATE:{
            RotateCommand rotate = cmd.rotate;
            if(!HasBehaviour(rotate.entity, CAN_ROTATE)){
                break;
            }
            rotate.entity->facing = rotate.to;
        }
        break;
    }
}
```

We've added our `case` for `CMD_TYPE::ROTATE` and we've also placed all the content of both of our cases inside curly braces `{}` we have to place these curly braces around our logic if we have more than 1 case and a case adds a new variable. In this case `MoveCommand mv` and `RotateCommand rotate`. The fact is that the compiler can't guarantee that a case won't fall into another case and as such all switch cases are actually in the same scope. This means that we could in theory reach `case CMD_TYPE::ROTATE` to work with `MoveCommand mv` without first having created it. Our compiler won't let us write code like this without the curly braces to set the bounds for each case. It's a bit messy but necessary.

We use our `Behaviour` flag `CAN_ROTATE` to ensure that only relevant entities

perform rotations.

Now besides actually doing our rotation (and move) we need a way of adding our ability to these events. We'll add three new functions to `entity.h/cpp`.

```
// entity.h
void PostMove(Entity* entity, LevelData* level);
void PostRotation(Entity* entity, LevelData* level, Direction from, Direction to);
void PreRotation(Entity* entity, LevelData* level, Direction from, Direction to);
```

As we add these function we are getting a compile error. Because to have `LevelData` in `entity.h` we need to `#include "levels.h"` but as `levels.h` includes `entity.h` we have a `circular include chain` that breaks compilation. But with `entity.h` not using the `LevelData` directly, we can `forward-declare` our `LevelData` to fix the circular dependency.

```
// entity.h
// #include "levels.h" <- removed
struct LevelData; // added
```

with `entity.h` having a struct declared with the same name we can have function declarations that use the struct. the header file itself doesn't use the struct so it doesn't need to know how it works. Then `entity.cpp` can `#include "levels.h"` and in their functions the `LevelData` will be mapped to the `LevelData` from he included `levels.h` header file.

right now the only logic we need to create is for our `Medusa` petrification ability, so lets go ahead and add our logic in `entity.cpp`

```
// entity.cpp

void PostMove(Entity *entity, LevelData* level){
    if(entity->id == ID::MEDUSA){
        Entity* entity_looked_at = RaycastFirstEntity(entity->x, entity->y,
↪ entity->facing, level);
        if(entity_looked_at != nullptr){
            if(!HasBehaviour(entity_looked_at, Behaviour::IS_PETRIFIED)){
                AddBehaviour(entity_looked_at, Behaviour::IS_PETRIFIED);
            }
        }
    }
}
```

we check if the entity that moved was **Medusa** and if so we use our new **Raycast** function along with her **facing** direction to get the first entity she's looking at. If we found an entity we check if it already was petrified, and if not we make it petrified.

```

// entity.cpp

void PostRotation(Entity* entity, LevelData* level, Direction from, Direction to){
    if(from == to){
        return;
    }
    if(entity->id == ID::MEDUSA){
        Entity* entity_looked_at = RaycastFirstEntity(entity->x, entity->y, to, level);
        if(entity_looked_at != nullptr){
            if(!HasBehaviour(entity_looked_at, Behaviour::IS_PETRIFIED)){
                AddBehaviour(entity_looked_at, Behaviour::IS_PETRIFIED);
            }
        }
    }
}

void PreRotation(Entity* entity, LevelData* level, Direction from, Direction to){
    if(from == to){
        return;
    }
    if(entity->id == ID::MEDUSA){
        Entity* entity_previously_looked_at = RaycastFirstEntity(entity->x, entity->y,
↪ from, level);
        if(entity_previously_looked_at != nullptr){
            if(HasBehaviour(entity_previously_looked_at, Behaviour::IS_PETRIFIED)){
                RemoveBehaviour(entity_previously_looked_at, Behaviour::IS_PETRIFIED);
            }
        }
    }
}

```

for our rotations we check if we actually had a change in rotation by comparing `from` and `to`. Then using `to` for `PostRotation` and `from` for `PreRotation` we raycast once again. and in the case of `PostRotation` we petrify the entity we found and for `PreRotation` we remove petrification from the entity as we know that after our rotation has completed we're no longer looking at that entity.

Make sure you pay attention to the fact that our two rotation functions are almost entirely similar except if they `Add` or `Remove` the flag and if the raycast uses `from` or `to`.

Now we can go to `command.cpp` and add these `function calls` to

Execute()

```
// command.cpp
void Execute(AnyCommand cmd, LevelData* level, bool from_redo = false){
    switch(cmd.command.type){
        case CMD_TYPE::NONE:
            break;
        case CMD_TYPE::MOVE: {
            MoveCommand mv = cmd.move;
            mv.entity->x_prev = mv.entity->x;
            mv.entity->y_prev = mv.entity->y;
            mv.entity->x += mv.xDir;
            mv.entity->y += mv.yDir;
            if(from_redo){
                mv.entity->progress_01 = 1;
            }
            PostMove(mv.entity, level); // <----- new
            break;
        }
        case CMD_TYPE::ROTATE:{
            RotateCommand rotate = cmd.rotate;
            if(!HasBehaviour(rotate.entity, CAN_ROTATE)){
                break;
            }
            PreRotation(rotate.entity, level, rotate.from, rotate.to); // new
            rotate.entity->facing = rotate.to;
            PostRotation(rotate.entity, level, rotate.from, rotate.to); // new
        }
        break;
    }
}
```

next we add the logic in `Undo()`

```

// command.cpp
void Undo(CommandBuffer* buffer){
    switch(cmd.command.type){
        case CMD_TYPE::NONE:
            break;
        case CMD_TYPE::MOVE:{
            // move undo logic hidden for clarity
            }
            break;
        case CMD_TYPE::ROTATE:{ // new
            RotateCommand rotate = cmd.rotate;
            if(!HasBehaviour(rotate.entity, CAN_ROTATE)){
                break;
            }
            rotate.entity->facing = rotate.from;
            break;
        }
    }
}

```

In `levelRenderer.cpp` we'll check if an entity is petrified and if so overwrite the its sprite to be the rock sprite instead

```

// levelRenderer.cpp
void RenderEntities(GameData* data, SDL_Renderer* renderer){
    // above code hidden for clarity

    Sprite* sprite = GetSpriteFromID(entity.id, data->spriteBuffer); // old

    if(HasBehaviour(&entity, Behaviour::IS_PETRIFIED)){ // new
        sprite = GetSpriteFromID(ID::ROCK, data->spriteBuffer); // new
    }

    // code hidden for clarity
}

```

Lastly we need to fix our Undo/Redo logic to work with the changes to `Behaviour`. Currently our code works as we want when moving and rotating our entities. but as we Undo our steps our game breaks.

This is because our `Add/Remove Behaviour` functions are not part of our Command structure yet. We're going to do some refactoring then resolve this.

As I was programming the BehaviourCommand logic I found myself passing `command_timestamp` to a bunch of functions to pass it along to the PostMove and PostRotate functions. This was not a dramatic issue but I still felt that it was unnecessarily prone to mistakes so lets go ahead and include our `command_timestamp` in our `CommandBuffer` struct and remove the `uint32_t command_timestamp` from our `GameData` struct

```
// gameState.h

struct GameData{
    // uint32_t command_timestamp // <---- removed
}
```

then add it in our `CommandBuffer` instead

```
// command.h

struct CommandBuffer{
    struct CommandBuffer{
        AnyCommand* allCommands;
        int capacity;
        int index;
        int head;
        uint32_t timestamp; // new
    };
};
```

Now of course all parts of our code base where we previously passed in `command_timestamp` will break and we need to fetch `timestamp` from our `CommandBuffer` struct instead. And any function declaration and parameters in `.h` and `.cpp` files need to remove the timestamp parameter. A list of affected functions

- TryMove() in `game.h/cpp`
- Push() in `command.h/cpp`
- Update() inside `game.cpp` increases timestamp by 1

Lets set up a new `Command` that will handle changes to `Behaviour`.

```
// command.h

struct ModifyBehaviourCommand : Command {
    enum Mode{
        ADD,
        REMOVE
    };

    Entity* entity;
    Behaviour flag;
    Mode mode;
    ModifyBehaviourCommand(Entity* entity, Behaviour flag, Mode mode){
        this->entity = entity;
        this->flag = flag;
        this->mode = mode;
        type = CMD_TYPE::MODIFY_BEHAVIOUR;
    }
};
```

the internal enum `Mode` is just to make it clearer if the command is adding or removing a flag. in a previous iteration it was just a bool. It worked but when constructing the Command the bool was not immediately understood.

I've opted for this approach instead of having an `AddBehaviourCommand` and a `RemoveBehaviourCommand` as that felt more prone to create divergent behaviour if one is updated and we forget to change the other.

As per usual with a new command we:

- 1) set it up
- 2) add a constructor to it
- 3) add another `CMD_TYPE`. in this case `MODIFY_BEHAVIOUR`
- 4) add the command to `union AnyCommand`
- 5) create a constructor inside `AnyCommand` that takes in our `ModifyBehaviourCommand`

```
// command.h
union AnyCommand {
    Command command;
    MoveCommand move;
    RotateCommand rotate;
    ModifyBehaviourCommand modify; // new

    AnyCommand(MoveCommand mov){
        move = mov;
    };
    AnyCommand(RotateCommand rot){
        rotate = rot;
    };
    AnyCommand(ModifyBehaviourCommand mod){ // new
        modify = mod;
    }
};
```

Now we need to pass along `CommandBuffer` to `PostMove()` , `PreRotate()` and `PostRotate()`

```
// entity.h

void PostMove(Entity* entity, LevelData* level, CommandBuffer* commandBuffer);
void PostRotation(Entity* entity, LevelData* level, CommandBuffer* commandBuffer,
    ⇨ Direction from, Direction to);
void PreRotation(Entity* entity, LevelData* level, CommandBuffer* commandBuffer,
    ⇨ Direction from, Direction to);
```

then inside those functions where we previously just called `Add/RemoveBehaviour` we now create our `ModifyBehaviourCommand` and push it.

```

// entity.cpp

void PostMove(Entity *entity, LevelData* level, CommandBuffer* commandBuffer){
    if(entity->id == ID::MEDUSA){
        Entity* entity_looked_at = RaycastFirstEntity(entity->x, entity->y,
↪ entity->facing, level);
        if(entity_looked_at != nullptr){
            if(!HasBehaviour(entity_looked_at, Behaviour::IS_PETRIFIED)){
                // new
                ModifyBehaviourCommand modify(entity_looked_at,
↪ Behaviour::IS_PETRIFIED, ModifyBehaviourCommand::ADD);
                Push(commandBuffer, modify, level);
            }
        }
    }
}

```

```

// entity.cpp
void PostRotation(Entity* entity, LevelData* level, CommandBuffer* commandBuffer,
↳ Direction from, Direction to){
    if(from == to){
        return;
    }
    if(entity->id == ID::MEDUSA){
        Entity* entity_looked_at = RaycastFirstEntity(entity->x, entity->y, to, level);
        if(entity_looked_at != nullptr){
            if(!HasBehaviour(entity_looked_at, Behaviour::IS_PETRIFIED)){
                // new
                ModifyBehaviourCommand modify(entity_looked_at,
↳ Behaviour::IS_PETRIFIED, ModifyBehaviourCommand::ADD);
                Push(commandBuffer, modify, level);
            }
        }
    }
}

void PreRotation(Entity* entity, LevelData* level, CommandBuffer* commandBuffer,
↳ Direction from, Direction to){
    if(from == to){
        return;
    }
    if(entity->id == ID::MEDUSA){
        Entity* entity_previously_looked_at = RaycastFirstEntity(entity->x, entity->y,
↳ from, level);
        if(entity_previously_looked_at != nullptr){
            if(HasBehaviour(entity_previously_looked_at, Behaviour::IS_PETRIFIED)){
                // new
                ModifyBehaviourCommand modify(entity_previously_looked_at,
↳ Behaviour::IS_PETRIFIED, ModifyBehaviourCommand::REMOVE);
                Push(commandBuffer, modify, level);
            }
        }
    }
}

```

Then we need to change `Execute()` to also have `CommandBuffer*` as a parameter so it can be passed to the three functions. Besides this we are also adding the new Command to `Execute()` and `Undo()`

```
// command.cpp
void Execute(AnyCommand cmd, LevelData* level, CommandBuffer* commandBuffer, bool
↪ from_redo = false){
    // code hidden for clarity
    case CMD_TYPE::MODIFY_BEHAVIOUR:{
        ModifyBehaviourCommand modify = cmd.modify;
        if(modify.mode == ModifyBehaviourCommand::ADD){
            AddBehaviour(modify.entity, modify.flag);
        }
        else{
            RemoveBehaviour(modify.entity, modify.flag);
        }
        break;
    }
}
```

because our enum is part of our struct we can only access it by specifying the struct name first `ModifyBehaviourCommand::ADD/REMOVE`.

Then we can just invert the `Add/RemoveBehaviour` in our `Undo()`

```
// command.cpp
void Undo(CommandBuffer* buffer){
    // code hidden for clarity

    case CMD_TYPE::MODIFY_BEHAVIOUR: {
        ModifyBehaviourCommand modify = cmd.modify;
        if(modify.mode == ModifyBehaviourCommand::ADD){
            // note that this is flipped compared to Execute()
            RemoveBehaviour(modify.entity, modify.flag);
        }
        else{
            AddBehaviour(modify.entity, modify.flag);
        }
        break;
    }
}
```

As a final step we'll return to our `Golem` and do something simple. Lets stop any entity from pushing him. We have to give him the `UNPUSHABLE` behaviour in `InitializeBaseBehaviour`

```
// entity.cpp
void InitializeBaseBehaviour(Entity* entity){
assert(entity->id != ID::NONE);
switch (entity->id) {
    default:
        // ...
        break;
    case ID::DEMON:
        // ...
        break;
    case ID::GOLEM:
        SetBehaviour(entity, (Behaviour)(CAN_ROTATE | CAN_MOVE | IS_PLAYER |
↪ RESPOND_TO_INPUT));
        AddBehaviour(entity, Behaviour::UNPUSHABLE); // <----- new
        entity->strength = 999;
        break;
    case ID::MEDUSA:
        // ...
        break;
    case ID::SIREN:
        // ...
        break;
    case ID::ROCK:
        // ...
}
}
```

and then its just a single line change inside `TryMove()`

```
// game.cpp

bool TryMove(Entity* mover, LevelData* level, CommandBuffer* cmd_buffer, int xDir, int
↪ yDir, int strength){
    // other code hidden for clarity

    // here we also check against `UNPUSHABLE`
    if(HasBehaviour(stepInto_entity, CAN_MOVE) && !HasBehaviour(stepInto_entity,
↪ UNPUSHABLE)){ // new
        if(TryMove(stepInto_entity, level, cmd_buffer, xDir, yDir, --strength)){ // old
            // ...
            // ...
        }
    }
}
```

That's it. Nice to end on a win.

This chapter was long, and probably more difficult, great job getting to the

end of it!

26 Animation Part II

It's time to start using our known gamestate to select appropriate sprites to render to the screen.

In the beginning of this course we set our tile size, both in our .PNGs and in `common.h` to 32. At this stage when we're adding our tileset and version 1.0 of our entities I've opted for 16x16 tiles as the base unit.

This means that the first step is to adjust `common.h` as well as grab all the .pngs from the .ZIP file `SOKOBAN_CHAPTER_027_SPRITES.zip` and replace the contents of `assets/sprites` with these new `.png` files.

```
// common.h
const int UPSCALE_FACTOR = 4;
const int CELL_SIZE_PX = 16 * UPSCALE_FACTOR;
```

we've increased `UPSCALE_FACTOR` to `4` to account for the smaller base tiles.

Our next issue is the fact that we're going to have entities that we

- a) want to place in the middle of a tile
- b) want to be larger than a 1x1 tile

Our entities will be standing in the middle of their tile and their heads and arms can reach outside of its own tile. If we decided to ensure that each entity lived exactly in its own tile then we would not have to worry about the next step - but that is very very uncommon artwise.

We need to give our `Sprite` and `SpriteDataEntry` both a `pivot_x` and `pivot_y` integer. These will be manually set by us. They will represent the pixel on our sprite that we want to have be put in the center of the tile.

```
// spritelibrary.h

struct Sprite{
    SDL_Texture* texture;
    int width;
    int height;
    int pivot_x; // new
    int pivot_y; // new
};
```

Then our `SpriteDataEntry` will have the same variables

```
// spritelibrary.h
const int NOT_SET = -1;

struct SpriteDataEntry{
    SPRITE_ID id;
    const char* path;
    int pivot_x = NOT_SET;
    int pivot_y = NOT_SET;
};
```

We create the `NOT_SET` constant as a way of flagging if the pivot variables were not set manually by us. This will allow us to catch these cases and programmatically set the pivot to the dead center of our sprite.

also inside `spritelibrary.h` we'll add `SPRITE_ID's` for our new sprites

```
// spritelibrary.h
enum class SPRITE_ID{
    Fallback,
    Ground,
    Ground_alt, // new
    Wall,
    Rock,
    Demon,
    Medusa_Idle_Side, // new
    Medusa_Idle_Front, // new
    Medusa_Idle_Back, // new
    Golem,
    Siren,
    Dropshadow // new
};
```

In `spritelibrary.cpp` we can now add our manually set pivots to our

static array of `SpriteDataEntry`

```
// spritelibrary.cpp
static const SpriteDataEntry all_sprite_data[] = {
    {SPRITE_ID::Fallback, FALLBACK_PATH,0,0},
    {SPRITE_ID::Wall, "assets/sprites/wall.png", 0, 0},
    {SPRITE_ID::Demon, "assets/sprites/player.png"},
    {SPRITE_ID::Rock, "assets/sprites/rock.png", 10, 20},
    {SPRITE_ID::Ground, "assets/sprites/ground.png", 0, 0},
    {SPRITE_ID::Ground_alt, "assets/sprites/ground_alt.png",0,0},
    {SPRITE_ID::Medusa_Idle_Side, "assets/sprites/medusa_idle_side.png", 12, 24},
    {SPRITE_ID::Medusa_Idle_Front, "assets/sprites/medusa_idle_front.png", 12, 24},
    {SPRITE_ID::Medusa_Idle_Back, "assets/sprites/medusa_idle_back.png", 12, 24},
    {SPRITE_ID::Dropshadow, "assets/sprites/dropshadow.png", 8, 8}
};
```

Our `Wall`, `Ground`, `Fallback` and newly added `Ground_alt` are all manually set to `(0, 0)`. For all our level tiles we'll make sure to have our pivot be in the top left corner. If we were to adjust our level tiles to have their pivots centered all of our entities would need to be adjusted by this same amount to not be offset. Fair disclosure, this issue stumped me for a pretty long while (ugh...).

As we can see, our `rock.png` is `20x20 px` and the pivot has been placed at the very bottom-center. The same is true for the `24x24 px medusa_idle_side/front/back`.

With custom pivots we can have sprites that are not `16x16 px` and with some clever math always have them centered on their tile.

I have opted to have `SPRITE_ID::Demon` without a manual pivot to showcase how our `NOT_SET sentinel` will be used. The term `sentinel` means a special reserved value that should never be part of the actual scope of the variable. Used as a substitute for (in this case) a `bool not_set` variable inside the struct itself. Though that would also work and if this `sentinel` logic is confusing you could easily swap to a `bool` inside the struct instead.

In `LoadSprite()` in `spritelibrary.cpp` we fetch this new pivot and check against our `sentinel`

```
// spritelibrary.cpp

void LoadSprite(Sprite* spriteBuffer, SpriteDataEntry entry, SDL_Renderer* renderer){
    SDL_Surface* surface = IMG_Load(entry.path);
    if(surface == nullptr){
        surface = IMG_Load(FALLBACK_PATH);
    }
    assert(surface != nullptr);
    SDL_Texture* texture = SDL_CreateTextureFromSurface(renderer, surface);
    Sprite* sprite = &spriteBuffer[(int)entry.id];
    sprite->texture = texture;
    sprite->height = texture->h;
    sprite->width = texture->w;

    if(entry.pivot_x == NOT_SET || entry.pivot_y == NOT_SET){ // new
        sprite->pivot_x = sprite->width / 2; // new
        sprite->pivot_y = sprite->height / 2; // new
    }
    else{ // new
        sprite->pivot_x = entry.pivot_x; // new
        sprite->pivot_y = entry.pivot_y; // new
    }

    SDL_DestroySurface(surface);
}
```

we take the `pivot_x/y` from our `SpriteDataEntry` and sets the pivot of our `Sprite`. If we found that our `SpriteDataEntry` had its pivot set to our default `sentinel` value of `NOT_SET` aka `-1` then we place the pivot in the middle of the sprite.

We are no longer just fetching a sprite from as little data as the `id` of the entity. instead we'll be using its `behaviour`, `facing Direction` and `progress_01` to get the correct sprite to render.

In `spritelibrary.h/.cpp` we'll add a new function

```
// spritelibrary.h
```

```
Sprite* GetSprite_FromEntityState(Entity* entity, Sprite* spritebuffer);
```

This will evaluate the variables inside `Entity` to select the appropriate sprite from the `spritebuffer`

```
// spritelibrary.cpp
```

```
Sprite* GetSprite_FromEntityState(Entity* entity, Sprite* spritebuffer){  
    if(HasBehaviour(entity, Behaviour::IS_PETRIFIED)){  
        return &spritebuffer[(int)SPRITE_ID::Rock];  
    }  
  
    switch (entity->id) {  
    case ID::MEDUSA:  
        switch (entity->facing) {  
        case Direction::RIGHT:  
        case Direction::LEFT:  
            return &spritebuffer[(int)SPRITE_ID::Medusa_Idle_Side];  
            break;  
        case Direction::DOWN:  
            return &spritebuffer[(int)SPRITE_ID::Medusa_Idle_Back];  
            break;  
        case Direction::UP:  
            return &spritebuffer[(int)SPRITE_ID::Medusa_Idle_Front];  
            break;  
        }  
        default:  
            return GetSpriteFromID(entity->id, spritebuffer);  
            break;  
    }  
  
    return nullptr;  
}
```

Right now we're heavily using the `GetSpriteFromID` as a fallback when we have not set up the specific logic for an entity. At this stage this function does two things.

- 1) checks if the entity `IS_PETRIFIED` then returns the `SPRITE_ID::Rock` in that case.
- 2) if it was `Medusa` we check its facing direction and pick the correct

sprite. We'll be flipping the `_Side` sprite along the x-axis to avoid having to add a mirrored sprite to our `assets/sprites` folder each time.

In our `GetSpriteFromID` I've opted to return `fallback` inside the `Medusa` case to signal that something has gone terribly wrong

```
// spritelibrary.cpp

Sprite* GetSpriteFromID(ID id, Sprite* spriteBuffer){
    Sprite* sprite_to_return = nullptr;

    switch (id) {
    case ID::NONE:
        sprite_to_return = nullptr;
        break;
    case ID::GROUND:
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Ground];
        break;
    case ID::WALL:
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Wall];
        break;
    case ID::DEMON:
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Demon];
        break;
    case ID::ROCK:
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Rock];
        break;
    case ID::MEDUSA:
        sprite_to_return = nullptr; // changed
        break;
    case ID::SIREN:
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Siren];
        break;
    case ID::GOLEM:
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Golem];
        break;
    }

    if(sprite_to_return == nullptr || sprite_to_return->texture == nullptr){
        sprite_to_return = &spriteBuffer[(int)SPRITE_ID::Fallback];
    }

    return sprite_to_return;
}
```

During development we want our program to fail and fail loudly. We want to catch bugs as easily and as often as possible. That's why it's a good idea to be pretty liberal with `asserts` and to not have this function `fail silently` by having us return for example the most neutral Medusa Sprite. We're not trying to make it so the problem is as discrete as possible. we WANT it to completely blow up.

Next we need to start using our `GetSprite_FromEntityState()` as well as drawing our new `dropshadow` sprite beneath the entity. but before we do we're going to make it so that our entities `jump` in a small parabola when getting to an empty square. We'll be adding two new `Behaviour` to `entity.h` to control this

```
// entity.h

enum Behaviour : uint32_t {
    NONE = 0,
    CAN_MOVE = 1 << 0,
    IS_PLAYER = 1 << 1,
    RESPOND_TO_INPUT = 1 << 2,
    IS_PETRIFIED = 1 << 3,
    CAN_ROTATE = 1 << 4,
    UNPUSHABLE = 1 << 5,
    JUMPS = 1 << 6, // new
    IS_PUSHING = 1 << 7 // new
};
```

We will only allow entities that have `JUMPS` and is not currently pushing something to perform the jump.

in `entity.cpp` we'll make `Medusa` have the new `JUMPS` behaviour

```
// entity.cpp

case ID::MEDUSA:
    SetBehaviour(entity, (Behaviour)(CAN_ROTATE | CAN_MOVE | IS_PLAYER |
↪ RESPOND_TO_INPUT));
    AddBehaviour(entity, Behaviour::JUMPS);
    entity->strength = 1;
    break;
```

Then in `TryMove()` and `Update()` inside `game.cpp` we'll be adding and removing `IS_PUSHING`.

```
// game.cpp

bool TryMove(Entity* mover, LevelData* level, CommandBuffer* cmd_buffer, int xDir, int
↪ yDir, int strength){
    // code above hidden for clarity

    if(HasBehaviour(stepInto_entity, CAN_MOVE) && !HasBehaviour(stepInto_entity,
↪ UNPUSHABLE)){ // old
        if(TryMove(stepInto_entity, level, cmd_buffer, xDir, yDir, --strength)){ // old
            MoveCommand mv(mover, xDir, yDir); // old
            AddBehaviour(mover, Behaviour::IS_PUSHING); // <----- new
            Push(cmd_buffer, mv, level); // old
            return true; // old
        }
    }
}
```

so if we found an entity and managed to move it with the recursive `TryMove()` then we know that our `Mover` performed a push and we can now add the behaviour.

in `Update()` where we loop over all of our entities once `are_entities_moving` is `false` we can reset this behaviour flag to `0`


```

for (int i = 0; i < data->GetCurrentLevel()->entityCount; i++) {
    Entity* entity = &data->GetCurrentLevel()->entityBuffer[i];
    if (HasBehaviour(entity, Behaviour::IS_PUSHING)) { // <--- new
        RemoveBehaviour(entity, Behaviour::IS_PUSHING); // <--- new
    }

    if (HasBehaviour(entity, (Behaviour)(RESPOND_TO_INPUT | CAN_MOVE))) { // old
        // code inside this if-statement hidden for clarity
    }
}

```

Now `IS_PUSHING` is only true for moving entities during the visualisation when they were pushing something.

Lets look at `levelRenderer.cpp` and update our `RenderEntities()` function

```
// levelRenderer.cpp

void RenderEntities(GameData* data, SDL_Renderer* renderer){
    LevelData lvl = data->levels[data->currentLevelIndex];
    for (int i = 0; i < lvl.entityCount; i++) {
        Entity entity = lvl.entityBuffer[i];
        if(entity.id == ID::NONE){
            continue;
        }

        Sprite* sprite = GetSprite_FromEntityState(&entity, data->spriteBuffer);
        if(HasBehaviour(&entity, Behaviour::IS_PETRIFIED)){
            sprite = GetSpriteFromID(ID::ROCK, data->spriteBuffer);
        }
        float x_animated = std::lerp(entity.x_prev, entity.x, entity.progress_01);
        float y_animated = std::lerp(entity.y_prev, entity.y, entity.progress_01);

        float dropshadow_y = y_animated;

        if(HasBehaviour(&entity, Behaviour::JUMPS) && !HasBehaviour(&entity,
            ↪ Behaviour::IS_PUSHING)){
            y_animated -= 0.5 * sinf(entity.progress_01 * 3.14);
        }

        Sprite* dropshadow = &data->spriteBuffer[(int)SPRITE_ID::Dropshadow];

        RenderEntity_OnTile(dropshadow, &lvl, renderer, &data->camera, x_animated,
        ↪ dropshadow_y, 1, 0.4, false);
        RenderEntity_OnTile(sprite, &lvl, renderer, &data->camera, x_animated, y_animated,
        ↪ 1, 1, entity.facing == Direction::RIGHT);
    }
}
```

It's not too complex, but I hope you see the use case for why we want to add features as we need them instead of trying to divine them as the function is first created. This allows us to only add what we need and keep code as simple as possible until a concrete need for change arrives.

Now we use `GetSprite_FromEntityState()` to retrieve the correct sprite. We've also removed the old logic here that checked `IS_PETRIFIED` as that is being taken care of by the `GetSprite_FromEntityState()` function. We then store the `y` position for our dropshadow before we update `y_animated` to account for an entity being allowed to jump.

the following expression `y_animated -= 0.5 * sinf(entity.progress_01 * 3.14);` is part of linear algebra but I had to remind myself on how it was written. What we've done is mapped our `Progress_01` to a parabola that goes from `0` to `0.5` and back to `0` creating an arc.

By multiplying `progress_01` with `PI` we get a value that goes between `0` and `PI`. When we map `0` to `PI` in a `sine` wave function we start at `0` go up to `1` at `sinf(PI/2)` then back to `0` at `sinf(PI)`.

This changes our `progress_01` mapping from `0.0 - 0.5 - 1.0` into `0.0 - 1.0 - 0.0` then we multiply this value by `0.5` as this is the amplitude (or height) of the arc we want to use. `0.5` being half a tile in height aka `50%`.

We then take away this jump height number from `y_animated` as negative `y` is upwards in `SDL`.

Linear algebra is a whole course in and of itself. If you can remember that a sine wave oscillates between `-1` and `1` over time and that it does so in smooth arcs then with a little practice and some refreshing online you'll be able to retrieve this function (and many like it).

Then we call a new `RenderEntity_OnTile()` function that we'll look at right now inside `rendering.h/.cpp`

```
// rendering.h

void RenderSprite_World(Sprite* sprite, SDL_Renderer* renderer, const Camera* camera,
    ↪ float x, float y, float scale = 1, float alpha = 1, bool flipped = false);
void RenderSprite_Grid(Sprite* sprite, LevelData* lvl, SDL_Renderer* renderer, const
    ↪ Camera* camera, float x, float y, float scale = 1, float alpha = 1, bool flipped =
    ↪ false);
void RenderEntity_OnTile(Sprite* sprite, LevelData* lvl, SDL_Renderer* renderer, const
    ↪ Camera* camera, float x, float y, float scale = 1, float alpha = 1, bool flipped =
    ↪ 1);
```

It has the same exact parameters as `RenderSprite_Grid()` . Also, note how all three of these functions now has a `bool flipped = false` parameter. Note that these are optional parameters so their values will be set to their defaults if not explicitly set.

Note: with the addition of a new parameter we need to update the function both in our `.h` and our `.cpp` file.

Lets look at the changes to `rendering.cpp`

```

// rendering.cpp

void RenderSprite_World(Sprite* sprite, SDL_Renderer* renderer, const Camera* camera,
↳ float x, float y, float scale, float alpha, bool flipped){
    SDL_FRect rect;
    rect.x = x;
    rect.y = y;
    float final_scale = UPSCALE_FACTOR * scale;
    rect.h = sprite->height * final_scale;
    rect.w = sprite->width * final_scale;
    rect.x -= sprite->pivot_x * final_scale;
    rect.y -= sprite->pivot_y * final_scale;
    rect.x -= camera->camera_x;
    rect.y -= camera->camera_y;
    SDL_SetTextureScaleMode(sprite->texture, SDL_SCALEMODE_PIXELART);
    SDL_SetTextureAlphaModFloat(sprite->texture, alpha);

    // SDL_RenderTexture(renderer, sprite->texture, NULL, &rect);
    SDL_RenderTextureRotated(renderer, sprite->texture, NULL, &rect, 0.0, NULL, flipped ?
↳ SDL_FlipMode::SDL_FLIP_HORIZONTAL : SDL_FlipMode::SDL_FLIP_NONE);
}

void RenderSprite_Grid(Sprite* sprite, LevelData* lvl, SDL_Renderer* renderer, const
↳ Camera* camera, float x, float y, float scale, float alpha, bool flipped){
    camera->GridToWorld(&x, &y, lvl);
    RenderSprite_World(sprite, renderer, camera, x, y, scale, alpha, flipped);
}

void RenderEntity_OnTile(Sprite* sprite, LevelData* lvl, SDL_Renderer* renderer, const
↳ Camera* camera, float x, float y, float scale, float alpha, bool flipped){
    camera->GridToWorld(&x, &y, lvl);
    x += CELL_SIZE_PX / 2.0;
    y += CELL_SIZE_PX / 2.0;
    RenderSprite_World(sprite, renderer, camera, x, y, scale, alpha, flipped);
}

```

In `RenderSprite_World()` we now call `SDL_SetTextureScaleMode` to make sure that our tiny tiny sprites are not rendered using `billinear filtering`. That is the most common type of filtering that blurs textures to avoid making sharp low-res textures in our game. But by setting our `SDL_SCALEMODE` to the provided `_PIXELART` we instead use `point filtering` meaning that no blurring happen. You can try and remove this line and see how bad the pixelart looks.

We also swap from `SDL_RenderTexture` to `SDL_RenderTextureRotated` as this version has a parameter for flipping the texture. We use a new operator to decide if we should use `SDL_FLIP_HORIZONTAL` or `SDL_FLIP_NONE`

```
// example
Boss BossToSpawn = difficulty >= DIFFICULTY::HARD ? Dragon : Sheep;
```

in this example we ask a question `difficulty > DIFFICULTY::HARD ?` then we need to provide a true and a false output separated by a `:`. So if the difficulty in this example is at least `HARD` then the boss becomes a dragon. If it is not it becomes a sheep.

This is the same code as

```
// example
Boss BossToSpawn;
if(difficulty >= DIFFICULTY::HARD){
    BossToSpawn = Dragon;
}
else{
    BossToSpawn = Sheep;
}
```

Its just some syntactic sugar to help us reduce code lines. And once you know the code structure of the `?` operator its pretty easy to parse.

we have also collected `UPSCALE_FACTOR * scale` into a temporary variable as we'll be using it in 4 places now.

```
rect.x -= sprite->pivot_x * final_scale;
rect.y -= sprite->pivot_y * final_scale;
```

This adjusts the position of the sprite based on the pivot set. This will put the pivot point at the top left corner of the tile (so not yet in the middle). What makes it adjust to be fully centered is the little piece of math that we do that differentiates `RenderSprite_Grid()` and `RenderEntity_OnTile()`

```

void RenderEntity_OnTile(Sprite* sprite, LevelData* lvl, SDL_Renderer* renderer, const
↪ Camera* camera, float x, float y, float scale, float alpha, bool flipped){
    camera::GridToWorld(&x, &y, lvl);
    x += CELL_SIZE_PX / 2.0;
    y += CELL_SIZE_PX / 2.0;
    RenderSprite_World(sprite, renderer, camera, x, y, scale, alpha, flipped);
}

```

by adjusting the `x` and `y` position when rendering an entity on a tile we shift the position by half the size of a tile. moving the rendering point from the upper left corner to the center of the tile.

Now we have made it so the default rendering point of an entity is the middle of a tile. Then we adjust the sprite to place its pivot point at this position. The result is an entity with its pivot right at the center of the tile.

The easiest way to check this is to start running the game then to comment out these lines and then call `reload project_name` from `powershell` to only rebuild the `.DLL` then you can tab back to the game and see what happens when we add each adjustment.

An optional step that I've added is to draw the walkable grid two different colors like a checkerboard to help visualize the grid. I do this by adding to `RenderLevel()`

```

// levelrenderer.cpp
//Sprite* sprite = GetSpriteFromID((ID)cellType, gameData->spriteBuffer); // <-- old
Sprite* sprite;

if(ID(cellType) == ID::GROUND){
    sprite = &gameData->spriteBuffer[(x + y) % 2 == 0 ? (int)SPRITE_ID::Ground :
↪ (int)SPRITE_ID::Ground_alt];
}
else{
    sprite = GetSpriteFromID((ID)cellType, gameData->spriteBuffer);
}

```

`(x + y) % 2` will flip-flop between 1 and 0. So by using the handy `?`

operator we can select `Ground` or `Ground_alt` alternating. Then if the `ID` was not `ground` we just go ahead and fetch the sprite as normal. This is a bit hacky and we'll most likely be refactoring it soon.

now we have crispy pixel art, that can be flipped along the x-axis and that leverages the pivot positions we've set to place the entity in the correct position.

27 Scratch Arena and Sprite Sorting

You might have noticed an issue where the order entities are being drawn to the screen is sometimes wrong. With a lower entity being drawn behind an entity above it.

We are going to make a copy of our EntityBuffer and sort it. This in regular C++ would require us to create a new array then `free` it. If we don't free it we are causing a stackoverflow due to us having assigned memory that we never allow our computer to recapture and reuse. We'll fix this need to create->free all together by using a `scratch arena`

a `scratch arena` is a memory arena that we allocate then reset during each tick. Meaning that all memory in it is freed in bulk instead of individually.

creating the `scratch arena` is as simple as creating a subarena from `arena_main` then calling `Reset()` at the beginning of our game-loop.

```
// gameState.h

struct GameData{
    // other variables hidden for clarity
    Memory::Arena* arena_scratch;
}

// main.cpp

gameData->arena_main = arena_main;

// other code hidden for clarity

gameData->arena_scratch = Memory::CreateSubArena(arena_main, KILOBYTES(256));
```

then in our `while(running)` game loop we reset the arena.

```
// main.cpp
while(running){
    DLL_CheckStatus(&dll); // old

    Reset(gameData->arena_scratch); // new

    CalculateDeltaTime(&dt); // old
```

I also think it's time to simplify our `Memory::Allocate` with a handy macro.

Inside `arena.h` we'll add the following code

```
// arena.h

#define ALLOC(arena, type) (type*)Memory::Allocate((arena), sizeof(type));
#define ALLOC_ARRAY(arena, type, count) (type*)Memory::Allocate((arena), sizeof(type) *
↪ count);
```

This will take `ALLOC(arena, type)` when written in our codebase and replace it with the code that follows it. We'll use `ALLOC` for single items and `ALLOC_ARRAY` for when we want to allocate an array. Inside `main.cpp` at all places where we call `Allocate()` we can now use our simplified macro.

```
// main.cpp

// old
GameData* gameData = (GameData*)Memory::Allocate(arena_main, sizeof(GameData));

// new
GameData* gameData = ALLOC(arena_main, GameData);

// and for example

// old
gameData->fps_buffer = (float*)Memory::Allocate(arena_main, sizeof(float) *
↪ gameData->fps_buffer_count);

// new
gameData->fps_buffer = ALLOC_ARRAY(arena_main, float, gameData->fps_buffer_count);
```

This makes the code easier to read and reduces the amount of mindless typing we have to do each time we want to allocate to an arena. I've gone ahead and substituted all call sites for `Memory::Allocate` with this macro. You're

free to do the same. But make sure to compile your program afterwards to ensure you didn't break something my mistyping.

Now we can refactor our `RenderEntities` to both fix a problem we had with accidentally copying data and to introduce our scratch arena to help with sorting. Previously we stored `levelData lvl` as the actual struct and not a pointer `LevelData* lvl`. This meant that we copied over the content each time, which will contribute to a potential `stack overflow`. We also made the same mistake when fetching the specific entity with `Entity entity = lvl.entityBuffer[i];` this should also have been a pointer instead.

With this in mind, lets look at the updated `RenderEntities`

```

// levelRenderer.cpp
void RenderEntities(GameData* data, SDL_Renderer* renderer){
    LevelData* lvl = &data->levels[data->currentLevelIndex];

    Entity** SortedEntities = ALLOC_ARRAY(data->arena_scratch, Entity*, lvl->entityCount);
    for (int i = 0; i < lvl->entityCount; i++) {
        SortedEntities[i] = &lvl->entityBuffer[i];
    }
    std::sort(SortedEntities, SortedEntities + lvl->entityCount,
        ↪ IsEntityBelowOtherEntity);

    for (int i = 0; i < lvl->entityCount; i++) {
        Entity* entity = SortedEntities[i]; // we grab this entity instead of the one from
        ↪ `lvl->entityBuffer[i]`
        if(entity->id == ID::NONE){
            continue;
        }
        Sprite* sprite = GetSprite_FromEntityState(entity, data->spriteBuffer);
        if(HasBehaviour(entity, Behaviour::IS_PETRIFIED)){
            sprite = GetSpriteFromID(ID::ROCK, data->spriteBuffer);
        }
        float x_animated = std::lerp(entity->x_prev, entity->x, entity->progress_01);
        float y_animated = std::lerp(entity->y_prev, entity->y, entity->progress_01);
        float dropshadow_y = y_animated;
        if(HasBehaviour(entity, Behaviour::JUMPS) && !HasBehaviour(entity,
            ↪ Behaviour::IS_PUSHING)){
            y_animated -= 0.5 * sinf(entity->progress_01 * 3.14);
        }

        Sprite* dropshadow = &data->spriteBuffer[(int)SPRITE_ID::Dropshadow];

        RenderEntity_OnTile(dropshadow, lvl, renderer, &data->camera, x_animated,
        ↪ dropshadow_y, 1, 0.4, false);
        RenderEntity_OnTile(sprite, lvl, renderer, &data->camera, x_animated, y_animated, 1,
        ↪ 1, entity->facing == Direction::RIGHT);
    }
}

```

We will be creating a new type of variable a pointer to a pointer. A bit strange, but all it is is a pointer that points to a place in memory where another pointer exists. We are going to be sorting pointers and to sort pointers we need an ordered list that points to them that we can sort.

The original pointers are layed out sequentially in our memory, but the correct draw order is not the same order as they are in memory. This is why another

array of pointers exist where each pointer-pointer points at a specific entry in the original array. Allowing for a remapping

```
// our original entity pointers in memory  
1-2-3-4-5
```

```
// our Entity** pointer-pointers in memory  
1-2-3-4-5
```

but the `1` pointer-pointer "points" to original entity pointer `4` like so

```
(1)4-(2)2-(3)1-(4)5-(5)3
```

this allows us to draw the entity with the lowest y-value (after sorting) first even though it was the fourth entity in the original memory block.

`std::sort` comes from `#include <algorithm>`. This is a standard library in C++ that give us a handy way of sorting a known array.

`std::sort` accepts 3 parameters 1) the first entry in the array 2) the last entry in the array 3) the way we want to sort them

We're creating a small function inside `levelRenderer.cpp` that we pass as an argument to the `std::sort`

```
// levelrenderer.cpp  
bool IsEntityBelowOtherEntity(Entity* a, Entity* b){  
    return a->y < b->y;  
}
```

note, this has to be placed above our `RenderEntities()` as it is not defined in our `.h` file.

in our `std::sort` the second parameter `SortedEntities + lvl->entityCount`

takes the known `Entity**` then moves down our memory block a number of `Entity*` long steps equal to `entityCount`. To arrive at the last element in the array.

We pass `IsEntityBelowOtherEntity` as the function itself, that's why we don't add `()` and parameters. We're not calling the function we're telling `sort` to call and use it. The function compares two `Entity*` and because this is what the array points to our compiler knows how to work with this.

With this we've added our scratch arena and added draw order to our entities!

28 Spawn Commands and active/inactive entities

Currently our game breaks if we move with a character then remove it from our dev menus. We don't get the figure back when we undo/redo. Lets fix that. The issue is that as we undo an action the unit that we spawned doesn't go away. It stays on the board and the undo no longer represent the actual game state we previously had.

We'll need two new `Commands` . `AddCommand` and `RemoveCommand` .

```
// command.h

enum class CMD_TYPE : uint8_t{
    NONE = 0,
    MOVE = 1,
    ROTATE = 2,
    MODIFY_BEHAVIOUR = 3,
    ADD = 4, // new
    REMOVE = 5 // new
};

// command.h
struct AddCommand : Command{
    int x;
    int y;
    ID id;

    AddCommand(int x, int y, ID id){
        this->x = x;
        this->y = y;
        this->id = id;
        type = CMD_TYPE::ADD;
    }
};
```

Our `AddCommand` is simpler than the `RemoveCommand` as we only need to store the `ID` of the entity we want to spawn. So the `AddCommand` does not store an `Entity` itself.

```

// command.h
struct RemoveCommand : Command{
    int x;
    int y;
    Behaviour storedBehaviour;
    ID storedID;

    RemoveCommand(Entity* entity){
        x = entity->x;
        y = entity->y;
        storedBehaviour = entity->behaviour;
        storedID = entity->id;
        type = CMD_TYPE::REMOVE;
    }
};

```

Here we need to save info about our `Entity` as we remove it. Lets say that we have petrified an Entity before removing it. To perserve our history we need to store this `Behaviour` so we can add it back.

As usual we add constructors to both `Add` and `Removee` then add them as variables and constructor parameters to `AnyCommand`.


```
// command.h
union AnyCommand {
    Command command;
    MoveCommand move;
    RotateCommand rotate;
    ModifyBehaviourCommand modify;
    AddCommand add; // new
    RemoveCommand remove; // new

    AnyCommand(MoveCommand mov){
        move = mov;
    };
    AnyCommand(RotateCommand rot){
        rotate = rot;
    };
    AnyCommand(ModifyBehaviourCommand mod){
        modify = mod;
    }
    AnyCommand(AddCommand add){ // new
        this->add = add;
    }
    AnyCommand(RemoveCommand rem){ // new
        remove = rem;
    }
};
```

Note, due to my honestly pretty substandard naming convention of my parameters I ended up with the same variable name for my addCommand and the parameter. forcing me to use `this->` to disambiguate. This is no issue really, but the syntax has a certain `smell` to it.

Next we'll first refactor a silly mistake in `Command.cpp` before we add our `Add/Remove` logic to `Execute()` and `Undo()`.

In our `switch` cases we get the relevant command by writing `CommandType theCommand = cmd.specifiCommand`. This should always have been a pointer so we don't create any new data. So instead we write `CommandType* theCommand = cmd.specifiCommand`. We can easily fix this issue that covers a lot of our lines inside `Execute()` and `Undo` by first pressing `v` to enter `selection mode` then we select the entire code block

by moving the caret down over each line. Then we press **s** type **cmd\.** and press **enter**. This will put a caret on each of the lines at the exact position where it found the text **cmd.** we use **** so that the **.** is not **escaped** and is actually evaluated as text.

Once we have all our cloned Carets we enter **insert mode** with **i** and delete the **.** and replace it with **->**. With this we've modified 10+ places with just one command. Learning this select and multi-edit command will drastically improve your speed when refactoring.

Now we can add the switch cases to our functions

```
// command.cpp
// inside Execute()
case CMD_TYPE::ADD:{
    AddCommand* add = &cmd.add;
    AddEntity(add->id, add->x, add->y, level);
    break;
}
case CMD_TYPE::REMOVE:{
    RemoveCommand* remove = &cmd.remove;
    RemoveEntity(remove->x, remove->y, level);
    break;
}
}
```

We encapsulate each case with **{}** then we call our old **AddEntity** and **RemoveEntity** using the parameters we stored in the commands.

```
// command.cpp
// inside Undo()

case CMD_TYPE::ADD:{
    AddCommand* add = &cmd.add;
    RemoveEntity(add->x, add->y, level);
    break;
}
case CMD_TYPE::REMOVE:{
    RemoveCommand* remove = &cmd.remove;
    AddEntity(remove->storedID, remove->x, remove->y, level);
    Entity* entity = GetEntity(level, remove->x, remove->y);
    SetBehaviour(entity, remove->storedBehaviour);
    break;
}
}
```

As both `RemoveEntity()` `AddEntity()` and `GetEntity()` require that we pass along `LevelData*` we need to change the parameter list of our `Undo()` function to receive a `LevelData*` this will require us to modify our `command.h` file to add this parameter as well as updating all of our callsites to pass along this variable.

```
// command.h

void Undo(CommandBuffer* buffer, LevelData* level); // `LevelData* level` is new
```

we call Undo from * game.cpp * dev_gui.cpp * command.cpp

so those three callsites are where you will need to add and pass along the `LevelData*` parameter.

Now in our `levelEditor.h/.cpp` We will be changing from adding/removing our `Entities` by calling those functions directly and instead creating then pushing our new commands to do the same actions. To push our commands we need to pass along our `commandBuffer`. To do this we need to update our parameters inside `levelEditor.h` to supply it.

```
// levelEditor.h

void PlaceObject(const int x, const int y, Editor* editor, LevelData* level,
↳ CommandBuffer* commandbuffer); // commandBuffer is new
void Update(Editor* editor, Input* input, LevelData* level, CommandBuffer* buffer); //
↳ commandBuffer is new
```

Then inside `levelEditor.cpp` we can make the necessary changes

```
// levelEditor.cpp
void PlaceObject(const int x, const int y, Editor* editor, LevelData* level,
↳ CommandBuffer* commandBuffer){
    if(editor->object_to_place_id == ID::GROUND || editor->object_to_place_id ==
↳ ID::WALL){
        level->cells[y * level->w + x] = (int)editor->object_to_place_id;
    }
    else{
        // AddEntity(editor->object_to_place_id, x, y, level); // old
        AddCommand add(x, y, editor->object_to_place_id); // new
        Push(commandBuffer, add, level); // new
    }
}
```

Then we update our callsite for `RemoveEntity()`

```

// levelEditor.cpp

void Update(Editor* editor, Input* input, LevelData* level, CommandBuffer* buffer){
    if(MousePressed(input, MouseButton::LEFT)){
        // code hidden for clarity
    }
}
else if(MousePressed(input, MouseButton::RIGHT)){
    if(camera::GetIsPointInsideGrid(input->mouse_x, input->mouse_y, level)){
        int x;
        int y;
        camera::WorldToGrid(input->mouse_x, input->mouse_y, &x, &y, level);
        Entity* entity = GetEntity(level, x, y);
        if(entity == nullptr){
            return;
        }
        // here is where the old call to RemoveEntity() was
        RemoveCommand remove(entity); // new
        Push(buffer, remove, level); // new
    }
}
}
}

```

Now our history works as intended with our add/remove. To clarify why this was important to do now.

We already were and will continue to test our game by making temporary levels using our levelEditor. It will be extremely bothersome to have our history malfunction and cause issues that we might confuse with mistakes in newly written code. That's why we make sure to squash this bug right away.

29 Scenes and transitions Part I

We can't start our game inside gameplay forever. We're going to create a titlescreen and transition between it and gameplay. We'll also lay some groundwork to simplify adding more of these scenes. (like game credits and a main menu).

Right now our `GameData` struct has everything the game could be interested in inside this growing monolithic struct. We're going to make some changes that will require updating a lot of our code. We're taking variables inside the struct that are part of the different scenes and breaking them into their own "substructs"

```
// gameState.h

struct GameData {
    // new
    SCENE_TYPES scene_current;
    SCENE_TYPES scene_previous;
    Scenes scenes;
    Transition transition;
    EditorData editor_data;

    // old
    Input input;
    Sprite* spriteBuffer;
    Memory::Arena* arena_main;
    Memory::Arena* arena_levels;
    Memory::Arena* arena_entities;
    Memory::Arena* arena_images;
    Memory::Arena* arena_commands;
    Memory::Arena* arena_input;
    Memory::Arena* arena_scratch;
    Camera camera;
    ImGuiContext* ImGui_context;
    const float* dt;
};
```

We have taken all the variables that are scene agnostic and kept them inside the main struct. Then we've added a new `enum` to help us keep track of the

current and previously active scenes. We'll need to know about both to help us with fade-in-and-out-from-black transitions between the scenes. All of our editor variables are also collected in a new `EditorData` struct.

There is also a new `Transition` struct. This will be filled with variables to help us cover the screen in black to hide our scene transitions. We'll look into it a bit later.

We've created a `Scenes` struct that will act as an intermediary, holding all of the new structs related to each scene.

```
// gameState.h
struct Scenes{
    Gameplay gameplay;
    MainMenu mainMenu;
    TitleScreen titlescreen;
    Credits credits;
};
```

Each of these new structs will need to be declared above our `Scenes` struct.

```
// gameState.h

struct Gameplay {
};

struct MainMenu {
};

struct TitleScreen {
};

struct Credits {
};
```

We'll fill these soon.

```
// gameState.h
enum class SCENE_TYPES : uint8_t{
    NONE,
    TITLES_SCREEN,
    MAINMENU,
    GAME,
    CREDITS,
};
```

We create our `enum` to have the same elements as our `Scenes` struct. As well as `NONE`. We're never going to be using it directly as a scene we go to. But we're using it along with `assert()` to catch bad code easier.

We're moving `GetCurrentLevel()` out of the function to below our `GameData` struct. This will require us to make some changes to it

```
// gameState.h
inline LevelData* GetCurrentLevel(Gameplay* game){
    return &game->levels[game->currentLevelIndex];
}
```

We now pass along a `Gameplay*` pointer instead of fetching this through the implicit connection between the structs data and its function.

We've set this function to be `inline` this means that it will be the same function for all files that implement this `.h` file. If we didn't have this each file that implemented it would get its own copy of the function and as soon as two of these functions included each other there would be a compilation conflict. The other option would be to create a `gameState.cpp` file and add this function to it. Totally legit, but as this is just a small helper function I've opted to have it live inside my `.h` file.

Lets look at `Gameplay`


```
// gameState.h

struct Gameplay {
    CommandBuffer* commandBuffer;
    LevelData* levels;
    int levelCount;
    int currentLevelIndex;
    Position* input_buffer;
    int input_buffer_capacity;
    int input_buffer_write_count;
    int input_buffer_read_count;
    bool initialized; // new
};
```

besides `initialized` all of the variables inside `Gameplay` are just the game-play specific variables previously found inside `GameData`.

These changes means that everywhere where we could previously write `data->commandBuffer` or `data->levels` now have to go through our intermediary `Scenes` struct then into the specific struct.

```
//example

data->levels // old

data->scenes.gameplay.levels // new
```

This feels like a lot more indirection. And it is. But we're allowing for this additional hurdle to help our project grow. With just a flat struct containing everything we'll have to be very careful with how we name files. And it will become easier and easier to misunderstand what the purpose of a variable is. But we have a lot of functions that we pass multiple variables from `data` to. These function calls will become extremely long if we need to go thorough `scenes` then `gameplay` for each variable.

This is how we'll fix this issue

```
// example

// old
Undo(data->commandBuffer, data->GetCurrentLevel())

// new
Gameplay* gameplay = &data->scenes.gameplay;
Undo(gameplay->commandBuffer, GetCurrentLevel(gameplay));
```

So, by fetching `Gameplay* gameplay` once we can collapse the function calls back to their original size. In this example we can also see the new way we need to call `GetCurrentLevel()`.

We're going to be a bit brutalistic at this stage and for `Gameplay` and `Titlescreen` (our first two scenes we'll be working with) we'll add some functions directly into `game.cpp` as these are not supposed to be able to be called by outside files.

```
// game.cpp

void InitializeGame(Gameplay* gameplay, Arena* arena_levels){
    // ... code will go here
}

void UpdateTitlescreen(TitleScreen* titlescreen, const float dt){
    // ... code will go here
}

void UpdateGame(Gameplay* gameplay, Input* input, const float dt){
    // ... code will go here
}
```

We can see how we pass along `Gameplay* gameplay` to `UpdateGame()` this means that the variables inside `TitleScreen` will not be accessible to this function as we have not passed along the complete `GameData* data`. This will help reduce complexity, improve readability and reduce the chance of us creating hard to understand bugs. I've also taken the time to make the `dt` (deltatime) parameter a `const` as we're not supposed to make any changes to it, just read its value.

Our `InitializeGame()` lives only inside our `game.cpp` and is called from our `Initialize()` function. Inside it we've just placed the gameplay specific code that previously lived inside `Initialize()`. This step was not strictly necessary but it will help with readability later in the project.

```
// game.cpp

void InitializeGame(Gameplay* gameplay, Arena* arena_levels){
    assert(gameplay->initialized == false);
    gameplay->currentLevelIndex = 1;
    CreateLevel(arena_levels, &gameplay->levels[0], "assets/levels/testLevel.tmj");
    CreateLevel(arena_levels, &gameplay->levels[1], "assets/levels/testLevel_box.tmj");
    gameplay->initialized = true;
}

void Initialize(GameData* data, SDL_Window* window, SDL_Renderer* renderer){
    DEV::Initialize(window, renderer);
    AssetManagement::LoadAllSprites(data->spriteBuffer, renderer);
    data->imGui_context = ImGui::GetCurrentContext();
    SDL_Texture* blackfade = GetSprite(SPRITE_ID:black_1x1,
↪ data->spriteBuffer)->texture;
    SDL_SetTextureBlendMode(blackfade, SDL_BLENDMODE_BLEND);
    InitializeGame(&data->scenes.gameplay, data->arena_levels);
    ChangeScene(data, SCENE_TYPES::GAME);
}
```

We'll be using a 1x1 size black pixel as our texture for our fade in/out from black. But due to it having no transparent pixels in the image itself SDL defaults to giving it `SDL_BLENDMODE_NONE`. Without us setting it to `SDL_BLENDMODE_BLEND` we won't be able to update its alpha value to make it transparent.

We're adding some new sprites, so first we'll add them as `SPRITE_IDS`, then we'll load them lastly we'll write the `GetSprite()` helper function. You'll find the required sprites in the `chapter 30 sprite assets.zip` file.

```

// spritelibrary.h
enum class SPRITE_ID{
    Fallback,
    Ground,
    Ground_alt,
    Wall,
    Rock,
    Demon,
    Medusa_Idle_Side,
    Medusa_Idle_Front,
    Medusa_Idle_Back,
    Golem,
    Siren,
    Dropshadow,
    titlescreen_background, // new
    black_1x1 // new
};

Sprite* GetSprite(SPRITE_ID sprite_id, Sprite* spriteBuffer); // new function

// spritelibrary.cpp
static const SpriteDataEntry all_sprite_data[] = {
    {SPRITE_ID::Fallback, FALLBACK_PATH,0,0},
    {SPRITE_ID::Wall, "assets/sprites/wall.png", 0, 0},
    {SPRITE_ID::Demon, "assets/sprites/player.png"},
    {SPRITE_ID::Rock, "assets/sprites/rock.png", 10, 20},
    {SPRITE_ID::Ground, "assets/sprites/ground.png", 0, 0},
    {SPRITE_ID::Ground_alt, "assets/sprites/ground_alt.png",0,0},
    {SPRITE_ID::Medusa_Idle_Side, "assets/sprites/medusa_idle_side.png", 12, 24},
    {SPRITE_ID::Medusa_Idle_Front, "assets/sprites/medusa_idle_front.png", 12, 24},
    {SPRITE_ID::Medusa_Idle_Back, "assets/sprites/medusa_idle_back.png", 12, 24},
    {SPRITE_ID::Dropshadow, "assets/sprites/dropshadow.png", 8, 8},
    {SPRITE_ID::black_1x1, "assets/sprites/1x1_black.png",0,0}, // new
    {SPRITE_ID::titlescreen_background, "assets/sprites/titlescreen.png",0,0} // new
};

Sprite* GetSprite(SPRITE_ID sprite_id, Sprite* spriteBuffer){
    return &spriteBuffer[(int)sprite_id];
}

```

We set the pivot of the new sprites to `0,0` as we do not want to shift them at all. The `GetSprite()` function is a one-liner and you could just as easily substitute and use the code directly. I am of two minds about these types of helper functions, but I've kept it as I often find students respond well to functions that help with contextualisation. The reason we can do this

simple lookup inside the `spriteBuffer` is because when we loaded the sprites we looped over them in `SPRITE_ID` order. Meaning that the `SPRITE_ID` with enum value `0` was put into `spriteBuffer[0]`.

Back in our `Initialize()` in `game.cpp` we grab the texture and update its `BLEND_MODE`. after that we call `InitializeGame()` and finally call `ChangeScene()`. We'll come back to `ChangeScene()` for now you can think of it as just setting our `current_scene` to the appropriate value.

Lets look at our `Update()` function that we call from `main.exe`

```
// game.cpp
// new Update() part 1

void Update(GameData* data, float dt){
    Gameplay* gameplay = &data->scenes.gameplay;
    TitleScreen* titlescreen = &data->scenes.titlescreen;
    EditorData* editorData = &data->editor_data;
    Transition* transition = &data->transition;

    if(KeyPressed(&data->input, SDL_SCANCODE_F2)){
        editorData->edit_level = !editorData->edit_level;
    }
    if(editorData->edit_level){ // old
        EDITOR::Update(&editorData->editor, &data->input, GetCurrentLevel(gameplay),
↪ gameplay->commandBuffer);
    }
    if(KeyPressed(&data->input, SDL_SCANCODE_5)){
        ChangeScene(data, SCENE_TYPES::TITLESCREEN);
        return;
    }

    // ... more code to follow
}
```

A lot of changes here. Lets break them down one by one

First we fetch pointer references to `Gameplay`, `TitleScreen`, `EditorData` and `Transition` to simplify passing their variables along. We update our old call sites to use the new way we find variables We also add a quick testbutton

5 to call `ChangeScene()` .

before moving forward we should look at our new `Transition` struct inside `gameState.h`

```
// gameState.h

struct Transition {
    enum States {
        Inactive,
        FadeTo,
        FadeFrom
    };
    States state;
    float fade_time_elapsed;
    float fade_time_duration = 1;
};
```

We've opted for one single struct that can handle both the fade-in and the fade-out. We also set `fade_duration` to have a default value of `1` . We'll be controlling the alpha of a black texture by comparing `time_elapsed` with `time_duration` . Note: `fade_time_elapsed` is a tiny bit excessive with the context that the variable lives inside `Transition` . If you want you can change the name of these to just `time_elapsed` and `fade_duration` . I'll keep the verbose versions.

The `States` enum helps us track what the Transition is supposed to be doing using simple `switch-statements` .

```

// game.cpp
// New Update() part 2

if(transition->state != Transition::Inactive){
    transition->fade_time_elapsed += dt;
    if(transition->fade_time_elapsed >= transition->fade_time_duration){
        transition->fade_time_elapsed = 0;
        switch (transition->state) {
            case Transition::Inactive:
                break;
            case Transition::FadeTo:
                transition->state = Transition::FadeFrom;
                break;
            case Transition::FadeFrom:
                transition->state = Transition::Inactive;
                break;
        }
    }
}

switch(data->scene_current){
case SCENE_TYPES::TITLES_SCREEN:
    UpdateTitlescreen(titlescreen, dt);
    if(AnyKeyPressed(&data->input)){
        if(transition->state == Transition::FadeTo || transition->state ==
           Transition::Inactive){
            ChangeScene(data, SCENE_TYPES::GAME);
        }
    }
    break;
case SCENE_TYPES::MAINMENU:
    break;
case SCENE_TYPES::GAME:
    UpdateGame(gameplay, &data->input, dt);
    break;
case SCENE_TYPES::CREDITS:
    break;
case SCENE_TYPES::NONE:
    assert(false);
    break;
}
}

```

We check if our `Transition` state is not `Inactive`. meaning that it is currently running a fade. If it is we

- 1) add `dt` to `time_elapsed` then if `time_elapsed` has reached our

`duration` we reset it and depending on the `State` of our `Transition` we either make the transition `Inactive` or transition from `FadeTo` to `FadeFrom`

after that we check which scene we're currently in and call the appropriate `Update` function.

`AnyKeyPressed()` is a new function that we need to add to `input.h/.cpp`.

```
// input.h
bool AnyKeyPressed(const Input* input);

// input.cpp
bool AnyKeyPressed(const Input *input){
    for (int i = 0; i < SDL_SCANCODE_COUNT; i++) {
        if(KeyPressed(input, (SDL_Scancode)i)){
            return true;
        }
    }
    return false;
}
```

It loops over the entire keyboard array and checks if any of the buttons were pressed that frame, if not it returns false.

Lets finally look at our `ChangeScene()` function

```
// game.h
void ChangeScene(GameData* data, SCENE_TYPES new_scene);
```



```
// game.cpp

void ChangeScene(GameData* data, SCENE_TYPES new_scene){
    assert(new_scene != data->scene_current);
    data->scene_previous = data->scene_current;
    data->scene_current = new_scene;
    data->transition.state = data->scene_previous == SCENE_TYPES::NONE ?
↪ Transition::FadeFrom : Transition::FadeTo;
    data->transition.fade_time_elapsed = 0;
    switch (data->scene_current) {
        case SCENE_TYPES::TITLES_SCREEN:
            data->transition.fade_time_duration = 1;
            break;
        case SCENE_TYPES::MAINMENU:
            break;
        case SCENE_TYPES::GAME:{
            data->transition.fade_time_duration = 0.5f;
            Gameplay* gameplay = &data->scenes.gameplay;
            assert(gameplay->initialized);
            StartLevel(gameplay, data->arena_commands, data->arena_entities);
            break;
        }
        case SCENE_TYPES::CREDITS:
            break;
        case SCENE_TYPES::NONE:
            assert(false);
            break;
    }
}
```

We assert that we didn't try and change the scene to the scene we were already in. This behaviour should never happen and we're fine with crashing the program at this point.

If we get past our `assert` then we know that `current` and `new` are different and we can then safely store the old version that `current` has at the moment in `previous` then update `current`.

We use our handy `?` operator to decide which Transition state to select based on if we are entering the first ever scene of the game whether or not the fade should instantly begin as fading out or if we should fade in first.

We reset `_time_elapsed` then depending on the scene we're entering we do

scene specific setups. We also `assert(false)` if we ever tried to change to `NONE`. a `(false)` assert will always crash our program.

We'll continue to add logic here as it becomes necessary.

`StartLevel()` is also a new function exclusive to `game.cpp` that does the following

```
// game.cpp
void StartLevel(Gameplay* gameplay, Arena* arena_commands, Arena* arena_entities){
    Reset(arena_commands);
    CreateEntities(&gameplay->levels[gameplay->currentLevelIndex], arena_entities);
}
```

We make sure we have no commands from a previous level sitting around in our `arena_command` then we create the entities for the level set by `currentLevelIndex`.

Our codebase is currently littered with error messages. All of these are due to the fact that we try and access our variables from `data` directly. These all require the same changes to begin working again.

- 1) we fetch a pointer to the struct that actually holds the variable
- 2) we substitute `data->` with `the_struct_we_fetched_in_step_01->`

This is simple but boring work. We can drastically speed up this process by making good use of the multi-caret editing from the previous chapter.

- 1) mark a block of code inside select-mode using `v`
- 2) press `s` to select based on the search phrase
- 3) press enter to finish selecting
- 4) make changes as normal
- 5) press `,` to remove all but the normal caret

There is very little in the way of creativity at this part of refactoring.

Next lets look at our `Draw()`

```
// game.cpp

void Draw(GameData* data, SDL_Renderer* renderer){
    DEV::PreDraw(data->ImGui_context);
    SDL_SetRenderDrawColor(renderer, 120, 70, 120, 255);
    SDL_RenderClear(renderer);

    switch(data->transition.state){
    case Transition::Inactive:
        DrawScene(data, data->scene_current, renderer);
        break;
    case Transition::FadeTo: {
        DrawScene(data, data->scene_previous, renderer);
        float alpha = data->transition.fade_time_elapsed /
            ↪ data->transition.fade_time_duration;
        RenderSprite_World(GetSprite(SPRITE_ID::black_1x1, data->spriteBuffer),
        ↪ renderer, &data->camera, 0, 0, SCREEN_WIDTH,alpha);
        break;
    }
    case Transition::FadeFrom:{
        DrawScene(data, data->scene_current, renderer);
        float alpha = 1 - data->transition.fade_time_elapsed /
            ↪ data->transition.fade_time_duration;
        RenderSprite_World(GetSprite(SPRITE_ID::black_1x1, data->spriteBuffer),
        ↪ renderer, &data->camera, 0, 0, SCREEN_WIDTH,alpha);
        break;
    }
    }

    DEV::Draw(data, renderer);
    SDL_RenderPresent(renderer);
}
```

We check the `state` of `transition` and depending on the current state we either just draw the current scene or we draw the previous-or-current scene along with an overlayed fade texture. We are actually grabbing our `1x1 black pixel` and scaling it up to be as alrge as our `SCREEN_WIDTH`. By doing this we ensure that it covers the entire screen. (at least for as long as our screen is wider than it is tall)

The `alpha` calculations inside `FadeTo` and `FadeFrom` are very similar,

except that for `FadeFrom` we take the value and we subtract it from `1`. Meaning that we start at 1 and go down towards zero, as opposed to counting up from zero.

Our `DrawScene()` takes the `scene_current` and draws the appropriate “stuff”

```
// game.cpp

void DrawScene(GameData* data, SCENE_TYPES scene, SDL_Renderer* renderer){
    switch(scene){
        case SCENE_TYPES::TITLES_SCREEN:{
            Sprite* background = GetSprite(SPRITE_ID::titlescreen_background,
↪ data->spriteBuffer);
            RenderSprite_World(background, renderer, &data->camera, 0, 0);
        }
        break;
        case SCENE_TYPES::MAINMENU:
        case SCENE_TYPES::GAME:
            RenderLevel(data, renderer);
            RenderEntities(data, renderer);
            break;
        case SCENE_TYPES::CREDITS:
            break;
        case SCENE_TYPES::NONE:
            assert(false);
            break;
    }
}
```

We can see how we’ve just lifted the `RenderLevel()` and `RenderEntities()` to the `GAME` case.

With these changes we can start our game from the titlescreen then press any key, watch the screen fade to black before putting us into gameplay!

30 Tilemap parsing

Graphics is very much a non-trivial part of game development. We'll be doing quite extensive refactoring to our codebase to work with a less fragile and more expressive output from Tiled.

You'll find a copy of the chapters `assets` folder in the course material named `chapter 31 assets.zip`. Replace your `assets` folder with this new one.

Inside you'll find a new `.tmj` file as well as a new `.tsj` file stored in a new subdirectory called `tilesets`.

I've also included `tiled project files chapter 31.zip`. This is the entire Tiled project used to produce the contents of the `assets` folder.

The goal of this chapter is to stop referencing ground and wall tiles directly by individual `SPRITE_IDS` and instead we grab a large `tileset` with multiple tiles all layed out next to each other. Then we select the appropriate tile to use based on the `uint16_t` id of our `cells[]` in `LevelData*`.

Right now we only have a single tilset `Dungeon` but this architecture is made to simplify the addition of more tilesets and more worlds down the line.

Previously we checked against the `ID` of a tile to figure out if we were allowed to walk on it. We did this with the naive `if-statement`

```
// example
if(cell_id == ID::GROUND){
    // ...
}
```

We are removing both our `ID::GROUND` and `ID::WALL` from our code. then we'll change the name of `ID` to `ENTITY_ID` as we are no longer storing IDs for our terrain. Now they are exclusive for our Entities. This will touch a lot

of our codebase. But at this point you should be familiar enough using Helix to search for, find and update these parts of the code once everything is set up in this chapter.

We can open Tiled project and inspect it. our `testing.tmx` is our level. It uses three tilesets to draw the level.

- 1) automap rules. This is a tileset from Tiled itself. It was used alongside a feature called `automapping` to help with level creation. The important thing to understand for us is that it is part of the project file.
- 2) `hell_of_a_time_dungeon_tileset`. Here we have all the tiles necessary to create our level. We have a few variations for walls but a lot of different ground tiles
- 3) Entities. This tileset has our `Medusa`, a `blue box` for `Demon` and our `Rock`. It should be noted that when we open `entities.tsx` we can find our three entities. In this chapter we will have a very brittle setup where the order of the entities in this `.tsx` file need to match the `ENTITY_ID` `enum` order. So if `Medusa` is the first `ENTITY_ID` then it also has to be the leftmost sprite in our `entities tileset`. We'll be creating a more robust connection between these at a later chapter.

Here we run across the fundamental increase in complexity as compared to what we did previously. We can no longer say that `ID 5` is for example `Ground` as we have many many tiles that represent Ground. The same goes for walls. We need a more robust way of categorising these. We are also interested potentially in having more behaviour associated with a tile besides whether or not we can walk on it. Additionally we can not even know for sure if `ID 5` is a ground tile or maybe a bush or a wall. This is because each tileset we add needs their own `ids` meaning that the first tileset starts from

0 then the next starts from the previous tilesets final id and goes from there.

We've decided to use Tiled in this project for two reasons

- 1) we want to work with JSON files
- 2) we want to show a pretty standard workflow for working with colleagues using other programs or file formats to produce content for our game. It would pretty simple to expand on our level editor to allow us to draw and create our levels right inside our game engine. But this would mean that we would need to take a whole detour into `tools programming`. This is the work of creating robust systems that anyone in the team can learn to use, demanding little to no programming knowledge. I mention this so your takeaway isn't that Tiled is some godsend software that we have no issues with.

Inside the project file `hell_of_a_time_dungeon_tileset.tsx` we have added what is called a custom property to each tile. This property labeled `walkable` is a bool that we set inside Tiled on a per tile basis to control if they should be walkable or not. When we export our `dungeon_tileset.tsj` file to our assets folder then this custom property will be stored inside alongside each `id`. We can reference this custom property when parsing our `.tsj` file in our game engine. the `.tsj` file is just a `JSON` file with a custom extension that Tiled has added. It's exactly the same as a normal JSON. The name is just for cataloguing.

We'll be setting up a `tilesetLibrary.h/.cpp` that will hook into our `AssetManagement` namespace and add a way for us to load each `tileset`

```

// tilesetlibrary.h

#pragma once
#include "Parsers/json.hpp"

using namespace nlohmann;

namespace Memory{
    struct Arena;
}

enum class TILESETS {
    NONE = 0,
    Dungeon = 1,
    COUNT = 2
};

struct Tileset{
    TILESETS type;
    bool* walkableBuffer;
};

struct TilesetDataEntry{
    TILESETS type;
    const char* path;
};

uint16_t GetLocalTileID(uint16_t id_global, const json& tmj_result);
uint16_t Get_Tileset_ID_Offset_From_Tilemap(int id_limit, const json& tmj_result);

namespace AssetManagement{
    void LoadAllTilesets(Tileset* tilesetBuffer, Memory::Arena* arena_images);
    void LoadTileset(TilesetDataEntry* entry, Tileset* tilesetBuffer, Memory::Arena*
        ↪ arena_images);
}

```

We forward declare our `Arena` struct so that this `.h` file is allowed to add it as a parameter to our functions. We could also `#include "arena.h"` but doing it this way will help avoid circular dependencies. But as we currently don't have any of those you can decide if you find the forward declaration hard to understand. In that case just substitute it for our normal include..

we also specify `using namespace nlohmann` this is the namespace holding our `Json parser` by adding this `using` we dont have to write `nlohmann::`

before being allowed to access the `json` struct.

We store our different tilesets in our `TILESETS` enum. Currently we just have `Dungeon` as well as some helpers values.

our `Tileset` struct is very basic. It knows its type then it stores an array of booleans. This array will live alongside our tileset's `IDs` to allow us to quickly and easily check if a tile is walkable.

`TilesetDataEntry` is a struct that we'll be manually filling out just as we did for `SpriteDataEntry`. We'll do this inside `tilesetLibrary.cpp`.

We also have two helper functions `GetLocalTileID()` and `Get_Tileset_ID_Offset_From_Tilemap()` (a bit of a mouthful...)

These are used to get a tiles ID/position within its own tileset, not caring about the fact that our `.tmj` might have had multiple tilesets used. This will help us find that the first tile in our tileset is a corner wall piece even if we normally can't know that based on the ID it was given inside our `.tmj`.

Then using our `AssetManagement` namespace we create two load functions. `LoadAllTilesets` will be calling `LoadTileset` once for each `TilesetDataEntry` that we've created inside `tilesetLibrary.cpp`

```
// tilesetLibrary.cpp
#include "Parsers/json.hpp"
#include "tilesetLibrary.h"
#include "arena.h"
#include <cassert>
#include "fstream"

using namespace nlhmann;
using namespace std;

static const TilesetDataEntry all_tilesets_data[]{
    {TILESETS::Dungeon, "assets/tilesets/dungeon_tileset.tsj"}
};
```

here we simplify accessing `nlohmann` and `std` then we set up our `static` array of `TileSetDataEntry`. Right now there is just one. But its contents is the `enum` as well as a `path` to the `.tsj` file holding our `tileset`. Remember, our `.tsj` file has both our `ids` in the tilesets local 0-count range and our `custom property` called `walkable` added inside `Tiled`.

```
// tilesetLibrary.cpp

uint16_t Get_Tileset_ID_Offset_From_Tilemap(int id_limit, const json& tmj_result){
    int highest_tilemap_start_id = 0;
    for (const json& tileset : tmj_result["tilesets"]) {
        int first_id = tileset["firstgid"].get<int>();
        if(first_id <= id_limit && first_id > highest_tilemap_start_id){
            highest_tilemap_start_id = first_id;
        }
    }
    return highest_tilemap_start_id;
}

uint16_t GetLocalTileID(uint16_t id_global, const json& tmj_result){
    return id_global - Get_Tileset_ID_Offset_From_Tilemap(id_global, tmj_result);
}
```

Our helper functions are a bit hard to parse since we are doing a bunch of things related to parsing a JSON file that we don't normally do for the rest of our codebase. The first thing you need to know is that our `.tmj` file has an array inside it called `tilesets` and each of these tilesets has what is called a `firstgid` this is the `id` of the first tile in that `tileset` but it's based on the tilesets that came before it.

// example

tileset_01 has 6 tiles and `firstgid` is 1

tileset_02 has 51 tiles and `firstgid` is 7

tileset_03 has 2 tiles and `firstgid` is 58

If the tilesets had been added in another order their `firstgid` values would also change.

`Get_Tileset_ID_Offset_From_Tilemap()` takes in a `cell id` as a parameter then finds the tileset with the highest `firstgid` that is still lower than the `id` we specified. This means that if we pass in `id 44` then we find that `firstgid` for `tileset_03` was too large and therefore the `id` doesn't belong to it. Then we have two tilesets left. One with `firstgid` of `1` and one with `7`. Both are lower than `id (44)` so we can safely select the highest one `tileset_02`. This allows us to find out the `firstgid` of the tileset that the tile belonged to by just specifying its `global id`.

`GetLocalTileID` takes the `id_global` and subtracts the `firstgid` that its own tileset uses to get the `id` that the tile has inside its own tileset only. inside `Get_Tileset_ID_Offset_From_Tilemap()` we use `const auto& tileset` it is very syntactically different from our normal code.

`const` means that we are not allowed to accidentally modify the values in `tileset`. `auto` means that we let our compiler figure out the `data type` of our tileset. We do this because the `nlohmann::json` parser uses more complex data structures behind the scenes to help us work with our JSON file. So we'll let the compiler handle the mapping. `&` put after `auto` will make the tileset be a reference to the data from `tmj_result` instead of creating a whole new copy of the data. The size of the data behind the scenes is pretty large, so we want to avoid creating a copy of it when there is no need.

This is not my favorite type of programming, but this syntax style will be contained to the parsing of json files for our game engine. If we can swallow the fact that we don't have full knowledge of the underlying data type hidden by `auto` and that as long as we know how to fetch data from the json using `["name_of_data"]` and `.get<variableType>()` then we can move

forward.

the strings `"tilesets"` and `"firstgid"` are the exact names that I found when opening the `.tmj` file in `Sublime Text` to inspect the data inside it.

```
// tilesetLibrary.cpp

namespace AssetManagement{
    void LoadAllTilesets(Tileset* tilesetBuffer, Memory::Arena* arena_images){
        for (TilesetDataEntry entry : all_tilesets_data) {
            LoadTileset(&entry, tilesetBuffer, arena_images);
        }
    }

    void LoadTileset(TilesetDataEntry* entry, Tileset* tilesetBuffer, Memory::Arena*
        ↪ arena_images){
        assert(entry->type != TILESETS::COUNT);
        assert(entry->type != TILESETS::NONE);

        Tileset* tileset = &tilesetBuffer[(int)entry->type];
        tileset->type = entry->type;
        ifstream stream(entry->path);
        auto jsonResult = json::parse(stream);
        int tile_count = jsonResult["tilecount"].get<int>();
        tileset->walkableBuffer = ALLOC_ARRAY(arena_images, bool, tile_count);

        auto& tiles = jsonResult["tiles"];

        for(const auto& tile : tiles){
            int tile_id = tile["id"].get<int>();

            for(const auto& tile_property : tile["properties"]){
                if(tile_property["name"] == "walkable"){
                    tileset->walkableBuffer[tile_id] = tile_property["value"].get<bool>();
                }
            }
        }
    }
}
```

`LoadAllTilesets()` loops over `TileDataEntry` array and calls `LoadTileset` for each of them. it also passes `Tileset* tilesetBuffer` that we will add to an pass in from `GameData` in `gameState.h`

`LoadTileset` then tries and find the file found at `entry->path` and uses

the `nlohmann json parser` to turn it from text into something we can use in code. We store the full parsed json data in `auto jsonResult`.

`tile_count` is fetched by taking our full json data and fetching the data held in `tilecount`. This is a number inside our `.tsj` file. Note. We are creating our tilesets from our `.tsj` file(s). Not from the actual level file which is our `.tmj`. The `.tsj` file is just our tiles and their `custom properties` exported from Tiled.

we allocate our `walkableBuffer` and make it as large as the amount of tiles in our tileset.

We then fetch the json array holding all our tiles by using `["tiles"]`. You can open our `.tsj` file in `Sublime Text` to learn what each array is called. A `JSON` file is so nice because it is human-readable.

We then use the same `const auto&` syntax to fetch each of the entries from the json array. We use `auto` everywhere when working with this parser so we can just ignore the underlying types.

Inside the `for-loop` we fetch the id of the tile then we loop over all properties stored on that tile. Currently we only store `walkable` but we can imagine having more attributes for a tile. When the property is `walkable` then we know we can read the value stored in that json array element to figure out if the tile is walkable or not. We assign the `walkableBuffer` array at that index to the parsed `value`. And as we know (because we've opened and inspected our `.tsj` file) the name was `walkable` the `type` was `bool` and the `value` was `true/false`. So using `.Get<bool>()` we on the `value` element we can get back `true` or `false`.

I will happily suggest spending extra time learning how to parse a JSON. But

as this step only happens during the Initialization of our program means that we know immediately if things have worked or not. And thankfully a LLM is a great help when figuring out how to parse a JSON like this. If you provide it the JSON file and what data you want to retrieve it will help you figure out how to get there.

So after `LoadTileset()` has run we have allocated a boolean array for the walkable parameter synced with the local id of each tile. We also assign the relevant `tileset* pointer` from `tilesetBuffer` to point to this data.

Lets look at `gameState.h` and add what we need to it

```
// gameState.h

struct GameData {
    // other variables hidden for clarity
    Tileset* tilesetBuffer;
```

That's it.

We'll start using the width/height of a tile in `tileset` space, meaning the actual size in pixels (not the scaled up size). We'll modify `common.h` to hold both a `_RAW` size and a `_SCALED` size.

```
// common.h

// other variables etc hidden for clarity

const int TILE_SIZE_PX_RAW = 16;
const int TILE_SIZE_PX_SCALED = TILE_SIZE_PX_RAW * UPSCALE_FACTOR;
```

I used `space+r` to rename `CELL_SIZE_PX` to `TILE_SIZE_PX_SCALED`. If I change the name using this renaming command it will automatically update throughout my codebase. I then add `TILE_SIZE_PX_RAW` and use it to calculate `_SCALED`.

In `entity.h` we're updating the `enum class ID` to

`enum class ENTITY_ID`. if we use `space+r` then it will similarly update throughout the codebase. We're also removing `GROUND` and `WALL` and reordering our enum to match the layout of entities from our `entities.tmx` file. (we'll create a more robust solution for this matching later)

```
// entity.h

enum class ENTITY_ID : uint8_t {
    MEDUSA = 0,
    DEMON = 1,
    ROCK = 2,
    SIREN = 3,
    GOLEM = 4,
};
```

We don't yet have the tiles in `entities.tmx` for `SIREN` and `GOLEM`. But we'll add these later so I'll keep them around.

With `ID::NONE/GROUND/WALL` removed we will get more than a few errors throughout our codebase as we previously had `switch-cases` for them. We're adding a bool `active` to our `Entity` struct to be used instead of our `ID == NONE` checks that we did earlier

```
// entity.h

struct Entity{
    ENTITY_ID id; // <---- name updated from `ID` to `ENTITY_ID`
    bool active; // <----- new
    Direction facing;
    int strength;
    int x;
    int y;
    int x_prev;
    int y_prev;
    float progress_01;
    Behaviour behaviour;
};
```

in `entity.cpp` we used to `assert` that `ID` was not `NONE`. We will change this to look at `active` instead

```
// entity.cpp
void InitializeBaseBehaviour(Entity* entity){
// assert(entity->id != ID::NONE); // old
assert(entity->active); // new
switch (entity->id) {
```

Now we need to make sure we actually load all (we just have one) tilesets from `game.cpp`

```
// game.cpp
void Initialize(GameData* data, SDL_Window* window, SDL_Renderer* renderer){
    DEV::Initialize(window, renderer);
    AssetManagement::LoadAllSprites(data->spriteBuffer, renderer);
    data->imGui_context = ImGui::GetCurrentContext();

    AssetManagement::LoadAllTilesets(data->tilesetBuffer, data->arena_images); // <---
    ↪ new

    SDL_Texture* blackfade = GetSprite(SPRITE_ID:black_1x1,
    ↪ data->spriteBuffer)->texture;
    SDL_SetTextureBlendMode(blackfade, SDL_BLENDMODE_BLEND);
    InitializeGame(&data->scenes.gameplay, data->arena_levels, data->tilesetBuffer); //
    ↪ new parameter
    ChangeScene(data, SCENE_TYPES::TITLES_SCREEN);
}
```

We also update our `InitializeGame()` to take our `tilesetBuffer` as a parameter

```
// game.cpp
void InitializeGame(Gameplay* gameplay, Arena* arena_levels, Tileset* tilesetBuffer){
    assert(gameplay->initialized == false);
    gameplay->currentLevelIndex = 0;
    CreateLevel(arena_levels, &gameplay->levels[0],
    ↪ &tilesetBuffer[(int)TILESETS::Dungeon], "assets/levels/testing.tmj");
    gameplay->initialized = true;
}
```

As you can see, our `CreateLevel()` now takes a `Tileset*` pointer as a parameter. This is the tileset the level is using to draw its contents.

`CreateLevel()` and `CreateEntities()` in `level.cpp` has changed a lot to work with our new tilemap logic.


```

// levels.cpp

void CreateLevel(Arena* arena, LevelData* level, Tileset* tileset, const char*
↪ level_name){
    fstream stream(level_name);
    auto result = json::parse(stream);

    bool found = false;
    vector<uint16_t> levelData;
    for (const auto& layer : result["layers"]) {
        if (layer["name"] == "level") {
            levelData = layer["data"].get<vector<uint16_t>>();
            found = true;
            break;
        }
    }
    assert(found);
    int first_non_zero_id = 0;
    for (int id : levelData) {
        if(id != 0){
            first_non_zero_id = id;
            break;
        }
    }
    int id_offset = Get_Tileset_ID_Offset_From_Tilemap(first_non_zero_id, result);

    level->w = result["width"].get<int>();
    level->h = result["height"].get<int>();
    level->level_path = level_name;
    level->tileset = tileset;
    level->cells = ALLOC_ARRAY(arena, uint16_t, level->w * level->h);
    for (int i = 0; i < level->w * level->h; i++) {
        int local_id = levelData[i] - id_offset;
        if(local_id < 0){
            local_id = 0;
        }
        level->cells[i] = local_id;
    }
}

```

We still parse a json file loaded from the specified path. But once we have that we need to find the actual layer in Tiled that we have used to draw our levels. In Tiled we currently have 3 layers

- blueprint
- level

- entities

To make sure we fetch our data from the correct place we loop over each layer until we find one named “level”. When we do we flip the `found` bool to `true` and collect all the `cell ids` stored in the “data” array.

Once again, these names are all just lifted from the `JSON` file. We open it in Sublime Text to easily inspect its contents.

we then loop over all `ids` until we find one that is not `0`. We will use this non-zero ID to get the `firstgid` offset of our tileset within our `.tmj` file.

Then we fetch the width and height without any changes compared to before the refactor.

The last change is that we assign `local_id` to the `level->cells[i]` by removing the `firstgid id_offset` from all numbers larger than 0. We only do this for non-zero values as 0 is not an actual tile ID but the info that there is nothing here. If we are not careful we could have created really odd behaviour with all zeroes suddenly becoming non-zero values.

You can see that we assign a pointer reference to `levelData` for `level->tileset` we have to update our `LevelData` struct to hold this pointer.

```
// levels.h

struct LevelData{
    int w;
    int h;
    uint16_t* cells; // updated from uint8_t to uint16_t
    const char* level_path;
    Entity* entityBuffer;
    int entityCount;
    const Tileset* tileset; // new
};

uint16_t GetCellID(LevelData* level ,int x, int y); // updated from uint8_t to uint16_t
```

We make sure the tileset is `const` as we are never going to want to adjust it when we pass it in alongside `LevelData` we have also updated `cells` to use `uint16_t` this is because `uint8_t` caps out at 255. And we could accidentally overshoot this value in a larger project with way more tiles. We also update our `GetCellID` in both `.h` and `.cpp` to match this new return value.

Lets look at `CreateEntities()` next

```

// levels.cpp

void CreateEntities(LevelData* lvl_data, Arena* arena){
    Reset(arena);
    lvl_data->entityCount = 0;
    lvl_data->entityBuffer = (Entity*)Memory::Allocate(arena, sizeof(Entity) * 256);

    fstream stream(lvl_data->level_path);
    auto result = json::parse(stream);

    vector<uint16_t> entities;
    bool found = false;
    for (const auto& layer : result["layers"]) {
        if (layer["name"] == "entities") {
            entities = layer["data"].get<vector<uint16_t>>();
            found = true;
            break;
        }
    }
    if(!found){
        return;
    }

    for (int i = 0; i < lvl_data->w * lvl_data->h; i++) {
        if(entities[i] == 0){
            continue;
        }
        uint16_t entity_id = GetLocalTileID(entities[i], result);
        int x = i % lvl_data->w;
        int y = i / lvl_data->w;
        AddEntity((ENTITY_ID)entity_id, x, y, lvl_data);
    }
}

```

We get the JSON file from the `level_path` that was stored during `CreateLevel()` then we search for the layer named “entities” instead of “level” as we did during the previous function. Once we have found that layer we can fetch all the entities from the data block called “data”. Similarly we flip the `found` bool to true.

But not similar to our `CreateLevel()` we don’t assert that `found` was true, instead we return early. I’ve opted for this as I want to be able to test that a tileset is working properly even if I have not added an entities layer inside

Tiled

Once we have our entities we go over each one and call `AddEntity()` as usual. But we make sure to fetch the tileset id using our helper function `GetLocalTileID()`. We could easily fetch the offset once and apply it to all the `ids` as we did in `CreateLevel()`. You're free to make this change if you want. But even rerunning this code for each entity has proven a non issue so far during testing.

Next its time to make `spriteLibrary.h/.cpp` have all the necessary info about a tileset in order to let us use it to render our level.

For now we're extending `Sprite` and `SpriteDataEntry` to hold variables related to the sprite being a tileset

```
// spriteLibrary.h

struct Sprite{
    SDL_Texture* texture;
    int width;
    int height;
    int pivot_x;
    int pivot_y;
    int tileset_cell_count_x;
    int tileset_cell_count_y;
};

const int NOT_SET = -1;
struct SpriteDataEntry{
    SPRITE_ID id;
    const char* path;
    int pivot_x = NOT_SET;
    int pivot_y = NOT_SET;
    int tileset_cell_count_x = NOT_SET;
    int tileset_cell_count_y = NOT_SET;
};
```

We might opt for having a different struct all together for our Tilesets later.

But for now I'm ok with expanding our `Sprite` struct.

We have also added our tilemap to our `SPRITE_ID` enum

```
// spriteLibrary.h

enum class SPRITE_ID{
    Fallback,
    // Ground, <- removed
    // Ground_alt, <- removed
    // Wall, <- removed
    Rock,
    Demon,
    Medusa_Idle_Side,
    Medusa_Idle_Front,
    Medusa_Idle_Back,
    Golem,
    Siren,
    Dropshadow,
    titlescreen_background,
    black_1x1,
    dungeon_tileset // new
};
```

Then we update our `spriteLibrary.cpp`

```
// spriteLibrary.cpp
static const SpriteDataEntry all_sprite_data[] = {
    {SPRITE_ID::Fallback, FALLBACK_PATH,0,0},
    {SPRITE_ID::Demon, "assets/sprites/player.png"},
    {SPRITE_ID::Rock, "assets/sprites/rock.png", 10, 20},
    {SPRITE_ID::Medusa_Idle_Side, "assets/sprites/medusa_idle_side.png", 12, 24},
    {SPRITE_ID::Medusa_Idle_Front, "assets/sprites/medusa_idle_front.png", 12, 24},
    {SPRITE_ID::Medusa_Idle_Back, "assets/sprites/medusa_idle_back.png", 12, 24},
    {SPRITE_ID::Dropshadow, "assets/sprites/dropshadow.png", 8, 8},
    {SPRITE_ID::black_1x1, "assets/sprites/1x1_black.png",0,0},
    {SPRITE_ID::titlescreen_background, "assets/sprites/titlescreen.png",0,0},
    {SPRITE_ID::dungeon_tileset, "assets/sprites/hell_of_a_time_dungeon_tileset.png",0,0,
        ↪ 9, 9} // new
};
```

because only `dungeon_tileset` is a tileset its the only one that has explicit values set for our new tileset variables. I counted the amount of rows and columns of the tileset by hand for this step. Later we might automate this as our `.tsj` file does know the amount of rows/columns it stores.

```
// spriteLibrary.cpp

void LoadSprite(Sprite* spriteBuffer, SpriteDataEntry entry, SDL_Renderer* renderer){
    SDL_Surface* surface = IMG_Load(entry.path);
    if(surface == nullptr){
        surface = IMG_Load(FALLBACK_PATH);
    }
    assert(surface != nullptr);
    SDL_Texture* texture = SDL_CreateTextureFromSurface(renderer, surface);
    Sprite* sprite = &spriteBuffer[(int)entry.id];
    sprite->texture = texture;
    sprite->height = texture->h;
    sprite->width = texture->w;
    if(entry.pivot_x == NOT_SET || entry.pivot_y == NOT_SET){
        sprite->pivot_x = sprite->width / 2;
        sprite->pivot_y = sprite->height / 2;
    }
    else{
        sprite->pivot_x = entry.pivot_x;
        sprite->pivot_y = entry.pivot_y;
    }

    sprite->tilesheet_cell_count_x = entry.tilesheet_cell_count_x; // new
    sprite->tilesheet_cell_count_y = entry.tilesheet_cell_count_y; // new

    SDL_DestroySurface(surface);
}
```

Then we update our `LoadSprite()` function to assign the tilesheet variables from the `SpriteDataEntry` to our `Sprite`.

in `TryMove()` in `game.cpp` and `RaycastFirstEntity()` in `Levels.cpp` we previously checked if we've reached a wall. Now that we no longer have an `ID::GROUND/WALL` to check against we need to instead use our `walkable` bool that we collected when we created our level.

In `Levels.h/.cpp` we'll add a small helper function

```
// levels.h
bool IsWalkable(int x, int y, LevelData* level);
```

```
// levels.cpp
bool IsWalkable(int x, int y, LevelData* level){
    uint16_t id = GetCellID(level, x, y);
    return level->tiles->>walkableBuffer[id];
}
```

because our `walkableBuffer` is in alignment with our local cell ids we can find the walkable status by just checking the array at the same index.

In `main.cpp` we need to allocate our `tiles->walkableBuffer`. And because we're adding it to `arena_images` we need to give it more memory as we previously had the memory footprint set to exactly the size needed to hold our `Sprite*` array.

```
// main.cpp
int SPRITE_COUNT = 256; // old
size_t IMAGE_ARENA_SIZE = MEGABYTES(1); // updated to a way too large size for now
gameData->arena_images = Memory::CreateSubArena(arena_main, IMAGE_ARENA_SIZE); // old
gameData->spriteBuffer = ALLOC_ARRAY(gameData->arena_images, Sprite, SPRITE_COUNT); //
↳ old
gameData->tiles->walkableBuffer = ALLOC_ARRAY(gameData->arena_images, Tiles,
↳ (int)TILES::COUNT); // new
```

We will need a new Render function to render our level using our `tiles` instead of individual `Sprite's`. We are going to use two `FRect`. One to tell SDL which box on the screen to draw our texture to. and the other what box/area within our `tiles` to fill the first `Rect` with. We're also removing `RenderSprite_Grid()` as after this refactor step no code is using it.

```
// rendering.h
void RenderTile_World(Sprite* tiles, int cell_id, LevelData* lvl, SDL_Renderer*
↳ renderer, const Camera* camera, float x, float y, float scale, float alpha);
```

we've added `cell_id` and removed `flipped` as compared to `RenderSprite_World()`

We'll be calling this function from `levelRenderer.cpp`


```

void RenderLevel(GameData* gameData, SDL_Renderer* renderer){
    Gameplay* gameplay = &gameData->scenes.gameplay;
    LevelData* level = &gameplay->levels[gameplay->currentLevelIndex];

    Sprite* tileset;
    switch(level->tileset->type){
        case TILESETS::Dungeon:
            tileset = GetSprite(SPRITE_ID::dungeon_tileset, gameData->spriteBuffer);
            break;
        case TILESETS::NONE:
        case TILESETS::COUNT:
            assert(false);
            break;
    }

    for(int x = 0; x < level->w; x++){
        for (int y = 0 ; y < level->h; y++) {
            uint16_t id = GetCellID(level, x, y);
            RenderTile_World(tileset, id, level, renderer, &gameData->camera, x, y, 1, 1);
        }
    }
}

```

Currently we `assert(false)` if the `TILESETS` enum value was wrong. We really should have this part of the code live inside our `tileset` struct or a helper function. But just to get things up and running we're doing the linking between the tileset enum and the necessary tileset sprite here.

We get the full tileset sprite then we pass it along with the id so we can calculate which tile to render from the grid that is the tileset.

Lets finally look at the render function inside `rendering.cpp` to see how we render the tiles of our level

```

void RenderTile_World(Sprite* tileset_atlas_sprite, int cell_id, LevelData* lvl,
↳ SDL_Renderer* renderer, const Camera* camera, float x, float y, float scale, float
↳ alpha){

    SDL_FRect tilesetRect;
    tilesetRect.w = TILE_SIZE_PX_RAW;
    tilesetRect.h = TILE_SIZE_PX_RAW;

    tilesetRect.x = (cell_id % tileset_atlas_sprite->tileset_cell_count_x) *
↳ TILE_SIZE_PX_RAW;
    tilesetRect.y = (cell_id / tileset_atlas_sprite->tileset_cell_count_x) *
↳ TILE_SIZE_PX_RAW;

    camera->GridToWorld(&x, &y, lvl);

    SDL_FRect rect;
    rect.x = x;
    rect.y = y;
    float final_scale = UPSCALE_FACTOR * scale;
    rect.h = TILE_SIZE_PX_RAW * final_scale;
    rect.w = TILE_SIZE_PX_RAW * final_scale;
    rect.x -= tileset_atlas_sprite->pivot_x * final_scale;
    rect.y -= tileset_atlas_sprite->pivot_y * final_scale;
    rect.x -= camera->camera_x;
    rect.y -= camera->camera_y;

    SDL_SetTextureScaleMode(tileset_atlas_sprite->texture, SDL_SCALEMODE_PIXELART);
    SDL_SetTextureAlphaModFloat(tileset_atlas_sprite->texture, alpha);
    SDL_RenderTexture(renderer, tileset_atlas_sprite->texture, &tilesetRect, &rect);
}

```

We create our `tilesetRect` then we set the width and height of the rect to the actual dimensions of a tile before any scaling is applied. We then find the x and y coordinate of the tile by doing our 1D to 2D transformation using modulo `%` and divided by `/`. We then multiply the coordinate by the size of a tile to slide our rect into position. This 16x16 area within our tilemap that we've calculated will be the pixels drawn into our `rect` created just below.

We convert our x and y coordinates to world space then we set up our destination rect as usual. Accounting for the pivot, camera, scale and global `UPSCALE_FACTOR`.

Finally we make sure to pass along `&tilesetRect` to our `SDL_RenderTexture` where we previously used `NULL`. Because `NULL` meant use the entire texture.

now, after this pretty intense chapter we are rewarded with some actually decent graphics to look at. And it makes such a difference! Now adding more tilesets and making changes to them in Tiled will be easy!

31 Sokoban Programming V

31.1 Control deltatime

We're going to continue working on the core of our game as we introduce some sprite animations, gameplay logic and refactoring to support our changes.

First, lets add a feature to allow us to speed up and slow down the entire game.

```
// gameState.h  
  
const float* dt;  
float* dt_scaler;
```

We will be multiplying `dt` with `dt_scaler` as we pass `dt` to our `.DLL` this will allow us to control the game speed. We'll use this to

- a) slow down the game to more easily check animations and other effects
- b) speed up the game to reach desired gamestates faster.

```
// main.cpp  
  
bool running = true;  
float dt;  
float dt_scaler = 1; // new  
gameData->dt = &dt;  
gameData->dt_scaler = &dt_scaler; // new
```

Then we modify `dt` in our boilerplate layer

```
void CalculateDeltaTime(float* dt, float scaler){  
    NOW = SDL_GetTicksNS();  
    *dt = NOW - PREV;  
    *dt = SDL_NS_TO_SECONDS(*dt);  
    *dt *= scaler; // new  
    PREV = NOW;  
}
```

Now lets add a slider to our `dev_gui.cpp`

```

// dev_gui.cpp

void DrawFPS(GameData* data){
    EditorData* editor = &data->editor_data;
    // here we need to multiply byu `dt_scaler` again or else we get the wrong numbers
    ↪ back
    editor->fps_buffer[editor->fps_buffer_index++] = 1.0 / *data->dt * *data->dt_scaler;
    editor->fps_buffer_index %= editor->fps_buffer_count;
    ImGui::PlotHistogram("fps", editor->fps_buffer, editor->fps_buffer_count,0,nullptr
    ↪ ,0,FPS, ImVec2(-1,35));
}

void DEV::Draw(GameData* data, SDL_Renderer* renderer){
    ImGui::Begin("Dev Tools");
    ImGui::Text("memory arena usage amount");

    Draw_ImGui_Arena_Usage(data->arena_main, "all memory");
    Draw_ImGui_Arena_Usage(data->arena_images, "images");
    Draw_ImGui_Arena_Usage(data->arena_levels, "levels");
    Draw_ImGui_Arena_Usage(data->arena_commands, "commands");
    Draw_ImGui_Arena_Usage(data->arena_entities, "entities");
    Draw_ImGui_Arena_Usage(data->arena_input, "input");
    Draw_ImGui_Arena_Usage(data->arena_scratch, "scratch"); // new

    DrawFPS(data);

    ImGui::SliderFloat("deltaTimeScaler", data->dt_scaler, 0.1, 3); // new

    ImGui::End();
}

```

With that we can change our game's speed with a simple slider

31.2 Select an active entity

Currently all of our entities that respond to inputs are allowed to move at the same time during a key press. This is good for some type of game, but not the one we're making. We want to press **X** to swap which entity is the one we're moving.

We're going to make use of our `arena_scratch` to set up a `pointer-pointer` array holding all relevant targets. We'll be recreating this array each frame rather than storing it alongside `entityBuffer` we're

doing this so that we never run the risk of having the two arrays drift out of sync by us forgetting to update one when we update the other.

```
struct Gameplay {
    CommandBuffer* commandBuffer;
    LevelData* levels;
    int levelCount;
    int currentLevelIndex;
    float undo_timer;
    Position* input_buffer;
    int input_buffer_capacity;
    int input_buffer_write_count;
    int input_buffer_read_count;
    bool initialized;

    int activePlayerIndex; // new
    Entity** activePlayerBuffer; // new
};
```

in `game.cpp` we'll find how many eligible entities we have then allocate that amount to our scratch arena.

```
// game.cpp

void UpdateGame(Gameplay* gameplay, Input* input, Arena* arena_scratch, const float
↳ dt){
    int player_count = 0;
    for (int i = 0; i < level->entityCount; i++) {
        if(entityBuffer[i].active == false){
            continue;
        }
        if(HasBehaviour(&level->entityBuffer[i], (Behaviour)(IS_PLAYER))){
            player_count++;
        }
    }

    int index = 0;
    gameplay->activePlayerBuffer = ALLOC_ARRAY(arena_scratch, Entity*, player_count);
    for (int i = 0; i < level->entityCount; i++) {
        if(entityBuffer[i].active == false){
            continue;
        }
        if(HasBehaviour(&level->entityBuffer[i], (Behaviour)(IS_PLAYER))){
            gameplay->activePlayerBuffer[index++] = &level->entityBuffer[i];
        }
    }
}
```

So this is an array that sits in sequence in memory (as all arrays do) but they point to pointers to entities that are not in an ordered sequence inside entityBuffer. Now each frame this array is recreated. Notice how we've added `Arena* arena_scratch` as a parameter to `UpdateGame`. So in `Update()` we must pass this along as well

```
// game.cpp  
// inside a switch-case inside Update()  
  
case SCENE_TYPES::GAME:  
    UpdateGame(gameplay, &data->input, data->arena_scratch, dt); // added parameter  
    break;
```

To hammer home the point, recreating this from “scratch” each frame means that there is no way that the data inside it could “go stale”. meaning that its referencing old data. It’s always automatically kept up to date.

Our goal now is to limit which character acts based on `activePlayerBuffer[activePlayerIndex]`. We are going to do a few things now.

- 1) create a command that shifts the `activePlayerIndex` forward
- 2) Push this command
- 3) Limit our top level TryMove() call to only work on this specific entity.

```
// command.h
struct SwapActiveEntityCommand : Command {
    int index_current;
    int index_previous;
    int* value_to_change;

    SwapActiveEntityCommand(int* activeEntityIndex, int limit){
        index_previous = *activeEntityIndex;
        index_current = *activeEntityIndex + 1;
        value_to_change = activeEntityIndex;
        index_current %= limit;
        type = CMD_TYPE::SWAP_ACTIVE;
    }
};
```

The `limit` is used to ensure that once we reach the end of our entity count we wrap back to 0 instead of going outside of the bounds of the array. This is also why we store `index_previous` as a separate value. It simplifies fetching the old value when we call `undo`. Though we could do some check to look at if we've reached 0 and wrap to limit during undo/execute I find this less appealing. Storing it inside our command is easy and simplifies the places where we use the command. the `value_to_change` is a bit more generic than necessary. But its a pointer that will point to the memory address of our `activeEntityIndex` so that we can modify it from `Execute()/Undo()`

Now we need to add the `SWAP_ACTIVE` enum to our list as well as creating a constructor for `AnyCommand` that accepts a `SwapActiveEntityCommand`. Though we have outlined this process multiple times in the course material. If you struggle with this step, return to earlier chapters on creating new commands and repeat those steps.


```
// game.cpp
if(KeyPressed(input, SDL_SCANCODE_X) && player_count > 0){
    SwapActiveEntityCommand swap(&gameplay->activePlayerIndex, player_count);
    Push(gameplay->commandBuffer, swap, GetCurrentLevel(gameplay));
    gameplay->commandBuffer->timestamp += 1;
}
```

so if we press **X** we create and push our swap command, then we progress **commandBuffer->timestamp** as we want to make sure that this command gets undone/redone in isolation and not part of other commands that comes after.

In **command.cpp** we add our **Execute()** case and **Undo** case. The setup is very similar to our other commands

```
// command.cpp
// Execute()
case CMD_TYPE::SWAP_ACTIVE:{
    SwapActiveEntityCommand* swap = &cmd.swap_active;
    *swap->value_to_change = swap->index_current;
    break;
}

// Undo()
case CMD_TYPE::SWAP_ACTIVE:{
    SwapActiveEntityCommand* swap = &cmd.swap_active;
    *swap->value_to_change = swap->index_previous;
    break;
}
```

Currently changing this pointer's value does absolutely nothing, but behind the scenes we can cycle between our player entities, which currently are all entities besides rocks...

Lets visualize our selected entity. To do this we'll actually go down a pretty deep rabbithole of refactoring. In the **Chapter 32 assets.zip** you'll find a new sprite **selection_marker.png** we'll put this ontop of our selected entity.

You can also see that we have deleted the `Medusa_Idle_side/front/back` and replaced them with a spritesheet called `medusa_rotate.png`. This is a sequence of sprites we'll use to create our first frame-by-frame animation. This will require some setup and to (eventually) make things easier we'll be refactoring our old naive rendering code.

```
// spriteLibrary.h

struct Sprite{
    SDL_Texture* texture;
    int width;
    int height;
    int pivot_x;
    int pivot_y;
    int sprite_count_x; // name changed
    int sprite_count_y; // name changed
};
```

we remove `tileset` from the name as we will use this for both tilesets and spritesheet animations.

```
// spriteLibrary.h

struct SpriteRenderInfo{
    Sprite* sprite;
    int frame;

    SpriteRenderInfo(){
        this->sprite = nullptr;
        this->frame = 0;
    }

    SpriteRenderInfo(int frame, Sprite* sprite){
        this->frame = frame;
        this->sprite = sprite;
    }

    SpriteRenderInfo(Sprite* sprite){
        this->sprite = sprite;
        this->frame = 0;
    }
};
```

Ok, this struct is a little strange. Mostly it just references a `Sprite` by

pointer. But it has a frame as well. We'll use the `frame` to get the appropriate sprite using our clever `1D to 2D` algorithm later.

We also have 3(!) constructors. One is the `default` constructor that accepts no parameters. when we start creating our own constructors we can no longer create one of these structs without passing some parameters along the compiler no longer creates a default constructor for us during compilation. By recreating this default constructor we get the ability to do so back.

The second constructor passes both the parameters and assigns them, this creates a fully formed `SpriteRenderInfo`. But with a constructor that accepts only a `Sprite*` we have actually created a way of passing a `Sprite*` as a parameter as a substitute for a full `SpriteRenderInfo`. This is really neat as this reduces the amount of code duplication and extra boilerplate we have to write. this is called an `implicit conversion constructor`

```
// spriteLibrary.cpp

static const SpriteDataEntry all_sprite_data[] = {
    // fallback pivot placed in the center due to later changes to rendering
    {SPRITE_ID::Fallback, FALLBACK_PATH,8,8},
    {SPRITE_ID::Demon, "assets/sprites/player.png"},
    {SPRITE_ID::Rock, "assets/sprites/rock.png", 10, 20},

    // replaces three old medusa elements
    {SPRITE_ID::Medusa_Rotate, "assets/sprites/medusa_rotate.png", 12, 24, 8, 1},
    {SPRITE_ID::Dropshadow, "assets/sprites/dropshadow.png", 8, 8},
    {SPRITE_ID::black_1x1, "assets/sprites/1x1_black.png",0,0},
    {SPRITE_ID::titlescreen_background, "assets/sprites/titlescreen.png",0,0},
    {SPRITE_ID::selection_marker, "assets/sprites/selection_marker.png",9,9},
    {SPRITE_ID::dungeon_tileset, "assets/sprites/hell_of_a_time_dungeon_tileset.png",0,0,
        ↪ 9, 9}
};
```

The Medusa rotate spritesheet has 8 frames layed out in a line, that's why we pass `8, 1` as the two last parameters. This is just as with `dungeon_tileset`. You should also cleanup `SPRITE_ID` and remove the old Medusa entries.

```
// spriteLibrary.h
```

```
Sprite* GetSprite(SPRITE_ID sprite_id, Sprite* spriteBuffer);  
SpriteRenderInfo GetSprite_FromEntityState(Entity* entity, Sprite* spritebuffer);
```

We're updating `GetSprite_FromEntityState` to return `SpriteRenderInfo` instead, this workhorse function will be responsible for picking the right frame of our animations based on the states of our entities. It will grow pretty huge pretty soon and we will for sure need to think about how we can manage its size. For this chapter we're going to be super messy and just ensure that the Medusa character works as she should.

Before we dive into `rendering.h/.cpp` we need to fix an issue we had that caused a bug during `Redo`. We need to place all `PostMove/PreRotate/PostRotate()` function calls inside an if-statement to make them only fire if we are not redoing our command. We're also going to be a bit more defensive with our `fromRedo` parameter as we can accidentally pass another variable by mistake and many variables can "decay" into bools. Meaning that they get converted to `true` if they are for example not a `nullptr`.

```

// command.cpp
enum class FromRedo {No, Yes};

void Execute(AnyCommand cmd, LevelData* level, CommandBuffer* commandBuffer, FromRedo
↪ fromRedo = FromRedo::No){
    switch(cmd.command.type){
        case CMD_TYPE::NONE:
            break;
        case CMD_TYPE::MOVE: {
            MoveCommand mv = cmd.move;
            mv.entity->x_prev = mv.entity->x;
            mv.entity->y_prev = mv.entity->y;
            mv.entity->x += mv.xDir;
            mv.entity->y += mv.yDir;
            if(fromRedo == FromRedo::Yes){
                mv.entity->progress_01 = 1;
            }
            mv.entity->action = Actions::MOVING;
            if(fromRedo == FromRedo::No){
                PostMove(mv.entity, level, commandBuffer);
            }
            break;
        }
        case CMD_TYPE::ROTATE:{
            RotateCommand* rotate = &cmd.rotate;
            if(!HasBehaviour(rotate->entity, CAN_ROTATE)){
                break;
            }
            if(fromRedo == FromRedo::Yes){
                rotate->entity->progress_01 = 1;
            }
            rotate->entity->action = Actions::ROTATING;
            if(fromRedo == FromRedo::No){
                PreRotation(rotate->entity, level, commandBuffer, rotate->from, rotate->to);
            }
            rotate->entity->facing_previous = rotate->from;
            rotate->entity->facing_current = rotate->to;
            if(fromRedo == FromRedo::No){
                PostRotation(rotate->entity, level, commandBuffer, rotate->from, rotate->to);
            }
            break;
        }
    }
    // other cases hidden for brevity

```

our `FromRedo` enum now forces us to pass it explicitly fixing the issue where a bool could decay. Note how each `PostMove` and `Rotation` call is held inside a `if FromRedo::No` and that our `progress_01 = 1` only happens on a

`FromRedo::Yes` .

We are also storing the `facing_previous` direction inside our `Entity` now.

We'll use it to help with animations later. This means that the old `facing` has been renamed to `facing_current` .

```
// Entity.h

struct Entity{
    Actions action;
    ENTITY_ID id;
    bool active;
    Direction facing_current; // renamed
    Direction facing_previous; // new
    int strength;
    int x;
    int y;
    int x_prev;
    int y_prev;
    float progress_01;
    Behaviour behaviour;
};
```

You can also see that we assign `entity->action` to `MOVING/ROTATING` depending on the command.

```

// entity.h
enum class Actions { // new
    NONE = 0,
    MOVING = 1,
    ROTATING = 2
};

struct Entity{
    Actions action; // new
    ENTITY_ID id;
    bool active;
    Direction facing_current;
    Direction facing_previous;
    int strength;
    int x;
    int y;
    int x_prev;
    int y_prev;
    float progress_01;
    Behaviour behaviour;
};

```

each `Entity` has an action enum variable we can assign and query against in other code. On its own this does nothing. But it tracks the status of the entity.

We also set `entity->action` to `NONE` during `AddEntity()` inside `levels.cpp`.

```
// levels.cpp

void AddEntity(ENTITY_ID entity_id, int x, int y, LevelData *level){
    Entity* entity = GetEntity(level, x, y);

    if(entity == nullptr){
        entity = GetNextAvailableEntity(level);
    }

    entity->active = true;
    entity->x = x;
    entity->y = y;
    entity->x_prev = x;
    entity->y_prev = y;
    entity->id = entity_id;
    entity->action = Actions::NONE; // new
    InitializeBaseBehaviour(entity);
}
```

our `rendering.h/.cpp` is getting a facelift. We're selecting better function names and removing a few functions as we can consolidate our calls down to three functions in total.

```
// rendering.h
// parameters layed out in three rows for clarity
void RenderTile(Sprite* tileset, int cell_id, LevelData* level,
                SDL_Renderer* renderer, const Camera* camera,
                float x, float y, float scale, float alpha);

void RenderSprite_World(SpriteRenderInfo tileset, SDL_Renderer* renderer,
                        const Camera* camera, float x, float y,
                        float scale = 1, float alpha = 1, bool flipped = false);

void RenderSprite_OnTile(SpriteRenderInfo spriteInfo, LevelData* level,
                        SDL_Renderer* renderer, const Camera* camera, float x,
                        float y, float scale = 1, float alpha = 1, bool flipped =
                        ↪ false);
```

these are our three rendering functions. Eventually all rendering calls go to `RenderSprite_World`.

You can also see how we use `SpriteRenderInfo` instead of `Sprite*`. As we allow a `Sprite*` to degrade into a `SpriteRenderInfo` with our third constructor made earlier we have opted for maximum clarity in the case of

`RenderTile`. Forcing us to specify the `cell_id` each time it's called.

`RenderTile` this accepts a Sprite with more than one tile inside it and a 1D `cell_id` that is then remapped to the correct spot.

`RenderSprite_OnTile` makes sure to offset the rendered entity correctly to make its origin correct.

```
// rendering.cpp

void RenderTile(Sprite* tileset, int cell_id /* other parameters hidden for brevity */){
    camera::GridToWorld(&x, &y, level);
    RenderSprite_World({cell_id, tileset}, renderer, camera, x, y, scale, alpha, false);
}

void RenderSprite_OnTile(/* parameters hidden for brevity */){
    camera::GridToWorld(&x, &y, level);
    x += TILE_SIZE_PX_SCALED / 2.0;
    y += TILE_SIZE_PX_SCALED / 2.0;
    RenderSprite_World(spriteInfo, renderer, camera, x, y, scale, alpha, flipped);
}
```

So both these functions call into `RenderSprite_World` but they modify `x` and `y` in different ways. We can also see how `RenderTile` constructs the `SpriteRenderInfo` using the shorthand `{}` and passes both `cell_id` and `tileset` into it.

ok, now we're going to look at the pretty large `RenderSprite_World()` function. I've added comments to break up the code into blocks

```

// rendering.cpp

void RenderSprite_World(SpriteRenderInfo spriteRenderInfo /* other parameters hidden for
↳ brevity */){
    // fetch some variables to make using them take less characters
    int frame = spriteRenderInfo.frame;
    Sprite* sprite = spriteRenderInfo.sprite;

    // Check if we are working with a tileset/spritesheet by calling `GetSpriteCount()` a
    ↳ new function
    SDL_FRect tilesetRect;
    if(GetSpriteCount(sprite) > 1){
        int width = sprite->width / sprite->sprite_count_x;
        int height = sprite->height / sprite->sprite_count_y;
        tilesetRect.w = width;
        tilesetRect.h = height;
        // 1D to 2D conversion of the frame to grid-space.
        tilesetRect.x = (frame % sprite->sprite_count_x) * width;
        tilesetRect.y = (frame / sprite->sprite_count_x) * height;
    }
    else{
        tilesetRect.w = sprite->width;
        tilesetRect.h = sprite->height;
        tilesetRect.x = 0;
        tilesetRect.y = 0;
    }

    // the usual offset based on size, pivot and camera
    SDL_FRect rect;
    rect.x = x;
    rect.y = y;
    float final_scale = UPSCALE_FACTOR * scale;
    rect.h = tilesetRect.w * final_scale;
    rect.w = tilesetRect.h * final_scale;
    rect.x -= sprite->pivot_x * final_scale;
    rect.y -= sprite->pivot_y * final_scale;
    rect.x -= camera->camera_x;
    rect.y -= camera->camera_y;

    SDL_SetTextureScaleMode(sprite->texture, SDL_SCALEMODE_PIXELART);
    SDL_SetTextureAlphaModFloat(sprite->texture, alpha);
    // made a variable for SDL_FlipMode to make the function call below shorter.
    SDL_FlipMode flip = flipped ? SDL_FlipMode::SDL_FLIP_HORIZONTAL :
↳ SDL_FlipMode::SDL_FLIP_NONE;
    SDL_RenderTextureRotated(renderer, sprite->texture, &tilesetRect, &rect, 0, 0, flip);
}

```

`GetSpriteCount()` is a new helper function in `spriteLibrary.h/`

```
// spriteLibrary.h

inline int GetSpriteCount(Sprite* sprite){
    if(sprite->sprite_count_x == NOT_SET) return 1;
    if(sprite->sprite_count_y == NOT_SET) return 1;
    return sprite->sprite_count_x * sprite->sprite_count_y;
}
```

I've inlined the function but if you find this weird you can always declare it in the .h file and then write the code in `spriteLibrary.cpp`.

The `RenderSprite_World` function has to do quite a lot, but its mostly just math. All parts of this function have existed in previous functions, we have just collected them into one.

Now we have to update our `levelRenderer.cpp` so it correctly uses these functions

```
// levelRenderer.cpp

void RenderLevel(GameData* gameData, SDL_Renderer* renderer){
    Gameplay* gameplay = &gameData->scenes.gameplay;
    LevelData* level = &gameplay->levels[gameplay->currentLevelIndex];

    Sprite* sprite;
    switch(level->tileset->type){
        case TILESETS::Dungeon:
            sprite = GetSprite(SPRITE_ID::dungeon_tileset, gameData->spriteBuffer);
            break;
        case TILESETS::NONE:
        case TILESETS::COUNT:
            assert(false);
            break;
    }

    for(int x = 0; x < level->w; x++){
        for (int y = 0 ; y < level->h; y++) {
            uint16_t id = GetCellID(level, x, y);
            RenderTile(sprite, id, level, renderer, &gameData->camera, x, y, 1, 1);
        }
    }
}
```

from `RenderLevel()` we call `RenderTile()`

```

// levelRenderer.cpp

void RenderEntities(GameData* data, SDL_Renderer* renderer){
    LevelData* lvl =
    ↪ &data->scenes.gameplay.levels[data->scenes.gameplay.currentLevelIndex];

    Entity** SortedEntities = ALLOC_ARRAY(data->arena_scratch, Entity*, lvl->entityCount);
    for (int i = 0; i < lvl->entityCount; i++) {
        SortedEntities[i] = &lvl->entityBuffer[i];
    }

    std::sort(SortedEntities, SortedEntities + lvl->entityCount,
    ↪ IsEntityBelowOtherEntity);

    Gameplay* gameplay = &data->scenes.gameplay;
    Entity* activeEntity = gameplay->activePlayerBuffer[gameplay->activePlayerIndex];

    for (int i = 0; i < lvl->entityCount; i++) {
        Entity* entity = SortedEntities[i];
        if(entity->active == false){
            continue;
        }
        SpriteRenderInfo sprite = GetSprite_FromEntityState(entity, data->spriteBuffer);
        if(HasBehaviour(entity, Behaviour::IS_PETRIFIED)){
            sprite = GetSprite(SPRITE_ID::Rock, data->spriteBuffer);
        }
        float x_animated = std::lerp(entity->x_prev, entity->x, entity->progress_01);
        float y_animated = std::lerp(entity->y_prev, entity->y, entity->progress_01);
        float ground_y = y_animated;
        if(entity->action == Actions::MOVING && HasBehaviour(entity, Behaviour::JUMPS) &&
        ↪ !HasBehaviour(entity, Behaviour::IS_PUSHING)){
            y_animated -= 0.5 * sinf(entity->progress_01 * 3.14);
        }

        Sprite* dropshadow = &data->spriteBuffer[(int)SPRITE_ID::Dropshadow];

        RenderSprite_OnTile(dropshadow, lvl, renderer, &data->camera, x_animated, ground_y,
    ↪ 1, 0.4, false);
        if(entity == activeEntity){
            SpriteRenderInfo selection_marker = GetSprite(SPRITE_ID::selection_marker,
    ↪ data->spriteBuffer);
            RenderSprite_OnTile(selection_marker, lvl, renderer, &data->camera, x_animated,
    ↪ ground_y);
        }
        RenderSprite_OnTile(sprite, lvl, renderer, &data->camera, x_animated, y_animated, 1,
    ↪ 1, false);
    }
}

```

This function is also growing longer. At the moment it's a non-issue, but with a few more edge-cases we would want to do something about it.

`GetSprite_FromEntityState()` now returns a `SpriteRenderInfo`. We also check if the entity we're rendering is the `ActiveEntity`. This is also

We have also added a `Actions` enum to our `Entity` struct. We'll be using this to help us simplify asking questions about what the Entity is doing. We'll look at how this is handled soon.

by storing `activeEntity` we can easily check if the currently rendered entity is the `activeEntity`. And if it is we go ahead and Render the `selection_marker` between the dropshadow and the entity itself. We use the `ground_y` so the selection stays on the ground. `ground_y` is just the old `dropshadow_y` that I have renamed.

for the dropshadow, selection marker and the entities themselves we use `RenderSprite_OnTile` and we sometimes pass just a `Sprite*` to the function (that converts to `SpriteRenderInfo`) and other times we pass the fully qualified `SpriteRenderInfo` from `GetSprite_FromEntityState()`

In `Game.cpp` we're doing 2 things:

- 1) forcing the game to wait to move an entity until they have finished rotating.
- 2) only allow the `activeEntity` to Rotate and TryMove

```

bool are_entities_acting = false;
LevelData *level = GetCurrentLevel(gameplay);
Entity* entityBuffer = level->entityBuffer;

for (int i = 0; i < level->entityCount; i++){
    if(IsActing(&entityBuffer[i])){
        are_entities_acting = true;
        break;
    }
}

for (int i = 0; i < level->entityCount; i++){
    Entity* entity = &entityBuffer[i];
    if(!entity->active) continue;
    switch(entity->action){
        case Actions::NONE:
            continue;
        case Actions::MOVING:
            entity->progress_01 += MOVE_SPEED * dt;
            break;
        case Actions::ROTATING:
            entity->progress_01 += 8 * dt;
            break;
    }
}

for (int i = 0; i < level->entityCount; i++){
    Entity* entity = &entityBuffer[i];
    if(entity->progress_01 >= 1){
        entity->x_prev = entity->x;
        entity->y_prev = entity->y;
        entity->facing_previous = entity->facing_current;
        entity->action = Actions::NONE;
        entity->progress_01 = 0;
        if(HasBehaviour(entity, Behaviour::IS_PUSHING)){
            RemoveBehaviour(entity, Behaviour::IS_PUSHING);
        }
    }
}

```

we've updated `are_entities_moving` to a more general `_acting`. We've also added a new function `IsActing()` to `entity.h/.cpp` to make checking their `acting` status easier.

```

//entity.h
bool IsActing(Entity* e);

// entity.cpp
bool IsActing(Entity* e){
    if(e->active == false) return false;
    return e->action != Actions::NONE;
}

```

We loop over each entity and checking the `Actions` enum they are using we control how fast `progress_01` should advance. We then loop over all of them again and makes sure that we have no stale references to old positions/rotations and reset `progress_01` and the relevant action to `NONE`. As this happens before any input has been parsed we are yet to update `current` to a new value. We have also moved the `IS_PUSHING` reset code from our `for-loop` that calls `TryMove()`.

```

// game.cpp
// at the end of UpdateGame()

    if(are_entities_acting){
        return;
    }
    if(gameplay->input_buffer_read_count == gameplay->input_buffer_write_count){
        return;
    }

    Entity* entity = GetActiveEntity(gameplay);

    if(!HasBehaviour(entity, (Behaviour)(RESPOND_TO_INPUT | CAN_MOVE))){
        return;
    }

    if(HasBehaviour(entity, Behaviour::IS_PETRIFIED)){
        return;
    }

    int xDir = gameplay->input_buffer[gameplay->input_buffer_read_count %
        ↳ gameplay->input_buffer_capacity].x;
    int yDir = gameplay->input_buffer[gameplay->input_buffer_read_count %
        ↳ gameplay->input_buffer_capacity].y;

    Direction new_facing = DirectionFromXY(xDir, yDir);
    if(new_facing != entity->facing_current){
        RotateCommand rotate(entity, entity->facing_current, new_facing);
        Push(gameplay->commandBuffer, rotate, level);
        return;
    }

    if(!IsActing(entity)){
        TryMove(entity, level, gameplay->commandBuffer, xDir, yDir, entity->strength);
        gameplay->commandBuffer->timestamp += 1;
        gameplay->input_buffer_read_count++;
    }

```

The biggest win of actually setting up the gameplay code we want to use is that we could entirely remove the old `for-loop` that iterated over all entities. Now that we are only interested in the `activeEntity` we can just fetch a pointer to it using our helper function `GetActiveEntity()`. Then we can check if it is supposed to rotate, and if yes we push a `RotateCommand` onto the stack. This will set `entity->action` to `Actions::Rotate`.

We have more than a few `if-statements` that we evaluate. And if the

condition is met we `return` early. Meaning that no code below it runs. This is the same as nesting `if-statements` inside each other but instead we have flipped the question to instead of allowing us in we keep the execution out with our `return` therefore avoiding the Inception style statement within statements.

It also has the added benefit of making the very (very) much easier to read and the flow is easier to understand at a glance.

By gating `TryMove()` behind an `IsActing()` we only progress the `input_buffer` if all previous actions have been resolved. Fair warning. There is some code-smell with the way Actions are set up. Part of me wishes to get rid of the entire enum. and work to clarify and make the input reading more robust. This might happen in a later chapter.

Our `GetSprite_FromEntityState()` function will require a large overhaul as we add more spritesheets and sprites for entities. Right now we're just getting the minimal code made to showcase the rotation animation of our Medusa character. Also, the math to select what frames of our looping rotate animation to pick based on `progress_01` took a fair bit of trial and error. It's not the easiest to wrap your mind around but as the case is very narrow it's not a bad idea to talk through the code with an LLM.

```

// spriteLibrary.cpp
// Part 1
SpriteRenderInfo GetSprite_FromEntityState(Entity* entity, Sprite* spritebuffer){
    if(HasBehaviour(entity, Behaviour::IS_PETRIFIED)){
        return GetSprite(SPRITE_ID::Rock, spritebuffer);
    }

    if(entity->id == ENTITY_ID::MEDUSA && entity->action == Actions::ROTATING){
        Sprite* spritesheet = GetSprite(SPRITE_ID::Medusa_Rotate, spritebuffer);

        int start = 0;
        int end = 0;
        switch(entity->facing_previous){
            case Direction::RIGHT:
                start = 6;
                break;
            case Direction::LEFT:
                start = 2;
                break;
            case Direction::UP:
                start = 4;
                break;
            case Direction::DOWN:
                start = 0;
                break;
        }

        switch(entity->facing_current){
            case Direction::RIGHT:
                end = 6;
                break;
            case Direction::LEFT:
                end = 2;
                break;
            case Direction::UP:
                end = 4;
                break;
            case Direction::DOWN:
                end = 0;
                break;
        }
    }
}

```

our first if statement just checks if we're petrified and returns the rock sprite. This will be modified to a cooler looking thing later. Then we do our brutalistic check that is exclusive to a rotating Medusa. We fetch the frame index for the final poses based on the layout of our `medusa_rotate.png`. We do this for

both `_current` and `_previous` facing directions as we want to interpolate between them.

```
// spriteLibrary.cpp  
// part 2  
  
int sprite_count = GetSpriteCount(spriteSheet);  
int forward = ((end - start) % sprite_count + sprite_count) % sprite_count;  
int backward = sprite_count - forward;  
end = (forward <= backward) ? (start + forward) : (start - backward);  
int current_frame = ((int)lerp(start, end, entity->progress_01) % sprite_count);  
return {current_frame, spriteSheet};  
}
```

Next we get how many sprites the full spriteSheet is then we do some fancy math to get the distance between the two selected frames going both in a forward direction and a backward direction. We then either set end to the start position plus the difference or minus the difference. The reason we have to do this is because we want our `end` to potentially go beyond our `sprite_count` as that is needed to loop over the right-side edge of the sheet back to frame 0. We then make a lerp that goes between our start and end based on `progress_01` and because our `end` can exist outside of our spriteSheet bounds we need to `modulo` it against our `sprite_count`.

```

//spriteLibrary.cpp
// part 3

switch (entity->id) {
    case ENTITY_ID::MEDUSA:{
        Sprite* sprite = GetSprite(SPRITE_ID::Medusa_Rotate, spritebuffer);
        switch (entity->facing_current) {
            case Direction::RIGHT:
                return {6, sprite};
                break;
            case Direction::LEFT:
                return {2, sprite};
                break;
            case Direction::DOWN:
                return {0, sprite};
                break;
            case Direction::UP:
                return {4, sprite};
                break;
        }
    }
    case ENTITY_ID::DEMON:
        return GetSprite(SPRITE_ID::Demon, spritebuffer);
    case ENTITY_ID::ROCK:
        return GetSprite(SPRITE_ID::Rock, spritebuffer);
    default:
        return GetSprite(SPRITE_ID::Fallback, spritebuffer);
}
}

```

Then if we weren't rotating we check if we were in fact Medusa but just not rotating. In that case we just return the frame that points in the right direction. If we were not Medusa we switch-case for two other Entities. And if we reach **default** we will display our **fallback** letting us know that we have cases missing for some new entity.

With all of this done we now have spritesheet animations for medusa rotating implemented as well as a delay that makes her rotate before jumping to her next position. But we also have the boilerplate in place to help us animate more entities later. we have also added gameplay logic to handle selecting which entity to use and we have simplified some code.

A lot of systems touched each other in this chapter! Good job getting through this one!

32 Buttons Part I

We're going to create the skeleton of a main menu in this chapter. We'll need buttons, a way to render them and the logic to allow us to press them.

Our `GameData` struct and the variables inside `gameState.h` have started to grow. And our refactoring step of putting scene specific variables in their own struct did help I want to go further. So at this stage, as we're adding the variables for our main menu, we'll be putting them inside its own `.h/.cpp` file. So to start with lets set up `mainmenu.h/.cpp`

```
//mainmenu.h

#pragma once

struct GameData;
struct SDL_Renderer;
struct Button;
struct Sprite;
namespace Memory{
    struct Arena;
}
```

We are including `mainmenu.h` in `gameState.h` and if we were to include `gameState.h` inside `mainmenu.h` we would get a chain of circular dependencies. To fix this we're `forward declaring` all the structs we'll be using in this file.

```
//mainmenu.h

struct MainMenu {
    Button* buttons;
    int button_count;
    int activeButtonIndex;
    Button** activeButtons;
    int activeButtonCount;
    bool initialized;
};

void InitializeMenu(MainMenu* mainmenu, Sprite* spriteBuffer, Memory::Arena*
    ↪ arena_main);
void UpdateMenu(GameData* data);
void DrawMenu(MainMenu* mainmenu, SDL_Renderer* renderer, Sprite* spriteBuffer);
```

`buttons` is an array with the size of `button_count`. `activeButtonIndex` will be used to control how our buttons are rendered and to make sure we press the correct button. our `activeButtons**` pointer pointer array will be a subset of `buttons*` that are all `is_active` set to `true`. `initialized` will be set during `InitializeMenu` and an `assert` will make sure we don't reinitialize our menu.

Right now our main menu is very similar to our titlescreen, with the addition of a new type of struct `Button`. We'll look at `button.h` right after this. Our main menu has `Initialize`, `Update` and `Draw` we'll be calling these when appropriate from `game.cpp`

```
// button.h
#pragma once
#include "SDL3/SDL_rect.h"
#include "SDL3/SDL_render.h"

struct Sprite;
struct GameData;
```

We do some normal inclusion of SDL3 headers and forward declare `Sprite` and `GameData`. To be honest I don't think we necessarily need to forward declare in this case.

```
// button.h

enum class ButtonMode {
    Centered,
    Raw
};

enum class ButtonType {
    NONE,
    START_GAME,
    QUIT
};

struct Button {
    ButtonType type;
    SDL_FRect rect;
    SDL_Texture* texture;
    bool is_active;
};
```

We create our `Button` struct, it has a type, a rectangle to be drawn in and a pointer to a Texture to render to the screen. `is_active` will allow us to add buttons to a scene without having them display until we're ready.

```
// button.h

void PressButton(Button* button, GameData* data);
int GetActiveButtonCount(Button* buttons, int count);
bool IsHoveredOver(Button* button, float x, float y);
void SetupButton(Button* button, ButtonType type, Sprite* spriteBuffer, SDL_FRect rect,
    ↪ ButtonMode mode);
```

We'll call `SetupButton` when we create a button. `IsHoveredOver` checks the current mouse position and does some basic collision bounds detection. `GetActiveButtonCount` is a small helper function to loop over all buttons and return the total count of buttons with `is_active` set to `true`.

We'll be checking our collision from `collision.h`. Our bounds collision detection is the most basic type of collision we can detect


```
// collision.h
#pragma once
#include "SDL3/SDL_rect.h"

bool CheckCollisionInsideBounds(SDL_FRect bounds, float x, float y);
```

Lets look at the implementation of this function

```
// collision.cpp
#include "collision.h"
bool CheckCollisionInsideBounds(SDL_FRect bounds, float x, float y){
    if(x > bounds.x + bounds.w) return false;
    if(x < bounds.x) return false;
    if(y > bounds.y + bounds.h) return false;
    if(y < bounds.y) return false;
    return true;
}
```

by checking the `x` and `y` positions and returning `false` if any of them are outside of the rectangles area we've implemented collision detection between a point in space and a rectangle! To check collision between two rectangles instead we use something called `AABB collision detection` but we wont be needing that for this game. Just wanted you to have heard about it.

```
// button.cpp

#include "button.h"
#include "collision.h"
#include "game.h"
#include "gameState.h"
#include "spriteLibrary.h"

bool IsHoveredOver(Button* button, float x, float y){
    if(button == nullptr) return false;
    if(button->is_active == false) return false;

    assert(button->type != ButtonType::NONE);

    return CheckCollisionInsideBounds(button->rect, x, y);
}
```

we make sure the button is correctly set up or at least not supposed to be triggered (null pointer or not active). Then we call our new collision detection

function and return the bool result.

```
// button.cpp

void SetupButton(Button* button, ButtonType type, Sprite* spriteBuffer, SDL_FRect rect,
    ↪ ButtonMode mode){
    assert(type != ButtonType::NONE);
    button->type = type;
    button->rect = rect;
    if(mode == ButtonMode::Centered){
        button->rect.x -= button->rect.w / 2;
        button->rect.y -= button->rect.h / 2;
    }
    button->is_active = true;
    switch(button->type){
    case ButtonType::START_GAME:
        button->texture = GetSprite(SPRITE_ID::Fallback, spriteBuffer)->texture;
        break;
    case ButtonType::QUIT:
        button->texture = GetSprite(SPRITE_ID::Fallback, spriteBuffer)->texture;
        break;
    default:
        button->texture = GetSprite(SPRITE_ID::Fallback, spriteBuffer)->texture;
        break;
    }
}
```

here we set all the internals of a new `Button` struct. currently we assign `fallback` as the texture for all of the buttons. in a later chapter we'll update these with some actual graphics. We are also doing a bit more defensive programming with `assert` in this chapter than usual. But it's good practice to do so.

```
// button.cpp

void PressButton(Button *button, GameData *data){
    if(button == nullptr){
        return;
    }
    assert(button->is_active);
    switch(button->type){
        case ButtonType::NONE:
            assert(false);
        case ButtonType::START_GAME:
            ChangeScene(data, SCENE_TYPES::GAME);
            break;
        case ButtonType::QUIT:
            data->running = false;
            break;
    }
}
```

It's a bit brutish to pass along the entire `GameData` pointer, but with it we can have our buttons really affect the entire program. (for good and for bad).

```
// button.cpp

int GetActiveButtonCount(Button *buttons, int count){
    if(count == 0){
        return 0;
    }
    int amount = 0;
    for (int i = 0; i < count; i++) {
        if(buttons[i].is_active){
            amount++;
        }
    }
    return amount;
}
```

lastly we have a function that loops over all buttons and increments `amount` whenever we find a button that is active. Finally it returns this value. Right now only our main menu have buttons inside the game. But once we add more we'll continue to create these helper functions to assist us with our boilerplate. But right now we could easily take our one callsite and just add the code inside this function to it. It is really only a good idea to create a

function when the code inside it is called from more than 1 place. Every function and file adds obsuscation to our code by not having the logic live next to each other.

We'll be using a bespoke `rendering` function for our buttons as they live in UI-space and not in game space. The camera should for example not affect a button at all.

```
// rendering.h
```

```
void RenderButton(Button* button, bool is_selected, SDL_Renderer* renderer);
```

and then the implementation

```
// rendering.cpp
```

```
void RenderButton(Button* button, bool is_selected, SDL_Renderer* renderer){
    SDL_Texture* texture =button->texture;
    SDL_SetTextureScaleMode(texture, SDL_SCALEMODE_PIXELART);
    uint8_t colorOverlay = is_selected ? 255: 230;
    SDL_SetTextureBlendMode(texture, SDL_BLENDMODE_BLEND);
    SDL_SetTextureColorMod(texture, colorOverlay, colorOverlay, colorOverlay);
    SDL_RenderTexture(renderer, button->texture, NULL, &button->rect);
}
```

we fetch the texture, set it to have no bilinear filtering. We then work with something we don't do usually. We are gonig to be adding color ontop of our button. We are either adding `(255,255,255)` on top or `(230,230,230)` this is the RGB values that are mapped to an `uint8_t` meaning that we have a precision of 0-255. giving us `255x255x255 = 16581375` unique colors in our color space. `255` in all channels means pure white - aka no added color at all. `230,230,230` adds a very light grey ontop, making the texture a little darker. We use this to make all buttons that are not the selected button a bit darker. We assing this overlaid color usign `SDL_SetTextureColorMod`. lastly we render the texture to the backbuffer.

We're simplifying access to our `running` boolean by passing it into our

GameData struct

```
//gameState.h

struct GameData {
    // other variables hidden for clarity

    bool running;
};
```

And in `main.cpp` we remove our local `running` and substitute `gameData->running`

```
// main.cpp

gameData->running = true;
float dt;
float dt_scaler = 1;
gameData->dt = &dt;
gameData->dt_scaler = &dt_scaler;
while(gameData->running){
```

Now lets take our logic and call it from `game.cpp`

```
// game.cpp
void Initialize(GameData* data, SDL_Window* window, SDL_Renderer* renderer){
    DEV::Initialize(window, renderer);
    AssetManagement::LoadAllSprites(data->spriteBuffer, renderer);
    data->imGui_context = ImGui::GetCurrentContext();

    AssetManagement::LoadAllTilesets(data->tilesetBuffer, data->arena_images);

    SDL_Texture* blackfade = GetSprite(SPRITE_ID::black_1x1,
    ↪ data->spriteBuffer)->texture;
    SDL_SetTextureBlendMode(blackfade, SDL_BLENDMODE_BLEND);
    InitializeGame(&data->scenes.gameplay, data->arena_levels, data->tilesetBuffer);
    InitializeMenu(&data->scenes.mainMenu, data->spriteBuffer, data->arena_main); // new
    ChangeScene(data, SCENE_TYPES::MAINMENU);
}
```

in `Update()` we call into `mainmenu`

```
// game.cpp
case SCENE_TYPES::MAINMENU:
    UpdateMenu(data);
    break;
```

And in `Draw()`

```
// game.cpp
case SCENE_TYPES::MAINMENU:
    DrawMenu(&data->scenes.mainMenu, renderer, data->spriteBuffer);
    break;
```

With everything set up we can fill out the different functions in `mainmenu.cpp`

```
// mainmenu.cpp

#include "mainmenu.h"
#include "SDL3/SDL_scancode.h"
#include "arena.h"
#include "button.h"
#include "common.h"
#include "gameState.h"
#include "input.h"
#include "rendering.h"
#include "spriteLibrary.h"
#include <cassert>

void InitializeMenu(MainMenu* mainmenu, Sprite* spriteBuffer, Memory::Arena*
↪ arena_main){
    assert(mainmenu->initialized == false);
    mainmenu->button_count = 2;
    mainmenu->buttons = ALLOC_ARRAY(arena_main, Button, mainmenu->button_count);

    SetupButton(&mainmenu->buttons[0], ButtonType::START_GAME, spriteBuffer, {SCREEN_WIDTH
↪ / 2.0, SCREEN_HEIGHT / 2.0, 200, 80}, ButtonMode::Centered);
    SetupButton(&mainmenu->buttons[1], ButtonType::QUIT, spriteBuffer, {SCREEN_WIDTH /
↪ 2.0, (SCREEN_HEIGHT / 2.0) + 100, 200, 80}, ButtonMode::Centered);

    mainmenu->initialized = true;
}
```

During initialization we make sure we haven't already done our initialization. Then we currently hard-code our `button_count` meaning that if we want to setup a new button we need to remember to increase this number. There are

a multitude of ways of refactoring out this error-prone step. We might do that later. Otherwise it's a simple exercise for you, the reader.

We allocate our buttons to our `arena_main` as we have no need of ever purging them from memory. We then call `SetupButton` passing along the relevant variables. The biggest parameter is our `SDL_FRect` that we pass with the condensed `{}` syntax.

I've added `ButtonMode` here to allow us to put our button centered on the position of our rect, or if set to `::Raw` it will get rendered from the top left corner.

lastly we set `initialized` to `true`.

```
//mainmenu.cpp

void DrawMenu(MainMenu* mainmenu, SDL_Renderer* renderer, Sprite* spriteBuffer){
    Sprite* background = GetSprite(SPRITE_ID::titlescreen_background, spriteBuffer);
    float scale = (SCREEN_HEIGHT / ((float)background->height * UPSCALE_FACTOR));
    RenderSprite_World(GetSprite(SPRITE_ID::titlescreen_background, spriteBuffer),
    ↪  renderer, NULL, SCREEN_WIDTH / 2.0, SCREEN_HEIGHT / 2.0, scale);
    for (int i = 0; i < mainmenu->activeButtonCount; i++) {
        Button* button = mainmenu->activeButtons[i];
        RenderButton(mainmenu->activeButtons[i], i == mainmenu->activeButtonIndex,
    ↪  renderer);
    }
}
```

first we're rendering our titlescreen background. We've made some changes to it in `spriteLibrary.cpp`

```
// spriteLibrary.cpp

{SPRITE_ID::titlescreen_background, "assets/sprites/titlescreen.png"},
```

we've opted to omit the pivot as we want our program to automatically set this to the dead center of the texture.

We pass `NULL` as our parameter for our `const Camera* camera` when

calling `RenderSprite_World()` this will currently break but we can easily allow for this behaviour by making a small adjustment to the function

```
// rendering.cpp

void RenderSprite_World(SpriteRenderInfo spriteRenderInfo, SDL_Renderer* renderer,
↪  const Camera* camera, float x, float y, float scale, float alpha, bool flipped){
    // code hidden for brevity

    if(GetSpriteCount(sprite) > 1){
        // hidden for brevity
    }
    else{
        // hidden for brevity
    }
    // old
    SDL_FRect rect;
    rect.x = x;
    rect.y = y;
    float final_scale = UPSCALE_FACTOR * scale;
    rect.w = tilesetRect.w * final_scale;
    rect.h = tilesetRect.h * final_scale;
    rect.x -= sprite->pivot_x * final_scale;
    rect.y -= sprite->pivot_y * final_scale;

    // new if-statement with old code inside
    if(camera != NULL){
        rect.x -= camera->camera_x;
        rect.y -= camera->camera_y;
    }

    // code hidden for brevity
}
```

We just make sure that if `camera` was `NULL` we don't try and use it to adjust our `rect's` position. Now we can safely pass `NULL` instead of our camera.

Back in `DrawMenu()` We create a `scale` variable that we'll use to scale the texture up and down depending on the size of the game window `SCREEN_HEIGHT`. This means that the larger the difference is between the game window and the height of the texture the more scale increases. This ensures that the titlescreen background fills the entire window.

When we call `RenderSprite_World()` we also pass along `SCREEN_WIDTH/HEIGHT / 2.0` to place the rendering origin in the center of the screen. Doing this with the pivot set to the center of the texture is what we need to do to center the entire thing.

lastly we loop over each of the `activeButtons` and call `RenderButton` by passing it along. `i == mainmenu->activeButtonIndex` will be true only for the active button and false for all others. Letting us add the light grey overlay color on all non selected buttons.

```

// mainmenu.cpp
// updateMenu part 1
void UpdateMenu(GameData* data){
    MainMenu* mainmenu = &data->scenes.mainMenu;
    Input* input = &data->input;
    mainmenu->activeButtonCount = GetActiveButtonCount(mainmenu->buttons,
↪ mainmenu->button_count);
    if(mainmenu->activeButtonCount == 0){
        return;
    }
    mainmenu->activeButtons = ALLOC_ARRAY(data->arena_scratch, Button*,
↪ mainmenu->activeButtonCount);

    int index = 0;
    for (int i = 0; i < mainmenu->button_count; i++) {
        Button* button = &mainmenu->buttons[i];
        if(button->is_active){
            mainmenu->activeButtons[index] = button;
            index += 1;
        }
    }

    int* buttonIndex = &mainmenu->activeButtonIndex;

    bool anyHoveredOver = false;
    bool mouseMoving = input->mouse_magnitude > 0.1;
    if(mouseMoving){
        for (int i = 0; i < mainmenu->activeButtonCount; i++) {
            Button* button = mainmenu->activeButtons[i];
            if(IsHoveredOver(button, input->mouse_x, input->mouse_y)){
                anyHoveredOver = true;
                *buttonIndex = i;
                break;
            }
        }
    }
}

```

passing the entire `GameData*` is currently necessary as we need to pass that same fat struct into `PressButton`. Fixing this would mean actually committing to the variables that `PressButton` needs and then walking up this call-chain fixing so we only send those variables as parameters. For now we'll pass everything

we collect `Input*` and `MainMenu*` pointers to reduce the length of each

line of code that references them. We then fetch how many active buttons we have and allocate our pointer pointer array of `activeButtons` to our `scratch arena`. This arena gets reset each frame (on purpose).

We then loop over all buttons and assign the active ones in order to our `activeButtons` array.

Next we're checking if any button is hovered over and if the mouse is currently in motion. To do this we need to expand our `Input` struct and add some new boilerplate to `main.cpp` if the mouse is in fact moving we loop over all buttons and if any of them are hovered over we set `anyHoveredOver` to true and set the `activeButtonIndex` to the value of `i`. This is how we get the relevant button for mouse inputs.

```
// input.h

struct Input{
    const bool* keys_current;
    const bool* keys_previous;
    float* keys_held_time;
    SDL_MouseButtonFlags mouse_current;
    SDL_MouseButtonFlags mouse_previous;
    float* mouse_held_time;
    float mouse_x;
    float mouse_y;
    float mouse_x_delta; // new
    float mouse_y_delta; // new
    double mouse_magnitude; // new
};
```

```
// main.cpp

gameData->input.keys_current = SDL_GetKeyboardState(nullptr); // old

float* delta_x = &gameData->input.mouse_x_delta;
float* delta_y = &gameData->input.mouse_y_delta;
*delta_x = gameData->input.mouse_x; // store last frames value
*delta_y = gameData->input.mouse_y; // store last frames value
gameData->input.mouse_current = SDL_GetMouseState(&gameData->input.mouse_x,
↪ &gameData->input.mouse_y);
*delta_x = gameData->input.mouse_x - *delta_x;
*delta_y = gameData->input.mouse_y - *delta_y;

float dx = *delta_x;
float dy = *delta_y;
gameData->input.mouse_magnitude = std::sqrt(dx * dx + dy * dy);

// printf("x: %.4f y: %.4f \n", *delta_x, *delta_y);
// printf("%.4f \n", gameData->input.mouse_magnitude);

dll.update(gameData, dt); // old
```

we fetch pointers to `mouse_x/y_delta`. A delta value is the value as a comparison between the current frame and the previous one. We already use this for `deltatime`. So we assign `delta_x/y` to the value from `mouse_x/y`. But we do this before we call `GetMouseState` for this frame, meaning that the values stored are the last frames values. We then update our `delta_x/y` to be the current position of the mouse minus last frames position. This gives us the length the mouse travelled in `x` and `y` since the last frame. After that we call `std::sqrt` that gives us a `vector` in space, the length of this is the total distance travelled in both the x and y direction, this is the pythagorean theorem.

I've saved two `printf` calls that you can uncomment if you want to have a look at what happens when you move the mouse around.

```

// mainmenu.cpp
// UpdateMenu() part 2

bool up = KeyPressed(input, SDL_SCANCODE_UP);
bool down = KeyPressed(input, SDL_SCANCODE_DOWN);

if(up || down){
    int direction = up ? 1 : -1;
    *buttonIndex += direction + mainmenu->activeButtonCount;
    *buttonIndex = *buttonIndex % mainmenu->activeButtonCount;
}

Button* selected = mainmenu->activeButtons[*buttonIndex];

if(selected != nullptr){
    if(KeyPressed(input, SDL_SCANCODE_RETURN)){
        PressButton(mainmenu->activeButtons[*buttonIndex], data);
        return;
    }

    if(IsHoveredOver(selected, input->mouse_x, input->mouse_y)){
        if(MousePressed(input, MouseButton::LEFT)){
            PressButton(selected, data);
            return;
        }
    }
}
}
}

```

Now that we can select a button using the mouse we can check if we are pressing the up or down key. If so we modify the `buttonIndex` pointer that points to the same place in memory as our `mainmenu->activeButtonIndex`. Instead of having two `if-statements` we use the `?` operator to select if we're going backwards or forwards. But with the Modulo `%` allowing for negative numbers we actually have to shift the index by the full count of `activeButtonCount` this does nothing to change the value as this full value addition will get stripped by the modulo operator - but it will ensure that even when we remove using `direction = -1` we never reach a negative number.

we then point to the button that is selected from `buttonIndex` and store it

in `selected`.

If we have a `selected` we check if we are pressing `enter` aka `return` or if we are hovering over the button and pressing left mouse button. In either case we call `PressButton`.

With this we've added buttons and the collision and pressing of said buttons!

33 Sokoban Programming VI

Before we add goal squares to our project we will be needing code that checks a new layer of our `.TMJ` file. Currently we have 2 places where we loop over every layer in our `auto result` from `CreateLevel()` and `CreateEntities()`. By adding a third place where we need to type this same `for-loop` structure we've hit a good benchmark for a helper function.

```
// levels.h
#include "Parsers/json.hpp"

namespace AssetManagement{
    std::vector<uint16_t> GetCellDataFromJsonLayer(nlohmann::json& parsedJson, const
        ↪ char* layerName, bool* wasFound);
    int GetFirstNonZeroCell(std::vector<uint16_t>* list);
}
```

So we'll create two helper functions inside our `AssetManagement namespace`. These will be created then substituted in the two currently places where we currently duplicate our code.

```

// levels.cpp
namespace AssetManagement{
std::vector<uint16_t> GetCellDataFromJsonLayer(nlohmann::json& parsedJson, const char*
↪ layerName, bool* wasFound){
    std::vector<uint16_t> result;
    *wasFound = false;
    for (const auto& layer : parsedJson["layers"]) {
        if (layer["name"] == layerName) {
            result = layer["data"].get<vector<uint16_t>>();
            *wasFound = true;
            break;
        }
    }
    return result;
}

int GetFirstNonZeroCell(std::vector<uint16_t> *list){
    for (int id : *list) {
        if(id != 0){
            return id;
        }
    }
    assert(false);
    return -1;
}
}

```

The code is lifted almost entirely from `levels.cpp` but now we pass in a bool by pointer to store whether or not we found what we were looking for. We pass `parsedJson` with a `&` instead of a `*`. When working with our json object doing this lets us keep the more familiar `parsedJson["layers"]` syntax rather than the noisier `(*parsedJson)["layers"]` that unfortunately would be required when using an explicit pointer. the `&` forces the variable to exist at the callsite and we pass it just by name. meaning that we don't know if its a copy or a reference before we look at the function we're using. I don't like this type of coding for our own code, but as we're working with this json parser and it has this `c++ structure` we're sorta forced into doing the same if we want to have a relatively clean code layout.

we also have a few callsites where we do a conversion from `1D` to `2D` by

using divided by and modulo operators. This is not the simplest code to remember and a small helper function would be very beneficial.

```
// common.h

inline void Expand1DTo2D(int flatIndex, int width, int* x, int* y){
    *x = flatIndex % width;
    *y = flatIndex / width;
}

inline void Expand1DTo2D(int flatIndex, int width, float* x, float* y){
    *x = (float)(flatIndex % width);
    *y = (float)(flatIndex / width);
}
```

This does the same work but we can just pass the required parameters and then it sets `x` and `y` to the correct values. As these are pointers the value will be updated for the variable living at the callsite.

now we can simplify `CreateLevel()` and `CreateEntities()`

```

// levels.cpp

void CreateLevel(Arena* arena, LevelData* level, Tileset* tileset, const char*
↪ level_name){
    fstream stream(level_name);
    auto result = json::parse(stream);

    bool found = false;
    vector<uint16_t> levelData = AssetManagement::GetCellDataFromJsonLayer(result,
↪ "level", &found);

    assert(found);

    int first_non_zero_id = AssetManagement::GetFirstNonZeroCell(&levelData);
    int id_offset = Get_Tileset_ID_Offset_From_Tilemap(first_non_zero_id, result);

    level->w = result["width"].get<int>();
    level->h = result["height"].get<int>();
    level->level_path = level_name;
    level->tileset = tileset;
    level->cells = ALLOC_ARRAY(arena, uint16_t, level->w * level->h);
    for (int i = 0; i < level->w * level->h; i++) {
        int local_id = levelData[i] - id_offset;
        if(local_id < 0){
            local_id = 0;
        }
        level->cells[i] = local_id;
    }
}

```

It's the same function in all regards but we now use our helper functions to reduce to volume of code.

```
// levels.cpp

void CreateEntities(LevelData* lvl_data, Arena* arena){
    Reset(arena);
    lvl_data->entityCount = 0;
    lvl_data->entityBuffer = (Entity*)Memory::Allocate(arena, sizeof(Entity) * 256);

    fstream stream(lvl_data->level_path);
    auto result = json::parse(stream);

    bool found = false;
    vector<uint16_t> entities = AssetManagement::GetCellDataFromJsonLayer(result,
↪  "entities", &found);

    if(!found){
        return;
    }

    for (int i = 0; i < lvl_data->w * lvl_data->h; i++) {
        if(entities[i] == 0){
            continue;
        }
        uint16_t entity_id = GetLocalTileID(entities[i], result);
        int x;
        int y;
        Expand1DTo2D(i, lvl_data->w, &x, &y);
        AddEntity((ENTITY_ID)entity_id, x, y, lvl_data);
    }
}
```

The same goes for our `CreateEntities()` function.

Then in `RenderSprite_World()` we use our helper function as well - not because it saves a lot of space, but it feels like a consistent practice.

```
// rendering.cpp

// tilesetRect.x = (frame % sprite->sprite_count_x) * width; // old
// tilesetRect.y = (frame / sprite->sprite_count_x) * height; // old
Expand1DTo2D(frame, sprite->sprite_count_x, &tilesetRect.x, &tilesetRect.y); // new
tilesetRect.x *= width; // new
tilesetRect.y *= height; // new
```

in the supplied assets `chapter 34 assets.zip` you'll find a new `.tmj` file called `testing_goal.tmj`. This level has a new layer inside `Tiled` called

`goals`. On this layer we exclusively place a new `goal tile`. This sits on top of the rendered level and in our game the victory condition for a level will be that each goal has a player entity standing on it. We place this not in our `level` layer as we want the graphics to sit on top of our level. And we don't place it in our `entity` layer as we want to have both a goal and an entity being able to overlap. We could label this layer `markers` and maybe have more than one thing in it. But as we live by the practice of just programming what we need and refactoring later we'll just have our goal tile in this layer.

We want to store these goals in a separate array inside our `LevelData`

```
// levels.h
struct Goal{
    int x;
    int y;
    float blink_timer;
};

struct LevelData{
    int w;
    int h;
    uint16_t* cells;
    Goal* goals; // new
    int goalCount; // new
    const char* level_path;
    Entity* entityBuffer;
    int entityCount;
    const Tileset* tileset;
};
```

Right now our `Goal` struct is just a `x` and `y` position and a timer. But it's still a valuable struct that might expand in the future. Another solution would be to have a `Position` struct and for all callsites, structs and functions where we currently pass `x` and `y` separately we instead pass in a `Position`. But this has little value currently so let's hold off. `blink_timer` will increase by `deltaTime` each frame for as long as an entity is standing on the goal square. Otherwise we'll reset it to 0.

With our helper functions we'll add more logic to the end of `CreateLevel()` to allocate our `goals` array.

```
// levels.cpp

// at the end of the CreateLevel() function

vector<uint16_t> onLevel = AssetManagement::GetCellDataFromJsonLayer(result,
↪ "on_level", &found);
if(found){
    int firstNonZero = AssetManagement::GetFirstNonZeroCell(&onLevel);
    id_offset = Get_Tileset_ID_Offset_From_Tilemap(firstNonZero, result);
    level->goalCount = 0;

    for (int i = 0; i < level->w * level->h; i++) {
        int local_id = onLevel[i] - id_offset + 1;
        if(local_id > 0){
            level->goalCount++;
        }
    }

    level->goals = ALLOC_ARRAY(arena, Goal, level->goalCount);
    int index = 0;
    for (int i = 0; i < level->w * level->h; i++) {
        int local_id = onLevel[i] - id_offset + 1;
        if(local_id < 0){
            local_id = 0;
        }
        if(local_id != 0){
            int x;
            int y;
            Expand1DTo2D(i, level->w, &x, &y);
            level->goals[index].x = x;
            level->goals[index].y = y;
            index++;
        }
    }
}
```

we find how many goals are on the layer by finding all non-zero cells. This means that we could in theory add any tile to the grid and it would get converted to a goal. This is not ideal but we can live with it for now. If we ever need more markers we can think about refactoring. With only one single tile in the tileset used for our `goal` we need to add `+1` to the `id_offset` as

our tilemap doesn't start with an empty zero-indexed tile. So in this example `.tmj` the only non-zero cell in our `goals` layer has an `id` of `90` and the `firstgid` is also `90`. Meaning that if we remove it outright those cells would turn into `90 - 90 = 0` and just show up as all empty cells do. This is a bit messy but with this being the only callsite we can afford it.

We call `Expand1DTo2D()` to get the x and y coordinates of the goal cell before assigning it to the relevant `level->goals[index]`. Note how we only advance `index` after we've found a goal cell in the level grid.

Now we can create our `SPRITE_ID` for our goal as well as making sure we render them during `RenderLevel()`

```
// spriteLibrary.h

enum class SPRITE_ID{
    // other hidden for brevity
    Goal
};

// spriteLibrary.cpp

static const SpriteDataEntry all_sprite_data[] = {
    {SPRITE_ID::Goal, "assets/sprites/goal.png", 8, 8, 8, 1}
};
```

As we can see, the goal sprite is a spritesheet with 8 frames in total, layed out in a single row. It also has it's pivot in the center of its 16x16 frame. You can inspect the `SpriteDataEntry` struct to more clearly see which number corresponds to which variable.

At the end of `RenderLevel()` in `levelRenderer` we can make sure we render all of our goals

```
// levelRenderer.cpp

for(int i = 0; i < level->goalCount; i++){
    Goal goal = level->goals[i];
    Sprite* sprite = GetSprite(SPRITE_ID::Goal, gameData->spriteBuffer);
    int frame = (int)(goal.blink_timer / 0.2) % (sprite->sprite_count_x *
    ↪ sprite->sprite_count_y);
    RenderSprite_OnTile({frame, sprite}, level, renderer, &gameData->camera, goal.x,
    ↪ goal.y);
}
```

We fetch our `goal` sprite and select the appropriate frame by taking `blink_timer` and keeping it within the spritesheet bounds using `%` then we divide it by 0.2, making to so it changes frame every 0.2 seconds.

`blink_timer` won't update on its own, inside our `UpdateGame()` we have to loop over all goals

```
// game.cpp

for (int i = 0; i < level->goalCount; i++) {
    Entity* entity = GetEntity(level, level->goals[i].x, level->goals[i].y);
    if(entity != nullptr && !IsActing(entity)){
        level->goals[i].blink_timer += dt;
    }
    else{
        level->goals[i].blink_timer = 0;
    }
}
```

This resets our goal back to frame 0 as soon as there is no entity on it. and we only increment `blink_timer` if an entity existed and it has finished whatever action it might have been taking. For example, walking onto the goal square.

Lastly, lets make sure we actually play the new level

```
// game.cpp

void InitializeGame(Gameplay* gameplay, Arena* arena_levels, Tileset* tilesetBuffer){
    assert(gameplay->initialized == false);
    gameplay->currentLevelIndex = 0;

    // we have updated the path to be `testing_goal.tmj`
    CreateLevel(arena_levels, &gameplay->levels[0],
    ↪ &tilesetBuffer[(int)TILESETS::Dungeon], "assets/levels/testing_goal.tmj");
    gameplay->initialized = true;
}
```

Before moving forward, we have to fix a very very small issue. Our `CalculateDeltaTime()` function in `main.cpp` should be the first code that runs each `while-loop`

```
// main.cpp
while(gameData->running){
    CalculateDeltaTime(&dt, dt_scaler);
    // the rest of the while loop
```

otherwise our fps will be off by a very small fraction as `DLL_CheckStatus()` and `Reset()` take a small amount of time to run and were previously not “seen” by the deltatime function.

Now we can see our goals rendered and they pulse if we stand on them. The next step is actually changing level when all entities are standing on a goal spot.

We’ll look over all goals and if we have met the condition for each we’ll advance our `CurrentLevelIndex` and call `StartLevel()`


```

// game.cpp
// inside UpdateGame()

if(level->goalCount > 0){
    int goals_reached = 0;
    for (int i = 0; i < level->goalCount; i++) {
        Goal goal = level->goals[i];
        Entity* entity = GetEntity(level, goal.x, goal.y);
        if(entity == nullptr){
            break;
        }
        else if (HasBehaviour(entity, Behaviour::IS_PLAYER)){
            goals_reached++;
        }
    }
    if(goals_reached == level->goalCount){
        gameplay->currentLevelIndex++;
        StartLevel(gameplay, arena_commands, arena_entities);
        return;
    }
}
}

```

We only allow for the level to actually change if we found a goal. Otherwise the level would just blink past. This will create a level that is unwinnable, but if we want later we can assert that `goalCount` is atleast a `1`. Note how we call `return` after `StartLevel()` as we don't want anything in `UpdateGame()` to be called this frame if we have changed level.

We loop over each `Goal` then we try and find a relevant entity ontop of it. Finally if we found an equal amount of entities to the `goalCount` we can progress `currentLevelIndex` and call `StartLevel()`. Because `StartLevel()` requires us to pass along `arena_commands` and `arena_entities` we need to include these as parameters in `UpdateGame()`

```

// game.cpp

void UpdateGame(Gameplay* gameplay, Input* input, Arena* arena_scratch, Arena*
↪ arena_commands, Arena* arena_entities, const float dt){

```

as well as pass these parameters in at the callsite in `Update()`

```
// game.cpp

case SCENE_TYPES::GAME:
    UpdateGame(gameplay, &data->input, data->arena_scratch, data->arena_commands,
    ↪ data->arena_entities, dt);
    break;
```

We also need to cleanup our `CommandBuffer` between levels

```
// command.h
void ResetCommandBuffer(CommandBuffer* buffer);

// command.cpp

void ResetCommandBuffer(CommandBuffer* buffer){
    buffer->index = 0;
    buffer->head = 0;
    buffer->timestamp = 0;
}
```

we then call this new function at the top of `StartLevel()`

```
// game.cpp

void StartLevel(Gameplay* gameplay, Arena* arena_commands, Arena* arena_entities){
    ResetCommandBuffer(gameplay->commandBuffer);
    Reset(arena_commands);
    CreateEntities(&gameplay->levels[gameplay->currentLevelIndex], arena_entities);
    gameplay->activePlayerIndex = 0;
}
```

we make sure to call `Reset` on our `arena_commands` to purge that memory block from containing stale data. Now it's all neatly zeroed-out.

The last step is to change what levels we have in the game. Included in the same assets `.ZIP` that you unpacked earlier are `level_01.tsj` and `level_02.tsj`.

Let's make those the levels we initialize

```
// game.cpp

void InitializeGame(Gameplay* gameplay, Arena* arena_levels, Tileset* tilesetBuffer){
    assert(gameplay->initialized == false);
    gameplay->currentLevelIndex = 0;
    CreateLevel(arena_levels, &gameplay->levels[0],
↪ &tilesetBuffer[(int)TILESETS::Dungeon], "assets/levels/level_01.tmj");
    CreateLevel(arena_levels, &gameplay->levels[1],
↪ &tilesetBuffer[(int)TILESETS::Dungeon], "assets/levels/level_02.tmj");
    gameplay->initialized = true;
}
```

With this we can now start the game, playing `level_01` then as soon as our character steps on the goal space we instantly load `level_02`. We be beat that level we crash the game as we don't currently handle `currentLevelIndex` going beyond the amount of levels we've added. We'll refactor this in an upcoming chapter.

And as an added bonus we'll quickly add `R` to reset the current level

```
// game.cpp

void UpdateGame(Gameplay* gameplay, Input* input, Arena* arena_scratch, Arena*
↪ arena_commands, Arena* arena_entities, const float dt){

    if(KeyPressed(input, SDL_SCANCODE_R)){ // restart level on `R`
        StartLevel(gameplay, arena_commands, arena_entities);
        return;
    }

    // other code hidden
```

34 FMOD and Audio

We'll be implementing audio. We are just going to dip our toes into audio systems programming - which is really a whole discipline in and of itself. And just as with VFX programming and animation systems there are a lot of terms and concepts that someone has to understand in order to fully grasp the boilerplate.

We'll be working with a `middleware` called `FMOD`. This is an industry standard tool that many (many) of the largest commercial titles use. There are two ways of using FMOD

- 1) FMOD core
- 2) FMOD studio

We'll be using FMOD core even though FMOD studio is far away the more common way of working with FMOD. We do this because FMOD studio is its own software with a bunch of buttons and menus that do not fit into the scope of this course. FMOD core is just a few `.h` files and `.lib` files that we can hook into to produce audio for our games.

To download FMOD core you need to create a free account over at <https://www.fmod.com/download#fmodengine>. Once that is done you should download `FMOD Engine` this contains the `core` package. once you have downloaded and run the `installer` you'll find that included in the install directory `FMOD SoundSystem\FMOD Studio API Windows\api\core` is the `core` folder with an `inc` and a `lib` folder. Copy over the contents of each into your project in the project folder with the same name. For my `include` folder I decided to put all `fmod .h files` into a `FMOD` subdirectory.

In the `lib` folder, look for the `x64` folder inside. In my own projects `lib` folder I've opted for putting the `FMOD .lib files` into a `FMOD` subdirectory as well.

Audio is a pretty difficult part of game development. and there are many pitfalls and ways to make your life harder. But FMOD honestly does a great job of having minimal boilerplate and handling most of our issues for us.

You can unzip the `chapter 35 assets.zip` and replace the old content inside your assets folder with it.

lets set up `audioSystem.h` that will manage FMOD

```

// audioSystem.h
#pragma once
#include "FMOD/fmod_common.h"

namespace Memory{
    struct Arena;
}

enum class SFX_ID{
    JUMP,

    COUNT
};

struct SoundDataEntry{
    SFX_ID id;
    const char* path;
};

struct AudioSystem{
    bool initialized;
    void* fmod_memory;
    FMOD_SYSTEM* sound_system;
    static const int CHANNEL_COUNT = 32;
    FMOD_CHANNEL* channels[CHANNEL_COUNT];
    FMOD_SOUND* soundEffects[(int)SFX_ID::COUNT];
};

// global promise
extern AudioSystem* g_audioSystem;

void PlaySFX(SFX_ID id, float volume = 1);
void InitializeAudioSystem(AudioSystem* audio, Memory::Arena* arena_main);
void UpdateAudio(AudioSystem* audio);

namespace AssetManagement{
    void LoadAllSFX(AudioSystem* audioSystem);
}

```

We're `#including` `fmod_common.h` making sure to have our path to the `.h` include our subdirectory created earlier. We then `forward declare` `Memory::Arena`.

`SoundDataEntry` will be used just like `SpriteDataEntry` in our `audioSystem.cpp` later. We'll use it to load our sound files.

`AudioSystem` is our main struct responsible for holding everything we need to play audio. We also do something slightly different with our arrays. As the size of these arrays will not change we can specify their size in advance. This has the effect that when we add `AudioSystem` to `GameData` we do not have to allocate our arrays in our memory arena ourselves, as this is done for us.

We'll be giving `FMOD` a section of our memory to work with, a pointer to this memory is held in our `void* fmod_memory`.

`FMOD_SYSTEM* sound_system` is the struct that we initialize and always pass in when we work with our audio. This holds all the relevant information that FMOD needs to function.

a `channel` or `voice` is an audio thread responsible for streaming and pushing audio to our sound card. We can not have an infinite amount of these as this will throttle our audio thread. Though I am no audio programmer I've found that 16-32 channels works for our needs. When a source of sound is going to be played we must provide it with a channel to take over. This means that we can't have more than 32 things playing at once.

`FMOD_SOUND` holds our actual audio file in memory. FMOD reads bytes from this and pushes that through the relevant channel. We make this array have the size of `SFX_ID::COUNT` as that makes sure that as long as `COUNT` is the last enum in the list we have allocated enough array elements for all sound effects.

Audio tends to be the type of system that easily worms its way throughout our codebase. To avoid having to pass `AudioSystem` everywhere we're making a `global pointer` to an `AudioSystem` available for every file that includes `audioSystem.h`. The `extern` keyword promises that the actual pointer will be found during compilation from another file. We'll keep this global

pointer synced to the `AudioSystem` we'll put inside `GameData` so that any file can point to it and use our audio system. every program has by its nature some global state.

We'll write a pointer with this exact name again inside `AudioSystem.cpp` as this is the actual variable. our `extern` in our `.h` file is just a way of accessing this by `#include`.

our three functions do exactly what we expect. Plays sound effects, initializes everything and keeps it updated.

in the `AssetManagement` namespace we add a `LoadAllSFX()`. We'll call our `InitializeAudioSystem` and `LoadAllSFX()` from our `Initialize()` function inside `game.cpp`. But first we must have a `AudioSystem` inside `GameData` to actually work with

```
// gameState.h
struct GameData {
    SCENE_TYPES scene_current;
    SCENE_TYPES scene_previous;
    Scenes scenes;
    Transition transition;
    EditorData editor_data;
    Input input;
    Sprite* spriteBuffer;
    Tileset* tilesetBuffer;
    AudioSystem audio; // new

    // other variables hidden for clarity
```

then inside `game.cpp`


```
// game.cpp
void Initialize(GameData* data, SDL_Window* window, SDL_Renderer* renderer){
    DEV::Initialize(window, renderer);
    InitializeAudioSystem(&data->audio, data->arena_main); // new
    AssetManagement::LoadAllSFX(&data->audio); // <----- new
    AssetManagement::LoadAllSprites(data->spriteBuffer, renderer);
    data->ImGui_context = ImGui::GetCurrentContext();

    AssetManagement::LoadAllTilsets(data->tilesetBuffer, data->arena_images);

    SDL_Texture* blackfade = GetSprite(SPRITE_ID::black_1x1, data->spriteBuffer)->texture;
    SDL_SetTextureBlendMode(blackfade, SDL_BLENDMODE_BLEND);
    InitializeGame(&data->scenes.gameplay, data->arena_levels, data->tilesetBuffer);
    InitializeMenu(&data->scenes.mainMenu, data->spriteBuffer, data->arena_main);
    ChangeScene(data, SCENE_TYPES::MAINMENU);
}
```

we'll need to allocate memory for our AudioSystem. This starts with deciding on the amount of memory to give it.

```
// common.h
constexpr size_t GAME_MEMORY_ALLOWANCE = MEGABYTES(14);
constexpr size_t AUDIO_MEMORY_ALLOWANCE = MEGABYTES(5); // new
```

Just make sure the audio segment is smaller than the game memory as we're creating a subarena for the audio from the main memory.

Lets look at our implementation in `audioSystem.cpp`

```

// audioSystem.cpp
// part 1

#include "audioSystem.h"
#include "FMOD/fmod.h"
#include "arena.h"
#include "common.h"
#include <cassert>

// global
AudioSystem* g_audioSystem;

static const SoundDataEntry all_sound_data[] = {
    {SFX_ID::FALLBACK, "assets/audio/sfx/fallback.wav"},
    {SFX_ID::JUMP, "assets/audio/sfx/fallback.wav"},
};

```

here we create our same `g_audioSystem`. Ensure it has the exact same name. the `g_` is called `hungarian notation` and signifies that the name of the variable tells us about its type. In this case `global_`.

We then set up a compile time known array of `SoundDataEntry`. For now we only have one `::JUMP` and our fallback that both link to `fallback.wav`. But later we'll add more.

```

// audioSystem.h
// part 2

void InitializeAudioSystem(AudioSystem* audio, Memory::Arena* arena_main){
    assert(audio->initialized == false);

    size_t memory_size = AUDIO_MEMORY_ALLOWANCE;
    audio->fmod_memory = Memory::CreateSubArena(arena_main, memory_size);
    FMOD_RESULT memory_init_ok = FMOD_Memory_Initialize(audio->fmod_memory, memory_size,
↪ nullptr, nullptr, nullptr, FMOD_MEMORY_ALL);
    assert(memory_init_ok == FMOD_OK);

    FMOD_RESULT system_creation_ok = FMOD_System_Create(&audio->sound_system,
↪ FMOD_VERSION);
    assert(system_creation_ok == FMOD_OK);

    FMOD_RESULT system_init_ok = FMOD_System_Init(audio->sound_system,
↪ AudioSystem::CHANNEL_COUNT, FMOD_INIT_NORMAL, nullptr);
    assert(system_init_ok == FMOD_OK);

    g_audioSystem = audio;

    audio->initialized = true;
}

```

Every FMOD function returns a `FMOD_RESULT` that tells us what happened during the function call. We can check this against `FMOD_OK` to make sure that it executed successfully. We use four `asserts` to help us catch errors with our audio initialization. First we create a `SubArena` from `arena_main` and assign the first memory address to `audio->fmod_memory`. This is of the type `void*` it is just a place in memory, not a pointer to a variable/struct. We then call `FMOD_Memory_Initialize` and pass along the point in memory and the `memory_size`. If everything went well we can continue.

We then create the `FMOD_SYSTEM` that we point to using `audio->sound_system`.

finally we initialize the `FMOD_SYSTEM` letting FMOD handle the setup behind the scenes. With all of this done we can assign our global pointer to point at our `audio` and flip our `initialized` flag to `true`.

That is the entire setup to have FMOD in our project. The next steps are about loading our sound effects, finding a free channel and actually playing audio

```
// audioSystem.cpp
// part 3

const int NOT_FOUND = -1;
int GetAvailableChannelIndex(AudioSystem* audio){
    for (int i = 0; i < AudioSystem::CHANNEL_COUNT; i++) {
        FMOD_CHANNEL* channel = audio->channels[i];
        if(channel == nullptr){ // never used before
            return i;
        }
        FMOD_BOOL is_playing = false;
        FMOD_Channel_IsPlaying(channel, &is_playing);
        if(is_playing == false){
            return i;
        }
    }

    return NOT_FOUND;
}
```

This is a function not found in our `audioSystem.h` as this is just a helper function that only `audioSystem.cpp` needs to be able to call. We can not return a pointer to a `FMOD_CHANNEL` directly because before we have ever used a channel it is actually `nullptr`. Therefore we use a `sentinel` value of `-1 aka NOT_FOUND` to represent not having an available channel. This function loops over all channels and checks through `FMOD_Channel_IsPlaying` if the channel is available. If yes it returns that index.

We could make a more advanced system later where audio has different priorities and if no channel is found it just grabs the channel containing the audio with the lowest priority. But for now we're going to ignore this.

```

// audioSystem.cpp
// Part 4

void PlaySFX(SFX_ID id, float volume){
    assert(id != SFX_ID::COUNT);
    FMOD_SOUND* sfx = g_audioSystem->soundEffects[(int)id];
    if(sfx == nullptr){
        assert(id != SFX_ID::FALLBACK);
        PlaySFX(SFX_ID::FALLBACK);
    }
    int channel_index = GetAvailableChannelIndex(g_audioSystem);
    if(channel_index == NOT_FOUND){
        return;
    }
    FMOD_CHANNEL** channel_slot = &g_audioSystem->channels[channel_index];

    FMOD_System_PlaySound(g_audioSystem->sound_system, sfx, nullptr, false, channel_slot);
    FMOD_Channel_SetVolume(*channel_slot, volume);
}

```

Playing sound effects becomes a very simple setup. We find the appropriate `FMOD_SOUND` from our `soundeffects` array, making sure it exists, otherwise we call `FALLBACK` instead. We then look for an available `channel`. If no such channel is found we can `return` early. Otherwise we fetch a pointer to the channel pointer from the `channel_index` and call `FMOD_System_PlaySound()` this connects a `FMOD_SOUND` to a `FMOD_CHANNEL` and tells `FMOD_SYSTEM` to begin streaming data from one to the other and into our computers sound card (in simplified terms). We also allow for a volume float to be passed in. calling `SetVolume` on our channel tells it how loud the sfx should be.

`FMOD` does not automatically keep itself working. We need to make sure we update it each frame.

```
// audioSystem.cpp
// part 5

void Update(AudioSystem* audio){
    if(g_audioSystem == nullptr || g_audioSystem != audio){
        g_audioSystem = audio;
    }
    assert(audio->initialized);
    FMOD_System_Update(audio->sound_system);
}
```

this function makes sure that `g_audioSystem` always point to our audio. We need this to ensure that nothing breaks after a `hot-reload` as `g_audioSystem` lives as part of our `.DLL`.

lastly we call `FMOD_System_Update` and with that our FMOD system will continue to function.

With all of these function calls you should refer to FMODs documentation or ask an LLM or quick-start guide online!

Lets look at loading our SFX now

```
// audioSystem.cpp
// part 6

namespace AssetManagement{
    void LoadAllSFX(AudioSystem* audioSystem){
        for (const SoundDataEntry& sound_data : all_sound_data) {
            FMOD_RESULT sound_created_ok = FMOD_System_CreateSound(audioSystem->sound_system,
↪ sound_data.path, FMOD_DEFAULT, nullptr,
↪ &audioSystem->soundEffects[(int)sound_data.id]);
            assert(sound_created_ok == FMOD_OK);
        }
    }
}
```

We loop over all `SoundDataEntry` from `all_sound_data`. to not have to worry about `for-looping` to a `_amount` variable we can use the other syntax for a for-loop that is “more modern” in c++. this will grab our array and intuit the size itself. And by putting `&` after the variable type we are

using it `by reference` instead of `by value` or `by pointer` meaning that the compiler does the heavy lifting and goes and grabs it for us.

as always, knowing why some parameters are `nullptr` and what they do is part of reading documentation, quick-start guides or discussing the implementation with LLMs.

Now to test our audio we can call `PlaySFX()` from our `UpdateGame()`. We'll check the result of `TryMove()` and if `true` we play `::JUMP`

```
// game.cpp

if(!IsActing(entity)){ // old
    TryMove(entity, level, gameplay->commandBuffer, xDir, yDir, entity->strength); //
↪ old
    // here we fetch the result
    bool moved = TryMove(entity, level, gameplay->commandBuffer, xDir, yDir,
↪ entity->strength);
    if(moved){
        PlaySFX(SFX_ID::JUMP); // play audio if the player managed to change position
    }
    gameplay->commandBuffer->timestamp += 1; // old
    gameplay->input_buffer_read_count++; // old
}
```

Now we can build our game and hear our fallback sound effect each time the player takes a step!

35 Animation III

Lets add some idle animations to Medusa!

in `chapter 36 asset.zip` you'll find the three new spritesheets we'll be working with. Lets set them up in our `spriteLibrary.h/.cpp`

```
// spriteLibrary.h
enum class SPRITE_ID{
    Fallback,
    Rock,
    Demon,
    Medusa_Rotate,
    Medusa_Idle_Left, // new
    Medusa_Idle_Front, // new
    Medusa_Idle_Back, // new
    Golem,
    Siren,
    Dropshadow,
    titlescreen_background,
    black_1x1,
    dungeon_tileset,
    selection_marker,
    Goal
};

// spriteLibrary.cpp

static const SpriteDataEntry all_sprite_data[] = {
    {SPRITE_ID::Fallback, FALLBACK_PATH,8,8},
    {SPRITE_ID::Demon, "assets/sprites/player.png"},
    {SPRITE_ID::Rock, "assets/sprites/rock.png", 10, 18},
    {SPRITE_ID::Medusa_Rotate, "assets/sprites/medusa_rotate.png", 12, 24, 8, 1},
    {SPRITE_ID::Medusa_Idle_Left, "assets/sprites/medusa_idle_left.png", 12, 24, 4, 1}, //
    ↪ new
    {SPRITE_ID::Medusa_Idle_Front, "assets/sprites/medusa_idle_front.png", 12, 24, 4, 1},
    ↪ // new
    {SPRITE_ID::Medusa_Idle_Back, "assets/sprites/medusa_idle_back.png", 12, 24, 4, 1}, //
    ↪ new
    {SPRITE_ID::Dropshadow, "assets/sprites/dropshadow.png", 8, 8},
    {SPRITE_ID::black_1x1, "assets/sprites/1x1_black.png",0,0},
    {SPRITE_ID::titlescreen_background, "assets/sprites/titlescreen.png"},
    {SPRITE_ID::selection_marker, "assets/sprites/selection_marker.png",9,9},
    {SPRITE_ID::dungeon_tileset, "assets/sprites/hell_of_a_time_dungeon_tileset.png",0,0,
    ↪ 9, 9},
    {SPRITE_ID::Goal, "assets/sprites/goal.png",8, 8, 8, 1}
};
```


Each of these animations are four frames long.

Then lets extend our `SpriteRenderInfo` struct to hold a value for if the sprite should be flipped along the `x-axis`.

```
// spriteLibrary.h

struct SpriteRenderInfo{
    Sprite* sprite;
    int frame;
    bool flipped_x; // new

    SpriteRenderInfo(){
        this->sprite = nullptr;
        this->frame = 0;
        this->flipped_x = false;
    }

    SpriteRenderInfo(int frame, Sprite* sprite){
        this->frame = frame;
        this->sprite = sprite;
        this->flipped_x = false;
    }

    SpriteRenderInfo(int frame, Sprite* sprite, bool flipped_x){ // new
        this->frame = frame;
        this->sprite = sprite;
        this->flipped_x = flipped_x;
    }

    SpriteRenderInfo(Sprite* sprite){
        this->sprite = sprite;
        this->frame = 0;
        this->flipped_x = false;
    }
};
```

We'll also set this to false for each of the constructors except a new fourth constructor that accepts a bool as a third parameter. This will allow us to skip adding this parameter and still pass `Sprite` as if it was a full `SpriteRenderInfo` struct with default values.

Because we want our animations to play all the time, even when the player has made no inputs we have to have some way of giving our

`GetSprite_FromEntityState` function knowledge about elapsed time. We're going to make a concession here, if the game can't be played at a stable framerate then this function will cause our animations to not run smoothly. but(!) not running at a stable framerate is not an option for a shipped title - so any safeguard here would be a silly bandaid for the actual solution which is to improve our performance until it can at least hit a stable 30fps. Right now on my machine we always hit our target of 240fps.

We'll be holding onto a `uint64_t` in `GameData`. This will increase by `1` each frame/tick. And because a `uint64_t` can hold fantastically huge numbers it will actually take hundreds of years before we get to the upper bounds of this variable at 240fps.

```
// gameState.h

struct GameData {
    SCENE_TYPES scene_current;
    SCENE_TYPES scene_previous;
    Scenes scenes;
    Transition transition;
    EditorData editor_data;
    Input input;
    Sprite* spriteBuffer;
    Tileset* tilesetBuffer;
    AudioSystem audio;
    uint64_t* ticks_total; // new
```

Then we need to allocate it from `main.cpp`

```
// main.cpp

gameData->ticks_total = ALLOC(arena_main, uint64_t);
```

Then in `game.cpp` we will give it a base value of `0` at Initialization then increase it by `1` each time we call `Update()`

```
// game.cpp

void Initialize(GameData* data, SDL_Window* window, SDL_Renderer* renderer){
    *data->ticks_total = 0; // <----- new
    DEV::Initialize(window, renderer);
    InitializeAudioSystem(&data->audio, data->arena_main);
    AssetManagement::LoadAllSFX(&data->audio);
    AssetManagement::LoadAllSprites(data->spriteBuffer, renderer);
    data->ImGui_context = ImGui::GetCurrentContext();

    AssetManagement::LoadAllTilesets(data->tilesetBuffer, data->arena_images);

    SDL_Texture* blackfade = GetSprite(SPRITE_ID::black_1x1,
    ↪ data->spriteBuffer)->texture;
    SDL_SetTextureBlendMode(blackfade, SDL_BLENDMODE_BLEND);
    InitializeGame(&data->scenes.gameplay, data->arena_levels, data->tilesetBuffer);
    InitializeMenu(&data->scenes.mainMenu, data->spriteBuffer, data->arena_main);
    ChangeScene(data, SCENE_TYPES::MAINMENU);
}
```

we have to make sure we assign `0` to the value being pointed to and not the pointer itself. As we're allowed to set the pointer to `0` causing it to become `nullptr` instead. So take note of the `*` before the name.

```
// game.cpp

void Update(GameData* data, float dt){
    *data->ticks_total += 1;
    // other code hidden for brevity
```

We have to remember the **dereference *** here as well or we'll make it `nullptr`.

Now we need to add this `tick_total` to our `GetSprite_FromEntityState()`

```
// spriteLibrary.h

SpriteRenderInfo GetSprite_FromEntityState(Entity* entity, Sprite* spritebuffer, const
    ↪ uint64_t* ticks_total);
```

and then use it in that function to calculate the relevant frame. Note, this function is still very messy.

```

// spriteLibrary.cpp
// in GetSprite_FromEntityState()

switch (entity->id) { // old
    case ENTITY_ID::MEDUSA:{
        Sprite* sprite = nullptr;
        int frame = 0;
        switch (entity->facing_current) {
            case Direction::RIGHT:
                sprite = GetSprite(SPRITE_ID::Medusa_Idle_Left, spritebuffer);
                frame = (int)((*ticks_total * 8) / FPS % 4);
                return {frame, sprite, true};
            case Direction::LEFT:
                sprite = GetSprite(SPRITE_ID::Medusa_Idle_Left, spritebuffer);
                frame = (int)((*ticks_total * 8) / FPS % 4);
                return {frame, sprite};
            case Direction::DOWN:
                sprite = GetSprite(SPRITE_ID::Medusa_Idle_Back, spritebuffer);
                frame = (int)((*ticks_total * 8) / FPS % 4);
                return {frame, sprite};
            case Direction::UP:
                sprite = GetSprite(SPRITE_ID::Medusa_Idle_Front, spritebuffer);
                frame = (int)((*ticks_total * 8) / FPS % 4);
                return {frame, sprite};
        }
    }
    case ENTITY_ID::DEMON:
        return GetSprite(SPRITE_ID::Demon, spritebuffer);
    case ENTITY_ID::ROCK:
        return GetSprite(SPRITE_ID::Rock, spritebuffer);
    default:
        return GetSprite(SPRITE_ID::Fallback, spritebuffer);
}

```

At the end of the function, after we have not entered the `if-statement` guarded by us having to have been in `Actions::ROTATING` we instead fetch the relevant idle animation, which for all entities other than Medusa is just a still frame.

For medusa we check the relevant Direction then fetch the corresponding spritesheet. We then calculate the correct frame by the following equation:

```
(total_ticks * framerate) / game_update_speed_in_frames % frames_in_animation.
```

This will for as long as we maintain a stable framerate select frame 0,1,2 or 3 as it evaluates `ticks_total`. In the case of `Direction::RIGHT` we give it a `true` as a third parameter before returning. This is our new `flipped_x` parameter. Meaning that when the character faces to the right we will render the left facing spritesheet flipped along the x-axis.

Now we should make our `RenderSprite_World()` function care about our `flipped_x` value

```
// rendering.cpp  
// at the bottom of the function  
SDL_FlipMode flip = (flipped || spriteRenderInfo.flipped_x) ?  
    ↪ SDL_FlipMode::SDL_FLIP_HORIZONTAL : SDL_FlipMode::SDL_FLIP_NONE;  
SDL_RenderTextureRotated(renderer, sprite->texture, &tilesetRect, &rect, 0, 0, flip); //  
    ↪ old
```

by doing `flipped || spriteRenderInfo.flipped_x` we will make `flip` evaluate to `true` if either value was set to `true`.

With this our Medusa idles as the game runs!

But we can do better. The magic number for `framerate` in our frame algorithm can be put into our `Sprite` struct and we can use the fact that we have our helper function `GetSpriteCount` to get the amount of frames.

```
// spriteLibrary.h

struct Sprite{
    SDL_Texture* texture;
    int width;
    int height;
    int pivot_x;
    int pivot_y;
    int sprite_count_x;
    int sprite_count_y;
    int framerate; // new
};

struct SpriteDataEntry{
    SPRITE_ID id;
    const char* path;
    int pivot_x = NOT_SET;
    int pivot_y = NOT_SET;
    int tileset_cell_count_x = NOT_SET;
    int tileset_cell_count_y = NOT_SET;
    int framerate = NOT_SET; // new
};
```

Then we need to assign it to `Sprite`

```
// spriteLibrary.cpp
void LoadSprite(Sprite* spriteBuffer, SpriteDataEntry entry, SDL_Renderer* renderer){
    // other code hidden for brevity
    sprite->framerate = entry.framerate;
}
```

And finally we need to assign it to our idle animations `SpriteDataEntry`

```
//spriteLibrary.cpp
{
    .id = SPRITE_ID::Medusa_Idle_Left,
    .path = "assets/sprites/medusa_idle_left.png",
    .pivot_x = 12,
    .pivot_y = 24,
    .tileset_cell_count_x = 4,
    .tileset_cell_count_y = 1,
    .framerate = 8
},
{SPRITE_ID::Medusa_Idle_Front, "assets/sprites/medusa_idle_front.png", 12, 24, 4, 1,
↵ 8},
{SPRITE_ID::Medusa_Idle_Back, "assets/sprites/medusa_idle_back.png", 12, 24, 4, 1, 8},
```

I've opted to show a different way of styling our `{}`. If we add a `.` and the

name of the variable we can designate them by name, making it easier to understand which variable we're actually adding to. This might become more necessary as our `Sprite` struct grows or maybe we should see this as a bit `smelly` and look for another way of simplifying/clarifying this.

Lets go back to our frame calculation algorithm

```
// spriteLibrary.cpp

switch (entity->id) {
case ENTITY_ID::MEDUSA:{
Sprite* sprite = nullptr;
int frame = 0;
switch (entity->facing_current) {
case Direction::RIGHT:
sprite = GetSprite(ENTITY_ID::MEDUSA_Idle_Left, spritebuffer);
frame = (int)((*ticks_total * sprite->framerate) / FPS % GetSpriteCount(sprite));
return {frame, sprite, true};
case Direction::LEFT:
sprite = GetSprite(ENTITY_ID::MEDUSA_Idle_Left, spritebuffer);
frame = (int)((*ticks_total * sprite->framerate) / FPS % GetSpriteCount(sprite));
return {frame, sprite};
case Direction::DOWN:
sprite = GetSprite(ENTITY_ID::MEDUSA_Idle_Back, spritebuffer);
frame = (int)((*ticks_total * sprite->framerate) / FPS % GetSpriteCount(sprite));
return {frame, sprite};
case Direction::UP:
sprite = GetSprite(ENTITY_ID::MEDUSA_Idle_Front, spritebuffer);
frame = (int)((*ticks_total * sprite->framerate) / FPS % GetSpriteCount(sprite));
return {frame, sprite};
}
}
```

now we have gotten rid of all our `magic numbers` and if we build and run everything still works just as before!

One tiny thing before we end this chapter. Our camera does not exactly place our level in the center of the screen. We need 1 more step

```
// camera.cpp

void camera::GridToWorld(float* x, float* y, const LevelData* lvl){
    *x *= TILE_SIZE_PX_SCALED;
    *x += TILE_SIZE_PX_SCALED; // new
    *x += SCREEN_WIDTH / 2.0;
    *x -= lvl->w * TILE_SIZE_PX_SCALED / 2.0;
    *y *= TILE_SIZE_PX_SCALED;
    *y += TILE_SIZE_PX_SCALED; // new
    *y += SCREEN_HEIGHT / 2.0;
    *y -= lvl->h * TILE_SIZE_PX_SCALED / 2.0;
}
```

that I believe should do the trick!

36 Music

With `FMOD core` implemented it's easy to set up music playback. But because a music track can be very large and a game might have a lot of tracks playing throughout the game we can't really pre-load all music as we do for our sound effects. Instead we'll be streaming the music a few bytes at a time.

```
// audioSystem.h

enum class SONG_ID{
    NONE,
    THEME
};

struct AudioSystem{
    bool initialized;
    void* fmod_memory;
    FMOD_SYSTEM* sound_system;
    static const int CHANNEL_COUNT = 32;
    FMOD_CHANNEL* channels[CHANNEL_COUNT];
    FMOD_SOUND* soundEffects[(int)SFX_ID::COUNT];

    SONG_ID song_id; // new
    FMOD_SOUND* song; // new
    FMOD_CHANNEL* song_channel; // new
};

void PlaySong(SONG_ID id);
```

We create a new enum to hold all of our songs, then we create a separate `FMOD_SOUND` and `FMOD_CHANNEL` that we'll use exclusively for music. a new function `PlaySong()` will be the only function call we'll need.

```

// audioSystem.cpp
void PlaySong(SONG_ID id){
    g_audioSystem->song_id = id;
    if(g_audioSystem->song != nullptr){
        FMOD_Channel_Stop(g_audioSystem->song_channel);
        FMOD_Sound_Release(g_audioSystem->song);
    }

    FMOD_SYSTEM* system = g_audioSystem->sound_system;
    const char* song_name;
    switch (id) {
    case SONG_ID::THEME:
        song_name = "assets/audio/music/hellofatime.mp3";
        break;
    case SONG_ID::NONE:
        break;
    }

    FMOD_System_CreateStream(system, song_name, FMOD_LOOP_NORMAL, nullptr,
↪ &g_audioSystem->song);
    int FOREVER = -1;
    FMOD_Sound_SetLoopCount(g_audioSystem->song, FOREVER);
    FMOD_System_PlaySound(system, g_audioSystem->song, nullptr, false,
↪ &g_audioSystem->song_channel);
}

```

We check if we have something playing, in that case we stop the channel and call `_Release()`. We grab our `FMOD_SYSTEM` from `g_audioSystem` and instead of our more elaborate `SoundDataEntry` we just pick the `song_name` directly from a switch case. We do this because even a huge game rarely has more than 30-50 songs. and we currently have 1...

We start a `Stream` that will pull in our `.mp3` piece by piece as needed. Then we set the `Loop Count` to `-1` which FMOD interprets as keep looping forever. we also use `FMOD_LOOP_NORMAL` instead of `FMOD_DEFAULT` when creating the stream to let FMOD know that we want this stream to loop once finished. `SetLoopCount` just controls how many times it's allowed to loop.

Then we call `PlaySound()` as normal.

In `Game.cpp` we call our play function

```

// game.cpp
void Initialize(GameData* data, SDL_Window* window, SDL_Renderer* renderer){
    *data->ticks_total = 0;
    DEV::Initialize(window, renderer);
    InitializeAudioSystem(&data->audio, data->arena_main);
    AssetManagement::LoadAllSFX(&data->audio);
    AssetManagement::LoadAllSprites(data->spriteBuffer, renderer);
    data->imGui_context = ImGui::GetCurrentContext();

    AssetManagement::LoadAllTilesets(data->tilesetBuffer, data->arena_images);

    SDL_Texture* blackfade = GetSprite(SPRITE_ID::black_1x1,
↪ data->spriteBuffer)->texture;
    SDL_SetTextureBlendMode(blackfade, SDL_BLENDMODE_BLEND);
    InitializeGame(&data->scenes.gameplay, data->arena_levels, data->tilesetBuffer);
    InitializeMenu(&data->scenes.mainMenu, data->spriteBuffer, data->arena_main);

    PlaySong(SONG_ID::THEME); // <----- new

    ChangeScene(data, SCENE_TYPES::MAINMENU);
}

```

That's it! Now we can play music by streaming our audio!

37 Parallax

Parallax is the effect of things further away not moving out of your field of view as fast as objects that are closer to you. Hold out a finger and slide your head from left to right. Your finger will move more relative to your head than the wall behind it.

We use Parallax to produce depth in 2D scenes. This is featured prominently in 2D sidescrollers. We'll be replicating the effect found in a game called **Arco** for our main menu. in the **chapter 38 assets.zip** you'll find five images that when stacked ontop of each other form our final main menu. Please add these to our **assets/sprites** folder.

We'll be referencing these in our **mainmenu.h/.cpp** as well as our **spriteLibrary.h/.cpp**

```
// spriteLibrary.h

enum class SPRITE_ID{
    // other enums hidden for brevity

    Menu_Horizon,
    Menu_Cloud_Back,
    Menu_Cloud_Front,
    Menu_Middle,
    Menu_Front
};

// spriteLibrary.cpp

static const SpriteDataEntry all_sprite_data[] = {

    // other SpriteDataEntries hidden for clarity

    {SPRITE_ID::Menu_Horizon, "assets/sprites/mainmenu_background.png"},
    {SPRITE_ID::Menu_Cloud_Back, "assets/sprites/mainmenu_cloud_back.png"},
    {SPRITE_ID::Menu_Cloud_Front, "assets/sprites/mainmenu_cloud_front.png"},
    {SPRITE_ID::Menu_Middle, "assets/sprites/mainmenu_middle.png"},
    {SPRITE_ID::Menu_Front, "assets/sprites/mainmenu_front.png"},
};
```

```

// mainmenu.h

struct MainMenu {
    Button* buttons;
    int button_count;
    int activeButtonIndex;
    Button** activeButtons;
    int activeButtonCount;
    bool initialized;
    Sprite* background_horizon; // new
    Sprite* background_cloud_back; // new
    Sprite* background_cloud_front; // new
    Sprite* background_middle; // new
    Sprite* background_front; // new
};

// new parameter
void DrawMenu(MainMenu* mainmenu, SDL_Renderer* renderer, Sprite* spriteBuffer, Input*
↪ input);

```

We add references to these five partial sprites in `MainMenu` and we will also be needing the mouse position to control the parallax effect in the main menu. To do that we've added `Input* input` to our `DrawMenu()` function.

this means that we have to update the callsite in `game.cpp`

```

// game.cpp
// in DrawScene()

case SCENE_TYPES::MAINMENU:
    DrawMenu(&data->scenes.mainMenu, renderer, data->spriteBuffer, &data->input);
break;

```

With this we can add the necessary logic to `mainmenu.cpp`

```

// mainmenu.cpp
void InitializeMenu(MainMenu* mainmenu, Sprite* spriteBuffer, Memory::Arena*
↪ arena_main){
    assert(mainmenu->initialized == false);
    mainmenu->button_count = 2;
    mainmenu->buttons = ALLOC_ARRAY(arena_main, Button, mainmenu->button_count);

    SetupButton(&mainmenu->buttons[0], ButtonType::START_GAME, spriteBuffer, {SCREEN_WIDTH
↪ / 2.0, SCREEN_HEIGHT / 2.0, 200, 80}, ButtonMode::Centered);
    SetupButton(&mainmenu->buttons[1], ButtonType::QUIT, spriteBuffer, {SCREEN_WIDTH /
↪ 2.0, (SCREEN_HEIGHT / 2.0) + 100, 200, 80}, ButtonMode::Centered);

    // new
    mainmenu->background_horizon = GetSprite(SPRITE_ID::Menu_Horizon, spriteBuffer);
    mainmenu->background_cloud_back = GetSprite(SPRITE_ID::Menu_Cloud_Back, spriteBuffer);
    mainmenu->background_cloud_front = GetSprite(SPRITE_ID::Menu_Cloud_Front,
↪ spriteBuffer);
    mainmenu->background_middle = GetSprite(SPRITE_ID::Menu_Middle, spriteBuffer);
    mainmenu->background_front = GetSprite(SPRITE_ID::Menu_Front, spriteBuffer);

    mainmenu->initialized = true;
}

```

we fetch the relevant sprites.

```
// mainmenu.cpp
void DrawMenu(MainMenu* mainmenu, SDL_Renderer* renderer, Sprite* spriteBuffer, Input*
↪ input){
    float scale = (SCREEN_HEIGHT / ((float)mainmenu->background_horizon->height *
↪ UPSCALE_FACTOR));
    scale *= 1.2;
    float mouse_x = input->mouse_x;
    float mouse_y = input->mouse_y;
    float center_x = SCREEN_WIDTH / 2.0;
    float center_y = SCREEN_HEIGHT / 2.0;
    float offset_x = center_x - mouse_x;
    float offset_y = center_y - mouse_y;
    RenderSprite_World(GetSprite(SPRITE_ID::Menu_Horizon, spriteBuffer),      renderer,
↪ NULL, center_x, center_y, scale);
    RenderSprite_World(GetSprite(SPRITE_ID::Menu_Cloud_Back, spriteBuffer),  renderer,
↪ NULL, center_x + (offset_x / 11), center_y + (offset_y / 11), scale);
    RenderSprite_World(GetSprite(SPRITE_ID::Menu_Cloud_Front, spriteBuffer), renderer,
↪ NULL, center_x + (offset_x / 9), center_y + (offset_y / 9), scale);
    RenderSprite_World(GetSprite(SPRITE_ID::Menu_Middle, spriteBuffer),      renderer,
↪ NULL, center_x + (offset_x / 7), center_y + (offset_y / 7), scale);
    RenderSprite_World(GetSprite(SPRITE_ID::Menu_Front, spriteBuffer),      renderer,
↪ NULL, center_x + (offset_x / 5), center_y + (offset_y / 5), scale);

    for (int i = 0; i < mainmenu->activeButtonCount; i++) {
        Button* button = mainmenu->activeButtons[i];
        RenderButton(mainmenu->activeButtons[i], i == mainmenu->activeButtonIndex,
↪ renderer);
    }
}
```

We then refactor our `DrawMenu()`. We now fetch the `x/y` of our mouse then calculate how far the mouse is from the dead center of the game window. With this offset we can render all of our menu art pieces at the center of the screen plus a portion of this offset. The sprites that we render first get offset by a smaller value than the objects closer to us. You can try and reverse this order for a different effect all together.

We also multiply `scale` by `1.2` as we need to be zoomed in a little to avoid our sprites cutting off at the edge of the 1920x1080 window. You can try and remove the scale multiplication and watch as we reach the edge of our sprites.

This is the basics of parallax. We adjust the position of something in relation-

ship to some movement of a camera based on how far away from the camera it is. Currently we just divide the offset by a value that will represent the depth of the object. The larger the fraction the further away the layer is. And no fraction at all means that it's at the horizon-distance.

That's it!

38 Text

We can of course already render text if each time we want to do so we just create a bespoke sprite and use that as a text-proxy. But this is not a good way of doing it and neither is it industry standard.

What we'll be doing is rendering text one character at a time by taking a font and converting it to a `SDL_Texture`.

The process will be us creating what is called a `Texture Atlas` a texture atlas is similar to a `spritesheet` because it has multiple individual things all layed out in a larger grid.

To convert a font into a texture we need to work with some external library (or spend a lot of effort coding our own). The easiest solution for us is to download `SDL_TTF` a library that helps us work with `TTF` files. a TTF file is a `True Text Font`. This is the same file format used by all your text-displaying software.

the `SDL_TTF` github is at: https://github.com/libsdl-org/SDL_ttf after navigating to `Releases` we're downloading `SDL3_ttf-devel-3.2.2-VC.zip` this will let us work with the `.h` files and `.dll` files directly as it our practice for this course.

after having unzipped our file we will find the `SDL_textengine.h` and `SDL_ttf.h` in `include` and add it to our own. I've opted to put these two `.h` files into their own subdirectory inside `include` that I've named `SDL_TTF`.

we also need `SDL3_ttf.dll` and `SDL3_ttf.lib`. These are going into their own subdirectory inside our `lib` folder. I've named their subdirectory `SDL_TTF` just as I did for our `include` folder.

Thankfully this will work out of the gate requiring no adjustments to our `cmakelists.txt`.

For this chapter I've downloaded a font called `ByteBounce` from: <https://www.1001fonts.com/bytebounce-font.html> I've put this `.ttf` file into a new subdirectory `assets/fonts`. We'll be using this path to load it later.

We want to store our text in a fashion to save us from having to re-create a texture each frame that we want to show text to the screen - that would be terribly slow.

We need to initialize `SDL_TTF` if we miss this step then even if we code everything correct afterwards then nothing will show up on our screen.

```
// main.cpp
void SDL_Setup(){
    SDL_Init(SDL_INIT_EVENTS);
    SDL_SetLogPriorities(SDL_LOG_PRIORITY_VERBOSE);

    TTF_Init(); // <----- new

    window = SDL_CreateWindow("hell of a time", SCREEN_WIDTH, SCREEN_HEIGHT, 0);
    renderer = SDL_CreateRenderer(window, NULL);
}
```

in a new `fontLibrary.h` we'll add structs to hold individual letters that we call `glyphs` when working with fonts as well as a struct to hold our `font atlas`

```
// fontLibrary.h

#pragma once

#include "SDL3/SDL_rect.h"
#include "SDL3/SDL_render.h"

struct Glyph{
    SDL_FRect atlasPosition;
};

struct FontAtlas {
    SDL_Texture* atlasTexture;
    static const int GLYPH_COUNT = 128;
    Glyph glyphs[GLYPH_COUNT];
};

namespace AssetManagement{
    void LoadFont(SDL_Renderer* renderer, const char* font_path, FontAtlas* fontAtlas,
        ↪ float ptsize);
}
```

`Glyph` just has a `SDL_FRect` inside of it now, meaning that we could ignore it and just work with the `SDL_FRect` struct directly. But I find the `Glyph` struct easy to understand at a glance and we might be adding more variables to this struct later. If you wish you can ignore creating the `Glyph` struct.

`FontAtlas` has a pointer to a `SDL_Texture` we'll be creating this texture at runtime. We're also working with an array of a known size. So we can specify the size as a `static const int` meaning that the value can't change and we'll be able to access this number by accessing the struct type itself.

Finally we create an array of `Glyphs` with the specified size. the `128` size will be enough to fetch all common numbers, letters and symbols on our keyboard. There is more to font handling and especially when it comes to localization. But for our needs and the font we've selected this will be plenty.

We've added a new function to our `AssetManagement` namespace. This

will grab a font and construct the contents of a `FontAtlas` based on it.

With only one function in our `.h` file we have only one function to write in a `fontLibrary.cpp`. We could have `inline` the function but we have not done so for other `xxxLibrary.h/.cpp` pairs so I'm opting for consistency.

```

// fontLibrary.cpp
#include "fontLibrary.h"
#include "SDL3/SDL_pixels.h"
#include "SDL3/SDL_render.h"
#include "SDL3/SDL_surface.h"
#include "SDL_TTF/SDL_ttf.h"
#include <cassert>

namespace AssetManagement{
    void LoadFont(SDL_Renderer* renderer, const char* font_path, FontAtlas* fontAtlas,
        ↪ float ptsize){
        TTF_Font* font = TTF_OpenFont(font_path, ptsize);
        assert(font != nullptr);
        SDL_Color white = {255,255,255,255};

        int atlas_size = 1024;
        SDL_Surface* atlas_surface;
        atlas_surface = SDL_CreateSurface(atlas_size, atlas_size, SDL_PIXELFORMAT_RGBA32);

        int draw_point_x = 0;
        int draw_point_y = 0;
        int tallest_glyph_in_row = 0;

        int FIRST_RELEVANT_GLYPH = 32;

        for (int i = FIRST_RELEVANT_GLYPH; i < FontAtlas::GLYPH_COUNT; i++) {
            SDL_Surface* glyph_surface = TTF_RenderGlyph_Blended(font, i, white);

            if(glyph_surface == nullptr){
                continue;
            }

            if(draw_point_x + glyph_surface->w > atlas_size){
                draw_point_x = 0;
                draw_point_y += tallest_glyph_in_row;
                tallest_glyph_in_row = 0;
            }

            if(tallest_glyph_in_row < glyph_surface->h){
                tallest_glyph_in_row = glyph_surface->h;
            }

            SDL_Rect glyph_position = {draw_point_x, draw_point_y, glyph_surface->w,
            ↪ glyph_surface->h};
            SDL_BlitSurface(glyph_surface, NULL, atlas_surface, &glyph_position);

            fontAtlas->glyphs[i].atlasPosition = {(float)glyph_position.x,
                                                    (float)glyph_position.y,
                                                    (float)glyph_position.w,
                                                    (float)glyph_position.h};

            draw_point_x += glyph_surface->w;
            SDL_DestroySurface(glyph_surface);
        }
        fontAtlas->atlasTexture = SDL_CreateTextureFromSurface(renderer, atlas_surface);
        SDL_DestroySurface(atlas_surface);
        TTF_CloseFont(font);
    }
}

```

Because `SDL_TTF` and the `base SDL` create some textures in memory outside of our memory arena we need to remember to call `Destroy()` and `Close()` functions. If we forget to do this we will not be able to recover this piece of memory for the entire runtime of the program even though we no longer need the memory. If we continue to not mark memory as available in a program that does not use memory arena structure we can eventually run out of memory completely and crash the program. This type of crash can happen at any time and can be very hard to track down.

We start by loading our font using `TTF_OpenFont()` this will allow us to fetch the font data. We then set up a new `SDL_Surface`. A `SDL_Surface` lives on the CPU and a `SDL_Texture` lives in `VRAM`. This makes `SDL_Texture` much for flexible and faster. But `SDL_Surface` is how we initially represent a texture before doing a conversion. the float `ptsize` control how large the font will draw the individual glyphs.

So we create a new `SDL_Surface` and give it a size of 1024x1024. This surface will be the proto-texture that we'll stamp each glyph onto. The goal is to stamp each glyph onto the surface in a grid, just like if it was a spritesheet.

`draw_point_x/y` will be the point on our surface that we will stamp the next glyph onto. As we continue stamping glyphs we'll move these two variables to ensure that each glyph has its own space on the spritesheet/atlas. `tallest_glyph_on_row` holds the height of the tallest glyph we've found since moving down to a new row. We need to shift down by (at least) this height in order to guarantee that the next row of glyph don't overlap with the once from the row above.

In a standard ascii table the glyphs from 0-31 are reserved for what is called `control codes` these are not meant to be rendered but instead used to help

control hardware and data. We have no use for these can they can only cause headaches. So we start our `for-loop` at the `32'nd` index instead. I put this in a named integer just to make the code `self-explanatory`.

We create the glyph as an `SDL_Surface` using `TTF_RenderGlyph_Blended` then sometimes a glyph can be missing from a font. In case it is `nullptr` we just continue to the next glyph.

```
if(draw_point_x + glyph_surface->w > atlas_size){
    draw_point_x = 0;
    draw_point_y += tallest_glyph_in_row;
    tallest_glyph_in_row = 0;
}
```

This if statement check if the stamp position along with the width of the glyph would go down beyond the right edge of our `atlas`. We it would we reset `x` back to the furthest left point aka `0`. We then shift the `draw_point_y` down by the height of the tallest glyph we've found so far. We then reset `tallest_glyph_in_row` as we're at the beginning of a new row.

```
if(tallest_glyph_in_row < glyph_surface->h){
    tallest_glyph_in_row = glyph_surface->h;
}
```

we check the current glyph against the tallest one we've found so far. And if the new glyph is taller we update our variable to reflect this.

```
SDL_Rect glyph_position = {draw_point_x, draw_point_y, glyph_surface->w,
    ↪ glyph_surface->h};
SDL_BlitSurface(glyph_surface, NULL, atlas_surface, &glyph_position);
```

We then construct a `SDL_Rect` that will be the square we stamp the glyph onto. It has both the position and the size of the rect itself.

We then call `BlitSurface`, this takes the pixels from one `SDL_Surface` and adds them to another `SDL_Surface` at a specified position. the `NULL` value

makes the program take the entire `glyph_surface` instead of a portion of it.

```
fontAtlas->glyphs[i].atlasPosition = {(float)glyph_position.x,  
                                     (float)glyph_position.y,  
                                     (float)glyph_position.w,  
                                     (float)glyph_position.h};
```

Our `fontAtlas` struct has our array of glyphs. We take the current index and ensure that it remembers this `glyph_position`. But you can see how we take each of the `x,y,w,h` and `cast` them to `float` this is because `atlasPosition` is a `SDL_FRect` instead of a `SDL_Rect`. We do this because the `SDL_Rendering` function we will use later demands that we work with `SDL_FRect`. Note how we're using the shorthand of `{}` to create the struct.

```
draw_point_x += glyph_surface->w;  
SDL_DestroySurface(glyph_surface);
```

As the last step of our `for-loop` we shift the `draw_point_x` the full width of the glyph. This will prepare the next loop of the `for-loop` to evaluate the next stamp position from just to the right of the last glyph.

`SDL_DestroySurface()` needs to be called now that we're done with this loop. We're never going to want to use this surface again, we've already stamped in onto our `atlas`.

We continue stamping glyphs until we've reached `GLYPH_COUNT`.

```
fontAtlas->atlasTexture = SDL_CreateTextureFromSurface(renderer, atlas_surface);  
SDL_DestroySurface(atlas_surface);  
TTF_CloseFont(font);
```

After that we take the now finished `SDL_Surface` `atlas_surface` and using `CreateTextureFromSurface` we store a pointer to a VRAM-living texture in `atlasTexture`. After that we destroy the now redundant `atlas_surface`

and call `CloseFont()` to free all the memory on the heap that we used to run this function.

We can add an enum to help us manage more fonts in the future. But for this chapter we'll only have the one. We'll need to store this in our `GameData`

```
// gameState.h
struct GameData {
    // other variables hidden for clarity

    FontAtlas font; // new
```

From `Initialize()` in `game.cpp` we'll call our `loadFont` function

```
// game.cpp

void Initialize(GameData* data, SDL_Window* window, SDL_Renderer* renderer){
    *data->ticks_total = 0;
    DEV::Initialize(window, renderer);
    InitializeAudioSystem(&data->audio, data->arena_main);
    AssetManagement::LoadAllSFX(&data->audio);
    AssetManagement::LoadAllSprites(data->spriteBuffer, renderer);
    data->imGui_context = ImGui::GetCurrentContext();

    AssetManagement::LoadFont(renderer, "assets/fonts/ByteBounce.ttf", &data->font, 48);
    ↪ // new
    AssetManagement::LoadAllTilesets(data->tilesetBuffer, data->arena_images);

    SDL_Texture* blackfade = GetSprite(SPRITE_ID::black_1x1, data->spriteBuffer)->te
```

We make sure to set a large enough `ptsize` to actually see the glyphs clearly.

`48` felt ok to me.

Rendering text will be a bit different. We'll add a bespoke function for it in `rendering.h/.cpp`

```
// rendering.h

struct Button;
struct FontAtlas;

enum class Alignment {
    Right,
    Centered
};

void RenderText(FontAtlas* atlas, const char* text, SDL_Renderer* renderer, Camera*
↪ camera, const float x, const float y, Alignment alignment);
```

`Alignment` is the same enum that we used to call `ButtonMode`. I've removed `ButtonMode` from `button.h` and placed this new `Alignment` enum in `rendering.h`. I've also forward-declared `Button` and `FontAtlas` to avoid circular dependencies.

The refactoring of `ButtonMode` to `Alignment` and the move from `button.h` to `rendering.h` means that we need to update our callsites that previously used `ButtonMode::`

This happens in `mainmenu.cpp` and `game.cpp` and `button.cpp`. Pressing `space+D` will list all of the current errors from `clangd`. You should be able to find each callsite by going through this list.

```
// rendering.cpp

static const char STOP_CHAR = '\\0';
void RenderText(FontAtlas* atlas, const char* text, SDL_Renderer* renderer, Camera*
↪ camera, const float x, const float y, Alignment mode){
    assert(atlas->atlasTexture != nullptr);
    float draw_position_x = x;
    float draw_position_y = y;
    if(camera != nullptr){
        draw_position_x -= camera->camera_x;
        draw_position_y -= camera->camera_y;
    }

    if(mode == Alignment::Centered){
        float totalWidth = 0;
        for (int i = 0; text[i] != STOP_CHAR; i++) {
            totalWidth += atlas->glyphs[text[i]].atlasPosition.w;
        }
        draw_position_x -= totalWidth / 2.0;
    }

    for (int i = 0; text[i] != STOP_CHAR; i++) {
        Glyph glyph = atlas->glyphs[text[i]];
        SDL_FRect renderRectangle = {draw_position_x, draw_position_y,
↪ glyph.atlasPosition.w, glyph.atlasPosition.h};
        SDL_RenderTexture(renderer, atlas->atlasTexture, &glyph.atlasPosition,
↪ &renderRectangle);

        draw_position_x += glyph.atlasPosition.w;
    }
}
```

A `char*` pointer array will always end with a `\\0` this is a special character that tells us that we have reached the end of the array. I've opted to place this in a variable to make the code easier to read.

This function loops over all the individual letters in `text` and select the appropriate rectangle in our atlas to render before shifting the position by its width and rendering the next letter.

if our `Alignment` is set to `Centered` then we first loop over all glyphs and collect their combined width. We can then shift `draw_position_x` by half that to have the text centered. We are also allow our `Camera*`

parameter being passed as `nullptr`. In that case we skip adjusting the `draw_position_x/y` by the camera offset.

our for loop responsible for rendering the text has `text[i] != STOP_CHAR` as its condition. This means that `i` will increase and the loops will continue until a `STOP_CHAR` has been reached. `char` also maps to `int` without any loss of information. This allows us to pass `text[i]` into our `glyphs` array to fetch the relevant `Glyph` struct - nifty!

Once we have the relevant glyph we use our `draw_position_x/y` along with the `glyph.atlasPosition.w/.h` to construct the position and size of the rectangle we will be drawing to the screen.

So we use the `glyph.atlasPosition` to find where on the `atlasTexture` our `Glyph` lives. Then we use `renderRectangle` to say where in the game we want to draw it.

Once we have drawn a `Glyph` we shift `draw_position_x` by its `width`.

This code could be further expanded to account for the text becoming too long and how to handle that. The most common case is shrinking it to fit or allowing the text to wrap down to a new line.

Now we can test our text rendering and write whatever we think is funny.

I've put a temporary call to `RenderText` in `DrawScene()`

```
// game.cpp  
// in the switch case inside DrawScene()  
  
case SCENE_TYPES::MAINMENU:  
    DrawMenu(&data->scenes.mainMenu, renderer, data->spriteBuffer, &data->input);  
    RenderText(&data->font, "hello sailor", renderer, &data->camera, SCREEN_WIDTH / 2.0,  
↪ SCREEN_HEIGHT / 2.0, Alignment::Centered);  
    break;
```

This is the basics of working with fonts and rendering text!

39 Buttons Part II

we could always use a fixed-size bespoke button texture. But when the text that we overlay on top of the button has different sizes we don't want our text to go outside of the bounds of the button texture. We could always make the text smaller. But a much more established way is to introduce **nine-slicing**. This means that we create our buttons as a 3x3 **atlas**. We then render each of the four corners at the appropriate positions then stretch the sprites between the corners until they fill the entire space.

This is not really “difficult” it just requires us to be a bit extra alert when programming the position code, there is a bunch of small equations to get the actual sizes and positions.

When creating this chapter I had to do quite a bit of testing to get things right. Just knowing how to write this without any hiccups is not how code like this usually goes.

We'll be adding a new variable to our **Button** struct

```
// button.h
struct Button {
    ButtonType type;
    SDL_FRect rect;
    Sprite* sprite;
    bool is_active;
    bool is_dynamic; // new
};
```

Previously our **Button** struct held a **SDL_Texture*** for its texture, but we've refactored this to be a **Sprite*** instead.

we use **is_dynamic** to control if we want to render our button using the normal method or a new render function we'll add to **rendering.h**

```

// rendering.h
void RenderButton_Dynamic(Button* button, bool is_selected, SDL_Renderer* renderer);

// mainmenu.cpp
void DrawMenu(MainMenu* mainmenu, SDL_Renderer* renderer, Sprite* spriteBuffer, Input*
↪ input){
    float scale = (SCREEN_HEIGHT / ((float)mainmenu->background_horizon->height *
↪ UPSCALE_FACTOR));
    scale *= 1.2;
    float mouse_x = input->mouse_x;
    float mouse_y = input->mouse_y;
    float center_x = SCREEN_WIDTH / 2.0;
    float center_y = SCREEN_HEIGHT / 2.0;
    float offset_x = center_x - mouse_x;
    float offset_y = center_y - mouse_y;
    RenderSprite_World(GetSprite(SPRITE_ID::Menu_Horizon, spriteBuffer),      renderer,
↪ NULL, center_x, center_y, scale);
    RenderSprite_World(GetSprite(SPRITE_ID::Menu_Cloud_Back, spriteBuffer),  renderer,
↪ NULL, center_x + (offset_x / 11), center_y + (offset_y / 11), scale);
    RenderSprite_World(GetSprite(SPRITE_ID::Menu_Cloud_Front, spriteBuffer),  renderer,
↪ NULL, center_x + (offset_x / 9), center_y + (offset_y / 9), scale);
    RenderSprite_World(GetSprite(SPRITE_ID::Menu_Middle, spriteBuffer),      renderer,
↪ NULL, center_x + (offset_x / 7), center_y + (offset_y / 7), scale);
    RenderSprite_World(GetSprite(SPRITE_ID::Menu_Front, spriteBuffer),      renderer,
↪ NULL, center_x + (offset_x / 5), center_y + (offset_y / 5), scale);
    for (int i = 0; i < mainmenu->activeButtonCount; i++) {
        Button* button = mainmenu->activeButtons[i];
        if(button->is_dynamic){ // <----- new
            RenderButton_Dynamic(button, i == mainmenu->activeButtonIndex, renderer);
        }
        else{
            RenderButton(button, i == mainmenu->activeButtonIndex, renderer);
        }
    }

    mainmenu->activeButtonCount = 0; // <----- new
}

```

We're also fixing a bug in our earlier code. When we press **Start Game** we stop calling **UpdateMenu** (on purpose). But the **activeButtonCount** and **activeButtons** are calculated in Update. This means that our activeButtons array when we stop calling Updates points at garbage memory. By setting **activeButtonCount** to **0** after we are done with a Draw call we can at least know that if we never call Update again then we will at least not have any value here to cause a loop through the garbage array. This is not a robust

fix, just a temporary measure so we can focus on what we're working on.

In `SetupButton()` we're assigning `is_dynamic` to our Start Game button.

```
// mainmenu.cpp
void InitializeMenu(MainMenu* mainmenu, Sprite* spriteBuffer, Memory::Arena*
↪ arena_main){
    assert(mainmenu->initialized == false);
    mainmenu->button_count = 2;
    mainmenu->buttons = ALLOC_ARRAY(arena_main, Button, mainmenu->button_count);

    // each parameter in its own line for clarity
    SetupButton(&mainmenu->buttons[0],
                ButtonType::START_GAME,
                spriteBuffer,
                {SCREEN_WIDTH / 2.0, SCREEN_HEIGHT / 2.0, 400, 170},
                Alignment::Centered,

    mainmenu->buttons[0].is_dynamic = true; // new

    SetupButton(&mainmenu->buttons[1], ButtonType::QUIT, spriteBuffer, {SCREEN_WIDTH /
↪ 2.0, (SCREEN_HEIGHT / 2.0) + 100, 200, 80}, Alignment::Centered);

    mainmenu->background_horizon      = GetSprite(SPRITE_ID::Menu_Horizon, spriteBuffer);
    mainmenu->background_cloud_back   = GetSprite(SPRITE_ID::Menu_Cloud_Back,
↪ spriteBuffer);
    mainmenu->background_cloud_front = GetSprite(SPRITE_ID::Menu_Cloud_Front,
↪ spriteBuffer);
    mainmenu->background_middle       = GetSprite(SPRITE_ID::Menu_Middle, spriteBuffer);
    mainmenu->background_front        = GetSprite(SPRITE_ID::Menu_Front, spriteBuffer);

    mainmenu->activeButtonIndex = 0;
    mainmenu->initialized = true;
}
```

With the change from `SDL_Texture*` to `Sprite*` we need to update our callsites that used the `SDL_Texture`. This was only two functions `SetupButton()` in `button.cpp` and `RenderButton` in `rendering.cpp`.

```

void SetupButton(Button* button, ButtonType type, Sprite* spriteBuffer, SDL_FRect rect,
↪ Alignment mode){
    assert(type != ButtonType::NONE);
    button->type = type;
    button->rect = rect;
    if(mode == Alignment::Centered){
        button->rect.x -= button->rect.w / 2;
        button->rect.y -= button->rect.h / 2;
    }
    button->is_active = true;
    switch(button->type){
    case ButtonType::START_GAME:
        button->sprite = GetSprite(SPRITE_ID::Button_Basic, spriteBuffer); // refactored to
↪ this + new ID
        break;
    case ButtonType::QUIT:
        button->sprite = GetSprite(SPRITE_ID::Fallback, spriteBuffer); // refactored to this
        break;
    default:
        button->sprite = GetSprite(SPRITE_ID::Fallback, spriteBuffer); // refactored to this
        break;
    }
}

```

We also make sure that our `START_GAME` button uses our `Button_Basic` `SPRITE_ID`.

```

// rendering.cpp

void RenderButton(Button* button, bool is_selected, SDL_Renderer* renderer){
    SDL_Texture* texture =button->sprite->texture; // refactored
    SDL_SetTextureScaleMode(texture, SDL_SCALEMODE_PIXELART);
    uint8_t colorOverlay = is_selected ? 255: 230;
    SDL_SetTextureBlendMode(texture, SDL_BLENDMODE_BLEND);
    SDL_SetTextureColorMod(texture, colorOverlay, colorOverlay, colorOverlay);
    SDL_RenderTexture(renderer, button->sprite->texture, NULL, &button->rect); //
↪ refactored
}

```

We are adding a new sprite to our assets folder


```
// spriteLibrary.h

enum class SPRITE_ID{
    // other SPRITE_IDS hidden for clarity
    Button_Basic
};
```

this is our 66x66 pixel square that we'll cut up to create our dynamic buttons

```
// spriteLibrary.cpp
const char* FALLBACK_PATH = "assets/sprites/fallback.png";

static const SpriteDataEntry all_sprite_data[] = {
    // other SpriteDataEntry hidden for clarity

    {SPRITE_ID::Button_Basic, "assets/sprites/basic_button.png",0,0,3,3},
};
```

Lets look at the pretty massive `RenderButton_Dynamic()`

```
// rendering.cpp
// RenderButton_Dynamic Part 1

void RenderButton_Dynamic(Button* button, bool is_selected, SDL_Renderer* renderer){
    assert(button->sprite->sprite_count_x == 3);
    assert(button->sprite->sprite_count_y == 3);

    uint8_t colorOverlay = is_selected ? 255: 230;
    SDL_Texture* texture = button->sprite->texture;
    SDL_FRect rect = button->rect;
```

we make sure that we're working with a `nine-slice` sprite using two `asserts`. We then fetch the `texture*` and `rect` from the `button->sprite` to make the callsites shorter. We also prepare our `colorOverlay` to darken the button if it is not the selected one.

```

// rendering.cpp
// RenderButton_Dynamic Part 2

float part_w = texture->w / 3.0;
float part_h = texture->h / 3.0;
float vertical_center_height = rect.h - (part_h * 2);
float horizontal_center_width = rect.w - (part_w * 2);

float right_x = rect.x + rect.w - part_w;
float bottom_y = rect.y + rect.h - part_h;
float center_y = rect.y + part_h;
float center_x = rect.x + part_w;

```

as we know our texture is a 3x3 atlas we can take the total width and height of the texture and divide it by 3. Then we get the size of one of the atlas pieces.

We then remove the size of two of these **parts** from the **width** and **height** of the buttons own size. This will give us the length of the stretching segments that will be used to fill the space between the corners of the button.

We'll be creating two pairs of **SDL_FRects** one list of **dst** aka destinations. These are the rects we'll render our texture into on the screen. and **src** aka source. These are the rects in our **atlas texture**. We'll be rendering 9 textures in this one function to reconstruct the entire button.

the **right_x, bottom_y etc** variables are used to calculate the x and y position of the different parts of the button.

```

// rendering.cpp
// RenderButton_Dynamic Part 3

SDL_FRect topLeftdst      = {rect.x,    rect.y,    part_w,                part_h
↪ };
SDL_FRect topRightdst     = {right_x,   rect.y,    part_w,                part_h
↪ };
SDL_FRect topCenterdst    = {center_x,   rect.y,    horizontal_center_width, part_h
↪ };
SDL_FRect bottomLeftdst   = {rect.x,     bottom_y, part_w,                part_h
↪ };
SDL_FRect bottomRightdst  = {right_x,    bottom_y, part_w,                part_h
↪ };
SDL_FRect bottomCenterdst = {center_x,    bottom_y, horizontal_center_width, part_h
↪ };
SDL_FRect centerLeftdst   = {rect.x,     center_y, part_w,
↪ vertical_center_height };
SDL_FRect centerRightdst  = {right_x,    center_y, part_w,
↪ vertical_center_height };
SDL_FRect centerdst       = {center_x,   center_y, horizontal_center_width,
↪ vertical_center_height };

```

I've aligned each parameter to make it easier to get an overview. Each of these 9 rects refer to a portion of the button that we'll be reconstructing on the screen. For example the `bottomRightdst` has its `x position` set to `right_x`, its `y` to `bottom_y` then its width and height is equal to `part_w/h`. We can see how only the stretching parts deviate from the `part_w/h` when it comes to setting the `width` and `height`. Try and intuit where the piece will be in the 3x3 grid and use the parameters to figure out how it would look.

```
// rendering.cpp
// RenderButton_Dynamic Part 4

SDL_FRect topLeftsrc      = {0,          0,          part_w, part_h};
SDL_FRect topRightsrc     = {part_w * 2, 0,          part_w, part_h};
SDL_FRect topCentersrc    = {part_w * 1, 0,          part_w, part_h};
SDL_FRect bottomLeftsrc   = {0,          part_h * 2, part_w, part_h};
SDL_FRect bottomRightsrc  = {part_w * 2, part_h * 2, part_w, part_h};
SDL_FRect bottomCentersrc = {part_w * 1, part_h * 2, part_w, part_h};
SDL_FRect centerLeftsrc   = {0,          part_h * 1, part_w, part_h};
SDL_FRect centerRightsrc  = {part_w * 2, part_h * 1, part_w, part_h};
SDL_FRect centersrc        = {part_w * 1, part_h * 1, part_w, part_h};
```

Because our `src` rectangles are not in the world itself it's easier to just select the appropriate square in the 3x3 grid by shifting along the `x` and `y` using `part_w/h`.

```
// rendering.cpp
// RenderButton_Dynamic Part 5

SDL_SetTextureScaleMode(texture, SDL_SCALEMODE_PIXELART);
SDL_SetTextureBlendMode(texture, SDL_BLENDMODE_BLEND);
SDL_SetTextureColorMod(texture, colorOverlay, colorOverlay, colorOverlay);
```

we really shouldn't be calling these each time but rather cache these settings when we create the Sprite in `spriteLibrary.cpp`. But we can worry about that later. These settings will help with adding the grey color and not getting blurry art when we scale the pixel art between the corners.

```
// rendering.cpp
// RenderButton_Dynamic Part 1

SDL_RenderTexture(renderer, texture, &topLeftsrc,      &topLeftdst      );
SDL_RenderTexture(renderer, texture, &topCentersrc,    &topCenterdst    );
SDL_RenderTexture(renderer, texture, &bottomCentersrc, &bottomCenterdst );
SDL_RenderTexture(renderer, texture, &centerLeftsrc,   &centerLeftdst   );
SDL_RenderTexture(renderer, texture, &centerRightsrc,  &centerRightdst  );
SDL_RenderTexture(renderer, texture, &bottomLeftsrc,   &bottomLeftdst   );
SDL_RenderTexture(renderer, texture, &topRightsrc,     &topRightdst     );
SDL_RenderTexture(renderer, texture, &bottomRightsrc,  &bottomRightdst  );
SDL_RenderTexture(renderer, texture, &centersrc,       &centerdst       );
```

Finally we take the `src` and `dst` pairs and render the `atlas` to the screen

in 9 steps. Each one responsible for one of the 3x3 squares. You can comment out these `RenderTexture()` calls to see what part of the button the draw. once you've unzipped `chapter 40 assets.zip` you can run the game and look at our non-blurry-non-bad-scaled start button!

The next step will be adding text ontop of the button

```
// button.h

struct Button {
    ButtonType type;
    SDL_FRect rect;
    Sprite* sprite;
    bool is_active;
    bool is_dynamic;
    FontAtlas* font; // new
    const char* text; // new
};
```

first lets put `STOP_CHAR` into `common.h` from `rendering.cpp` along with a small helper function.

```
// common.h
static const char STOP_CHAR = '\0';
inline bool IsStringEmpty(const char* str){
    return str == nullptr || str[0] == STOP_CHAR;
}
```

we'll use this to check if we've actually added any text to the button before we try and render this text.

At the bottom of `RenderButton()` and `RenderButton_Dynamic()` we'll add a call to `RenderText()`

```
// rendering.cpp
if(!IsStringEmpty(button->text)){
    float glyph_height = button->font->glyphs['H'].atlasPosition.h / 2.0;
    RenderText(button->font, button->text, renderer, nullptr, rect.x + (rect.w / 2.0),
↵ rect.y + (rect.h / 2.0) - glyph_height, Alignment::Centered);
}
```

we do some short equations to find the center of the button. Then we shift the text upwards by half the height of a “random” uppercase letter.

Lets refactor our `SetupButton()` to accept `FontAtlas*` and `const char*` as optional parameters

```
// button.h
void SetupButton(Button* button,
                 ButtonType type,
                 Sprite* spriteBuffer,
                 SDL_FRect rect,
                 Alignment mode,
                 FontAtlas* font = nullptr,
                 const char* text = nullptr);
```

the need to put the parameters on different lines to avoid the line becoming very very long is a sign that the function accepts to many variables. But that is not in itself a sign that we need to change it. But we should keep an eye on it

```

// button.cpp
void SetupButton(Button* button, ButtonType type, Sprite* spriteBuffer, SDL_FRect rect,
    ↪ Alignment mode, FontAtlas* font, const char* text){
    assert(type != ButtonType::NONE);
    button->type = type;
    button->rect = rect;
    if(mode == Alignment::Centered){
        button->rect.x -= button->rect.w / 2;
        button->rect.y -= button->rect.h / 2;
    }
    button->is_active = true;
    switch(button->type){
    case ButtonType::START_GAME:
        button->sprite = GetSprite(SPRITE_ID::Button_Basic, spriteBuffer);
        break;
    case ButtonType::QUIT:
        button->sprite = GetSprite(SPRITE_ID::Fallback, spriteBuffer);
        break;
    default:
        button->sprite = GetSprite(SPRITE_ID::Fallback, spriteBuffer);
        break;
    }

    // new code below
    bool hasText = !IsStringEmpty(text);

    if(font == nullptr){
        assert(!hasText);
    }
    if(hasText){
        assert(font != nullptr);
    }

    button->font = font;
    button->text = text;
}

```

We do two asserts to ensure that both the font and the text was set if either was assigned.

Then we assign `button->font/text` to the supplied parameters.

To have our font available at the callsite in `InitializeMenu()` we need to add it as a parameter

```

// mainmenu.h

void InitializeMenu(MainMenu* mainmenu, Sprite* spriteBuffer, FontAtlas* font,
↳ Memory::Arena* arena_main);

// mainmenu.cpp

void InitializeMenu(MainMenu* mainmenu, Sprite* spriteBuffer, FontAtlas* font,
↳ Memory::Arena* arena_main){
    assert(mainmenu->initialized == false);
    mainmenu->button_count = 2;
    mainmenu->buttons = ALLOC_ARRAY(arena_main, Button, mainmenu->button_count);

    SetupButton(&mainmenu->buttons[0],
                ButtonType::START_GAME,
                spriteBuffer,
                {SCREEN_WIDTH / 2.0, SCREEN_HEIGHT / 2.0, 300, 110},
                Alignment::Centered,
                font, // new
                "Start Game"); // new

    mainmenu->buttons[0].is_dynamic = true;

    SetupButton(&mainmenu->buttons[1], ButtonType::QUIT, spriteBuffer, {SCREEN_WIDTH /
↳ 2.0, (SCREEN_HEIGHT / 2.0) + 200, 200, 80}, Alignment::Centered, font, "Quit");

    mainmenu->background_horizon = GetSprite(SPRITE_ID::Menu_Horizon, spriteBuffer);
    mainmenu->background_cloud_back = GetSprite(SPRITE_ID::Menu_Cloud_Back, spriteBuffer);
    mainmenu->background_cloud_front = GetSprite(SPRITE_ID::Menu_Cloud_Front,
↳ spriteBuffer);
    mainmenu->background_middle = GetSprite(SPRITE_ID::Menu_Middle, spriteBuffer);
    mainmenu->background_front = GetSprite(SPRITE_ID::Menu_Front, spriteBuffer);

    mainmenu->activeButtonIndex = 0;
    mainmenu->initialized = true;
}

```

and add it when calling `InitializeMenu()`


```
// game.cpp

void Initialize(GameData* data, SDL_Window* window, SDL_Renderer* renderer){
    *data->ticks_total = 0;
    DEV::Initialize(window, renderer);
    InitializeAudioSystem(&data->audio, data->arena_main);
    AssetManagement::LoadAllSFX(&data->audio);
    AssetManagement::LoadAllSprites(data->spriteBuffer, renderer);
    data->imGui_context = ImGui::GetCurrentContext();
    AssetManagement::LoadFont(renderer, "assets/fonts/ByteBounce.ttf", &data->font, 48);
    AssetManagement::LoadAllTilesets(data->tilesetBuffer, data->arena_images);

    SDL_Texture* blackfade = GetSprite(SPRITE_ID::black_1x1,
↪ data->spriteBuffer)->texture;
    SDL_SetTextureBlendMode(blackfade, SDL_BLENDMODE_BLEND);
    InitializeGame(&data->scenes.gameplay, data->arena_levels, data->tilesetBuffer);
    // font parameter added
    InitializeMenu(&data->scenes.mainMenu, data->spriteBuffer, &data->font,
↪ data->arena_main);
    PlaySong(SONG_ID::THEME);
    ChangeScene(data, SCENE_TYPES::MAINMENU);
}
```

Now we can give a button text to render. That's cool!

40 Intermission I - Creating a release candidate

Sometime, not very likely for this project, we will want to be able to collect only the files we want to ship to our consumers. Our cache folder, ninja output and cmake files that are generated alongside our `.dll` and `exe` are not something we should ship to our consumers.

We can increase the capabilities of our `CMakeLists.txt` and our `CMakePresets.json` to give us access to a new parameter `--install` that we can call when compiling our project.

the `--install` parameter tells our compiler to run the `install()` function in our `CMakeLists.txt`, and that being responsible for copying over files into a specified directory. The `install()` function is used to tell cmake what files to consider for an install-version of our program as opposed to a debug version.

```
install(TARGETS ${PROJECT_NAME} DESTINATION .)
install(TARGETS ${DLL_NAME} RUNTIME DESTINATION .)
install(FILES ${DLL_FILES} DESTINATION .)
install(DIRECTORY ${CMAKE_BINARY_DIR}/assets DESTINATION .)
```

We're adding the following four `install()` function calls at the very bottom of our `CMakeLists.txt` these functions only run if the `--install` parameter has been passed to the compiler. Meaning that if we run `build game_name` as we usually do, these won't fire.

`TARGETS` are references to things that Cmake builds, in this case our `.exe` and `.dll` are added. We also add the `RUNTIME` parameter to `${DLL_NAME}` so cmake doesn't also add the `"nameofgame"_game.lib` the `.lib` files is only used during linking to give access to the contents of the `.dll` once

linking is finished this is not a file that is needed. `DESTINATION .` (note the ‘.’) means that the place where the files will show up is at the same folder that we will specify when calling `--install` from a newly created function inside our `powershell $profile`

`FILES` are any files found on disk not created during compilation, in this case we fetch `${DLL_FILES}` which are the collection of `.DLLs` we specified earlier in our `cmakelists.txt`.

`DIRECTORY` tells the `install()` function to copy an entire folder. We have to specify the path to it using `CMAKE_BINARY_DIR` a built in path variable that points to our `build` folder. We then add `/assets` to arrive on the assets folder.

In our `CMakepresets.json` we will be adding a new `configurePresets` and a new `buildPresets`. We do this by putting a comma ‘,’ at the end of the square brackets for the default presets then copying the entire preset block and changing the contents inside.

```

{
  "version": 10,
  "configurePresets": [
    {
      "name": "default",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "CMAKE_CXX_COMPILER": "clang++"
      }
    },
    {
      "name": "release",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Release",
        "CMAKE_CXX_COMPILER": "clang++",
        "CMAKE_INTERPROCEDURAL_OPTIMIZATION": "TRUE"
      }
    }
  ],
  "buildPresets": [
    {
      "name": "default",
      "configurePreset": "default"
    },
    {
      "name": "release",
      "configurePreset": "release"
    }
  ]
}

```

Note: previously we had our `version` set to `3`. The install of `cmake` that we grabbed at the beginning of the course has support for (at least) version `10`. This update is not strictly necessary but small performance increases are likely to have occurred between version 3 and 10.

our `release` preset still uses `/build` as the `binaryDir`. This is on purpose as we want to first build to our build folder then use our `install()` to copy over the relevant stuff.

```
"CMAKE_BUILD_TYPE": "Release",
```

This strips the debugging logic from our project, meaning that we will lose the ability to meaningfully check its behaviour in for example visual studio. But doing this will decrease the final program in size and make it run faster (just like that)

```
"CMAKE_INTERPROCEDURAL_OPTIMIZATION": "TRUE"
```

This one liner tells the compiler to perform optimization on our `.cpp` files, looking at all of them instead of individually. This will help boost performance with a tradeoff of making compilation slower. So for a release build it is perfect. We don't need the performance boost when we are building debug versions but would happily double our compile time if we can get a 5-15% performance boost for the consumer when building our release candidate.

```
"buildPresets": [  
  {  
    "name": "default",  
    "configurePreset": "default"  
  },  
  {  
    "name": "release",  
    "configurePreset": "release"  
  }  
]
```

We also need a buildPreset so we can tell which `configurePreset` should be used when calling `cmake` from our `powershell build function`

lastly we need to create our `release()` function inside `powershell $profile`

```

function release {
    param([Parameter(Mandatory=$true)][string]$project)

    goto $project

    $config = GetConfig $project
    $sourceDir = $config.path
    $buildDir = "$SourceDir/build"
    $releaseDir = "$SourceDir/release"

    Write-Host "begin creating release candidate..."
    Remove-Item -Recurse -Force $buildDir -ErrorAction SilentlyContinue
    Remove-Item -Recurse -Force $releaseDir -ErrorAction SilentlyContinue

    cmake --preset "release"
    if ($LASTEXITCODE -ne 0) {
        Write-Host "fetching cmake preset failed - aborting.";
        return
    }

    cmake --build --preset "release"
    if ($LASTEXITCODE -ne 0) {
        Write-Host "build failed - aborting.";
        return
    }

    cmake --install build --prefix "$sourceDir/release"
    if ($LASTEXITCODE -ne 0) {
        Write-Host "install commands failed - aborting.";
        return
    }
    Write-Host "release candidate successfully created."
}

```

we jump to our project directory then store paths into our temporary cache Variables

```

goto $project

$config = GetConfig $project
$sourceDir = $config.path
$buildDir = "$SourceDir/build"
$releaseDir = "$SourceDir/release"

```

```
Remove-Item -Recurse -Force $buildDir -ErrorAction SilentlyContinue
Remove-Item -Recurse -Force $releaseDir -ErrorAction SilentlyContinue
```

we remove everything from our `build` and `release` folders so we are sure that every file is brand new and nothing is lying around and causing issues

```
cmake --preset "release"
if ($LASTEXITCODE -ne 0) {
    Write-Host "fetching cmake preset failed - aborting.";
    return
}
```

we tell cmake to use our new `release` `buildPresets`. And if that function exited with an exit code that was `not equal` `-ne` to 0 we stop the rest of the function as we have failed to load our preset.

```
cmake --build --preset "release"
if ($LASTEXITCODE -ne 0) {
    Write-Host "build failed - aborting.";
    return
}
```

we then build our project putting everything into our `/build` folder. And once again, if we failed to build, then we don't move on to the next part of the function

```
cmake --install build --prefix "$releaseDir"
if ($LASTEXITCODE -ne 0) {
    Write-Host "install commands failed - aborting.";
    return
}
```

then we do the new part, we tell `cmake` to `--install` from our `build` folder and the `--prefix` parameter tells `cmake` to place the installed content inside the specified path.

the `install()` functions in `cmakelists.txt` our new `buildPresets` and `configurePresets` in `cmakePresets.json` and this

new `release` function are all the things we need to create our optimized and stripped down release candidate.

41 Intermission II - Debugging in Visual Studio

We want to be able to understand the flow of our code, and peek at variables to look at their values. We do this by using a debugger. We will be downloading the IDE `Visual Studio` and installing its `Community` version.

Download link: <https://visualstudio.microsoft.com/>

It's important that we download `Visual Studio` and not the smaller lightweight `Visual Studio Code`.

`Visual Studio` as a complete IDE `Integrated Development Environment` meaning that is a one-stop-shop for everything someone “would need” when working with programming. But it's bloated, slow and cumbersome. We will be using its debugging tools though.

Meaning that our day-to-day text editor is `Helix` and our debugger is `Visual Studio`.

With visual studio installed we can chose to open our `root folder` inside `Visual Studio`, then on the right-side pane we can right click our games `.exe` from our build folder and select `set as startup item`, this will tell Visual Studio to run that `.exe` when we press the green button.

Now we can add `breakpoints` to our code. A `breakpoint` is a notice to pause code execution once we hit a specific line of code. This means that we can pause our program at a critical moment to explore the state of our variables as well as stepping through our code as it runs, line by line.

We add breakpoints by pressing the leftmost side of a line of code. When done properly a little red circle will indicate the `breakpoint`. We can press

that red circle again to remove it.

With a breakpoint set we can press “play” or **F5** and once our code hits the breakpoint it will pause. At this point we can hover our cursor above variables to evaluate their content. We can stop our debugging by pressing **shift+F5** or by pressing the **red stop button**.

Once our program has paused on a breakpoint we can use **F11** and **F10** to move the program forward

F10 steps to the next visible line below the current one **F11** goes to the next piece of code being executed, jumping into a function if necessary. **F10** does not enter functions instead it runs the entire function as if it was a single line of code.

We also have **Shift+F11** that goes back out of a function that **F11** dove into. Meaning that we can get back to the place where the function was called.

By using **F10** and **F11** we can learn how our code flows and find bugs that would otherwise be very hard to reason about.

We can also hover our cursor just to the left of a line that is below our current one, a small green play button will appear. Pressing this will run code to that point before stopping again. This is very useful if we want to jump past a for-loop that is going to run the same code 100+ times. Sparing us clicking **F10** A LOT.

We might also want to pause execution on a line of code, but only if a certain variable has a specific value. For example a **TakeDamage()** function might only be something we want to evaluate if the damage taken would kill the player. For this we have **conditional breakpoints**. After we have created

a breakpoint we can right-click it and select `condition` inside this new window we can add a little boolean logic like `health <= damage`. Then the code will only pause if the incoming damage would reduce the health variable to 0 or below.

There is more we can do with `breakpoints` but this covers the fundamentals!

42 Intermission III - Github Part I

What if we chucked our laptop into the sea? Then everything we had been working on would be lost. This won't do.

We could use an external harddrive or store backups of our project on a cloud service like Dropbox, and for a solo-made game that, honestly, could work. But on larger or more serious projects we can leverage the Git ecosystem to keep our project saved on the cloud, up-to-date and synced across multiple computers.

Github is the platform where your project is hosted, using the **git** architecture. We use a series of commands to let **Git** know which files we want to **push** aka **upload** to our Github **repository**. A **repository** is the online storage of our files, as well as their changes in a timeline.

When we **push** a file to **Github**, if it was the first time we did so, we send the entire file. From this point forward, when we **push** our file we only push the latest changes, meaning that the amount (in bytes) being uploaded will be far smaller than if we had to reupload the entire thing each time. Git tracks our changes for us.

A bundle of changed files are called a **commit**. We give each **commit** a name and a non-mandatory description to help us (and our team) know what has changed. These commits then live as a timeline of changes, allowing us to **revert** back to an old version of our project if we would like.

If someone on our team has **pushed** a **commit** to our **repository** on **Github** using **Git** then we in turn can **fetch** the **commits** on **github** that are not yet synced to our computer. A **fetch** checks the difference between our machine and our **repository** then allowing us to **pull** aka

download those commits.

When we have changes we commit them then push them. When we want to download the latest changes from github then we fetch them to see if any existed, then pull them into our machine.

We can use Git in one of two ways

- 1) We can use powershell to send commands to Git directly
- 2) We can use a software like Github Desktop to do the same stuff, but with some nice helpful buttons instead of code.

<https://desktop.github.com/download/>

Besides the Github Desktop client we will also need an account on github.com.

Note: This account will be your portfolio, this if maintained nicely will be a huge asset for you when applying for internships and work. So please pick a sensible account name.

Once our account is set up we can log in to Github Desktop.

Now we can use file->new repository to start working on a new project. Or if we have the URL to a github project that we've been invited to collaborate on we can clone that repository from file->clone repository

Once we have our repository locally we can start committing changes and pushing and pulling those commits to and from Github.

For a more in-depth look, check out the documentation:
<https://docs.github.com/en/desktop>

With this we can do the very basics in Github. Later you will learn about branches and pull requests and merge conflicts.